

secs_q84

Generated by Doxygen 1.14.0

1 Topic Index	1
1.1 Topics	1
2 Data Structure Index	3
2.1 Data Structures	3
3 File Index	5
3.1 File List	5
4 Topic Documentation	7
4.1 ADC3: Analog-to-Digital Converter with Context	7
4.1.1 Detailed Description	9
4.1.2 Typedef Documentation	9
4.1.2.1 adc_result_t	9
4.1.3 Enumeration Type Documentation	9
4.1.3.1 ADC_channel_t	9
4.1.3.2 ADC_context_t	10
4.1.4 Function Documentation	10
4.1.4.1 ADC_ClearAccumulator()	10
4.1.4.2 ADC_DefineSetPoint()	10
4.1.4.3 ADC_DisableChannelScan()	11
4.1.4.4 ADC_DisableChannelSequencer()	11
4.1.4.5 ADC_DisableChargePump()	11
4.1.4.6 ADC_DisableContinuousConversion()	12
4.1.4.7 ADC_DischargeSampleCapacitor()	12
4.1.4.8 ADC_EnableChannelScan()	13
4.1.4.9 ADC_EnableChannelSequencer()	13
4.1.4.10 ADC_EnableChargePump()	14
4.1.4.11 ADC_EnableContinuousConversion()	14
4.1.4.12 ADC_EnableDoubleSampling()	15
4.1.4.13 ADC_GetAccumulatorValue()	15
4.1.4.14 ADC_GetConversionResult()	16
4.1.4.15 ADC_GetConversionStageStatus()	16
4.1.4.16 ADC_GetCurrentCountofConversions()	17
4.1.4.17 ADC_GetErrorCalculation()	17
4.1.4.18 ADC_GetFilterValue()	18
4.1.4.19 ADC_GetPreviousResult()	18
4.1.4.20 ADC_GetSingleConversion()	19
4.1.4.21 ADC_HasAccumulatorOverflowed()	19
4.1.4.22 ADC_HasErrorCrossedLowerThreshold()	20
4.1.4.23 ADC_HasErrorCrossedUpperThreshold()	20
4.1.4.24 ADC_Initialize()	21
4.1.4.25 ADC_IsConversionDone()	21

4.1.4.26 ADC_LoadAcquisitionRegister()	22
4.1.4.27 ADC_SelectContext()	22
4.1.4.28 ADC_SetActiveClockTuningInterruptHandler()	23
4.1.4.29 ADC_SetADIIInterruptHandler()	23
4.1.4.30 ADC_SetContext1ThresholdInterruptHandler()	24
4.1.4.31 ADC_SetContext2ThresholdInterruptHandler()	24
4.1.4.32 ADC_SetContext3ThresholdInterruptHandler()	25
4.1.4.33 ADC_SetContext4ThresholdInterruptHandler()	25
4.1.4.34 ADC_SetLowerThreshold()	26
4.1.4.35 ADC_SetPrechargeTime()	26
4.1.4.36 ADC_SetRepeatCount()	27
4.1.4.37 ADC_SetStopOnInterrupt()	27
4.1.4.38 ADC_SetUpperThreshold()	28
4.1.4.39 ADC_StartChannelSequencer()	28
4.1.4.40 ADC_StartConversion()	29
4.1.4.41 ADC_StopConversion()	29
5 Data Structure Documentation	31
5.1 B_type Struct Reference	31
5.1.1 Detailed Description	31
5.1.2 Field Documentation	31
5.1.2.1 dim_delay	31
5.1.2.2 display_update	31
5.1.2.3 one_sec_flag	31
5.2 block10 Union Reference	32
5.2.1 Detailed Description	32
5.2.2 Field Documentation	32
5.2.2.1 b	32
5.2.2.2 block	32
5.3 block10_type Struct Reference	32
5.3.1 Detailed Description	32
5.3.2 Field Documentation	33
5.3.2.1 bidh	33
5.3.2.2 bidl	33
5.3.2.3 didh	33
5.3.2.4 didl	33
5.3.2.5 ebit	33
5.3.2.6 function	33
5.3.2.7 rbit	33
5.3.2.8 stream	34
5.3.2.9 systemb	34
5.3.2.10 wbit	34

5.4 D_data Struct Reference	34
5.4.1 Detailed Description	34
5.4.2 Field Documentation	34
5.4.2.1 last_info	34
5.4.2.2 lcd	35
5.4.2.3 vterm	35
5.5 gem_display_type Struct Reference	35
5.5.1 Detailed Description	35
5.5.2 Field Documentation	35
5.5.2.1 block	35
5.5.2.2 delay	35
5.5.2.3 message	36
5.5.2.4 stack	36
5.6 gem_message_type Struct Reference	36
5.6.1 Detailed Description	36
5.6.2 Field Documentation	36
5.6.2.1 block	36
5.6.2.2 delay	36
5.6.2.3 message	37
5.6.2.4 stack	37
5.7 header10 Struct Reference	37
5.7.1 Detailed Description	37
5.7.2 Field Documentation	37
5.7.2.1 block	37
5.7.2.2 checksum	37
5.7.2.3 length	38
5.8 header12 Struct Reference	38
5.8.1 Detailed Description	38
5.8.2 Field Documentation	38
5.8.2.1 block	38
5.8.2.2 checksum	38
5.8.2.3 data	38
5.8.2.4 length	39
5.9 header13 Struct Reference	39
5.9.1 Detailed Description	39
5.9.2 Field Documentation	39
5.9.2.1 block	39
5.9.2.2 checksum	39
5.9.2.3 data	39
5.9.2.4 length	40
5.10 header14 Struct Reference	40
5.10.1 Detailed Description	40

5.10.2 Field Documentation	40
5.10.2.1 block	40
5.10.2.2 checksum	40
5.10.2.3 data	40
5.10.2.4 length	41
5.11 header153 Struct Reference	41
5.11.1 Detailed Description	41
5.11.2 Field Documentation	41
5.11.2.1 block	41
5.11.2.2 checksum	41
5.11.2.3 data	41
5.11.2.4 length	42
5.12 header17 Struct Reference	42
5.12.1 Detailed Description	42
5.12.2 Field Documentation	42
5.12.2.1 block	42
5.12.2.2 checksum	42
5.12.2.3 data	42
5.12.2.4 length	43
5.13 header18 Struct Reference	43
5.13.1 Detailed Description	43
5.13.2 Field Documentation	43
5.13.2.1 block	43
5.13.2.2 checksum	43
5.13.2.3 data	43
5.13.2.4 length	44
5.14 header24 Struct Reference	44
5.14.1 Detailed Description	44
5.14.2 Field Documentation	44
5.14.2.1 block	44
5.14.2.2 checksum	44
5.14.2.3 data	44
5.14.2.4 datam	45
5.14.2.5 length	45
5.15 header254 Struct Reference	45
5.15.1 Detailed Description	45
5.15.2 Field Documentation	45
5.15.2.1 block	45
5.15.2.2 checksum	45
5.15.2.3 data	46
5.15.2.4 length	46
5.16 header26 Struct Reference	46

5.16.1 Detailed Description	46
5.16.2 Field Documentation	46
5.16.2.1 block	46
5.16.2.2 checksum	46
5.16.2.3 data	47
5.16.2.4 datam	47
5.16.2.5 length	47
5.17 header27 Struct Reference	47
5.17.1 Detailed Description	47
5.17.2 Field Documentation	47
5.17.2.1 block	47
5.17.2.2 checksum	48
5.17.2.3 data	48
5.17.2.4 length	48
5.18 header33 Struct Reference	48
5.18.1 Detailed Description	48
5.18.2 Field Documentation	48
5.18.2.1 block	48
5.18.2.2 checksum	49
5.18.2.3 data	49
5.18.2.4 length	49
5.19 response_type Struct Reference	49
5.19.1 Detailed Description	49
5.19.2 Field Documentation	49
5.19.2.1 header	49
5.19.2.2 length	50
5.19.2.3 reply	50
5.19.2.4 reply_length	50
5.19.2.5 respond	50
5.20 ringBufS_t Struct Reference	50
5.20.1 Detailed Description	50
5.20.2 Field Documentation	50
5.20.2.1 buf	50
5.20.2.2 count	51
5.20.2.3 head	51
5.20.2.4 tail	51
5.21 spi_configuration_t Struct Reference	51
5.21.1 Detailed Description	51
5.21.2 Field Documentation	52
5.21.2.1 baud	52
5.21.2.2 con0	52
5.21.2.3 con1	52

5.21.2.4 con2	52
5.21.2.5 operation	52
5.22 spi_link_type Struct Reference	52
5.22.1 Detailed Description	53
5.22.2 Field Documentation	53
5.22.2.1 config	53
5.22.2.2 delay	53
5.22.2.3 int_count	53
5.22.2.4 LCD_DATA	53
5.22.2.5 LCD_TIMER	53
5.22.2.6 SPI_AUX	53
5.22.2.7 SPI_LCD	54
5.22.2.8 txbuf	54
5.23 terminal_type Struct Reference	54
5.23.1 Detailed Description	54
5.23.2 Field Documentation	54
5.23.2.1 ack	54
5.23.2.2 ceid	55
5.23.2.3 cmdlen	55
5.23.2.4 help_temp	55
5.23.2.5 host_display_ack	55
5.23.2.6 info	55
5.23.2.7 log_num	55
5.23.2.8 log_seq	55
5.23.2.9 mcode	55
5.23.2.10 mesgid	56
5.23.2.11 mparm	56
5.23.2.12 TID	56
5.24 uart1_status_t Union Reference	56
5.24.1 Detailed Description	56
5.24.2 Field Documentation	57
5.24.2.1 ferr	57
5.24.2.2 oerr	57
5.24.2.3 perr	57
5.24.2.4 reserved	57
5.24.2.5 status	57
5.25 uart2_status_t Union Reference	57
5.25.1 Detailed Description	58
5.25.2 Field Documentation	58
5.25.2.1 ferr	58
5.25.2.2 oerr	58
5.25.2.3 perr	58

5.25.2.4 reserved	58
5.25.2.5 status	59
5.26 V_data Struct Reference	59
5.26.1 Detailed Description	60
5.26.2 Field Documentation	60
5.26.2.1 abort	60
5.26.2.2 alarm	60
5.26.2.3 all_errors	61
5.26.2.4 br_total	61
5.26.2.5 brn_total	61
5.26.2.6 bt_total	61
5.26.2.7 btn_total	61
5.26.2.8 buf	61
5.26.2.9 checksum_error	61
5.26.2.10 debug	61
5.26.2.11 e_types	62
5.26.2.12 ebit	62
5.26.2.13 equip_timeout	62
5.26.2.14 error	62
5.26.2.15 uart	62
5.26.2.16 failed_receive	62
5.26.2.17 failed_send	62
5.26.2.18 flipper	62
5.26.2.19 function	63
5.26.2.20 g_state	63
5.26.2.21 help	63
5.26.2.22 help_id	63
5.26.2.23 info	63
5.26.2.24 llid	63
5.26.2.25 m_l_state	63
5.26.2.26 mode_pwm	63
5.26.2.27 msg_error	64
5.26.2.28 msg_ret	64
5.26.2.29 ping	64
5.26.2.30 ping_count	64
5.26.2.31 queue	64
5.26.2.32 r_checksum	64
5.26.2.33 r_l_state	64
5.26.2.34 rbit	64
5.26.2.35 rerror	65
5.26.2.36 response	65
5.26.2.37 rx_rs232	65

5.26.2.38 rx_total	65
5.26.2.39 rx_volts	65
5.26.2.40 s_state	65
5.26.2.41 sequences	65
5.26.2.42 set_sequ	65
5.26.2.43 sid	66
5.26.2.44 speed_spin	66
5.26.2.45 stack	66
5.26.2.46 stream	66
5.26.2.47 systemb	66
5.26.2.48 t_checksum	66
5.26.2.49 t_l_state	66
5.26.2.50 terminal	66
5.26.2.51 testing	67
5.26.2.52 ticker	67
5.26.2.53 ticks	67
5.26.2.54 timer_error	67
5.26.2.55 tx_rs232	67
5.26.2.56 tx_total	67
5.26.2.57 tx_volts	67
5.26.2.58 uart	67
5.26.2.59 uart_speed_fast	68
5.26.2.60 ui_state	68
5.26.2.61 ui_sw	68
5.26.2.62 v_rx_line	68
5.26.2.63 v_tx_line	68
5.26.2.64 vterm	68
5.26.2.65 vterm_switch	68
5.26.2.66 wbit	69
5.27 V_help Struct Reference	69
5.27.1 Detailed Description	69
5.27.2 Field Documentation	69
5.27.2.1 display	69
5.27.2.2 extrams	69
5.27.2.3 message	69
6 File Documentation	71
6.1 0805L075WR.c	71
6.2 FUSC2114X125N.d	71
6.3 eadog.p1.d	72
6.4 gemsecs.p1.d	72
6.5 main.p1.d	72

6.6 adc.p1.d	73
6.7 device_config.p1.d	73
6.8 dma1.p1.d	73
6.9 interrupt_manager.p1.d	73
6.10 mcc.p1.d	73
6.11 memory.p1.d	74
6.12 pin_manager.p1.d	74
6.13 pwm2_16bit.p1.d	74
6.14 spi1.p1.d	74
6.15 tmr2.p1.d	74
6.16 tmr5.p1.d	74
6.17 tmr6.p1.d	74
6.18 uart1.p1.d	74
6.19 uart2.p1.d	75
6.20 mconfig.p1.d	75
6.21 rs232.p1.d	75
6.22 timers.p1.d	75
6.23 BZT52B5V6-TP.c	75
6.24 SOD3716X135N.d	76
6.25 doprnt.d	76
6.26 eadog.c	77
6.27 eadog.h	82
6.28 gemsecs.c	84
6.29 gemsecs.h	104
6.30 secs_q84.X/main.c File Reference	106
6.30.1 Detailed Description	107
6.30.2 Macro Definition Documentation	107
6.30.2.1 __DEFINED_int24_t	107
6.30.2.2 I_TRACE	107
6.30.2.3 M_TRACE	108
6.30.3 Typedef Documentation	108
6.30.3.1 int24_t	108
6.30.4 Function Documentation	108
6.30.4.1 main()	108
6.30.4.2 onesecc_io()	108
6.30.4.3 spinners()	108
6.30.5 Variable Documentation	108
6.30.5.1 B	108
6.30.5.2 build_date	109
6.30.5.3 build_time	109
6.30.5.4 D	109
6.30.5.5 faker	109

6.30.5.6 GEM_TEXT	109
6.30.5.7 H10	109
6.30.5.8 H12	109
6.30.5.9 H13	110
6.30.5.10 H14	110
6.30.5.11 H153	110
6.30.5.12 H17	110
6.30.5.13 H254	111
6.30.5.14 H26	111
6.30.5.15 H33	111
6.30.5.16 HC33	111
6.30.5.17 mode_sw	112
6.30.5.18 r_block	112
6.30.5.19 S	112
6.30.5.20 spi_link	112
6.30.5.21 tickCount	112
6.30.5.22 V	112
6.31 main.c	113
6.32 adc.c	128
6.33 adc.h	133
6.34 clc1.c	135
6.35 clc1.h	136
6.36 clkref.c	136
6.37 clkref.h	137
6.38 device_config.c	137
6.39 device_config.h	140
6.40 dma1.c	140
6.41 dma1.h	142
6.42 dma2.c	143
6.43 dma2.h	144
6.44 dsm.c	145
6.45 dsm.h	146
6.46 ext_int.c	147
6.47 ext_int.h	148
6.48 interrupt_manager.c	150
6.49 interrupt_manager.h	150
6.50 mcc.c	151
6.51 mcc.h	152
6.52 memory.c	153
6.53 memory.h	156
6.54 nco1.c	157
6.55 nco1.h	158

6.56 pin_manager.c	158
6.57 pin_manager.h	159
6.58 pwm2_16bit.c	163
6.59 pwm2_16bit.h	166
6.60 pwm8.c	167
6.61 pwm8.h	167
6.62 spi1.c	168
6.63 spi1.h	170
6.64 tmr2.c	170
6.65 tmr2.h	172
6.66 tmr5.c	176
6.67 tmr5.h	178
6.68 tmr6.c	179
6.69 tmr6.h	181
6.70 uart1.c	186
6.71 uart1.h	190
6.72 uart2.c	191
6.73 uart2.h	196
6.74 mconfig.c	197
6.75 mconfig.h	201
6.76 msg_text.h	201
6.77 mydisplay.h	202
6.78 ringbufs.c	203
6.79 ringbufs.h	204
6.80 rs232.c	204
6.81 rs232.h	205
6.82 timers.c	206
6.83 timers.h	206
6.84 secs_q84.X/vconfig.h File Reference	207
6.84.1 Detailed Description	210
6.84.2 Macro Definition Documentation	210
6.84.2.1 BDELAY	210
6.84.2.2 DBUG_VTERM	210
6.84.2.3 DDELAY	210
6.84.2.4 DEFAULT_TID	210
6.84.2.5 DFLIP	210
6.84.2.6 DLED	211
6.84.2.7 E_OSCREEN	211
6.84.2.8 E_SSCREEN	211
6.84.2.9 ERROR_CHECKSUM	211
6.84.2.10 ERROR_COMM	211
6.84.2.11 HBTL	211

6.84.2.12 HBTS	211
6.84.2.13 HELP_VTERM	211
6.84.2.14 INFO_VTERM	212
6.84.2.15 LDELAY	212
6.84.2.16 LINE_LIMIT_H	212
6.84.2.17 LINE_LIMIT_LOW	212
6.84.2.18 LINE_LIMIT_MARK	212
6.84.2.19 LINE_LIMIT_OPEN	212
6.84.2.20 LINE_MARK_V	212
6.84.2.21 LINE_OPEN_V	212
6.84.2.22 LINE_RECV_V	213
6.84.2.23 LINE_SPACE_V	213
6.84.2.24 MAIN_VTERM	213
6.84.2.25 MAX_BUF	213
6.84.2.26 MAX_HELP_STRLEN	213
6.84.2.27 MAX_INFO	213
6.84.2.28 MAX_LCD_LINES	213
6.84.2.29 MAX_LINE	213
6.84.2.30 MAX_SHAPES	214
6.84.2.31 MAX_STRLEN	214
6.84.2.32 MAX_TERM	214
6.84.2.33 MAX_TEXT	214
6.84.2.34 MAX_VTERM	214
6.84.2.35 NHD	214
6.84.2.36 PINGER	214
6.84.2.37 RTY	214
6.84.2.38 S10F3_STR_POS	215
6.84.2.39 S10F3_TID_POS	215
6.84.2.40 SDELAY	215
6.84.2.41 SEQDELAY	215
6.84.2.42 SLED	215
6.84.2.43 SPIN_CHAR	215
6.84.2.44 SPIN_VAL_UPDATE	215
6.84.2.45 SWITCH_DURATION	216
6.84.2.46 SWITCH_VTERM	216
6.84.2.47 T1	216
6.84.2.48 T2	216
6.84.2.49 T3	216
6.84.2.50 T4	216
6.84.2.51 TDELAY	216
6.84.2.52 TRACE	216
6.84.2.53 TX_RESERVE	217

6.84.2.54 UART_SPEED_EADR	217
6.84.2.55 UART_SPEED_LOCK_EADR	217
6.84.2.56 V_OSCREEN	217
6.84.2.57 V_SSCREEN	217
6.84.2.58 VER	217
6.84.2.59 Y2KTIME	217
6.84.3 Enumeration Type Documentation	218
6.84.3.1 D_CODES	218
6.84.3.2 GEM_EQUIP	218
6.84.3.3 GEM_STATES	218
6.84.3.4 LINK_ERRORS	218
6.84.3.5 LINK_STATES	218
6.84.3.6 MSG_ERRORS	218
6.84.3.7 P_CODES	218
6.84.3.8 RECV_ERRORS	219
6.84.3.9 SEND_ERRORS	219
6.84.3.10 SEQ_STATES	219
6.84.3.11 TICKER_VAL	219
6.84.3.12 UI_STATES	219
6.84.4 Function Documentation	219
6.84.4.1 spinners()	219
6.84.4.2 UART1_Initialize19200()	219
6.84.4.3 UART2_Initialize19200()	220
6.84.5 Variable Documentation	220
6.84.5.1 B	220
6.84.5.2 build_date	220
6.84.5.3 build_time	220
6.84.5.4 GEM_TEXT	220
6.84.5.5 spin	220
6.85 vconfig.h	221
7 Examples	225
7.1 void	225
7.2 Use	225
7.3 uint16_t	226
7.4 /root/vtouch_v2/secs_q84.X/mcc_generated_files/pwm2_16bit.h	226
Index	229

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

ADC3: Analog-to-Digital Converter with Context	7
--	-------------------

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

B_type	31
block10	32
block10_type	32
D_data	34
gem_display_type	35
gem_message_type	36
header10	37
header12	38
header13	39
header14	40
header153	41
header17	42
header18	43
header24	44
header254	45
header26	46
header27	47
header33	48
response_type	49
ringBufS_t	50
spi1_configuration_t	
SPI1 Generated Driver File	51
spi_link_type	52
terminal_type	54
uart1_status_t	
Section: Data Type Definitions	56
uart2_status_t	
Section: Data Type Definitions	57
V_data	59
V_help	69

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

secs_q84.X/ eadog.c	77
secs_q84.X/ eadog.h	82
secs_q84.X/ gemsecs.c	84
secs_q84.X/ gemsecs.h	104
secs_q84.X/ main.c	
Lets get going with the code	106
secs_q84.X/ mconfig.c	197
secs_q84.X/ mconfig.h	201
secs_q84.X/ msg_text.h	201
secs_q84.X/ mydisplay.h	202
secs_q84.X/ ringbufs.c	203
secs_q84.X/ ringbufs.h	204
secs_q84.X/ rs232.c	204
secs_q84.X/ rs232.h	205
secs_q84.X/ timers.c	206
secs_q84.X/ timers.h	206
secs_q84.X/ vconfig.h	
Software version and a brief doc for each version changes	207
secs_q84.X/0805L075WR/PADS/ 0805L075WR.c	71
secs_q84.X/0805L075WR/PADS/ FUSC2114X125N.d	71
secs_q84.X/build/default/production/ eadog.p1.d	72
secs_q84.X/build/default/production/ gemsecs.p1.d	72
secs_q84.X/build/default/production/ main.p1.d	72
secs_q84.X/build/default/production/ mconfig.p1.d	75
secs_q84.X/build/default/production/ rs232.p1.d	75
secs_q84.X/build/default/production/ timers.p1.d	75
secs_q84.X/build/default/production/mcc_generated_files/ adc.p1.d	73
secs_q84.X/build/default/production/mcc_generated_files/ device_config.p1.d	73
secs_q84.X/build/default/production/mcc_generated_files/ dma1.p1.d	73
secs_q84.X/build/default/production/mcc_generated_files/ interrupt_manager.p1.d	73
secs_q84.X/build/default/production/mcc_generated_files/ mcc.p1.d	73
secs_q84.X/build/default/production/mcc_generated_files/ memory.p1.d	74
secs_q84.X/build/default/production/mcc_generated_files/ pin_manager.p1.d	74
secs_q84.X/build/default/production/mcc_generated_files/ pwm2_16bit.p1.d	74
secs_q84.X/build/default/production/mcc_generated_files/ spi1.p1.d	74

secs_q84.X/build/default/production/mcc_generated_files/tmr2.p1.d	74
secs_q84.X/build/default/production/mcc_generated_files/tmr5.p1.d	74
secs_q84.X/build/default/production/mcc_generated_files/tmr6.p1.d	74
secs_q84.X/build/default/production/mcc_generated_files/uart1.p1.d	74
secs_q84.X/build/default/production/mcc_generated_files/uart2.p1.d	75
secs_q84.X/BZT52B5V6-TP/PADS/BZT52B5V6-TP.c	75
secs_q84.X/BZT52B5V6-TP/PADS/SOD3716X135N.d	76
secs_q84.X/dist/default/production/doprnt.d	76
secs_q84.X/mcc_generated_files/adc.c	128
secs_q84.X/mcc_generated_files/adc.h	133
secs_q84.X/mcc_generated_files/clc1.c	135
secs_q84.X/mcc_generated_files/clc1.h	136
secs_q84.X/mcc_generated_files/clkref.c	136
secs_q84.X/mcc_generated_files/clkref.h	137
secs_q84.X/mcc_generated_files/device_config.c	137
secs_q84.X/mcc_generated_files/device_config.h	140
secs_q84.X/mcc_generated_files/dma1.c	140
secs_q84.X/mcc_generated_files/dma1.h	142
secs_q84.X/mcc_generated_files/dma2.c	143
secs_q84.X/mcc_generated_files/dma2.h	144
secs_q84.X/mcc_generated_files/dsm.c	145
secs_q84.X/mcc_generated_files/dsm.h	146
secs_q84.X/mcc_generated_files/ext_int.c	147
secs_q84.X/mcc_generated_files/ext_int.h	148
secs_q84.X/mcc_generated_files/interrupt_manager.c	150
secs_q84.X/mcc_generated_files/interrupt_manager.h	150
secs_q84.X/mcc_generated_files/mcc.c	151
secs_q84.X/mcc_generated_files/mcc.h	152
secs_q84.X/mcc_generated_files/memory.c	153
secs_q84.X/mcc_generated_files/memory.h	156
secs_q84.X/mcc_generated_files/nco1.c	157
secs_q84.X/mcc_generated_files/nco1.h	158
secs_q84.X/mcc_generated_files/pin_manager.c	158
secs_q84.X/mcc_generated_files/pin_manager.h	159
secs_q84.X/mcc_generated_files/pwm2_16bit.c	163
secs_q84.X/mcc_generated_files/pwm2_16bit.h	166
secs_q84.X/mcc_generated_files/pwm8.c	167
secs_q84.X/mcc_generated_files/pwm8.h	167
secs_q84.X/mcc_generated_files/spi1.c	168
secs_q84.X/mcc_generated_files/spi1.h	170
secs_q84.X/mcc_generated_files/tmr2.c	170
secs_q84.X/mcc_generated_files/tmr2.h	172
secs_q84.X/mcc_generated_files/tmr5.c	176
secs_q84.X/mcc_generated_files/tmr5.h	178
secs_q84.X/mcc_generated_files/tmr6.c	179
secs_q84.X/mcc_generated_files/tmr6.h	181
secs_q84.X/mcc_generated_files/uart1.c	186
secs_q84.X/mcc_generated_files/uart1.h	190
secs_q84.X/mcc_generated_files/uart2.c	191
secs_q84.X/mcc_generated_files/uart2.h	196

Chapter 4

Topic Documentation

4.1 ADC3: Analog-to-Digital Converter with Context

ADC Generated Driver API Header File.

Typedefs

- typedef uint16_t [adc_result_t](#)
This typedef should be used for result of A/D conversion.

Enumerations

- enum [ADC_channel_t](#) {
 channel_ANA1 = 0x1 , **channel_ANA2** = 0x2 , **channel_VSS** = 0x3B , **channel_Temp** = 0x3C ,
 channel_DAC1 = 0x3D , **channel_FVR_Buffer1** = 0x3E , **channel_FVR_Buffer2** = 0x3F }
This enumeration contains available ADC channels.
- enum [ADC_context_t](#) { **CONTEXT_1** }
This enumeration contains available ADC contexts.

Functions

- void [ADC_Initialize](#) (void)
This API initializes the ADC module.
- void [ADC_EnableChannelSequencer](#) (void)
This API enables ADC channel sequencer.
- void [ADC_DisableChannelSequencer](#) (void)
This API disables ADC channel sequencer.
- void [ADC_StartChannelSequencer](#) (void)
This API starts the channel sequencer by enabling GO bit.
- void [ADC_SelectContext](#) ([ADC_context_t](#) context)
This API is used to select an ADC context to perform read/write operations on context specific registers.
- void [ADC_EnableChannelScan](#) ([ADC_context_t](#) context)
This API is used to include a channel context in the scan sequence.
- void [ADC_DisableChannelScan](#) ([ADC_context_t](#) context)

- This API is used to exclude a channel context from the scan sequence.*

 - void [ADC_StartConversion](#) ([ADC_channel_t](#) channel)

This API starts A/D conversion on selected channel.
- bool [ADC_IsConversionDone](#) (void)

This API checks if ongoing A/D conversion is complete.
- [adc_result_t](#) [ADC_GetConversionResult](#) (void)

This API is used to retrieve the result of latest A/D conversion.
- [adc_result_t](#) [ADC_GetSingleConversion](#) ([ADC_channel_t](#) channel)

This API is used to retrieve the result of single A/D conversion on given channel.
- void [ADC_StopConversion](#) (void)

This API is used to stop ongoing A/D conversion.
- void [ADC_SetStopOnInterrupt](#) (void)

This API is used enable Stop On Interrupt bit for selected A/D context.
- void [ADC_DischargeSampleCapacitor](#) (void)

This API is used to discharge input sample capacitor for selected context by setting the channel to AVss.
- void [ADC_LoadAcquisitionRegister](#) ([uint16_t](#) acquisitionValue)

This API is used to load ADC Acquisition Time Control register with specified value.
- void [ADC_SetPrechargeTime](#) ([uint16_t](#) prechargeTime)

This API is used to load ADC Precharge Time Control register with specified value.
- void [ADC_SetRepeatCount](#) ([uint8_t](#) repeatCount)

This API is used to load repeat counter for given context with specified value.
- [uint8_t](#) [ADC_GetCurrentCountofConversions](#) (void)

This API retrieves the current value of ADC Repeat Count register for selected context.
- void [ADC_ClearAccumulator](#) (void)

This API clears the accumulator of selected context.
- [uint24_t](#) [ADC_GetAccumulatorValue](#) (void)

This API retrieves 18-bit value of ADC accumulator of selected context.
- bool [ADC_HasAccumulatorOverflowed](#) (void)

This API is used to determine whether ADC accumulator has overflowed for selected context.
- [uint16_t](#) [ADC_GetFilterValue](#) (void)

This API retrieves the value of ADC Filter register of selected context.
- [uint16_t](#) [ADC_GetPreviousResult](#) (void)

This API retrieves the value of ADC Previous register of selected context.
- void [ADC_DefineSetPoint](#) ([uint16_t](#) setPoint)

This API used to set value of ADC Threshold Set-point of selected context.
- void [ADC_SetUpperThreshold](#) ([uint16_t](#) upperThreshold)

This API used to set value of ADC Upper Threshold register of selected context.
- void [ADC_SetLowerThreshold](#) ([uint16_t](#) lowerThreshold)

This API used to set value of ADC Lower Threshold register of selected context.
- [uint16_t](#) [ADC_GetErrorCalculation](#) (void)

This API retrieves the value of ADC Set-point Error register of selected context.
- void [ADC_EnableDoubleSampling](#) (void)

This API enables double-sampling bit for selected context.
- void [ADC_EnableContinuousConversion](#) (void)

This API enables continuous conversion for selected context.
- void [ADC_DisableContinuousConversion](#) (void)

This API disables continuous conversion for selected context.
- bool [ADC_HasErrorCrossedUpperThreshold](#) (void)

This API is used to determine if ADC error has crossed the upper threshold for selected context.
- bool [ADC_HasErrorCrossedLowerThreshold](#) (void)

This API is used to determine if ADC error is less than the lower threshold for selected context.

- `uint8_t ADC_GetConversionStageStatus` (void)
This API is used retrieve the multi-stage status for selected context.
- `void ADC_EnableChargePump` (void)
This API is used to enable ADC charge pump.
- `void ADC_DisableChargePump` (void)
This API is used to disable ADC charge pump.
- `void ADC_SetADInterruptHandler` (void(*InterruptHandler)(void))
Setter for A/D conversion ISR.
- `void ADC_SetActiveClockTuningInterruptHandler` (void(*InterruptHandler)(void))
Setter for A/D Active Clock Tuning ISR.
- `void ADC_SetContext1ThresholdInterruptHandler` (void(*InterruptHandler)(void))
Setter for Context-1 threshold interrupt.
- `void ADC_SetContext2ThresholdInterruptHandler` (void(*InterruptHandler)(void))
Setter for Context-2 threshold interrupt.
- `void ADC_SetContext3ThresholdInterruptHandler` (void(*InterruptHandler)(void))
Setter for Context-3 threshold interrupt.
- `void ADC_SetContext4ThresholdInterruptHandler` (void(*InterruptHandler)(void))
Setter for Context-4 threshold interrupt.

4.1.1 Detailed Description

ADC Generated Driver API Header File.

@Company Microchip Technology Inc.

@File Name [adc.h](#)

@Summary This is the generated header file for the ADC driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides APIs for driver for ADC. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00

This file contains API prototypes and other datatypes for ADC module.

4.1.2 Typedef Documentation

4.1.2.1 `adc_result_t`

```
typedef uint16_t adc\_result\_t
```

This typedef should be used for result of A/D conversion.

Definition at line 65 of file [adc.h](#).

4.1.3 Enumeration Type Documentation

4.1.3.1 `ADC_channel_t`

```
enum ADC\_channel\_t
```

This enumeration contains available ADC channels.

Definition at line 72 of file [adc.h](#).

4.1.3.2 ADC_context_t

enum [ADC_context_t](#)

This enumeration contains available ADC contexts.

Definition at line 88 of file [adc.h](#).

4.1.4 Function Documentation

4.1.4.1 ADC_ClearAccumulator()

```
void ADC_ClearAccumulator (
    void ) [inline]
```

This API clears the accumulator of selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_ClearAccumulator();

    //Application code

    while(1);
}
```

Definition at line 308 of file [adc.c](#).

4.1.4.2 ADC_DefineSetPoint()

```
void ADC_DefineSetPoint (
    uint16_t setPoint)
```

This API used to set value of ADC Threshold Set-point of selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>setPoint</i>	- 16-bit value for set point.
----	-----------------	-------------------------------

Returns

void

```
void main(void)
{
    uint16_t setPoint = 90;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_DefineSetPoint (setPoint);

    //Application code

    while(1);
}
```

Definition at line 333 of file [adc.c](#).

4.1.4.3 ADC_DisableChannelScan()

```
void ADC_DisableChannelScan (
    ADC_context_t context)
```

This API is used to exclude a channel context from the scan sequence.

Correct context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>context</i>	- Context which needs to be excluded from scan. Refer ADC_context_t for available contexts
----	----------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_EnableChannelScan(CONTEXT_1);
    ADC_DisableChannelScan(CONTEXT_2);
    ADC_DisableChannelScan(CONTEXT_3);
    ADC_EnableChannelScan(CONTEXT_4);

    //Application code

    while(1);
}
```

Definition at line 211 of file [adc.c](#).

4.1.4.4 ADC_DisableChannelSequencer()

```
void ADC_DisableChannelSequencer (
    void ) [inline]
```

This API disables ADC channel sequencer.

ADC module should be initialized using [ADC_Initialize](#) before calling this API.

Parameters

void	
------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_DisableChannelSequencer();    //Disable scanner
    ADC_SelectContext(CONTEXT_1);
    convertedValue = ADC_GetSingleConversion(channel_ANA0);

    while(1);
}
```

Definition at line 184 of file [adc.c](#).

4.1.4.5 ADC_DisableChargePump()

```
void ADC_DisableChargePump (
    void ) [inline]
```

This API is used to disable ADC charge pump.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_DisableChargePump();
    ADC_StartConversion(channel_ANA0);

    //Application code

    while(1);
}
```

Definition at line 391 of file [adc.c](#).

4.1.4.6 ADC_DisableContinuousConversion()

```
void ADC_DisableContinuousConversion (
    void ) [inline]
```

This API disables continuous conversion for selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_DisableContinuousConversion();
    ADC_StartConversion(channel_ANA0);

    //Application code

    while(1);
}
```

Definition at line 366 of file [adc.c](#).

4.1.4.7 ADC_DischargeSampleCapacitor()

```
void ADC_DischargeSampleCapacitor (
    void ) [inline]
```

This API is used to discharge input sample capacitor for selected context by setting the channel to AVss.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_DischargeSampleCapacitor();
    ADC_StartConversion (channel_ANA0);
    while (!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    //Application code

    while(1);
}
```

Definition at line 280 of file [adc.c](#).

4.1.4.8 ADC_EnableChannelScan()

```
void ADC_EnableChannelScan (
    ADC_context_t context)
```

This API is used to include a channel context in the scan sequence.

Correct context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>context</i>	- Context which needs to be included in scan. Refer ADC_context_t for available contexts
----	----------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_EnableChannelScan (CONTEXT_1);
    ADC_EnableChannelScan (CONTEXT_2);

    //Application code

    while(1);
}
```

Definition at line 199 of file [adc.c](#).

4.1.4.9 ADC_EnableChannelSequencer()

```
void ADC_EnableChannelSequencer (
    void ) [inline]
```

This API enables ADC channel sequencer.

ADC module should be initialized using [ADC_Initialize](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue1, convertedValue2, convertedValue3, convertedValue4;
    ADC_Initialize();
    ADC_EnableChannelSequencer();    //Enable auto-scanning if not already enabled
    ADC_StartChannelSequencer();    // Software starts the sequencer

    while(!ADC_IsConversionDone());
    ADC_SelectContext (CONTEXT_1);
    convertedValue1 = ADC_GetConversionResult();
    ADC_SelectContext (CONTEXT_2);
    convertedValue2 = ADC_GetConversionResult();
    ADC_SelectContext (CONTEXT_3);
    convertedValue3 = ADC_GetConversionResult();
    ADC_SelectContext (CONTEXT_4);
    convertedValue4 = ADC_GetConversionResult();

    while(1);
}
```

Definition at line 179 of file [adc.c](#).

4.1.4.10 ADC_EnableChargePump()

```
void ADC_EnableChargePump (
    void )    [inline]
```

This API is used to enable ADC charge pump.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_EnableChargePump();
    ADC_StartConversion(channel_ANA0);

    //Application code

    while(1);
}
```

Definition at line 386 of file [adc.c](#).

4.1.4.11 ADC_EnableContinuousConversion()

```
void ADC_EnableContinuousConversion (
    void )    [inline]
```

This API enables continuous conversion for selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<code>void</code>	
-------------------	--

Returns

`void`

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_EnableContinuousConversion();
    ADC_StartConversion (channel_ANA0);

    //Application code

    while (1);
}
```

Definition at line 361 of file [adc.c](#).

4.1.4.12 ADC_EnableDoubleSampling()

```
void ADC_EnableDoubleSampling (
    void ) [inline]
```

This API enables double-sampling bit for selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<code>void</code>	
-------------------	--

Returns

`void`

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_EnableDoubleSampling();
    ADC_StartConversion (channel_ANA0);

    //Application code

    while (1);
}
```

Definition at line 356 of file [adc.c](#).

4.1.4.13 ADC_GetAccumulatorValue()

```
uint24_t ADC_GetAccumulatorValue (
    void )
```

This API retrieves 18-bit value of ADC accumulator of selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Value of ADC accumulator.

```
void main(void)
{
    uint24_t accumulatorValue;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    accumulatorValue = ADC_GetAccumulatorValue();

    //Application code

    while(1);
}
```

Definition at line 313 of file [adc.c](#).

4.1.4.14 ADC_GetConversionResult()

```
adc_result_t ADC_GetConversionResult (
    void )
```

This API is used to retrieve the result of latest A/D conversion.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

The result of A/D conversion. Refer [adc_result_t](#)

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);
    while(!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    //Application code

    while(1);
}
```

Definition at line 240 of file [adc.c](#).

4.1.4.15 ADC_GetConversionStageStatus()

```
uint8_t ADC_GetConversionStageStatus (
    void )
```

This API is used retrieve the multi-stage status for selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Contents of ADC STATUS register.

```
void main(void)
{
    uint8_t status;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    status = ADC_GetConversionStageStatus ();

    //Application code

    while (1);
}
```

Definition at line 381 of file [adc.c](#).

4.1.4.16 ADC_GetCurrentCountofConversions()

```
uint8_t ADC_GetCurrentCountofConversions (
    void )
```

This API retrieves the current value of ADC Repeat Count register for selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Current value of ADC Repeat Count register

```
void main(void)
{
    uint8_t conversionCount;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    conversionCount = ADC_GetCurrentCountofConversions ();

    //Application code

    while (1);
}
```

Definition at line 303 of file [adc.c](#).

4.1.4.17 ADC_GetErrorCalculation()

```
uint16_t ADC_GetErrorCalculation (
    void )
```

This API retrieves the value of ADC Set-point Error register of selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

16-bit value obtained from ADERRH and ADERRL registers.

```
void main(void)
{
    uint16_t errorCalc;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    errorCalc = ADC_GetErrorCalculation();

    //Application code

    while(1);
}
```

Definition at line 351 of file [adc.c](#).

4.1.4.18 ADC_GetFilterValue()

```
uint16_t ADC_GetFilterValue (
    void )
```

This API retrieves the value of ADC Filter register of selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

16-bit value obtained from ADFLTRH and ADFLTRL registers.

```
void main(void)
{
    uint16_t filterValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    filterValue = ADC_GetFilterValue();

    //Application code

    while(1);
}
```

Definition at line 323 of file [adc.c](#).

4.1.4.19 ADC_GetPreviousResult()

```
uint16_t ADC_GetPreviousResult (
    void )
```

This API retrieves the value of ADC Previous register of selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

16-bit value obtained from ADPREVH and ADPREVL registers.

```
void main(void)
{
    uint16_t previousResult;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    previousResult = ADC_GetPreviousResult();

    //Application code

    while(1);
}
```

Definition at line 328 of file [adc.c](#).

4.1.4.20 ADC_GetSingleConversion()

```
adc_result_t ADC_GetSingleConversion (
    ADC_channel_t channel)
```

This API is used to retrieve the result of single A/D conversion on given channel.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>channel</i>	- Analog channel number on which A/D conversion has to be applied. Refer ADC_channel_t for available channels
----	----------------	---

Returns

The result of A/D conversion. Refer [adc_result_t](#)

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_DisableChannelSequencer();    //Disable scanner
    ADC_SelectContext (CONTEXT_1);
    convertedValue = ADC_GetSingleConversion(channel_ANA0);

    //Application code

    while(1);
}
```

Definition at line 246 of file [adc.c](#).

4.1.4.21 ADC_HasAccumulatorOverflowed()

```
bool ADC_HasAccumulatorOverflowed (
    void )
```

This API is used to determine whether ADC accumulator has overflowed for selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Status of accumulator. true - ADC accumulator has overflowed. false - ADC accumulator has not overflowed.

```
void main(void)
{
    bool hasAccOvf;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    hasAccOvf = ADC_HasAccumulatorOverflowed();

    //Application code

    while (1);
}
```

Definition at line 318 of file [adc.c](#).

4.1.4.22 ADC_HasErrorCrossedLowerThreshold()

```
bool ADC_HasErrorCrossedLowerThreshold (
    void )
```

This API is used to determine if ADC error is less than the lower threshold for selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Status of operation. true - if $ERR < LTH$. false - if $ERR \geq LTH$.

```
void main(void)
{
    bool ulThr;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    ulThr = ADC_HasErrorCrossedLowerThreshold();

    //Application code

    while (1);
}
```

Definition at line 376 of file [adc.c](#).

4.1.4.23 ADC_HasErrorCrossedUpperThreshold()

```
bool ADC_HasErrorCrossedUpperThreshold (
    void )
```

This API is used to determine if ADC error has crossed the upper threshold for selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

Status of operation. true - if $ERR > UTH$ false - if $ERR \leq UTH$

```
void main(void)
{
    bool uThr;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    uThr = ADC_HasErrorCrossedUpperThreshold();

    //Application code

    while (1);
}
```

Definition at line 371 of file [adc.c](#).

4.1.4.24 ADC_Initialize()

```
void ADC_Initialize (
    void )
```

This API initializes the ADC module.

This routine must be called before any other ADC routine.

Parameters

<i>void.</i>	
--------------	--

Returns

void.

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    while (!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    while (1);
}
```

Definition at line 65 of file [adc.c](#).

4.1.4.25 ADC_IsConversionDone()

```
bool ADC_IsConversionDone (
    void )
```

This API checks if ongoing A/D conversion is complete.

Parameters

<i>void</i>	
-------------	--

Returns

bool - Status of A/D conversion. true - if conversion is complete. false - if conversion is ongoing.

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    while (!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    //Application code
    while(1);
}
```

Definition at line 235 of file [adc.c](#).

4.1.4.26 ADC_LoadAcquisitionRegister()

```
void ADC_LoadAcquisitionRegister (
    uint16_t acquisitionValue)
```

This API is used to load ADC Acquisition Time Control register with specified value.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>acquisitionValue</i>	- Value to be loaded in the acquisition time control register.
----	-------------------------	--

Returns

void

```
void main(void)
{
    uint16_t acquisitionValue = 98;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_LoadAcquisitionRegister (acquisitionValue);

    //Application code
    while(1);
}
```

Definition at line 286 of file [adc.c](#).

4.1.4.27 ADC_SelectContext()

```
void ADC_SelectContext (
    ADC_context_t context) [inline]
```

This API is used to select an ADC context to perform read/write operations on context specific registers.

Parameters

in	<i>context</i>	- Context to be selected. Refer ADC_context_t for available contexts
----	----------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    while (!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    while (1);
}
```

Definition at line 194 of file [adc.c](#).

4.1.4.28 ADC_SetActiveClockTuningInterruptHandler()

```
void ADC_SetActiveClockTuningInterruptHandler (
    void(* InterruptHandler ) (void))
```

Setter for A/D Active Clock Tuning ISR.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```
void customACTIHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetActiveClockTuningInterruptHandler(customACTIHandler);
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);

    //Replace with your application code
    while (1)
    {
    }
}
```

Definition at line 428 of file [adc.c](#).

4.1.4.29 ADC_SetADIIInterruptHandler()

```
void ADC_SetADIIInterruptHandler (
    void(* InterruptHandler ) (void))
```

Setter for A/D conversion ISR.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```
void customADIHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetADIInterruptHandler(customADIHandler);
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}
```

Definition at line 418 of file [adc.c](#).

4.1.4.30 ADC_SetContext1ThresholdInterruptHandler()

```
void ADC_SetContext1ThresholdInterruptHandler (
    void(* InterruptHandler ) (void))
```

Setter for Context-1 threshold interrupt.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```
void customADCH1InterruptHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetContext1ThresholdInterruptHandler(customADCH1InterruptHandler);
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}
```

Definition at line 423 of file [adc.c](#).

4.1.4.31 ADC_SetContext2ThresholdInterruptHandler()

```
void ADC_SetContext2ThresholdInterruptHandler (
    void(* InterruptHandler ) (void))
```

Setter for Context-2 threshold interrupt.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```

void customADCH2InterruptHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetContext2ThresholdInterruptHandler(customADCH1InterruptHandler);
    ADC_SelectContext (CONTEXT_2);
    ADC_StartConversion (channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}

```

4.1.4.32 ADC_SetContext3ThresholdInterruptHandler()

```

void ADC_SetContext3ThresholdInterruptHandler (
    void(* InterruptHandler ) (void))

```

Setter for Context-3 threshold interrupt.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```

void customADCH3InterruptHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetContext3ThresholdInterruptHandler(customADCH1InterruptHandler);
    ADC_SelectContext (CONTEXT_3);
    ADC_StartConversion (channel_ANA0);

    //Replace with your application code
    while(1)
    {
    }
}

```

4.1.4.33 ADC_SetContext4ThresholdInterruptHandler()

```

void ADC_SetContext4ThresholdInterruptHandler (
    void(* InterruptHandler ) (void))

```

Setter for Context-4 threshold interrupt.

Parameters

in	<i>InterruptHandler</i>	- Pointer to custom ISR.
----	-------------------------	--------------------------

Returns

void

```

void customADCH4InterruptHandler(void)
{
    //Custom ISR code
}

void main(void)
{
    ADC_Initialize();
    ADC_SetContext4ThresholdInterruptHandler(customADCH1InterruptHandler);
    ADC_SelectContext (CONTEXT_4);
    ADC_StartConversion (channel_ANA0);

    //Replace with your application code
    while (1)
    {
    }
}

```

4.1.4.34 ADC_SetLowerThreshold()

```

void ADC_SetLowerThreshold (
    uint16_t lowerThreshold)

```

This API used to set value of ADC Lower Threshold register of selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>lowerThreshold</i>	- 16-bit value for lower threshold.
----	-----------------------	-------------------------------------

Returns

void

```

void main(void)
{
    uint16_t lowerThreshold = 500;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_SetLowerThreshold(lowerThreshold);

    //Application code

    while (1);
}

```

Definition at line 345 of file [adc.c](#).

4.1.4.35 ADC_SetPrechargeTime()

```

void ADC_SetPrechargeTime (
    uint16_t prechargeTime)

```

This API is used to load ADC Precharge Time Control register with specified value.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>prechargeTime</i>	- Value to be loaded in the precharge time control register.
----	----------------------	--

Returns

void

```

void main(void)
{
    uint16_t prechargeTime = 98;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_SetPrechargeTime (prechargeTime);

    //Application code

    while (1);
}

```

Definition at line 292 of file [adc.c](#).

4.1.4.36 ADC_SetRepeatCount()

```

void ADC_SetRepeatCount (
    uint8_t repeatCount) [inline]

```

This API is used to load repeat counter for given context with specified value.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>repeatCount</i>	- Value to be loaded to repeat counter.
----	--------------------	---

Returns

void

```

void main(void)
{
    uint8_t repeatCount = 32;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_SetRepeatCount (repeatCount);

    //Application code

    while (1);
}

```

Definition at line 298 of file [adc.c](#).

4.1.4.37 ADC_SetStopOnInterrupt()

```

void ADC_SetStopOnInterrupt (
    void ) [inline]

```

This API is used enable Stop On Interrupt bit for selected A/D context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_EnableContinuousConversion();
    ADC_SetStopOnInterrupt();

    //Application code

    while (1);
}
```

Definition at line 275 of file [adc.c](#).

4.1.4.38 ADC_SetUpperThreshold()

```
void ADC_SetUpperThreshold (
    uint16_t upperThreshold)
```

This API used to set value of ADC Upper Threshold register of selected context.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>upperThreshold</i>	- 16-bit value for upper threshold.
----	-----------------------	-------------------------------------

Returns

void

```
void main(void)
{
    uint16_t upperThreshold = 5000;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_SetUpperThreshold(upperThreshold);

    //Application code

    while (1);
}
```

Definition at line 339 of file [adc.c](#).

4.1.4.39 ADC_StartChannelSequencer()

```
void ADC_StartChannelSequencer (
    void ) [inline]
```

This API starts the channel sequencer by enabling GO bit.

Channel sequencer should be enabled using [ADC_EnableChannelSequencer](#) before calling this API.

Parameters

<code>void</code>	
-------------------	--

Returns

`void`

```
void main(void)
{
    adc_result_t convertedValue1, convertedValue2, convertedValue3, convertedValue4;
    ADC_Initialize();
    ADC_EnableChannelSequencer();    //Enable auto-scanning if not already enabled
    ADC_StartChannelSequencer();    // Software starts the sequencer

    while(!ADC_IsConversionDone());
    ADC_SelectContext(CONTEXT_1);
    convertedValue1 = ADC_GetConversionResult();
    ADC_SelectContext(CONTEXT_2);
    convertedValue2 = ADC_GetConversionResult();
    ADC_SelectContext(CONTEXT_3);
    convertedValue3 = ADC_GetConversionResult();
    ADC_SelectContext(CONTEXT_4);
    convertedValue4 = ADC_GetConversionResult();

    while(1);
}
```

Definition at line 189 of file [adc.c](#).

4.1.4.40 ADC_StartConversion()

```
void ADC_StartConversion (
    ADC_channel_t channel)
```

This API starts A/D conversion on selected channel.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

in	<i>channel</i>	- Analog channel number on which A/D conversion has to be applied. Refer ADC_channel_t for available list of channels
----	----------------	---

Returns

`void`

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext(CONTEXT_1);
    ADC_StartConversion(channel_ANA0);
    while(!ADC_IsConversionDone());
    convertedValue = ADC_GetConversionResult();

    //Application code

    while(1);
}
```

Definition at line 223 of file [adc.c](#).

4.1.4.41 ADC_StopConversion()

```
void ADC_StopConversion (
    void )    [inline]
```

This API is used to stop ongoing A/D conversion.

Correct ADC context should be selected using [ADC_SelectContext](#) before calling this API.

Parameters

<i>void</i>	
-------------	--

Returns

void

```
void main(void)
{
    adc_result_t convertedValue;
    ADC_Initialize();
    ADC_SelectContext (CONTEXT_1);
    ADC_StartConversion (channel_ANA0);
    ADC_StopConversion();

    //Application code

    while(1);
}
```

Definition at line 269 of file [adc.c](#).

Chapter 5

Data Structure Documentation

5.1 B_type Struct Reference

Data Fields

- volatile bool [one_sec_flag](#)
- volatile uint16_t [dim_delay](#)
- volatile bool [display_update](#)

5.1.1 Detailed Description

Definition at line [158](#) of file [vconfig.h](#).

5.1.2 Field Documentation

5.1.2.1 dim_delay

```
volatile uint16_t dim_delay
```

Definition at line [160](#) of file [vconfig.h](#).

5.1.2.2 display_update

```
volatile bool display_update
```

Definition at line [161](#) of file [vconfig.h](#).

5.1.2.3 one_sec_flag

```
volatile bool one_sec_flag
```

Definition at line [159](#) of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/vconfig.h](#)

5.2 block10 Union Reference

Data Fields

- `uint8_t b [sizeof(block10_type)]`
- `block10_type block`

5.2.1 Detailed Description

Definition at line 50 of file [gemsecs.h](#).

5.2.2 Field Documentation

5.2.2.1 b

```
uint8_t b[sizeof(block10_type)]
```

Definition at line 51 of file [gemsecs.h](#).

5.2.2.2 block

```
block10_type block
```

Definition at line 52 of file [gemsecs.h](#).

The documentation for this union was generated from the following file:

- `secs_q84.X/gemsecs.h`

5.3 block10_type Struct Reference

Data Fields

- `uint32_t systemb`
- `uint8_t bidl`
- `uint8_t bidh: 7`
- `uint8_t ebit: 1`
- `uint8_t function`
- `uint8_t stream: 7`
- `uint8_t wbit: 1`
- `uint8_t didl`
- `uint8_t didh: 7`
- `uint8_t rbit: 1`

5.3.1 Detailed Description

Definition at line 37 of file [gemsecs.h](#).

5.3.2 Field Documentation

5.3.2.1 bidh

`uint8_t bidh`

Definition at line 40 of file [gemsecs.h](#).

5.3.2.2 bidl

`uint8_t bidl`

Definition at line 39 of file [gemsecs.h](#).

5.3.2.3 didh

`uint8_t didh`

Definition at line 46 of file [gemsecs.h](#).

5.3.2.4 didl

`uint8_t didl`

Definition at line 45 of file [gemsecs.h](#).

5.3.2.5 ebit

`uint8_t ebit`

Definition at line 41 of file [gemsecs.h](#).

5.3.2.6 function

`uint8_t function`

Definition at line 42 of file [gemsecs.h](#).

5.3.2.7 rbit

`uint8_t rbit`

Definition at line 47 of file [gemsecs.h](#).

5.3.2.8 stream

```
uint8_t stream
```

Definition at line 43 of file [gemsecs.h](#).

5.3.2.9 systemb

```
uint32_t systemb
```

Definition at line 38 of file [gemsecs.h](#).

5.3.2.10 wbit

```
uint8_t wbit
```

Definition at line 44 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/gemsecs.h](#)

5.4 D_data Struct Reference

Data Fields

- char [lcd](#) [MAX_VTERM][MAX_LCD_LINES][MAX_BUF+1]
- uint8_t [vterm](#): 2
- D_CODES [last_info](#)

5.4.1 Detailed Description

Definition at line 6 of file [mconfig.c](#).

5.4.2 Field Documentation

5.4.2.1 last_info

```
D_CODES last_info
```

Definition at line 9 of file [mconfig.c](#).

5.4.2.2 `lcd`

```
char lcd[MAX_VTERM][MAX_LCD_LINES][MAX_BUF+1]
```

Definition at line 7 of file [mconfig.c](#).

5.4.2.3 `vterm`

```
uint8_t vterm
```

Definition at line 8 of file [mconfig.c](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/mconfig.c](#)

5.5 `gem_display_type` Struct Reference

Data Fields

- [header153](#) message
- [response_type](#) block
- [uint16_t](#) delay
- [uint8_t](#) stack

5.5.1 Detailed Description

Definition at line 155 of file [gemsecs.h](#).

5.5.2 Field Documentation

5.5.2.1 `block`

```
response\_type block
```

Definition at line 157 of file [gemsecs.h](#).

5.5.2.2 `delay`

```
uint16_t delay
```

Definition at line 158 of file [gemsecs.h](#).

5.5.2.3 message

[header153](#) message

Definition at line [156](#) of file [gemsecs.h](#).

5.5.2.4 stack

uint8_t stack

Definition at line [159](#) of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/gemsecs.h](#)

5.6 gem_message_type Struct Reference

Data Fields

- [header33](#) message
- [response_type](#) block
- [uint16_t](#) delay
- [uint8_t](#) stack

5.6.1 Detailed Description

Definition at line [148](#) of file [gemsecs.h](#).

5.6.2 Field Documentation

5.6.2.1 block

[response_type](#) block

Definition at line [150](#) of file [gemsecs.h](#).

5.6.2.2 delay

uint16_t delay

Definition at line [151](#) of file [gemsecs.h](#).

5.6.2.3 message

`header33` message

Definition at line 149 of file [gemsecs.h](#).

5.6.2.4 stack

`uint8_t` stack

Definition at line 152 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- `secs_q84.X/gemsecs.h`

5.7 header10 Struct Reference

Data Fields

- `uint16_t` [checksum](#)
- [block10](#) `block`
- `uint8_t` [length](#)

5.7.1 Detailed Description

Definition at line 55 of file [gemsecs.h](#).

5.7.2 Field Documentation

5.7.2.1 block

[block10](#) `block`

Definition at line 57 of file [gemsecs.h](#).

5.7.2.2 checksum

`uint16_t` `checksum`

Definition at line 56 of file [gemsecs.h](#).

5.7.2.3 length

`uint8_t length`

Definition at line 58 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- `secs_q84.X/gemsecs.h`

5.8 header12 Struct Reference

Data Fields

- `uint16_t checksum`
- `uint8_t data` [2]
- `block10 block`
- `uint8_t length`

5.8.1 Detailed Description

Definition at line 61 of file [gemsecs.h](#).

5.8.2 Field Documentation

5.8.2.1 block

`block10 block`

Definition at line 64 of file [gemsecs.h](#).

5.8.2.2 checksum

`uint16_t checksum`

Definition at line 62 of file [gemsecs.h](#).

5.8.2.3 data

`uint8_t data` [2]

Definition at line 63 of file [gemsecs.h](#).

5.8.2.4 length

`uint8_t length`

Definition at line 65 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- `secs_q84.X/gemsecs.h`

5.9 header13 Struct Reference

Data Fields

- `uint16_t checksum`
- `uint8_t data` [3]
- `block10 block`
- `uint8_t length`

5.9.1 Detailed Description

Definition at line 68 of file [gemsecs.h](#).

5.9.2 Field Documentation

5.9.2.1 block

`block10 block`

Definition at line 71 of file [gemsecs.h](#).

5.9.2.2 checksum

`uint16_t checksum`

Definition at line 69 of file [gemsecs.h](#).

5.9.2.3 data

`uint8_t data` [3]

Definition at line 70 of file [gemsecs.h](#).

5.9.2.4 length

`uint8_t length`

Definition at line 72 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- `secs_q84.X/gemsecs.h`

5.10 header14 Struct Reference

Data Fields

- `uint16_t checksum`
- `uint8_t data` [4]
- `block10 block`
- `uint8_t length`

5.10.1 Detailed Description

Definition at line 75 of file [gemsecs.h](#).

5.10.2 Field Documentation

5.10.2.1 block

`block10 block`

Definition at line 78 of file [gemsecs.h](#).

5.10.2.2 checksum

`uint16_t checksum`

Definition at line 76 of file [gemsecs.h](#).

5.10.2.3 data

`uint8_t data` [4]

Definition at line 77 of file [gemsecs.h](#).

5.10.2.4 length

`uint8_t length`

Definition at line 79 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- `secs_q84.X/gemsecs.h`

5.11 header153 Struct Reference

Data Fields

- `uint16_t checksum`
- `uint8_t data` [143]
- `block10 block`
- `uint8_t length`

5.11.1 Detailed Description

Definition at line 126 of file [gemsecs.h](#).

5.11.2 Field Documentation

5.11.2.1 block

`block10 block`

Definition at line 129 of file [gemsecs.h](#).

5.11.2.2 checksum

`uint16_t checksum`

Definition at line 127 of file [gemsecs.h](#).

5.11.2.3 data

`uint8_t data`[143]

Definition at line 128 of file [gemsecs.h](#).

5.11.2.4 length

`uint8_t length`

Definition at line 130 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- `secs_q84.X/gemsecs.h`

5.12 header17 Struct Reference

Data Fields

- `uint16_t checksum`
- `uint8_t data` [7]
- `block10 block`
- `uint8_t length`

5.12.1 Detailed Description

Definition at line 82 of file [gemsecs.h](#).

5.12.2 Field Documentation

5.12.2.1 block

`block10 block`

Definition at line 85 of file [gemsecs.h](#).

5.12.2.2 checksum

`uint16_t checksum`

Definition at line 83 of file [gemsecs.h](#).

5.12.2.3 data

`uint8_t data` [7]

Definition at line 84 of file [gemsecs.h](#).

5.12.2.4 length

`uint8_t length`

Definition at line 86 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- `secs_q84.X/gemsecs.h`

5.13 header18 Struct Reference

Data Fields

- `uint16_t checksum`
- `uint8_t data` [8]
- `block10 block`
- `uint8_t length`

5.13.1 Detailed Description

Definition at line 89 of file [gemsecs.h](#).

5.13.2 Field Documentation

5.13.2.1 block

`block10 block`

Definition at line 92 of file [gemsecs.h](#).

5.13.2.2 checksum

`uint16_t checksum`

Definition at line 90 of file [gemsecs.h](#).

5.13.2.3 data

`uint8_t data` [8]

Definition at line 91 of file [gemsecs.h](#).

5.13.2.4 length

`uint8_t length`

Definition at line 93 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/gemsecs.h](#)

5.14 header24 Struct Reference

Data Fields

- `uint16_t` [checksum](#)
- `uint8_t` [data](#) [12]
- `uint8_t` [datam](#) [2]
- [block10](#) [block](#)
- `uint8_t` [length](#)

5.14.1 Detailed Description

Definition at line 96 of file [gemsecs.h](#).

5.14.2 Field Documentation

5.14.2.1 block

[block10](#) [block](#)

Definition at line 100 of file [gemsecs.h](#).

5.14.2.2 checksum

`uint16_t` [checksum](#)

Definition at line 97 of file [gemsecs.h](#).

5.14.2.3 data

`uint8_t` [data](#)[12]

Definition at line 98 of file [gemsecs.h](#).

5.14.2.4 datam

```
uint8_t datam[2]
```

Definition at line 99 of file [gemsecs.h](#).

5.14.2.5 length

```
uint8_t length
```

Definition at line 101 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- secs_q84.X/gemsecs.h

5.15 header254 Struct Reference

Data Fields

- uint16_t [checksum](#)
- uint8_t [data](#) [244]
- [block10](#) [block](#)
- uint8_t [length](#)

5.15.1 Detailed Description

Definition at line 133 of file [gemsecs.h](#).

5.15.2 Field Documentation

5.15.2.1 block

```
block10 block
```

Definition at line 136 of file [gemsecs.h](#).

5.15.2.2 checksum

```
uint16_t checksum
```

Definition at line 134 of file [gemsecs.h](#).

5.15.2.3 data

```
uint8_t data[244]
```

Definition at line 135 of file [gemsecs.h](#).

5.15.2.4 length

```
uint8_t length
```

Definition at line 137 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- secs_q84.X/gemsecs.h

5.16 header26 Struct Reference

Data Fields

- uint16_t [checksum](#)
- uint8_t [data](#) [14]
- uint8_t [datam](#) [2]
- [block10](#) [block](#)
- uint8_t [length](#)

5.16.1 Detailed Description

Definition at line 104 of file [gemsecs.h](#).

5.16.2 Field Documentation

5.16.2.1 block

```
block10 block
```

Definition at line 108 of file [gemsecs.h](#).

5.16.2.2 checksum

```
uint16_t checksum
```

Definition at line 105 of file [gemsecs.h](#).

5.16.2.3 data

```
uint8_t data[14]
```

Definition at line 106 of file [gemsecs.h](#).

5.16.2.4 datam

```
uint8_t datam[2]
```

Definition at line 107 of file [gemsecs.h](#).

5.16.2.5 length

```
uint8_t length
```

Definition at line 109 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/gemsecs.h](#)

5.17 header27 Struct Reference

Data Fields

- [uint16_t checksum](#)
- [uint8_t data](#) [17]
- [block10 block](#)
- [uint8_t length](#)

5.17.1 Detailed Description

Definition at line 112 of file [gemsecs.h](#).

5.17.2 Field Documentation

5.17.2.1 block

```
block10 block
```

Definition at line 115 of file [gemsecs.h](#).

5.17.2.2 checksum

```
uint16_t checksum
```

Definition at line 113 of file [gemsecs.h](#).

5.17.2.3 data

```
uint8_t data[17]
```

Definition at line 114 of file [gemsecs.h](#).

5.17.2.4 length

```
uint8_t length
```

Definition at line 116 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/gemsecs.h](#)

5.18 header33 Struct Reference

Data Fields

- [uint16_t checksum](#)
- [uint8_t data](#) [23]
- [block10 block](#)
- [uint8_t length](#)

5.18.1 Detailed Description

Definition at line 119 of file [gemsecs.h](#).

5.18.2 Field Documentation

5.18.2.1 block

```
block10 block
```

Definition at line 122 of file [gemsecs.h](#).

5.18.2.2 checksum

```
uint16_t checksum
```

Definition at line 120 of file [gemsecs.h](#).

5.18.2.3 data

```
uint8_t data[23]
```

Definition at line 121 of file [gemsecs.h](#).

5.18.2.4 length

```
uint8_t length
```

Definition at line 123 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- secs_q84.X/gemsecs.h

5.19 response_type Struct Reference

Data Fields

- uint8_t * [header](#)
- uint8_t [length](#)
- uint8_t * [reply](#)
- uint8_t [reply_length](#)
- uint8_t [respond](#): 1

5.19.1 Detailed Description

Definition at line 140 of file [gemsecs.h](#).

5.19.2 Field Documentation

5.19.2.1 header

```
uint8_t* header
```

Definition at line 141 of file [gemsecs.h](#).

5.19.2.2 length

```
uint8_t length
```

Definition at line 142 of file [gemsecs.h](#).

5.19.2.3 reply

```
uint8_t* reply
```

Definition at line 143 of file [gemsecs.h](#).

5.19.2.4 reply_length

```
uint8_t reply_length
```

Definition at line 144 of file [gemsecs.h](#).

5.19.2.5 respond

```
uint8_t respond
```

Definition at line 145 of file [gemsecs.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/gemsecs.h](#)

5.20 ringBufS_t Struct Reference

Data Fields

- [uint8_t buf](#) [RBUF_SIZE]
- [uint8_t head](#)
- [uint8_t tail](#)
- [uint8_t count](#)

5.20.1 Detailed Description

Definition at line 19 of file [ringbufs.h](#).

5.20.2 Field Documentation

5.20.2.1 buf

```
uint8_t buf[RBUF_SIZE]
```

Definition at line 20 of file [ringbufs.h](#).

5.20.2.2 count

uint8_t count

Definition at line 23 of file [ringbufs.h](#).

5.20.2.3 head

uint8_t head

Definition at line 21 of file [ringbufs.h](#).

5.20.2.4 tail

uint8_t tail

Definition at line 22 of file [ringbufs.h](#).

The documentation for this struct was generated from the following file:

- secs_q84.X/ringbufs.h

5.21 spi1_configuration_t Struct Reference

SPI1 Generated Driver File.

Data Fields

- uint8_t [con0](#)
- uint8_t [con1](#)
- uint8_t [con2](#)
- uint8_t [baud](#)
- uint8_t [operation](#)

5.21.1 Detailed Description

SPI1 Generated Driver File.

@Company Microchip Technology Inc.

@File Name [spi1.c](#)

@Summary This is the generated driver implementation file for the SPI1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides implementations for driver APIs for SPI1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00

Definition at line 50 of file [eadog.h](#).

5.21.2 Field Documentation

5.21.2.1 baud

`uint8_t baud`

Definition at line 54 of file [eadog.h](#).

5.21.2.2 con0

`uint8_t con0`

Definition at line 51 of file [eadog.h](#).

5.21.2.3 con1

`uint8_t con1`

Definition at line 52 of file [eadog.h](#).

5.21.2.4 con2

`uint8_t con2`

Definition at line 53 of file [eadog.h](#).

5.21.2.5 operation

`uint8_t operation`

Definition at line 55 of file [eadog.h](#).

The documentation for this struct was generated from the following files:

- `secs_q84.X/eadog.h`
- `secs_q84.X/mcc_generated_files/spi1.c`

5.22 spi_link_type Struct Reference

Data Fields

- `uint8_t SPI_LCD`: 1
- `uint8_t SPI_AUX`: 1
- `uint8_t LCD_TIMER`: 1
- `volatile uint8_t LCD_DATA`: 1
- `uint16_t delay`
- `uint8_t config`
- `uint8_t * txbuf`
- `volatile int32_t int_count`

5.22.1 Detailed Description

Definition at line 147 of file [vconfig.h](#).

5.22.2 Field Documentation

5.22.2.1 config

```
uint8_t config
```

Definition at line 153 of file [vconfig.h](#).

5.22.2.2 delay

```
uint16_t delay
```

Definition at line 152 of file [vconfig.h](#).

5.22.2.3 int_count

```
volatile int32_t int_count
```

Definition at line 155 of file [vconfig.h](#).

5.22.2.4 LCD_DATA

```
volatile uint8_t LCD_DATA
```

Definition at line 151 of file [vconfig.h](#).

5.22.2.5 LCD_TIMER

```
uint8_t LCD_TIMER
```

Definition at line 150 of file [vconfig.h](#).

5.22.2.6 SPI_AUX

```
uint8_t SPI_AUX
```

Definition at line 149 of file [vconfig.h](#).

5.22.2.7 SPI_LCD

```
uint8_t SPI_LCD
```

Definition at line 148 of file [vconfig.h](#).

5.22.2.8 txbuf

```
uint8_t* txbuf
```

Definition at line 154 of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- secs_q84.X/[vconfig.h](#)

5.23 terminal_type Struct Reference

Data Fields

- uint8_t [ack](#) [32]
- uint8_t [mesgid](#)
- uint8_t [TID](#)
- uint8_t [mcode](#)
- uint8_t [mparm](#)
- uint8_t [cmdlen](#)
- uint8_t [log_seq](#)
- uint8_t [host_display_ack](#): 1
- D_CODES [info](#)
- D_CODES [help_temp](#)
- uint16_t [ceid](#)
- uint16_t [log_num](#)

5.23.1 Detailed Description

Definition at line 197 of file [vconfig.h](#).

5.23.2 Field Documentation

5.23.2.1 ack

```
uint8_t ack[32]
```

Definition at line 198 of file [vconfig.h](#).

5.23.2.2 ceid

uint16_t ceid

Definition at line 202 of file [vconfig.h](#).

5.23.2.3 cmdlen

uint8_t cmdlen

Definition at line 199 of file [vconfig.h](#).

5.23.2.4 help_temp

D_CODES help_temp

Definition at line 201 of file [vconfig.h](#).

5.23.2.5 host_display_ack

uint8_t host_display_ack

Definition at line 200 of file [vconfig.h](#).

5.23.2.6 info

D_CODES info

Definition at line 201 of file [vconfig.h](#).

5.23.2.7 log_num

uint16_t log_num

Definition at line 203 of file [vconfig.h](#).

5.23.2.8 log_seq

uint8_t log_seq

Definition at line 199 of file [vconfig.h](#).

5.23.2.9 mcode

uint8_t mcode

Definition at line 199 of file [vconfig.h](#).

5.23.2.10 mesgid

uint8_t mesgid

Definition at line 198 of file [vconfig.h](#).

5.23.2.11 mparm

uint8_t mparm

Definition at line 199 of file [vconfig.h](#).

5.23.2.12 TID

uint8_t TID

Definition at line 199 of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- secs_q84.X/[vconfig.h](#)

5.24 uart1_status_t Union Reference

Section: Data Type Definitions.

```
#include <uart1.h>
```

Data Fields

- struct {
 - unsigned [perr](#): 1
 - unsigned [ferr](#): 1
 - unsigned [oerr](#): 1
 - unsigned [reserved](#): 5
- uint8_t [status](#)

5.24.1 Detailed Description

Section: Data Type Definitions.

Definition at line 74 of file [uart1.h](#).

5.24.2 Field Documentation

5.24.2.1 ferr

`unsigned ferr`

Definition at line 77 of file [uart1.h](#).

5.24.2.2 oerr

`unsigned oerr`

Definition at line 78 of file [uart1.h](#).

5.24.2.3 perr

`unsigned perr`

Definition at line 76 of file [uart1.h](#).

5.24.2.4 reserved

`unsigned reserved`

Definition at line 79 of file [uart1.h](#).

5.24.2.5 status

`uint8_t status`

Definition at line 81 of file [uart1.h](#).

The documentation for this union was generated from the following file:

- `secs_q84.X/mcc_generated_files/uart1.h`

5.25 uart2_status_t Union Reference

Section: Data Type Definitions.

```
#include <uart2.h>
```

Data Fields

- struct {
 unsigned [perr](#): 1
 unsigned [ferr](#): 1
 unsigned [oerr](#): 1
 unsigned [reserved](#): 5
};
- [uint8_t status](#)

5.25.1 Detailed Description

Section: Data Type Definitions.

Definition at line [74](#) of file [uart2.h](#).

5.25.2 Field Documentation

5.25.2.1 ferr

`unsigned ferr`

Definition at line [77](#) of file [uart2.h](#).

5.25.2.2 oerr

`unsigned oerr`

Definition at line [78](#) of file [uart2.h](#).

5.25.2.3 perr

`unsigned perr`

Definition at line [76](#) of file [uart2.h](#).

5.25.2.4 reserved

`unsigned reserved`

Definition at line [79](#) of file [uart2.h](#).

5.25.2.5 status

uint8_t status

Definition at line 81 of file [uart2.h](#).

The documentation for this union was generated from the following file:

- [secs_q84.X/mcc_generated_files/uart2.h](#)

5.26 V_data Struct Reference

Data Fields

- SEQ_STATES [s_state](#)
- UI_STATES [ui_state](#)
- GEM_STATES [g_state](#)
- GEM_EQUIP [e_types](#)
- LINK_STATES [m_l_state](#)
- LINK_STATES [r_l_state](#)
- LINK_STATES [t_l_state](#)
- char [buf](#) [MAX_BUF+1]
- char [terminal](#) [MAX_TERM+1]
- char [info](#) [MAX_INFO+1]
- uint32_t [ticks](#)
- uint32_t [systemb](#)
- uint32_t [tx_total](#)
- uint32_t [rx_total](#)
- uint32_t [bt_total](#)
- uint32_t [br_total](#)
- uint32_t [brn_total](#)
- uint32_t [btn_total](#)
- int32_t [testing](#)
- uint8_t [stream](#)
- uint8_t [function](#)
- uint8_t [error](#)
- uint8_t [abort](#)
- uint8_t [msg_error](#)
- uint8_t [msg_ret](#)
- uint8_t [alarm](#)
- UI_STATES [ui_sw](#)
- uint16_t [r_checksum](#)
- uint16_t [t_checksum](#)
- uint16_t [checksum_error](#)
- uint16_t [timer_error](#)
- uint16_t [ping](#)
- uint16_t [mode_pwm](#)
- uint16_t [equip_timeout](#)
- uint16_t [sequences](#)
- uint16_t [all_errors](#)
- uint8_t [rbit](#): 1
- uint8_t [wbit](#): 1

- `uint8_t ebit`: 1
- `uint8_t failed_send`: 4
- `uint8_t failed_receive`: 4
- `terminal_type` response
- `uint8_t uart`
- `uint8_t llid`
- `uint8_t sid`
- `uint8_t ping_count`
- `uint8_t euart`
- `uint8_t vterm`
- `uint8_t vterm_switch`
- `uint8_t uart_speed_fast`
- volatile `uint8_t ticker`
- `bool flipper`
- `bool queue`
- `bool debug`
- `bool help`
- `bool stack`
- `bool help_id`
- `bool rerror`
- `bool speed_spin`
- `bool set_sequ`
- `adc_result_t v_tx_line`
- `adc_result_t v_rx_line`
- `int16_t tx_volts`
- `int16_t rx_volts`
- `char tx_rs232`
- `char rx_rs232`

5.26.1 Detailed Description

Definition at line 300 of file `vconfig.h`.

5.26.2 Field Documentation

5.26.2.1 abort

```
uint8_t abort
```

Definition at line 311 of file `vconfig.h`.

5.26.2.2 alarm

```
uint8_t alarm
```

Definition at line 311 of file `vconfig.h`.

5.26.2.3 all_errors

```
uint16_t all_errors
```

Definition at line 313 of file [vconfig.h](#).

5.26.2.4 br_total

```
uint32_t br_total
```

Definition at line 309 of file [vconfig.h](#).

5.26.2.5 brn_total

```
uint32_t brn_total
```

Definition at line 309 of file [vconfig.h](#).

5.26.2.6 bt_total

```
uint32_t bt_total
```

Definition at line 309 of file [vconfig.h](#).

5.26.2.7 btn_total

```
uint32_t btn_total
```

Definition at line 309 of file [vconfig.h](#).

5.26.2.8 buf

```
char buf[MAX_BUF+1]
```

Definition at line 308 of file [vconfig.h](#).

5.26.2.9 checksum_error

```
uint16_t checksum_error
```

Definition at line 313 of file [vconfig.h](#).

5.26.2.10 debug

```
bool debug
```

Definition at line 318 of file [vconfig.h](#).

5.26.2.11 e_types

GEM_EQUIP e_types

Definition at line 304 of file [vconfig.h](#).

5.26.2.12 ebit

uint8_t ebit

Definition at line 314 of file [vconfig.h](#).

5.26.2.13 equip_timeout

uint16_t equip_timeout

Definition at line 313 of file [vconfig.h](#).

5.26.2.14 error

uint8_t error

Definition at line 311 of file [vconfig.h](#).

5.26.2.15 uart

uint8_t uart

Definition at line 316 of file [vconfig.h](#).

5.26.2.16 failed_receive

uint8_t failed_receive

Definition at line 314 of file [vconfig.h](#).

5.26.2.17 failed_send

uint8_t failed_send

Definition at line 314 of file [vconfig.h](#).

5.26.2.18 flipper

bool flipper

Definition at line 318 of file [vconfig.h](#).

5.26.2.19 function

`uint8_t function`

Definition at line 311 of file [vconfig.h](#).

5.26.2.20 g_state

`GEM_STATES g_state`

Definition at line 303 of file [vconfig.h](#).

5.26.2.21 help

`bool help`

Definition at line 318 of file [vconfig.h](#).

5.26.2.22 help_id

`bool help_id`

Definition at line 318 of file [vconfig.h](#).

5.26.2.23 info

`char info[MAX_INFO+1]`

Definition at line 308 of file [vconfig.h](#).

5.26.2.24 llid

`uint8_t llid`

Definition at line 316 of file [vconfig.h](#).

5.26.2.25 m_l_state

`LINK_STATES m_l_state`

Definition at line 305 of file [vconfig.h](#).

5.26.2.26 mode_pwm

`uint16_t mode_pwm`

Definition at line 313 of file [vconfig.h](#).

5.26.2.27 msg_error

uint8_t msg_error

Definition at line 311 of file [vconfig.h](#).

5.26.2.28 msg_ret

uint8_t msg_ret

Definition at line 311 of file [vconfig.h](#).

5.26.2.29 ping

uint16_t ping

Definition at line 313 of file [vconfig.h](#).

5.26.2.30 ping_count

uint8_t ping_count

Definition at line 316 of file [vconfig.h](#).

5.26.2.31 queue

bool queue

Definition at line 318 of file [vconfig.h](#).

5.26.2.32 r_checksum

uint16_t r_checksum

Definition at line 313 of file [vconfig.h](#).

5.26.2.33 r_l_state

LINK_STATES r_l_state

Definition at line 306 of file [vconfig.h](#).

5.26.2.34 rbit

uint8_t rbit

Definition at line 314 of file [vconfig.h](#).

5.26.2.35 error

`bool error`

Definition at line 318 of file [vconfig.h](#).

5.26.2.36 response

`terminal_type response`

Definition at line 315 of file [vconfig.h](#).

5.26.2.37 rx_rs232

`char rx_rs232`

Definition at line 321 of file [vconfig.h](#).

5.26.2.38 rx_total

`uint32_t rx_total`

Definition at line 309 of file [vconfig.h](#).

5.26.2.39 rx_volts

`int16_t rx_volts`

Definition at line 320 of file [vconfig.h](#).

5.26.2.40 s_state

`SEQ_STATES s_state`

Definition at line 301 of file [vconfig.h](#).

5.26.2.41 sequences

`uint16_t sequences`

Definition at line 313 of file [vconfig.h](#).

5.26.2.42 set_sequ

`bool set_sequ`

Definition at line 318 of file [vconfig.h](#).

5.26.2.43 sid

```
uint8_t sid
```

Definition at line 316 of file [vconfig.h](#).

5.26.2.44 speed_spin

```
bool speed_spin
```

Definition at line 318 of file [vconfig.h](#).

5.26.2.45 stack

```
bool stack
```

Definition at line 318 of file [vconfig.h](#).

5.26.2.46 stream

```
uint8_t stream
```

Definition at line 311 of file [vconfig.h](#).

5.26.2.47 systemb

```
uint32_t systemb
```

Definition at line 309 of file [vconfig.h](#).

5.26.2.48 t_checksum

```
uint16_t t_checksum
```

Definition at line 313 of file [vconfig.h](#).

5.26.2.49 t_l_state

```
LINK_STATES t_l_state
```

Definition at line 307 of file [vconfig.h](#).

5.26.2.50 terminal

```
char terminal[MAX_TERM+1]
```

Definition at line 308 of file [vconfig.h](#).

5.26.2.51 testing

```
int32_t testing
```

Definition at line 310 of file [vconfig.h](#).

5.26.2.52 ticker

```
volatile uint8_t ticker
```

Definition at line 317 of file [vconfig.h](#).

5.26.2.53 ticks

```
uint32_t ticks
```

Definition at line 309 of file [vconfig.h](#).

5.26.2.54 timer_error

```
uint16_t timer_error
```

Definition at line 313 of file [vconfig.h](#).

5.26.2.55 tx_rs232

```
char tx_rs232
```

Definition at line 321 of file [vconfig.h](#).

5.26.2.56 tx_total

```
uint32_t tx_total
```

Definition at line 309 of file [vconfig.h](#).

5.26.2.57 tx_volts

```
int16_t tx_volts
```

Definition at line 320 of file [vconfig.h](#).

5.26.2.58 uart

```
uint8_t uart
```

Definition at line 316 of file [vconfig.h](#).

5.26.2.59 uart_speed_fast

`uint8_t uart_speed_fast`

Definition at line 316 of file [vconfig.h](#).

5.26.2.60 ui_state

`UI_STATES ui_state`

Definition at line 302 of file [vconfig.h](#).

5.26.2.61 ui_sw

`UI_STATES ui_sw`

Definition at line 312 of file [vconfig.h](#).

5.26.2.62 v_rx_line

`adc_result_t v_rx_line`

Definition at line 319 of file [vconfig.h](#).

5.26.2.63 v_tx_line

`adc_result_t v_tx_line`

Definition at line 319 of file [vconfig.h](#).

5.26.2.64 vterm

`uint8_t vterm`

Definition at line 316 of file [vconfig.h](#).

5.26.2.65 vterm_switch

`uint8_t vterm_switch`

Definition at line 316 of file [vconfig.h](#).

5.26.2.66 wbit

```
uint8_t wbit
```

Definition at line 314 of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/vconfig.h](#)

5.27 V_help Struct Reference

Data Fields

- const char [message](#) [MAX_HELP_STRLEN]
- const char [display](#) [MAX_HELP_STRLEN]
- const char [extrams](#) [MAX_HELP_STRLEN]

5.27.1 Detailed Description

Definition at line 324 of file [vconfig.h](#).

5.27.2 Field Documentation

5.27.2.1 display

```
const char display[MAX_HELP_STRLEN]
```

Definition at line 325 of file [vconfig.h](#).

5.27.2.2 extrams

```
const char extrams[MAX_HELP_STRLEN]
```

Definition at line 325 of file [vconfig.h](#).

5.27.2.3 message

```
const char message[MAX_HELP_STRLEN]
```

Definition at line 325 of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- [secs_q84.X/vconfig.h](#)

Chapter 6

File Documentation

6.1 0805L075WR.c

```
00001 *PADS-LIBRARY-SCH-DECALS-V9*
00002
00003 0805L075WR          32000 32000 100 10 100 10 4 2 0 2 24
00004 TIMESTAMP 2020.05.27.13.18.59
00005 "Default Font"
00006 "Default Font"
00007 550   250   0 8 100 10 "Default Font"
00008 REF-DES
00009 550   150   0 8 100 10 "Default Font"
00010 PART-TYPE
00011 350  -100   0 12 100 10 "Default Font"
00012 *
00013 350  -200   0 12 100 10 "Default Font"
00014 *
00015 CLOSED 5 10 0 -1
00016 200    50
00017 500    50
00018 500   -50
00019 200   -50
00020 200    50
00021 OPEN  2 10 0 -1
00022 200    0
00023 500    0
00024 T0      0      0 0 0      10      0 0 0      -10      0 32 PIN
00025 P-520   0      0 2 -80     0      0 2 0
00026 T700    0      0 2 0      10      0 0 0      -10      0 32 PIN
00027 P-520   0      0 2 -80     0      0 2 0
00028
00029 *END*
```

6.2 FUSC2114X125N.d

```
00001 *PADS-LIBRARY-PCB-DECALS-V9*
00002
00003 FUSC2114X125N M 0 0 0 2 3 0 2 1 0
00004 TIMESTAMP 2025.02.13.17.02.33
00005 0 0 0 0 1.27 0.127 1 0 34 "Regular <Romansim Stroke Font>"
00006 REF-DES
00007 0 0 0 0 1.27 0.127 1 32 35 "Regular <Romansim Stroke Font>"
00008 PART-TYPE
00009 CLOSED 5 0.05 20 -1
00010 -1.715 1.01
00011 1.715 1.01
00012 1.715 -1.01
00013 -1.715 -1.01
00014 -1.715 1.01
00015 CLOSED 5 0.1 27 -1
00016 -1.05 0.675
00017 1.05 0.675
00018 1.05 -0.675
00019 -1.05 -0.675
00020 -1.05 0.675
00021 OPEN 2 0.2 26 -1
```

```

00022 0 0.575
00023 0 -0.575
00024 T-1 0 -1 0 1
00025 T1 0 1 0 2
00026 PAD 0 3 N 0
00027 -2 0.93 RF 0 90 1.52 0
00028 -1 0 R
00029 0 0 R
00030
00031 *END*
00032 *REMARK* AP
00033 0805L075WR.stp/0/0/0/0/0/0

```

6.3 eadog.p1.d

```

00001 build/default/production/eadog.p1: \
00002 eadog.c \
00003 eadog.h \
00004 vconfig.h \
00005 mcc_generated_files/adc.h \
00006 mcc_generated_files/spi1.h \
00007 mcc_generated_files/pin_manager.h \
00008 mcc_generated_files/mcc.h \
00009 mcc_generated_files/device_config.h \
00010 mcc_generated_files/interrupt_manager.h \
00011 mcc_generated_files/tmr5.h \
00012 mcc_generated_files/tmr6.h \
00013 mcc_generated_files/dma1.h \
00014 mcc_generated_files/tmr2.h \
00015 mcc_generated_files/memory.h \
00016 mcc_generated_files/uart2.h \
00017 mcc_generated_files/uart1.h \
00018 mcc_generated_files/pwm2_16bit.h

```

6.4 gemsecs.p1.d

```

00001 build/default/production/gemsecs.p1: \
00002 gemsecs.c \
00003 gemsecs.h \
00004 vconfig.h \
00005 mcc_generated_files/adc.h \
00006 mcc_generated_files/spi1.h \
00007 mcc_generated_files/pin_manager.h \
00008 mcc_generated_files/mcc.h \
00009 mcc_generated_files/device_config.h \
00010 mcc_generated_files/interrupt_manager.h \
00011 mcc_generated_files/tmr5.h \
00012 mcc_generated_files/tmr6.h \
00013 mcc_generated_files/dma1.h \
00014 mcc_generated_files/tmr2.h \
00015 mcc_generated_files/memory.h \
00016 mcc_generated_files/uart2.h \
00017 mcc_generated_files/uart1.h \
00018 mcc_generated_files/pwm2_16bit.h \
00019 timers.h \
00020 mydisplay.h \
00021 eadog.h \
00022 msg_text.h \
00023 mconfig.h

```

6.5 main.p1.d

```

00001 build/default/production/main.p1: \
00002 main.c \
00003 mcc_generated_files/mcc.h \
00004 mcc_generated_files/device_config.h \
00005 mcc_generated_files/pin_manager.h \
00006 mcc_generated_files/interrupt_manager.h \
00007 mcc_generated_files/adc.h \
00008 mcc_generated_files/tmr5.h \
00009 mcc_generated_files/tmr6.h \
00010 mcc_generated_files/dma1.h \
00011 mcc_generated_files/tmr2.h \

```



```

00012 mcc_generated_files/memory.h \
00013 mcc_generated_files/uart2.h \
00014 mcc_generated_files/uart1.h \
00015 mcc_generated_files/spi1.h \
00016 mcc_generated_files/pwm2_16bit.h \
00017 eadog.h \
00018 vconfig.h \
00019 gemsecs.h \
00020 timers.h \
00021 mydisplay.h \
00022 msg_text.h \
00023 mconfig.h \
00024 rs232.h

```

6.6 adc.p1.d

```

00001 build/default/production/mcc_generated_files/adc.p1: \
00002 mcc_generated_files/adc.c \
00003 mcc_generated_files/adc.h

```

6.7 device_config.p1.d

```

00001 build/default/production/mcc_generated_files/device_config.p1: \
00002 mcc_generated_files/device_config.c

```

6.8 dma1.p1.d

```

00001 build/default/production/mcc_generated_files/dma1.p1: \
00002 mcc_generated_files/dma1.c \
00003 mcc_generated_files/dma1.h

```

6.9 interrupt_manager.p1.d

```

00001 build/default/production/mcc_generated_files/interrupt_manager.p1: \
00002 mcc_generated_files/interrupt_manager.c \
00003 mcc_generated_files/interrupt_manager.h \
00004 mcc_generated_files/mcc.h \
00005 mcc_generated_files/device_config.h \
00006 mcc_generated_files/pin_manager.h \
00007 mcc_generated_files/adc.h \
00008 mcc_generated_files/tmr5.h \
00009 mcc_generated_files/tmr6.h \
00010 mcc_generated_files/dma1.h \
00011 mcc_generated_files/tmr2.h \
00012 mcc_generated_files/memory.h \
00013 mcc_generated_files/uart2.h \
00014 mcc_generated_files/uart1.h \
00015 mcc_generated_files/spi1.h \
00016 mcc_generated_files/pwm2_16bit.h

```

6.10 mcc.p1.d

```

00001 build/default/production/mcc_generated_files/mcc.p1: \
00002 mcc_generated_files/mcc.c \
00003 mcc_generated_files/mcc.h \
00004 mcc_generated_files/device_config.h \
00005 mcc_generated_files/pin_manager.h \
00006 mcc_generated_files/interrupt_manager.h \
00007 mcc_generated_files/adc.h \
00008 mcc_generated_files/tmr5.h \
00009 mcc_generated_files/tmr6.h \
00010 mcc_generated_files/dma1.h \
00011 mcc_generated_files/tmr2.h \
00012 mcc_generated_files/memory.h \
00013 mcc_generated_files/uart2.h \
00014 mcc_generated_files/uart1.h \
00015 mcc_generated_files/spi1.h \
00016 mcc_generated_files/pwm2_16bit.h

```

6.11 memory.p1.d

```
00001 build/default/production/mcc_generated_files/memory.p1: \
00002 mcc_generated_files/memory.c \
00003 mcc_generated_files/memory.h
```

6.12 pin_manager.p1.d

```
00001 build/default/production/mcc_generated_files/pin_manager.p1: \
00002 mcc_generated_files/pin_manager.c \
00003 mcc_generated_files/pin_manager.h
```

6.13 pwm2_16bit.p1.d

```
00001 build/default/production/mcc_generated_files/pwm2_16bit.p1: \
00002 mcc_generated_files/pwm2_16bit.c \
00003 mcc_generated_files/pwm2_16bit.h
```

6.14 spi1.p1.d

```
00001 build/default/production/mcc_generated_files/spi1.p1: \
00002 mcc_generated_files/spi1.c \
00003 mcc_generated_files/spi1.h
```

6.15 tmr2.p1.d

```
00001 build/default/production/mcc_generated_files/tmr2.p1: \
00002 mcc_generated_files/tmr2.c \
00003 mcc_generated_files/tmr2.h
```

6.16 tmr5.p1.d

```
00001 build/default/production/mcc_generated_files/tmr5.p1: \
00002 mcc_generated_files/tmr5.c \
00003 mcc_generated_files/tmr5.h \
00004 mcc_generated_files/interrupt_manager.h
```

6.17 tmr6.p1.d

```
00001 build/default/production/mcc_generated_files/tmr6.p1: \
00002 mcc_generated_files/tmr6.c \
00003 mcc_generated_files/tmr6.h \
00004 mcc_generated_files/interrupt_manager.h \
00005 mcc_generated_files/../timers.h
```

6.18 uart1.p1.d

```
00001 build/default/production/mcc_generated_files/uart1.p1: \
00002 mcc_generated_files/uart1.c \
00003 mcc_generated_files/uart1.h \
00004 mcc_generated_files/interrupt_manager.h
```

6.19 uart2.p1.d

```
00001 build/default/production/mcc_generated_files/uart2.pl: \
00002 mcc_generated_files/uart2.c \
00003 mcc_generated_files/uart2.h \
00004 mcc_generated_files/interrupt_manager.h
```

6.20 mconfig.p1.d

```
00001 build/default/production/mconfig.pl: \
00002 mconfig.c \
00003 mconfig.h \
00004 mcc_generated_files/mcc.h \
00005 mcc_generated_files/device_config.h \
00006 mcc_generated_files/pin_manager.h \
00007 mcc_generated_files/interrupt_manager.h \
00008 mcc_generated_files/adc.h \
00009 mcc_generated_files/tmr5.h \
00010 mcc_generated_files/tmr6.h \
00011 mcc_generated_files/dma1.h \
00012 mcc_generated_files/tmr2.h \
00013 mcc_generated_files/memory.h \
00014 mcc_generated_files/uart2.h \
00015 mcc_generated_files/uart1.h \
00016 mcc_generated_files/spi1.h \
00017 mcc_generated_files/pwm2_16bit.h \
00018 vconfig.h \
00019 mydisplay.h \
00020 eadog.h \
00021 timers.h \
00022 msg_text.h
```

6.21 rs232.p1.d

```
00001 build/default/production/rs232.pl: \
00002 rs232.c \
00003 rs232.h \
00004 vconfig.h \
00005 mcc_generated_files/adc.h \
00006 mcc_generated_files/spi1.h \
00007 mcc_generated_files/pin_manager.h
```

6.22 timers.p1.d

```
00001 build/default/production/timers.pl: \
00002 timers.c \
00003 vconfig.h \
00004 mcc_generated_files/adc.h \
00005 mcc_generated_files/spi1.h \
00006 mcc_generated_files/pin_manager.h \
00007 timers.h
```

6.23 BZT52B5V6-TP.c

```
00001 *PADS-LIBRARY-SCH-DECALS-V9*
00002
00003 BZT52B5V6-TP      32000 32000 100 10 100 10 4 6 0 2 8
00004 TIMESTAMP 2020.10.26.07.11.30
00005 "Default Font"
00006 "Default Font"
00007 400 350 0 8 100 10 "Default Font"
00008 REF-DES
00009 400 250 0 8 100 10 "Default Font"
00010 PART-TYPE
00011 400 -200 0 0 100 10 "Default Font"
00012 *
00013 400 -300 0 0 100 10 "Default Font"
00014 *
```

```

00015 COPCLS 4 10 0 -1
00016 200 0
00017 400 100
00018 400 -100
00019 200 0
00020 OPEN 2 10 0 -1
00021 200 80
00022 200 -80
00023 OPEN 2 10 0 -1
00024 200 80
00025 240 100
00026 OPEN 2 10 0 -1
00027 160 -100
00028 200 -80
00029 OPEN 2 10 0 -1
00030 100 0
00031 200 0
00032 OPEN 2 10 0 -1
00033 500 0
00034 400 0
00035 T0 0 0 0 60 10 0 2 140 10 0 16 PINSHORT
00036 P-520 0 0 2 -80 0 0 2 0
00037 T600 0 0 2 60 10 0 2 140 10 0 16 PINSHORT
00038 P-520 0 0 2 -80 0 0 2 0
00039
00040 *END*

```

6.24 SOD3716X135N.d

```

00001 *PADS-LIBRARY-PCB-DECALS-V9*
00002
00003 SOD3716X135N M 0 0 0 2 5 0 2 1 0
00004 TIMESTAMP 2025.02.23.02.02.25
00005 0 0 0 0 1.27 0.127 1 0 34 "Regular <Romansim Stroke Font>"
00006 REF-DES
00007 0 0 0 0 1.27 0.127 1 32 35 "Regular <Romansim Stroke Font>"
00008 PART-TYPE
00009 CLOSED 5 0.05 20 -1
00010 -2.575 1.675
00011 2.575 1.675
00012 2.575 -1.675
00013 -2.575 -1.675
00014 -2.575 1.675
00015 CLOSED 5 0.1 27 -1
00016 -1.35 0.8
00017 1.35 0.8
00018 1.35 -0.8
00019 -1.35 -0.8
00020 -1.35 0.8
00021 OPEN 2 0.1 27 -1
00022 -1.35 0.275
00023 -0.825 0.8
00024 OPEN 2 0.2 26 -1
00025 -2.325 0.8
00026 1.35 0.8
00027 OPEN 2 0.2 26 -1
00028 -1.35 -0.8
00029 1.35 -0.8
00030 T-1.8 0 -1.8 0 1
00031 T1.8 0 1.8 0 2
00032 PAD 0 3 N 0
00033 -2 0.75 RF 0 0 1.05 0
00034 -1 0 R
00035 0 0 R
00036
00037 *END*
00038 *REMARK* AP
00039 BZT52B5V6-TP.stp/0/0/0/0/0/0

```

6.25 doprnt.d

```

00001 dist/default/production/doprnt.pl: \
00002 /opt/microchip/xc8/v3.00/pic/sources/c99/common/doprnt.c

```

6.26 eadog.c

```

00001 #include <string.h>
00002 #include "eadog.h"
00003 #include "vconfig.h"
00004 #include "mcc_generated_files/mcc.h"
00005
00006
00007 #ifdef TRACE
00008 #define E_TRACE DEBUG1_Toggle()
00009 #else
00010 #define E_TRACE ""
00011 #endif
00012
00013 #ifdef CAN_REMOTE // SPI DMA wait timeout counts
00014 #define DONE_DELAY 99999
00015 #else
00016 #define DONE_DELAY 9999
00017 #endif
00018
00019 volatile struct spi_link_type spi_link = {
00020     .LCD_DATA = false,
00021 };
00022
00023 static const spil_configuration_t spil_configuration[] = {
00024     {0x83, 0x20, 0x3, 0x4, 0}
00025 };
00026
00027 static char Sstr[NSB][LSB];
00028 static volatile bool scroll_lock = false, powerup = true;
00029 static volatile uint8_t scroll_line_pos = 4;
00030
00031 static void send_lcd_cmd_long(const uint8_t); // for display init only
00032 static void send_lcd_data(const uint8_t);
00033 static void send_lcd_cmd(const uint8_t);
00034 static void spi_byte(void);
00035 static void wdtdelay(const uint32_t);
00036
00037 /*
00038  * Init the NHD-0420D3Z-NSW-BBW-V3 in 8-bit serial mode
00039  * channel 1 DMA if configured
00040  */
00041 bool init_display(void)
00042 {
00043     spi_link.txbuf = lcd_dma_buf; // use MCC DMA buffer variable
00044     memset(Sstr, ' ', NSB * LSB); // clear scroll buffer of junk
00045
00046 #ifdef USE_LCD_DMA
00047     DMA1_SetSCNTInterruptHandler(clear_lcd_done);
00048     DMA1_SetORInterruptHandler(spi_byte);
00049     DMA1_SetDMAPriority(2);
00050 #endif
00051 #ifdef NHD // uses MODE 3 on the Q84,
00052     https://newhavendisplay.com/content/specs/NHD-0420D3Z-NSW-BBW-V3.pdf
00053 #ifdef USEMCC_SPI
00054 #else
00055     SPI1CON0bits.EN = 0;
00056     // mode 3
00057     SPI1CON1 = 0x20;
00058     // SSET disabled; RXR suspended if the RxFIFO is full; TXR required for a transfer;
00059     SPI1CON2 = 0x03;
00060     // BAUD 0;
00061     SPI1BAUD = 0x04; // 50kHz SCK
00062     // CLKSEL MFINTOSC;
00063     SPI1CLK = 0x02;
00064     // BMODE every byte; LSBF MSb first; EN enabled; MST bus master;
00065     SPI1CON0 = 0x83;
00066     SPI1CON0bits.EN = 1;
00067 #endif
00068     if (powerup) {
00069         wdtdelay(LCD_PWR_DELAY); // > 400ms power up delay
00070     }
00071
00072 #ifdef USE_LCD_DMA
00073     SPI1INTFbits.SPI1TXUIF = 0;
00074     DMASELECT = 0; // use DMA1
00075     DMA1CON0bits.EN = 0;
00076     SPI1CON0bits.EN = 0;
00077     SPI1CON2 = 0x02; // Received data is not stored in the FIFO
00078     SPI1CON0bits.EN = 1;
00079     DMA1CON1bits.DMODE = 0;
00080     DMA1CON1bits.DSTP = 0;
00081     DMA1CON1bits.SMODE = 1;
00082     DMA1CON1bits.SMR = 0;
00083     DMA1CON1bits.SSTP = 1;
00084     DMA1SSA = (uint24_t) spi_link.txbuf;

```

```

00084     DMAAnCON0bits.DGO = 0;
00085     DMAAnCON0bits.EN = 1; /* enable DMA */
00086     SPI1INTFbits.SPI1TXUIF = 1;
00087     send_lcd_cmd_dma(LCD_CMD_BRI); // set back-light level
00088     send_lcd_data_dma(NHD_BL_MED);
00089     send_lcd_cmd_dma(LCD_CMD_CONT); // set display contrast
00090     send_lcd_data_dma(NHD_CONT);
00091     send_lcd_cmd_dma(LCD_CMD_ON); // display on
00092     send_lcd_cmd_dma(LCD_CMD_CLR); // clear screen
00093     wdtdelay(NHD_L_DELAY);
00094     DMA1_StopTransfer();
00095 #else
00096     send_lcd_cmd(LCD_CMD_BRI); // set back-light level
00097     send_lcd_data(NHD_BL_LOW);
00098     send_lcd_cmd(LCD_CMD_CONT); // set display contrast
00099     send_lcd_data(NHD_CONT);
00100     send_lcd_cmd(LCD_CMD_OFF); // display on
00101     send_lcd_cmd(LCD_CMD_CLR); // clear screen
00102     wdtdelay(NHD_L_DELAY);
00103     DMA1_StopTransfer();
00104 #endif
00105 #endif
00106     powerup = false; // only of the first display init call
00107     return true;
00108 }
00109
00110 #ifdef NHD
00111
00112 /*
00113  * R2 short on LCD NHD-0420D3Z-NSW-BBW-V3 board
00114  */
00115
00116 static void send_lcd_data(const uint8_t data)
00117 {
00118     CS_SetLow();
00119     SPI1_ExchangeByte(data);
00120     wdtdelay(NHD_T_DELAY);
00121 }
00122
00123 static void send_lcd_cmd(const uint8_t cmd)
00124 {
00125     CS_SetLow();
00126     SPI1_ExchangeByte(NHD_CMD);
00127     wdtdelay(NHD_T_DELAY);
00128     SPI1_ExchangeByte(cmd);
00129     wdtdelay(NHD_T_DELAY);
00130 }
00131
00132 static void send_lcd_cmd_long(const uint8_t cmd)
00133 {
00134     CS_SetLow();
00135     SPI1_ExchangeByte(NHD_CMD);
00136     wdtdelay(NHD_T_DELAY);
00137     SPI1_ExchangeByte(cmd);
00138     wdtdelay(NHD_L_DELAY);
00139 }
00140
00141 /*
00142  * CAN use DMA channel 1 for transfers
00143  */
00144 void eaDogM_WriteString(char *strPtr)
00145 {
00146     uint8_t len = (uint8_t) strlen(strPtr);
00147
00148     E_TRACE;
00149     wait_lcd_done();
00150     wait_lcd_set();
00151     CS_SetLow(); /* SPI select display */
00152     if (len > (uint8_t) MAX_STRLEN) {
00153         len = MAX_STRLEN;
00154     }
00155     memcpy(spi_link.txbuf, strPtr, len);
00156 #ifndef USE_LCD_DMA
00157     DMAAnCON0bits.EN = 0; /* disable DMA to change source count */
00158     DMA1_SetSourceSize(len);
00159     DMA1_SetDestinationSize(1);
00160     DMAAnCON0bits.EN = 1; /* enable DMA */
00161 #else
00162     SPI1_ExchangeBlock(spi_link.txbuf, len);
00163 #endif
00164     start_lcd(); // start DMA transfer
00165 }
00166
00167 /*
00168  * uses DMA channel 1 for transfers
00169  */
00170 void send_lcd_cmd_dma(const uint8_t strPtr)

```

```

00171 {
00172     send_lcd_data_dma(NHD_CMD); //prefix
00173     send_lcd_data_dma(strPtr); // cmd code
00174 }
00175
00176 /*
00177  * uses DMA channel 1 for transfers
00178  */
00179 void send_lcd_data_dma(const uint8_t strPtr)
00180 {
00181     E_TRACE;
00182     wait_lcd_done();
00183     wait_lcd_set();
00184     CS_SetLow(); /* SPI select display */
00185     spi_link.txbuf[0] = strPtr;
00186     DMA1_CON0bits.EN = 0; /* disable DMA to change source count */
00187     DMA1_SetSourceSize(1);
00188     DMA1_SetDestinationSize(1);
00189     DMA1_CON0bits.EN = 1; /* enable DMA */
00190     start_lcd(); // start DMA transfer
00191 }
00192
00193 /*
00194  * send three byte command string via DMA
00195  */
00196 void send_lcd_pos_dma(const uint8_t strPtr)
00197 {
00198     E_TRACE;
00199     wait_lcd_done();
00200     wait_lcd_set();
00201     CS_SetLow(); /* SPI select display */
00202     spi_link.txbuf[0] = NHD_CMD;
00203     spi_link.txbuf[1] = NHD_POS;
00204     spi_link.txbuf[2] = strPtr;
00205     DMA1_CON0bits.EN = 0; /* disable DMA to change source count */
00206     DMA1_SetSourceSize(3);
00207     DMA1_SetDestinationSize(1);
00208     DMA1_CON0bits.EN = 1; /* enable DMA */
00209     start_lcd(); // start DMA transfer
00210 }
00211
00212 void eaDogM_WriteStringAtPos(const uint8_t r, const uint8_t c, char *strPtr)
00213 {
00214     uint8_t row;
00215
00216 #ifndef CAN_REMOTE
00217     if (scroll_lock) { // don't update LCD text when in scroll mode
00218         return;
00219     }
00220 #endif
00221
00222     switch (r) {
00223     case LCD1:
00224         row = 0x40;
00225         break;
00226     case LCD2:
00227         row = 0x14;
00228         break;
00229     case LCD3:
00230         row = 0x54;
00231         break;
00232     case LCD0:
00233         row = 0x00;
00234         break;
00235     default:
00236         row = 0x00;
00237         break;
00238     }
00239
00240 #ifndef USE_LCD_DMA
00241     send_lcd_pos_dma(row + c);
00242     wtdelay(NHD_S_DELAY); // display command processing delay
00243     E_TRACE;
00244 #else
00245     send_lcd_cmd(0x45);
00246     send_lcd_data(row + c);
00247 #endif
00248 #ifndef USE_CAN
00249     can_fd_lcd_mirror(r, strPtr);
00250 #endif
00251     eaDogM_WriteString(strPtr);
00252 }
00253
00254 void eaDogM_WriteIntAtPos(const uint8_t r, const uint8_t c, const uint8_t i)
00255 {
00256
00257 }

```

```

00258
00259 void eaDogM_SetPos(const uint8_t r, const uint8_t c)
00260 {
00261 }
00262 }
00263
00264 void eaDogM_ClearRow(const uint8_t r)
00265 {
00266 }
00267 }
00268
00269 void eaDogM_WriteByteToCGRAM(const uint8_t ndx, const uint8_t data)
00270 {
00271 }
00272 }
00273
00274 #else
00275 #endif
00276
00277 void eaDogM_WriteCommand(const uint8_t cmd)
00278 {
00279 #ifdef USE_LCD_DMA
00280     send_lcd_cmd_dma(cmd);
00281 #else
00282     send_lcd_cmd(cmd);
00283 #endif
00284 }
00285
00286 void eaDogM_WriteChr(const int8_t value)
00287 {
00288 #ifdef USE_LCD_DMA
00289     send_lcd_data_dma((uint8_t) value);
00290 #else
00291     send_lcd_data((uint8_t) value);
00292 #endif
00293 }
00294
00295 /*
00296  * Trigger the SPI DMA transfer to the LCD display
00297  */
00298 void start_lcd(void)
00299 {
00300 #ifdef USE_LCD_DMA
00301     DMA1_StartTransferWithTrigger();
00302 #endif
00303 }
00304
00305 void wait_lcd_set(void)
00306 {
00307     spi_link.LCD_DATA = true;
00308 }
00309
00310 bool wait_lcd_check(void)
00311 {
00312     return spi_link.LCD_DATA;
00313 }
00314
00315 void wait_lcd_done(void)
00316 {
00317 #ifdef USE_LCD_DMA
00318     uint32_t delay = 0;
00319     while (spi_link.LCD_DATA) {
00320         if (delay++ > DONE_DELAY) {
00321             MLED_SetHigh();
00322             return;
00323         }
00324     };
00325     delay = 0;
00326     while (!SPI1STATUSbits.TXBE) {
00327         if (delay++ > DONE_DELAY) {
00328             MLED_SetHigh();
00329             return;
00330         }
00331     };
00332     MLED_SetLow();
00333 #endif
00334 }
00335
00336 /*
00337  * in DMA mode this is a ISR that runs when the source count is complete
00338  */
00339 void clear_lcd_done(void)
00340 {
00341     spi_link.LCD_DATA = false;
00342     E_TRACE;
00343     E_TRACE;
00344     E_TRACE;

```



```

00345     E_TRACE;
00346 }
00347
00348 void spi_rec_done(void)
00349 {
00350     #ifdef USE_LCD_DMA
00351     #endif
00352 }
00353
00354 /*
00355  * SPI buffer overrun ISR
00356  */
00357 static void spi_byte(void)
00358 {
00359     MLED_Toggle();
00360 }
00361
00362 /*
00363  * auto scrolls up the string on the display
00364  */
00365 char * eaDogM_Scroll_String(char *strPtr)
00366 {
00367     scroll_lock = true;
00368     memcpy((void *) &Sstr[4][0], &Sstr[0][0], MAX_STRLEN); // move top line to old line buffer
00369     memcpy((void *) &Sstr[0][0], &Sstr[1][0], MAX_STRLEN);
00370     memcpy((void *) &Sstr[1][0], &Sstr[2][0], MAX_STRLEN);
00371     memcpy((void *) &Sstr[2][0], &Sstr[3][0], MAX_STRLEN);
00372     memcpy((void *) &Sstr[3][0], strPtr, MAX_STRLEN); // place new text on the bottom line
00373     scroll_line_pos = 4;
00374     return &Sstr[4][0];
00375 }
00376
00377 /*
00378  * mainline info scroll updater, should run a ~1 second ticks
00379  */
00380 void eaDogM_Scroll_Task(void)
00381 {
00382     if (!scroll_lock) {
00383         return;
00384     }
00385
00386     if (scroll_line_pos == 0) { // wait for last line display time period
00387         scroll_lock = false;
00388         scroll_line_pos = 4;
00389         return;
00390     }
00391
00392     scroll_lock = false;
00393     --scroll_line_pos;
00394     eaDogM_WriteStringAtPos(scroll_line_pos, 0, &Sstr[scroll_line_pos][0]);
00395     scroll_lock = true;
00396 }
00397 }
00398
00399 void no_dma_set_lcd(void)
00400 {
00401     send_lcd_cmd(LCD_CMD_BRI); // set back-light level
00402     send_lcd_data(NHD_BL_HIGH);
00403     send_lcd_cmd(LCD_CMD_CON); // set display contrast
00404     send_lcd_data(NHD_CONT);
00405     send_lcd_cmd(LCD_CMD_ON); // display on
00406     send_lcd_cmd(LCD_CMD_CLR); // clear screen
00407     wdtDelay(NHD_L_DELAY);
00408 }
00409
00410 void check_lcd_dim(const bool dim)
00411 {
00412     if (B.display_update) {
00413         B.display_update = false;
00414         B.dim_delay = 0;
00415     #ifdef USE_LCD_DMA
00416         if (dim) {
00417             send_lcd_cmd_dma(LCD_CMD_BRI); // set back-light level
00418             send_lcd_data_dma(NHD_BL_OFF);
00419         } else {
00420             send_lcd_cmd_dma(LCD_CMD_BRI); // set back-light level
00421             send_lcd_data_dma(NHD_BL_HIGH);
00422         }
00423     #else
00424         if (dim) {
00425             send_lcd_cmd(LCD_CMD_BRI); // set back-light level
00426             send_lcd_data(NHD_BL_LOW);
00427         } else {
00428             send_lcd_cmd(LCD_CMD_BRI); // set back-light level
00429             send_lcd_data(NHD_BL_HIGH);
00430         }
00431     #endif

```

```

00432     }
00433 }
00434
00435 void set_lcd_dim(const bool dim)
00436 {
00437     if (B.display_update) {
00438         B.display_update = false;
00439         B.dim_delay = 0;
00440 #ifndef USE_LCD_DMA
00441         if (dim) {
00442             send_lcd_cmd_dma(LCD_CMD_BRI); // set back-light level
00443             send_lcd_data_dma(NHD_BL_OFF);
00444         } else {
00445             send_lcd_cmd_dma(LCD_CMD_BRI); // set back-light level
00446             send_lcd_data_dma(NHD_BL_HIGH);
00447         }
00448 #else
00449         if (dim) {
00450             send_lcd_cmd(LCD_CMD_BRI); // set back-light level
00451             send_lcd_data(NHD_BL_LOW);
00452         } else {
00453             send_lcd_cmd(LCD_CMD_BRI); // set back-light level
00454             send_lcd_data(NHD_BL_HIGH);
00455         }
00456 #endif
00457     }
00458
00459     if (B.dim_delay++ >= DIM_DELAY) {
00460         B.dim_delay = 0;
00461 #ifndef USE_LCD_DMA
00462         send_lcd_cmd_dma(LCD_CMD_BRI); // set back-light level
00463         send_lcd_data_dma(NHD_BL_OFF);
00464 #else
00465         send_lcd_cmd(LCD_CMD_BRI); // set back-light level
00466         send_lcd_data(NHD_BL_LOW);
00467 #endif
00468     }
00469 }
00470
00471 /*
00472  * busy loop delay with WDT reset
00473  */
00474 static void wdtdelay(const uint32_t delay)
00475 {
00476     uint32_t dcount;
00477
00478     for (dcount = 0; dcount <= delay; dcount++) { // delay a bit
00479         ClrWdt(); // reset the WDT timer
00480     };
00481 }

```

6.27 eadog.h

```

00001 /*
00002  * File:   eadog.h
00003  * Author: root
00004  *
00005  * Created on July 27, 2015, 2:05 PM
00006  */
00007
00008 /* Parts taken from:
00009  * Modified for XC8 and with SPI using ring buffers with interrupts
00010  * or DMA directly from buffer memory to the SPI device
00011  *
00012  *      file: EA-DOGM_MIO.c
00013  *      version: 2.03
00014  *      description: Multi I/O driver for EA DOGM displays
00015  *      : Uses 8Bit, SPI HW or SPI SW (bitbang)
00016  *      written by : Michael Bradley (mbradley@mcu-labs.com)
00017  *      contributions: Imaginos (CCS forum), Emil Nad (8Bit testing)
00018  *      jgschmidt (CCS forum)
00019  *
00020  *
00021  * For NHD-0420D3Z-NSW-BBW-V3 20X4 CHARACTER display
00022  Pin No. Symbol External Connection Function Description
00023 1 SPISS MPU SPI Slave Select (NC in I2C mode)
00024 2 SDO NC No Connect
00025 3 SCK/SCL MPU Serial Clock
00026 4 SDI/SDA MPU Serial Data In (SPI) / Serial Data (I2C)
00027 5 VSS Power Supply Ground
00028 6 VDD Power Supply Power Supply for logic (+5.0V)
00029  *
00030 R1 R2 Protocol Description

```

```

00031 Short Short TEST Self-test
00032 Open Short SPI 100KHz max clock
00033 Short Open I2C 50KHz max clock
00034 Open Open RS-232 5V, TTL signal
00035 */
00036
00037 #ifndef EADOG_H
00038 #define EADOG_H
00039
00040 #ifdef __cplusplus
00041 extern "C" {
00042 #endif
00043 #include "vconfig.h"
00044
00045 #define USEMCC_SPI
00046 #ifndef CAN_REMOTE_NODMA
00047 #define USE_LCD_DMA
00048 #endif
00049
00050     typedef struct {
00051         uint8_t con0;
00052         uint8_t con1;
00053         uint8_t con2;
00054         uint8_t baud;
00055         uint8_t operation;
00056     } spil_configuration_t;
00057
00058 #define EADOGM_CMD_CLR 1
00059 #define EADOGM_CMD_CURSOR_ON 0b00001111
00060 #define EADOGM_CMD_CURSOR_OFF 0b00001100
00061 #define EADOGM_CMD_DISPLAY_ON 0b00001100
00062 #define EADOGM_CMD_DISPLAY_OFF 0b00001000
00063 #define EADOGM_CMD_DDRAM_ADDR 0b10000000
00064 #define EADOGM_CMD_CGRAM_ADDR 0b01000000
00065 #define EADOGM_CMD_SELECT_R0 0b00011000
00066 #define EADOGM_CMD_SELECT_R1 0b00010000
00067 #define EADOGM_CMD_SET_TABLE2 0b00101010
00068 #define EADOGM_COLSPAN 16
00069
00070 #define LCD_CMD_MASK 0x01
00071 #define LCD_CMD_SET 0x100
00072 #define LCD_CLEAR_HOME 0x04
00073 #define LCD_CMD_ON 0x41
00074 #define LCD_CMD_OFF 0x42
00075 #define LCD_CMD_HOME 0x46
00076 #define LCD_CMD_CLR 0x51
00077 #define LCD_CMD_CONT 0x52
00078 #define LCD_CMD_BRI 0x53
00079 #define NHD_CMD 0xFE
00080 #define NHD_POS 0x45
00081 #define NHD_BL_OFF 1
00082 #define NHD_BL_LOW 2
00083 #define NHD_BL_MED 5
00084 #define NHD_BL_HIGH 8
00085 #define NHD_CONT 35
00086
00087 #define NHD_T_DELAY 8
00088 #define NHD_S_DELAY 400
00089 #define NHD_L_DELAY 1500
00090
00091 #define LCD0 0
00092 #define LCD1 1
00093 #define LCD2 2
00094 #define LCD3 3
00095
00096 #define DIM_DELAY 6
00097
00098 #define NSB 5
00099 #define LSB 21
00100
00101 #define LCD_PWR_DELAY 600000
00102
00103     bool init_display(void);
00104     void no_dma_set_lcd(void);
00105     void send_lcd_data_dma(const uint8_t);
00106     void send_lcd_cmd_dma(const uint8_t);
00107     void send_lcd_pos_dma(const uint8_t);
00108     void start_lcd(void);
00109     void wait_lcd_set(void);
00110     bool wait_lcd_check(void);
00111     void wait_lcd_done(void);
00112     void eaDogM_WriteChr(const int8_t);
00113     void eaDogM_WriteCommand(const uint8_t);
00114     void eaDogM_SetPos(const uint8_t, const uint8_t);
00115     void eaDogM_ClearRow(const uint8_t);
00116     void eaDogM_WriteString(char *);
00117     void eaDogM_WriteStringAtPos(const uint8_t, const uint8_t, char *);

```

```

00118     void eaDogM_WriteIntAtPos(const uint8_t, const uint8_t, const uint8_t);
00119     void eaDogM_WriteByteToCGRAM(const uint8_t, const uint8_t);
00120     void set_lcd_dim(const bool);
00121     void check_lcd_dim(const bool);
00122
00123     char * eaDogM_Scroll_String(char *);
00124     void eaDogM_Scroll_Task(void);
00125
00126     // DMA complete flag
00127     void clear_lcd_done(void);
00128     void spi_rec_done(void);
00129 #ifndef USE_CAN
00130     extern void can_fd_lcd_mirror(const uint8_t, char *);
00131 #endif
00132
00133 #define eaDogM_Cls()          eaDogM_WriteCommand(EADOGM_CMD_CLR)
00134 #define eaDogM_CursorOn()    eaDogM_WriteCommand(EADOGM_CMD_CURSOR_ON)
00135 #define eaDogM_CursorOff()   eaDogM_WriteCommand(EADOGM_CMD_CURSOR_OFF)
00136 #define eaDogM_DisplayOn()   eaDogM_WriteCommand(EADOGM_CMD_DISPLAY_ON)
00137 #define eaDogM_DisplayOff()  eaDogM_WriteCommand(EADOGM_CMD_DISPLAY_OFF)
00138
00139
00140 #ifdef __cplusplus
00141 }
00142 #endif
00143
00144 #endif /* EADOG_H */
00145

```

6.28 gemsecs.c

```

00001 /*
00002  * the main secs/gem protocol functions
00003  */
00004 #include "gemsecs.h"
00005
00006 extern struct V_data V;
00007 extern header10 r_block;
00008 extern struct header10 H10[];
00009 extern struct header12 H12[];
00010 extern struct header13 H13[];
00011 extern struct header14 H14[];
00012 extern struct header17 H17[];
00013 extern struct header18 H18[];
00014 extern struct header24 H24[];
00015 extern struct header26 H26[];
00016 extern struct header27 H27[];
00017 extern struct header33 H33[];
00018 extern const header33 HC33[];
00019 extern struct header153 H153[];
00020 extern header254 H254[];
00021 extern gem_message_type S[10];
00022 extern gem_display_type D[2];
00023
00024 static bool secs_send(uint8_t *, const uint8_t, const bool, const uint8_t);
00025
00026 /*
00027  * Checksum for message and header block after length byte
00028  */
00029 uint16_t block_checksum(uint8_t *byte_block, const uint16_t byte_count)
00030 {
00031     uint16_t sum = 0, i;
00032
00033     for (i = 0; i < byte_count; i++) {
00034         sum += byte_block[i];
00035     }
00036
00037     if (V.error && (rand() > ERROR_CHECKSUM)) {
00038         sum++;
00039     }
00040
00041     return sum;
00042 }
00043
00044 /*
00045  * Checksum for data stream
00046  */
00047 uint16_t run_checksum(const uint8_t byte_block, const bool clear)
00048 {
00049     static uint16_t sum = 0;
00050
00051     if (clear)
00052         sum = 0;

```

```

00053
00054     sum += byte_block;
00055     return sum;
00056 }
00057
00058 LINK_STATES m_protocol(LINK_STATES *m_link)
00059 {
00060     uint8_t rxData;
00061     static uint8_t rxData_l = 0, *b_block = (uint8_t*) & H254[0];
00062
00063     switch (*m_link) {
00064     case LINK_STATE_IDLE:
00065 #ifdef DB1
00066         WaitMs(50);
00067 #endif
00068         if (UART1_is_rx_ready()) {
00069             rxData = UART1_Read();
00070             V.rx_total++;
00071             DLED_Toggle();
00072             if (rxData == ENQ) {
00073                 V.uart = HOST_UART;
00074                 StartTimer(TMR_T2, T2);
00075                 V.error = LINK_ERROR_NONE; // reset error status
00076                 *m_link = LINK_STATE_ENQ;
00077             }
00078         }
00079         if (UART2_is_rx_ready()) {
00080             rxData = UART2_Read();
00081             V.rx_total++;
00082             DLED_Toggle();
00083             if (rxData == ENQ) {
00084                 V.uart = EQUIP_UART;
00085                 StartTimer(TMR_T2, T2);
00086                 V.error = LINK_ERROR_NONE; // reset error status
00087                 *m_link = LINK_STATE_ENQ;
00088             }
00089         }
00090         break;
00091     case LINK_STATE_ENQ:
00092         rxData_l = 0;
00093         if (TimerDone(TMR_T2)) {
00094             V.error = LINK_ERROR_T2;
00095             V.all_errors++;
00096             V.timer_error++;
00097             V.failed_receive = RECV_ERROR_T2;
00098             *m_link = LINK_STATE_NAK;
00099             MLED_SetHigh();
00100         } else {
00101 #ifdef DB2
00102             WaitMs(1);
00103             if (V.uart == HOST_UART)
00104                 if (V.rerror && (rand() < ERROR_COMM)) {
00105                     secs_send((uint8_t*) & H27[0], sizeof(header27), true, V.uart);
00106                 }
00107             if (V.uart == EQUIP_UART)
00108                 if (V.rerror && rand() < ERROR_COMM) {
00109                     secs_send((uint8_t*) & H10[0], sizeof(header10), true, V.uart);
00110                 }
00111             V.error = LINK_ERROR_NONE; // reset error status
00112             *m_link = LINK_STATE_EOT;
00113             StartTimer(TMR_T2, T2);
00114 #else
00115             if (UART1_is_rx_ready()) {
00116                 rxData = UART1_Read();
00117                 V.rx_total++;
00118                 DLED_Toggle();
00119                 if (rxData == EOT) {
00120                     StartTimer(TMR_T2, T2);
00121                     V.error = LINK_ERROR_NONE; // reset error status
00122                     *m_link = LINK_STATE_EOT;
00123                 }
00124             }
00125             if (UART2_is_rx_ready()) {
00126                 rxData = UART2_Read();
00127                 V.rx_total++;
00128                 DLED_Toggle();
00129                 if (rxData == EOT) {
00130                     StartTimer(TMR_T2, T2);
00131                     V.error = LINK_ERROR_NONE; // reset error status
00132                     *m_link = LINK_STATE_EOT;
00133                 }
00134             }
00135 #endif
00136         }
00137         break;
00138     case LINK_STATE_EOT:
00139         if (TimerDone(TMR_T2)) {

```

```

00140         V.error = LINK_ERROR_T2;
00141         V.timer_error++;
00142         V.all_errors++;
00143         V.failed_receive = RECV_ERROR_T2;
00144         *m_link = LINK_STATE_NAK;
00145         MLED_SetHigh();
00146     } else {
00147         if (UART1_is_rx_ready()) {
00148             rxData = UART1_Read();
00149             V.rx_total++;
00150             DLED_Toggle();
00151             if (rxData_l == 0) { // start header reads
00152                 r_block.length = rxData; // header+message bytes
00153                 run_checksum(0, true);
00154                 rxData_l++;
00155                 b_block[sizeof(header254) - rxData_l] = rxData; // buffer the message
00156             } else {
00157                 /*
00158                  * skip possible message data
00159                  */
00160                 if (rxData_l <= sizeof(block10)) // save header only
00161                 {
00162                     H10[1].block.b[sizeof(block10) - rxData_l] = rxData;
00163                 }
00164                 if (rxData_l <= r_block.length) // generate checksum from data stream
00165                 {
00166                     V.r_checksum = run_checksum(rxData, false);
00167                 }
00168
00169                 if (rxData_l == r_block.length + 1) // checksum high byte
00170                 {
00171                     H10[1].checksum = (uint16_t) rxData << 8;
00172                 }
00173                 if (rxData_l == r_block.length + 2) // checksum low byte
00174                 {
00175                     H10[1].checksum += rxData;
00176                 }
00177
00178                 rxData_l++;
00179                 b_block[sizeof(header254) - rxData_l] = rxData;
00180                 if (rxData_l > (r_block.length + 2)) { // end of total data stream
00181                     if (V.r_checksum == H10[1].checksum) {
00182                         *m_link = LINK_STATE_ACK;
00183                     } else { // bad checksum
00184                         while (UART1_is_rx_ready()) { // dump receive buffer of bad data
00185                             rxData = UART1_Read();
00186                             V.rx_total++;
00187                         }
00188                         WaitMs(T1); // inter-character timeout
00189                         V.error = LINK_ERROR_CHECKSUM;
00190                         V.checksum_error++;
00191                         V.all_errors++;
00192                         V.failed_receive = RECV_ERROR_CKSUM;
00193                         *m_link = LINK_STATE_NAK;
00194                         MLED_SetHigh();
00195                     }
00196                 }
00197             }
00198         }
00199     }
00200     if (UART2_is_rx_ready()) {
00201         rxData = UART2_Read();
00202         V.rx_total++;
00203         if (rxData_l == 0) { // start header reads
00204             r_block.length = rxData; // header+message bytes
00205             run_checksum(0, true);
00206             rxData_l++;
00207             b_block[sizeof(header254) - rxData_l] = rxData; // buffer the message
00208         } else {
00209             /*
00210              * skip possible message data
00211              */
00212             if (rxData_l <= sizeof(block10)) // save header only
00213             {
00214                 H10[1].block.b[sizeof(block10) - rxData_l] = rxData;
00215             }
00216             if (rxData_l <= r_block.length) // generate checksum from data stream
00217             {
00218                 V.r_checksum = run_checksum(rxData, false);
00219             }
00220
00221             if (rxData_l == r_block.length + 1) // checksum high byte
00222             {
00223                 H10[1].checksum = (uint16_t) rxData << 8;
00224             }
00225             if (rxData_l == r_block.length + 2) // checksum low byte
00226             {
00227                 H10[1].checksum += rxData;
00228             }
00229
00230             rxData_l++;
00231             b_block[sizeof(header254) - rxData_l] = rxData;
00232             if (rxData_l > (r_block.length + 2)) { // end of total data stream
00233                 if (V.r_checksum == H10[1].checksum) {
00234                     *m_link = LINK_STATE_ACK;
00235                 }
00236             }
00237         }
00238     }

```

```

00227             } else { // bad checksum
00228                 while (UART2_is_rx_ready()) { // dump receive buffer of bad data
00229                     rxData = UART2_Read();
00230                     V.rx_total++;
00231                 }
00232                 WaitMs(T1); // inter-character timeout
00233                 V.error = LINK_ERROR_CHECKSUM;
00234                 V.checksum_error++;
00235                 V.all_errors++;
00236                 V.failed_receive = RECV_ERROR_NAK;
00237                 *m_link = LINK_STATE_NAK;
00238                 MLED_SetHigh();
00239             }
00240         }
00241     }
00242 }
00243 }
00244 break;
00245 case LINK_STATE_ACK:
00246 #ifdef DB1
00247     WaitMs(1);
00248 #endif
00249     V.stream = H10[1].block.block.stream;
00250     V.function = H10[1].block.block.function;
00251     V.systemb = H10[1].block.block.systemb;
00252     V.rbit = H10[1].block.block.rbit;
00253     V.wbit = H10[1].block.block.wbit;
00254     V.ebit = H10[1].block.block.ebit;
00255     V.failed_receive = RECV_ERROR_NONE;
00256     secs_II_monitor_message(V.stream, V.function, LDELAY); // log selected messages
00257     V.g_state = secs_gem_state(V.stream, V.function);
00258     *m_link = LINK_STATE_DONE;
00259     break;
00260 case LINK_STATE_NAK:
00261     *m_link = LINK_STATE_ERROR;
00262     V.all_errors++;
00263     while (UART1_DataReady) { // dump the receive buffer
00264         UART1_Read();
00265         V.rx_total++;
00266     }
00267     while (UART2_DataReady) { // dump the receive buffer
00268         UART2_Read();
00269         V.rx_total++;
00270     }
00271     break;
00272 case LINK_STATE_ERROR:
00273 #ifdef FAKER
00274     snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "LINK_STATE_ERROR M ");
00275 #endif
00276     MLED_SetHigh();
00277     break;
00278 case LINK_STATE_DONE: // normally we don't execute this code state
00279     V.failed_receive = RECV_ERROR_NONE;
00280 default:
00281     *m_link = LINK_STATE_IDLE;
00282     break;
00283 }
00284
00285 return *m_link;
00286 }
00287
00288 LINK_STATES r_protocol(LINK_STATES * r_link)
00289 {
00290     uint8_t rxData;
00291     static uint8_t rxData_l = 0, retry = RTY, *b_block, d = 1;
00292
00293     switch (*r_link) {
00294     case LINK_STATE_IDLE:
00295         if (UART1_is_rx_ready() || UART2_is_rx_ready()) {
00296             if (UART1_is_rx_ready()) {
00297                 rxData = UART1_Read();
00298                 V.rx_total++;
00299             }
00300             if (UART2_is_rx_ready()) {
00301                 rxData = UART2_Read();
00302                 V.rx_total++;
00303             }
00304             DLED_Toggle();
00305             if (rxData == ENQ) {
00306                 DEBUG1_SetHigh();
00307                 V.error = LINK_ERROR_NONE; // reset error status
00308                 *r_link = LINK_STATE_ENQ;
00309 #ifdef FAKER
00310                 V.g_state = GEM_STATE_ONLINE;
00311 #endif
00312                 if (TimerDone(TMR_HBIO)) {
00313                     StartTimer(TMR_HBIO, HBTS); // add short idle time

```

```

00314         }
00315     }
00316     if (rxData == EOT) {
00317         DEBUG1_SetHigh();
00318         V.error = LINK_ERROR_NONE; // reset error status
00319         *r_link = LINK_STATE_EOT;
00320 #ifndef FAKER
00321         V.g_state = GEM_STATE_COMM;
00322 #endif
00323         if (TimerDone(TMR_HBIO)) {
00324             StartTimer(TMR_HBIO, HBTS); // add short idle time
00325         }
00326     }
00327 }
00328 break;
00329 case LINK_STATE_ENQ:
00330     rxData_l = 0; // message byte counter
00331     d = 1; // data byte counter
00332     b_block = (uint8_t*) & H254[0];
00333     UART1_Write(EOT);
00334     V.tx_total++;
00335 #ifndef FAKER
00336     UART2_Write(EOT);
00337     V.tx_total++;
00338 #endif
00339     StartTimer(TMR_T2, T2);
00340     *r_link = LINK_STATE_EOT;
00341 #ifndef DB2
00342     WaitMs(1);
00343     //H27[0].block.block.systemb = V.ticks; // make distinct, testing S1F13
00344     //secs_send((uint8_t*) & H27[0], sizeof(header27), true, 1);
00345     H10[3].block.block.systemb = V.ticks; // make distinct, testing S1F1
00346     secs_send((uint8_t*) & H10[3], sizeof(header10), true, 1);
00347 #endif
00348 #ifndef FAKER
00349     H10[3].block.block.systemb = V.ticks; // make distinct, testing S1F1
00350     secs_send((uint8_t*) & H10[3], sizeof(header10), false, 2);
00351 #endif
00352     break;
00353 case LINK_STATE_EOT:
00354     if (TimerDone(TMR_T2)) {
00355         V.timer_error++;
00356         V.all_errors++;
00357         if (!retry--) { // check for stalls
00358             V.error = LINK_ERROR_T2;
00359             V.failed_receive = RECV_ERROR_T2;
00360             V.all_errors++;
00361             *r_link = LINK_STATE_NAK;
00362             MLED_SetHigh();
00363         } else {
00364             *r_link = LINK_STATE_IDLE; // retry
00365         }
00366     } else {
00367         if (UART1_is_rx_ready() || UART2_is_rx_ready()) {
00368             if (UART1_is_rx_ready()) {
00369                 rxData = UART1_Read();
00370                 V.rx_total++;
00371             }
00372             if (UART2_is_rx_ready()) {
00373                 rxData = UART2_Read();
00374                 V.rx_total++;
00375             }
00376             DLED_Toggle();
00377             if (rxData_l == 0) { // start header reads
00378                 r_block.length = rxData; // header+message bytes
00379                 run_checksum(0, true);
00380                 rxData_l++;
00381                 b_block[sizeof(header254) - rxData_l] = rxData; // buffer the message
00382             } else {
00383                 /*
00384                  * skip possible message data
00385                  */
00386                 if (rxData_l <= sizeof(block10)) // save header only
00387                     H10[1].block.b[sizeof(block10) - rxData_l] = rxData;
00388                 if (d <= 16) {
00389                     if (rxData_l == sizeof(block10) + d) { // save possible data format codes
00390                         V.response.ack[d - 1] = rxData;
00391                         d++;
00392                     }
00393                 }
00394             }
00395         }
00396         if (rxData_l <= r_block.length) // generate checksum from data stream
00397             V.r_checksum = run_checksum(rxData, false);
00398         if (rxData_l == r_block.length + 1) // checksum high byte
00399             H10[1].checksum = (uint16_t) rxData << 8;
00400

```



```

00401         if (rxData_l == r_block.length + 2) // checksum low byte
00402             H10[1].checksum += rxData;
00403
00404         rxData_l++;
00405         b_block[sizeof(header254) - rxData_l] = rxData;
00406         if (rxData_l > (r_block.length + 2)) { // end of total data stream
00407             if (V.r_checksum == H10[1].checksum) {
00408                 *r_link = LINK_STATE_ACK;
00409                 DEBUG1_SetHigh();
00410             } else { // bad checksum
00411                 while (UART1_is_rx_ready()) { // dump receive buffer of bad data
00412                     rxData = UART1_Read();
00413                     V.rx_total++;
00414                 }
00415                 while (UART2_is_rx_ready()) { // dump receive buffer of bad data
00416                     rxData = UART2_Read();
00417                     V.rx_total++;
00418                 }
00419                 WaitMs(T1); // inter-character timeout
00420                 V.error = LINK_ERROR_CHECKSUM;
00421                 V.checksum_error++;
00422                 V.all_errors++;
00423                 V.failed_receive = RECV_ERROR_NAK;
00424                 *r_link = LINK_STATE_NAK;
00425                 MLED_SetHigh();
00426             }
00427         }
00428     }
00429 }
00430 }
00431 break;
00432 case LINK_STATE_ACK:
00433     UART1_Write(ACK);
00434     V.tx_total++;
00435 #ifdef FAKER
00436     UART2_Write(ACK);
00437     V.tx_total++;
00438 #endif
00439     V.stream = H10[1].block.block.stream;
00440     V.function = H10[1].block.block.function;
00441     V.systemb = H10[1].block.block.systemb;
00442     V.rbit = H10[1].block.block.rbit;
00443     V.wbit = H10[1].block.block.wbit;
00444     V.ebit = H10[1].block.block.ebit;
00445     secs_II_monitor_message(V.stream, V.function, SDELAY); // log selected messages
00446     V.g_state = secs_gem_state(V.stream, V.function);
00447     V.failed_receive = RECV_ERROR_NONE;
00448     *r_link = LINK_STATE_DONE;
00449     V.abort = LINK_ERROR_NONE;
00450     V.br_total++;
00451     break; // normally we don't execute LINK_STATE_DONE commands
00452 case LINK_STATE_NAK:
00453     #ifdef FAKER
00454         snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "LINK_STATE_NACK R    ");
00455     #endif
00456     UART1_Write(NAK);
00457     V.tx_total++;
00458     #ifdef FAKER
00459     UART2_Write(NAK);
00460     V.tx_total++;
00461     #endif
00462     *r_link = LINK_STATE_ERROR;
00463     V.all_errors++;
00464     while (UART1_DataReady) { // dump the receive buffer
00465         UART1_Read();
00466         V.rx_total++;
00467     }
00468     while (UART2_DataReady) { // dump the receive buffer
00469         UART2_Read();
00470         V.rx_total++;
00471     }
00472     retry = RTY;
00473     V.brn_total++;
00474     break;
00475 case LINK_STATE_ERROR:
00476     #ifdef FAKER
00477         snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "LINK_STATE_ERROR R    ");
00478     #endif
00479     MLED_SetHigh();
00480     break;
00481 case LINK_STATE_DONE: // auto move to idle to receive data from link
00482     V.failed_receive = RECV_ERROR_NONE;
00483     V.abort = LINK_ERROR_NONE;
00484     default:
00485         *r_link = LINK_STATE_IDLE;
00486         break;
00487 }

```

```

00488
00489     return *r_link;
00490 }
00491
00492 LINK_STATES t_protocol(LINK_STATES * t_link)
00493 {
00494     uint8_t rxData, uart_num = 1;
00495     static uint8_t retry, requeue = false;
00496     static response_type block;
00497
00498     switch (*t_link) {
00499     case LINK_STATE_IDLE:
00500 #ifdef FAKER
00501         snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "LINK_STATE_IDLE T ");
00502 #endif
00503         V.error = LINK_ERROR_NONE; // reset error status
00504         retry = RTY;
00505         UART1_Write(ENQ);
00506         V.tx_total++;
00507 #ifdef FAKER
00508         UART2_Write(ACK);
00509         V.tx_total++;
00510         V.stream = 1;
00511         V.function = 1; // S1F1 host ping
00512         uart_num = 2;
00513 #endif
00514         StartTimer(TMR_T2, T2);
00515         *t_link = LINK_STATE_ENQ;
00516 #ifdef DB3
00517         WaitMs(1);
00518         UART1_put_buffer(EOT);
00519 #endif
00520         break;
00521     case LINK_STATE_ENQ:
00522         if (TimerDone(TMR_T2)) {
00523             V.timer_error++;
00524             V.all_errors++;
00525             if (!retry--) { // check for stalls
00526                 V.error = LINK_ERROR_T2;
00527                 V.all_errors++;
00528                 V.failed_send = SEND_ERROR_T2;
00529                 *t_link = LINK_STATE_NAK;
00530                 MLED_SetHigh();
00531             } else {
00532                 StartTimer(TMR_T2, T2); // try again
00533             }
00534         } else {
00535             if (UART1_is_rx_ready()) {
00536                 rxData = UART1_Read();
00537                 V.rx_total++;
00538                 if (rxData == EOT) {
00539                     StartTimer(TMR_T3, T3);
00540                     *t_link = LINK_STATE_EOT;
00541                 }
00542                 if (rxData == ENQ) { // contention with equipment master
00543                     *t_link = LINK_STATE_DONE;
00544                 }
00545             }
00546             if (UART2_is_rx_ready()) {
00547                 rxData = UART2_Read();
00548                 V.rx_total++;
00549                 if (rxData == EOT) {
00550                     StartTimer(TMR_T3, T3);
00551                     *t_link = LINK_STATE_EOT;
00552                 }
00553                 if (rxData == ENQ) { // contention with equipment master
00554                     *t_link = LINK_STATE_DONE;
00555                 }
00556             }
00557         }
00558     }
00559     break;
00560 case LINK_STATE_EOT: // transmit the message block
00561     if (!requeue) {
00562         block = secs_II_message(V.stream, V.function); // parse proper response
00563     }
00564
00565     if (V.abort == LINK_ERROR_ABORT) {
00566         secs_send((uint8_t*) block.header, block.length, false, uart_num);
00567         V.failed_send = SEND_ERROR_ABORT;
00568         *t_link = LINK_STATE_ERROR;
00569         V.all_errors++;
00570         MLED_SetHigh();
00571     } else {
00572         if (!requeue) {
00573             secs_send((uint8_t*) block.header, block.length, false, uart_num);

```

```

00575         if (V.queue) {
00576             requeue = true; // send a reply message
00577         }
00578     } else {
00579         requeue = false;
00580         V.queue = false;
00581         secs_send((uint8_t*) block.reply, block.reply_length, false, uart_num);
00582     }
00583     if (V.error == LINK_ERROR_NONE) {
00584         *t_link = LINK_STATE_ACK;
00585 #ifdef FAKER
00586         UART2_Write(ACK);
00587         V.tx_total++;
00588 #endif
00589     } else {
00590         V.failed_send = SEND_ERROR_EOT;
00591         *t_link = LINK_STATE_ERROR;
00592         V.all_errors++;
00593         MLED_SetHigh();
00594     }
00595 }
00596 #ifdef DB4
00597 WaitMs(1);
00598 if (V.rerror && (rand() < ERROR_COMM)) {
00599     UART1_put_buffer(ACK);
00600 }
00601 #endif
00602 break;
00603 case LINK_STATE_ACK:
00604 #ifdef FAKER
00605     sprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "LINK_STATE_ACK T ");
00606 #endif
00607 if (TimerDone(TMR_T3)) {
00608     V.timer_error++;
00609     V.error = LINK_ERROR_T3;
00610     V.all_errors++;
00611     V.failed_send = SEND_ERROR_T3;
00612     *t_link = LINK_STATE_NAK;
00613 } else {
00614     if (UART1_is_rx_ready()) {
00615         rxData = UART1_Read();
00616         V.rx_total++;
00617         if (rxData == ACK) {
00618             V.failed_send = SEND_ERROR_NONE;
00619             *t_link = LINK_STATE_DONE;
00620             V.abort = LINK_ERROR_NONE;
00621             V.bt_total++;
00622         }
00623     }
00624     if (UART2_is_rx_ready()) {
00625         rxData = UART2_Read();
00626         V.rx_total++;
00627         if (rxData == ACK) {
00628             V.failed_send = SEND_ERROR_NONE;
00629             *t_link = LINK_STATE_DONE;
00630             V.abort = LINK_ERROR_NONE;
00631             V.bt_total++;
00632         }
00633     }
00634 }
00635 break;
00636 case LINK_STATE_NAK: // send failure
00637 #ifdef FAKER
00638     sprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "LINK_STATE_NAK T ");
00639 #endif
00640 *t_link = LINK_STATE_ERROR;
00641 V.all_errors++;
00642 while (UART1_DataReady) { // dump the receive buffer
00643     UART1_Read();
00644     V.rx_total++;
00645 }
00646 while (UART2_DataReady) { // dump the receive buffer
00647     UART2_Read();
00648     V.rx_total++;
00649 }
00650 V.btn_total++;
00651 break;
00652 case LINK_STATE_ERROR:
00653 #ifdef FAKER
00654     sprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "LINK_STATE_ERROR T ");
00655 #endif
00656 MLED_SetHigh();
00657 break;
00658 case LINK_STATE_DONE: // normally we don't execute this code
00659     V.failed_send = SEND_ERROR_NONE;
00660     V.abort = LINK_ERROR_NONE;
00661     break;

```

```

00662     default:
00663         *t_link = LINK_STATE_IDLE;
00664         break;
00665     }
00666
00667     return *t_link;
00668 }
00669
00670 /* send the whole sequence including length and checksum bytes with the variable length */
00671 static bool secs_send(uint8_t *byte_block, const uint8_t length, const bool fake, const uint8_t
s_uart)
00672 {
00673     uint8_t i, *k;
00674     uint16_t checksum;
00675
00676     DLED_Toggle();
00677     k = (uint8_t *) byte_block;
00678
00679     V.error = LINK_ERROR_NONE;
00680     if ((length - 3) != k[length - 1]) { // check header length field byte
00681         V.error = LINK_ERROR_SEND;
00682         V.all_errors++;
00683         V.failed_send = SEND_ERROR_DATA;
00684         MLED_SetHigh();
00685         return false; // don't send and return mismatch error
00686     }
00687     ++V.ticks; // transaction ID for host master messages
00688
00689     /*
00690     * space up from bottom two bytes and don't include last byte
00691     * for checksums
00692     * we send C structures from the max byte to first in uC memory
00693     */
00694     checksum = block_checksum(&k[2], length - 3);
00695     k[0] = checksum & 0xff;
00696     k[1] = (checksum >> 8) & 0xff;
00697     V.t_checksum = checksum;
00698
00699     switch (s_uart) {
00700     case 2:
00701         while (!UART2_is_tx_ready()); // wait for tx buffer to drain
00702         for (i = length; i > 0; i--) {
00703             if (fake) {
00704                 UART2_put_buffer(k[i - 1]);
00705             } else {
00706                 UART2_Write(k[i - 1]); // -1 for array memory addressing
00707                 V.tx_total++;
00708             }
00709         }
00710         break;
00711     case 1:
00712     default:
00713         while (!UART1_is_tx_ready()); // wait for tx buffer to drain
00714         for (i = length; i > 0; i--) {
00715             if (fake) {
00716                 UART1_put_buffer(k[i - 1]);
00717             } else {
00718                 UART1_Write(k[i - 1]); // -1 for array memory addressing
00719                 V.tx_total++;
00720             }
00721             #ifndef FAKER
00722                 UART2_Write(k[i - 1]); // -1 for array memory addressing
00723                 V.tx_total++;
00724             #endif
00725         }
00726         break;
00727     }
00728
00729     return true;
00730 }
00731
00732 /*
00733 * send the host message
00734 */
00735 void hb_message()
00736 {
00737     V.ping++;
00738     V.s_state = SEQ_STATE_TX;
00739     V.failed_send = SEND_ERROR_NONE;
00740     V.t_l_state = LINK_STATE_IDLE;
00741     if (V.msg_error == MSG_ERROR_NONE) { // set ping message, not for streams
00742         V.stream = 1;
00743         V.function = 2; // S1F1 host ping
00744     } else {
00745         V.stream = 1;
00746         V.function = 14; // S1F13 host ping
00747     }

```

```

00748 }
00749
00750 /*
00751  * send messages as a machine operation sequence
00752  */
00753 bool sequence_messages(const uint8_t sid)
00754 {
00755     V.msg_error = MSG_ERROR_NONE;
00756     switch (sid) {
00757     case 1:
00758         S[0].stack = 6; // number of commands
00759         S[0].message = HC33[1]; // open doors
00760         S[1].message = HC33[1];
00761         S[2].message = HC33[1];
00762         S[3].message = HC33[0]; // close doors
00763         S[4].message = HC33[0];
00764         S[5].message = HC33[0];
00765
00766         S[0].message.data[0] = 0x01;
00767         S[1].message.data[0] = 0x02;
00768         S[2].message.data[0] = 0x03;
00769         S[3].message.data[0] = 0x01;
00770         S[4].message.data[0] = 0x02;
00771         S[5].message.data[0] = 0x03;
00772
00773         S[0].delay = 10000; // set delay between commands
00774         S[1].delay = 10000;
00775         S[2].delay = 10000;
00776         S[3].delay = 10000;
00777         S[4].delay = 10000;
00778         S[5].delay = 10000;
00779
00780         S[0].block.header = (uint8_t*) & S[0].message; // S6F41
00781         S[0].block.length = sizeof(header33);
00782         S[1].block.header = (uint8_t*) & S[1].message;
00783         S[1].block.length = sizeof(header33);
00784         S[2].block.header = (uint8_t*) & S[2].message;
00785         S[2].block.length = sizeof(header33);
00786         S[3].block.header = (uint8_t*) & S[3].message;
00787         S[3].block.length = sizeof(header33);
00788         S[4].block.header = (uint8_t*) & S[4].message;
00789         S[4].block.length = sizeof(header33);
00790         S[5].block.header = (uint8_t*) & S[5].message;
00791         S[5].block.length = sizeof(header33);
00792         V.stack = S[0].stack; // queue up 10 messages, pop off the top of stack
00793         StartTimer(TMR_HBIO, S[V.stack - 1].delay); // restart sequence timer
00794         break;
00795     case 10: // write to equipment display sequence
00796     case 11:
00797         D[0].stack = 1; // number of commands
00798         D[0].message = H153[0]; // send a host display command
00799         D[0].delay = 5000; // set delay between commands
00800         D[0].block.header = (uint8_t*) & D[0].message; // S10F3
00801         D[0].block.length = sizeof(header153);
00802         V.stack = D[0].stack; // queue up 2 displays, pop off the top of stack
00803         StartTimer(TMR_HBIO, D[V.stack - 1].delay); // restart sequence timer
00804         break;
00805     default:
00806         V.stack = 0;
00807         return false;
00808         break;
00809     }
00810
00811     return true;
00812 }
00813
00814 /*
00815  * standard text to the V.terminal string buffer
00816  */
00817 void terminal_format(DISPLAY_TYPES t_format)
00818 {
00819     switch (t_format) {
00820     case display_message:
00821         snprintf(V.terminal, MAX_TERM, msg0,
00822             V.all_errors, V.r_l_state, V.failed_receive, V.t_l_state, V.failed_send, V.checksum_error,
00823             V.sequences, VER);
00824         break;
00825     case display_online:
00826         snprintf(V.terminal, MAX_TERM, msg1,
00827             V.all_errors, V.r_l_state, V.failed_receive, V.t_l_state, V.failed_send, V.checksum_error,
00828             V.sequences, VER);
00829         break;
00830     case display_remote:
00831         snprintf(V.terminal, MAX_TERM, msg2, msg_gemremote, V.sequences, VER);
00832         break;
00833     case display_gemhelp:
00834         snprintf(V.terminal, MAX_TERM, msg_gemhelp, msg_gemcmds, VER);
00835     }
00836 }

```

```

00833         break;
00834     case display_free:
00835         snprintf(V.terminal, MAX_TERM, msg_free, msg_freecmds, VER);
00836         break;
00837     default:
00838         snprintf(V.terminal, MAX_TERM, msg99,
00839             V.all_errors, V.r_l_state, V.failed_receive, V.t_l_state, V.failed_send, V.checksum_error,
00840             VER);
00841         break;
00842     }
00843 }
00844
00845 /*
00846  * format S10F3 Terminal Display, Single in H153[0]
00847  * move string data into a terminal H153 type GEM message
00848  */
00849 uint16_t format_display_text(const char *data)
00850 {
00851     int16_t j, i = 0, z = 0;
00852     uint16_t k;
00853
00854     k = (uint16_t) strlen(data);
00855
00856     if (!k) // check for null string
00857     {
00858         return k;
00859     }
00860
00861     /*
00862      * shift string into Terminal text array
00863      */
00864     for (j = S10F3_STR_POS; j >= z; j--) {
00865         if (i < k) {
00866             H153[0].data[j] = data[i++];
00867         } else {
00868             H153[0].data[j] = ' ';
00869         }
00870     }
00871     return k;
00872 }
00873
00874 /*
00875  * parse load-clock terminal command for port number
00876  */
00877 static void parse_ll(void)
00878 {
00879     if (V.response.cmdlen > 1) {
00880         switch (V.response.mparam) { // port selection
00881             case '1':
00882             case '2':
00883             case '3':
00884             case 'A':
00885             case 'B':
00886             case 'C':
00887             case 'a':
00888             case 'b':
00889             case 'c':
00890                 H33[0].data[0] = V.response.mparam & 0x03;
00891                 break;
00892             default:
00893                 H33[0].data[0] = 0x01;
00894                 break;
00895         }
00896     } else {
00897         H33[0].data[0] = 0x01;
00898     }
00899     V.llid = H33[0].data[0];
00900 }
00901
00902 /*
00903  * parse sequence command for sequence number from operator screen text
00904  */
00905 static void parse_sid(void)
00906 {
00907     if (V.response.cmdlen > 1) {
00908         switch (V.response.mparam) { // port selection
00909             case '1':
00910             case '2':
00911             case '3':
00912             case 'A':
00913             case 'B':
00914             case 'C':
00915             case 'a':
00916             case 'b':
00917             case 'c':
00918                 V.sid = V.response.mparam & 0x03;

```

```

00919         break;
00920     default:
00921         V.sid = 0x01;
00922         break;
00923     }
00924 } else {
00925     V.sid = 0x01;
00926 }
00927 }
00928
00929 /*
00930  * terminal response types for equipment
00931  */
00932 P_CODES s10f1_opcmd(void)
00933 {
00934     snprintf(V.info, MAX_INFO, " Terminal          ");
00935     set_vterm(V.vterm);
00936     V.response.cmdlen = V.response.ack[6]; // length of command string
00937     V.response.TID = V.response.ack[4]; // TID of equipment message
00938     V.response.mcode = V.response.ack[7]; // first char of equipment message
00939     V.response.mparm = V.response.ack[8]; // second char
00940
00941     if (V.response.cmdlen == 0) {
00942         snprintf(V.info, MAX_INFO, " S10F1 CMD Error          ");
00943         return CODE_ERR;
00944     }
00945
00946     if (V.response.mcode == 'M' || V.response.mcode == 'm') {
00947         snprintf(V.info, MAX_INFO, " Message Terminal          ");
00948         return CODE_TS;
00949     }
00950
00951     if (V.response.mcode == 'C' || V.response.mcode == 'c') { // ready cassette load-lock control
00952         parse_ll();
00953         snprintf(V.info, MAX_INFO, " Cassette Ready          ");
00954         switch (V.e_types) {
00955             case GEM_VII80:
00956                 H33[0].data[18] = '1';
00957                 H33[0].data[17] = '6';
00958                 break;
00959             case GEM_E220:
00960                 H33[0].data[18] = '1';
00961                 H33[0].data[17] = '0';
00962                 break;
00963             default:
00964                 break;
00965         }
00966
00967         return CODE_LOAD;
00968     }
00969
00970     if (V.response.mcode == 'R' || V.response.mcode == 'r') { // close door load-lock control
00971         parse_ll();
00972         snprintf(V.info, MAX_INFO, " Ready Door          ");
00973         switch (V.e_types) {
00974             case GEM_VII80:
00975                 H33[0].data[18] = '1';
00976                 H33[0].data[17] = '7';
00977                 break;
00978             case GEM_E220:
00979                 H33[0].data[18] = '1';
00980                 H33[0].data[17] = '0';
00981                 break;
00982             default:
00983                 break;
00984         }
00985
00986         return CODE_LOAD;
00987     }
00988
00989     if (V.response.mcode == 'P' || V.response.mcode == 'p') { // close door, rough and hivac load-lock
control
00990         parse_ll();
00991         snprintf(V.info, MAX_INFO, " Pump          ");
00992         switch (V.e_types) {
00993             case GEM_VII80:
00994                 H33[0].data[18] = '1';
00995                 H33[0].data[17] = '8';
00996                 break;
00997             case GEM_E220:
00998                 H33[0].data[18] = '1';
00999                 H33[0].data[17] = '0';
01000                 break;
01001             default:
01002                 break;
01003         }
01004

```

```

01005         return CODE_PUMP;
01006     }
01007
01008     if (V.response.mcode == 'O' || V.response.mcode == 'o') { // vent & open load-lock control
01009         parse_ll();
01010         snprintf(V.info, MAX_INFO, " Open                               ");
01011         switch (V.e_types) {
01012             case GEM_VII80:
01013                 H33[0].data[18] = '1';
01014                 H33[0].data[17] = '2';
01015                 break;
01016             case GEM_E220:
01017                 H33[0].data[18] = '1';
01018                 H33[0].data[17] = '0';
01019                 break;
01020             default:
01021                 break;
01022         }
01023
01024         return CODE_UNLOAD;
01025     }
01026
01027     if (V.response.mcode == 'F' || V.response.mcode == 'f') {
01028         snprintf(V.info, MAX_INFO, " Speed Lock reset                ");
01029         return CODE_FREE;
01030     }
01031
01032     if (V.response.mcode == 'L' || V.response.mcode == 'l') {
01033         snprintf(V.info, MAX_INFO, " Log file reset                    ");
01034         return CODE_LOG;
01035     }
01036
01037     if (V.response.mcode == 'S' || V.response.mcode == 's') {
01038         snprintf(V.info, MAX_INFO, " Sequence                        ");
01039         switch (V.e_types) {
01040             case GEM_VII80:
01041                 break;
01042             case GEM_E220:
01043                 break;
01044             default:
01045                 break;
01046         }
01047         return CODE_SEQUENCE;
01048     }
01049
01050     if (V.response.mcode == 'D' || V.response.mcode == 'd') {
01051         snprintf(V.info, MAX_INFO, " Debug Toggle                      ");
01052         return CODE_DEBUG;
01053     }
01054
01055     if (V.response.mcode == 'E' || V.response.mcode == 'e') {
01056         snprintf(V.info, MAX_INFO, " Error Toggle                        ");
01057         return CODE_ERROR;
01058     }
01059
01060     if (V.response.mcode == 'H' || V.response.mcode == 'h' || V.response.mcode == '?') {
01061         snprintf(V.info, MAX_INFO, " Help Toggle                          ");
01062         return CODE_HELP;
01063     }
01064
01065     return CODE_TS;
01066 }
01067
01068 /*
01069  * decode offline, online ceid codes
01070  */
01071 uint16_t s6f11_opcmd(void)
01072 {
01073     V.response.ceid = V.response.ack[9]; // CEID
01074     V.response.ceid = H254[0].data[(sizeof(H254[0].data) - 1) - 9]; // get CEID using full message
01075     block buffer
01076     V.testing = (sizeof(H254[0].data) - 1) - 9;
01077     return V.response.ceid;
01078 }
01079
01080 /*
01081  * load block structure with the correct sequence message to send
01082  */
01083 bool gem_messages(response_type *block, const uint8_t sid)
01084 {
01085     if (V.stack == 0) {
01086         return false;
01087     }
01088
01089     switch (sid) {
01090         case 1: // close doors sequence

```



```

01091     StartTimer(TMR_HBIO, S[V.stack - 1].delay);
01092     *block = S[V.stack - 1].block; // shallow contents copy
01093     S[V.stack - 1].message.block.block.systemb = V.ticks;
01094     V.l1id = S[V.stack - 1].message.data[0];
01095     break;
01096 case 10: // equipment display sequence
01097     StartTimer(TMR_HBIO, D[V.stack - 1].delay);
01098     *block = D[V.stack - 1].block; // shallow contents copy
01099     D[V.stack - 1].message.block.block.systemb = V.ticks;
01100     break;
01101 default:
01102     StartTimer(TMR_HBIO, S[V.stack - 1].delay);
01103     *block = S[V.stack - 1].block; // shallow contents copy
01104     S[V.stack - 1].message.block.block.systemb = V.ticks;
01105     V.l1id = S[V.stack - 1].message.data[0];
01106     break;
01107 }
01108
01109 V.stack--;
01110 return true;
01111 }
01112
01113 /*
01114  * parse stream and response codes into a message pointer and length to send message in response
01115  */
01116 response_type secs_II_message(const uint8_t stream, const uint8_t function)
01117 {
01118     static response_type block;
01119     uint16_t i = 0;
01120
01121     V.abort = LINK_ERROR_NONE;
01122     V.queue = false;
01123     block.respond = false;
01124
01125     if (TimerDone(TMR_HBIO)) { // hold sequences during equipment messages
01126         if (V.stack > 0) {
01127             gem_messages(&block, V.sid);
01128             if (V.sid >= 10) {
01129                 set_display_info(DIS_SEQUENCE_M);
01130             } else {
01131                 set_display_info(DIS_SEQUENCE);
01132             }
01133             vterm_sequence();
01134             StartTimer(TMR_INFO, TDELAY);
01135             V.set_sequ = true;
01136             return(block);
01137         }
01138         StartTimer(TMR_HBIO, HBTS); // add short idle time
01139     }
01140
01141     switch (stream) { // make proper response blocks
01142     case 1:
01143         switch (function) { // from equipment
01144         case 1: // S1F2 host response and S1F1 send
01145             block.header = (uint8_t*) & H12[0];
01146             block.length = sizeof(header12);
01147             H12[0].block.block.systemb = V.systemb;
01148             H10[0].block.block.systemb = V.ticks;
01149             block.respond = true;
01150             block.reply = (uint8_t*) & H10[0]; // S1F1 send queue
01151             block.reply_length = sizeof(header10);
01152             V.queue = true;
01153             break;
01154         case 2: // S1F1 host heartbeat send to equipment
01155             block.header = (uint8_t*) & H10[0];
01156             block.length = sizeof(header10);
01157             H10[0].block.block.systemb = V.systemb;
01158             break;
01159         case 3: // S1F4
01160             block.header = (uint8_t*) & H14[0];
01161             block.length = sizeof(header14);
01162             H14[0].block.block.systemb = V.systemb;
01163             break;
01164         case 13: // S1F14 response and S1F13 send
01165             block.header = (uint8_t*) & H17[0];
01166             block.length = sizeof(header17);
01167             H17[0].block.block.systemb = V.systemb;
01168             H12[1].block.block.systemb = V.ticks;
01169             block.respond = true;
01170             block.reply = (uint8_t*) & H12[1]; // S1F13 send queue
01171             block.reply_length = sizeof(header12);
01172             V.queue = true;
01173             break;
01174         // case 14: // model and rev information
01175         //     block.header = (uint8_t*) & H12[1];
01176         //     block.length = sizeof(header12);
01177         //     H12[1].block.block.systemb = V.systemb;

```

```

01178         //          break;
01179     default: // S1F0 abort
01180         H10[2].block.block.stream = stream;
01181         block.header = (uint8_t*) & H10[2];
01182         block.length = sizeof(header10);
01183         H10[2].block.block.systemb = V.systemb;
01184         V.abort = LINK_ERROR_ABORT;
01185         V.all_errors++;
01186         break;
01187     }
01188     break;
01189     case 2:
01190         switch (function) {
01191             case 17: // S2F18
01192 #ifdef Y2KTIME
01193                 block.header = (uint8_t*) & H26[0];
01194                 block.length = sizeof(header26);
01195                 H26[0].block.block.systemb = V.systemb;
01196 #else
01197                 block.header = (uint8_t*) & H24[0];
01198                 block.length = sizeof(header24);
01199                 H24[0].block.block.systemb = V.systemb;
01200 #endif
01201                 break;
01202             case 25: // S2F26
01203                 block.header = (uint8_t*) & H13[3];
01204                 block.length = sizeof(header13);
01205                 H13[3].block.block.systemb = V.systemb;
01206                 break;
01207             default: // S2F0 abort
01208                 H10[2].block.block.stream = stream;
01209                 block.header = (uint8_t*) & H10[2];
01210                 block.length = sizeof(header10);
01211                 H10[2].block.block.systemb = V.systemb;
01212                 V.abort = LINK_ERROR_ABORT;
01213                 V.all_errors++;
01214                 break;
01215         }
01216         break;
01217     case 5:
01218         switch (function) {
01219             case 1: // S5F2
01220                 block.header = (uint8_t*) & H13[2];
01221                 block.length = sizeof(header13);
01222                 H13[2].block.block.systemb = V.systemb;
01223                 break;
01224             default: // S5F0 abort
01225                 H10[2].block.block.stream = stream;
01226                 block.header = (uint8_t*) & H10[2];
01227                 block.length = sizeof(header10);
01228                 H10[2].block.block.systemb = V.systemb;
01229                 V.abort = LINK_ERROR_ABORT;
01230                 V.all_errors++;
01231                 break;
01232         }
01233         break;
01234     case 6:
01235         switch (function) {
01236             uint32_t ceid = 0;
01237             case 11: // S6F12
01238                 ceid = s6f11_opcmd(); // look for event codes for equipment types
01239                 switch (V.e_types) {
01240                     case GEM_VII80:
01241                         if (ceid == V.OSCREEN || ceid == V.SSCREEN) {
01242                             V.response.host_display_ack = true;
01243                             V.sid = 10;
01244                         }
01245                         break;
01246                     case GEM_E220:
01247                         if (ceid == E.OSCREEN) {
01248                             V.response.host_display_ack = true;
01249                             V.sid = 10;
01250                         }
01251                         break;
01252                     default:
01253                         break;
01254                 }
01255                 if (ceid == V.OSCREEN || ceid == V.SSCREEN) {
01256                     V.response.host_display_ack = true;
01257                 }
01258                 block.header = (uint8_t*) & H13[0];
01259                 block.length = sizeof(header13);
01260                 H13[0].block.block.systemb = V.systemb;
01261                 H13[0].block.block.function = 12;
01262                 break;
01263             case 13: // S6F14
01264                 block.header = (uint8_t*) & H13[0];

```

```

01265         block.length = sizeof(header13);
01266         H13[0].block.block.systemb = V.systemb;
01267         H13[0].block.block.function = 14;
01268         break;
01269     case 25: // S6F26
01270         block.header = (uint8_t*) & H13[0];
01271         block.length = sizeof(header13);
01272         H13[0].block.block.systemb = V.systemb;
01273         H13[0].block.block.function = 26;
01274         break;
01275     default: // S6F0 abort
01276         H10[2].block.block.stream = stream;
01277         block.header = (uint8_t*) & H10[2];
01278         block.length = sizeof(header10);
01279         H10[2].block.block.systemb = V.systemb;
01280         V.abort = LINK_ERROR_ABORT;
01281         V.all_errors++;
01282         break;
01283     }
01284     break;
01285 case 9:
01286     switch (function) {
01287     case 1:
01288         break;
01289     case 3:
01290         break;
01291     case 5:
01292         break;
01293     case 7:
01294         break;
01295     case 9:
01296         V.equip_timeout++;
01297         break;
01298     case 11:
01299         break;
01300     case 13:
01301         break;
01302     default: // S9F0 abort
01303         H10[2].block.block.stream = stream;
01304         block.header = (uint8_t*) & H10[2];
01305         block.length = sizeof(header10);
01306         H10[2].block.block.systemb = V.systemb;
01307         V.abort = LINK_ERROR_ABORT;
01308         V.all_errors++;
01309         break;
01310     }
01311     break;
01312 case 10:
01313     V.set_sequ = true;
01314     switch (function) {
01315     case 1: // S10F2 Terminal Request Acknowledge
01316         block.header = (uint8_t*) & H13[1];
01317         block.length = sizeof(header13);
01318         H13[1].block.block.systemb = V.systemb;
01319         H153[0].block.block.systemb = V.ticks;
01320         set_display_info(DIS_TERM);
01321
01322         switch (sl0f1_opcmd()) {
01323         case CODE_TM:
01324             block.respond = true;
01325             block.reply = (uint8_t*) & H153[1]; // S10F5 send Terminal Display, Multi-line, queue
01326             block.reply_length = sizeof(header153);
01327             H153[1].data[S10F3_TID_POS] = V.response.TID;
01328             V.queue = true;
01329             break;
01330         case CODE_LOAD:
01331             block.respond = true;
01332             block.reply = (uint8_t*) & H33[0]; // S6F41 send load lock ready command
01333             block.reply_length = sizeof(header33);
01334             V.queue = true;
01335             set_display_info(DIS_LOAD);
01336             break;
01337         case CODE_UNLOAD:
01338             block.respond = true;
01339             block.reply = (uint8_t*) & H33[0]; // S6F41 send load lock unload command
01340             block.reply_length = sizeof(header33);
01341             V.queue = true;
01342             set_display_info(DIS_UNLOAD);
01343             break;
01344         case CODE_PUMP:
01345             block.respond = true;
01346             block.reply = (uint8_t*) & H33[0]; // S6F41 send load lock pump command
01347             block.reply_length = sizeof(header33);
01348             V.queue = true;
01349             set_display_info(DIS_PUMP);
01350             break;
01351         case CODE_SEQUENCE:

```

```

01352         parse_sid();
01353         sequence_messages(V.sid);
01354         set_display_info(DIS_SEQUENCE);
01355         V.sequences++;
01356         break;
01357     case CODE_TS:
01358         block.respond = true;
01359         block.reply = (uint8_t*) & H153[0]; // S10F3 send Terminal Display, Single, queue
01360         block.reply_length = sizeof(header153);
01361         H153[0].data[S10F3_TID_POS] = V.response.TID;
01362         terminal_format(display_message);
01363         format_display_text(V.terminal);
01364         V.queue = true;
01365         break;
01366     case CODE_FREE:
01367         DATAEE_WriteByte(UART_SPEED_LOCK_EADR, true);
01368         set_display_info(DIS_FREE);
01369         terminal_format(display_free);
01370         format_display_text(V.terminal);
01371         V.queue = true;
01372         break;
01373     case CODE_LOG:
01374         do {
01375             // DATAEE_WriteByte(i, 0xff);
01376             } while (++i <= 764); // overwrite EEPROM data but leave program data intact in the
top 256 bytes
01377         V.response.log_num = 0;
01378         V.response.log_seq = 0;
01379         set_display_info(DIS_LOG);
01380         break;
01381     case CODE_ERROR:
01382         V.error = !V.error;
01383         if (V.debug) {
01384             V.ticker = TICKER_HIGH;
01385             V.vterm_switch = 0;
01386             refresh_lcd();
01387         } else {
01388             V.vterm = MAIN_VTERM;
01389             V.ticker = TICKER_ZERO;
01390             V.vterm_switch = 0;
01391             refresh_lcd();
01392         }
01393     case CODE_HELP:
01394         V.help = !V.help;
01395         if (V.debug) {
01396             V.ticker = TICKER_HIGH;
01397             V.vterm_switch = 0;
01398             refresh_lcd();
01399         } else {
01400             V.vterm = MAIN_VTERM;
01401             V.ticker = TICKER_ZERO;
01402             V.vterm_switch = 0;
01403             refresh_lcd();
01404         }
01405         block.respond = true;
01406         block.reply = (uint8_t*) & H153[0]; // S10F3 send Terminal Display, Single, queue
01407         block.reply_length = sizeof(header153);
01408         H153[0].data[S10F3_TID_POS] = V.response.TID;
01409         terminal_format(display_gemhelp);
01410         format_display_text(V.terminal);
01411         V.queue = true;
01412     case CODE_DEBUG:
01413         V.debug = !V.debug;
01414         if (V.debug) {
01415             V.ticker = TICKER_HIGH;
01416             V.vterm_switch = 0;
01417             refresh_lcd();
01418         } else {
01419             V.vterm = MAIN_VTERM;
01420             V.ticker = TICKER_ZERO;
01421             V.vterm_switch = 0;
01422             refresh_lcd();
01423         }
01424     default:
01425         break;
01426     }
01427     set_temp_display_help(display_info());
01428     vterm_sequence();
01429     StartTimer(TMR_INFO, TDELAY);
01430     break;
01431 default: // S10F0 abort
01432     H10[2].block.block.stream = stream;
01433     block.header = (uint8_t*) & H10[2];
01434     block.length = sizeof(header10);
01435     H10[2].block.block.systemb = V.systemb;
01436     V.abort = LINK_ERROR_ABORT;
01437     V.all_errors++;

```

```

01438         break;
01439     }
01440     break;
01441     default: // SxF0 abort
01442         H10[2].block.block.stream = stream;
01443         block.header = (uint8_t*) & H10[2];
01444         block.length = sizeof(header10);
01445         H10[2].block.block.systemb = V.systemb;
01446         V.abort = LINK_ERROR_ABORT;
01447         V.all_errors++;
01448         break;
01449     }
01450
01451     return(block);
01452 }
01453
01454 /*
01455  * logger helper
01456  */
01457 static void ee_logger(const uint8_t stream, const uint8_t function, const uint16_t dtype, uint8_t
    *msg_data)
01458 {
01459     uint16_t i = 0;
01460
01461     do {
01462         DATAEE_WriteByte(i + (uint16_t) ((V.response.log_seq & 0x03) << (uint8_t) 8), msg_data[254 + 2
    - i]);
01463     } while (++i <= 255);
01464
01465     snprintf(V.info, MAX_INFO, "Saved S%dF%d %d      ", stream, function, V.response.log_num);
01466     StartTimer(TMR_INFO, dtype);
01467     V.response.info = DIS_LOG;
01468     V.response.log_num++;
01469     V.response.log_seq++;
01470     if (V.response.log_seq >= 3) // limit saving to first three positions, last position is for
    program settings
01471     {
01472         V.response.log_seq = 0;
01473     }
01474 }
01475
01476 /*
01477  * parse stream and response codes for log function to EEPROM
01478  */
01479 void secs_II_monitor_message(const uint8_t stream, const uint8_t function, const uint16_t dtype)
01480 {
01481     uint8_t * msg_data = (uint8_t*) & H254[0];
01482     static uint8_t store1_1 = true, store1_13 = true, store6_11 = true;
01483
01484     ++V.ticks; // message sequence
01485     switch (stream) { // from equipment
01486     case 1:
01487         switch (function) { // from equipment
01488         case 1:
01489             if (!store1_1) {
01490                 break;
01491             }
01492             store1_1 = false;
01493             // ee_logger(stream, function, dtype, msg_data);
01494             break;
01495         case 13:
01496             if (!store1_13) {
01497                 break;
01498             }
01499             store1_13 = false;
01500             // ee_logger(stream, function, dtype, msg_data);
01501             break;
01502         default:
01503             break;
01504         }
01505     }
01506     break;
01507     case 2:
01508         switch (function) {
01509         case 41: // S2F41 remote command from host
01510         case 42: // S2F42 response from equipment
01511             /* always store this message */
01512             // ee_logger(stream, function, dtype, msg_data);
01513             if (function == 42) { // check for failed command
01514                 V.msg_ret = 0;
01515                 V.msg_error = MSG_ERROR_NONE;
01516                 V.response.info = DIS_STR;
01517             }
01518             break;
01519         default:
01520             break;
01521         }
01522     }

```

```

01522     case 6:
01523         switch (function) {
01524             case 11: // S6F11 // from host
01525                 if (!store6_11) {
01526                     break;
01527                 }
01528                 store6_11 = false;
01529                 // ee_logger(stream, function, dttime, msg_data);
01530                 break;
01531             default:
01532                 break;
01533         }
01534     }
01535     break;
01536 default:
01537     break;
01538 }
01539 }
01540
01541 /*
01542  * parse received stream and response codes for host operational state and
01543  * equipment model types
01544  * Be permissive, almost any stream command makes us online
01545  */
01546 GEM_STATES secs_gem_state(const uint8_t stream, const uint8_t function)
01547 {
01548     static GEM_STATES block = GEM_STATE_DISABLE;
01549     static GEM_EQUIP equipment = GEM_GENERIC;
01550     static response_type res_block;
01551
01552     V.abort = LINK_ERROR_NONE;
01553     V.queue = false;
01554     res_block.respond = false;
01555
01556     switch (stream) { // from equipment
01557     case 1:
01558         switch (function) {
01559             #if defined(DB2) || defined(FAKER) // basic comm exchange sets GEM state to remote
01560             case 1:
01561             #endif
01562             case 2:
01563                 if (block != GEM_STATE_REMOTE) {
01564                     if (TimerDone(TMR_HBIO)) {
01565                         StartTimer(TMR_HBIO, HBTL); // restart the heartbeat
01566                     }
01567                     terminal_format(display_online);
01568                     format_display_text(V.terminal);
01569                     V.response.mesgid = 1;
01570                     V.sequences++;
01571                     V.sid = 10;
01572                     sequence_messages(V.sid); // send a hello text message
01573                     set_display_info(DIS_SEQUENCE_M);
01574                 }
01575
01576                 if (block != GEM_STATE_REMOTE) {
01577                     block = GEM_STATE_ONLINE;
01578                     V.ticker = TICKER_ZERO;
01579                 }
01580
01581             break;
01582             case 13: // parse equipment model from comm request response
01583                 switch (V.response.ack[4]) {
01584                     case 'V':
01585                         switch (V.response.ack[5]) {
01586                             case 'I': // VII80 for Varian Viision 80
01587                                 equipment = GEM_VII80;
01588                                 break;
01589                             default:
01590                                 equipment = GEM_GENERIC;
01591                                 break;
01592                         }
01593                     break;
01594                     case 'E':
01595                         switch (V.response.ack[5]) {
01596                             case '2':
01597                                 equipment = GEM_E220;
01598                                 break;
01599                             case '5': // E220 or E500
01600                                 equipment = GEM_E220;
01601                                 break;
01602                             default:
01603                                 equipment = GEM_GENERIC;
01604                                 break;
01605                         }
01606                     break;
01607                     default:
01608                         equipment = GEM_GENERIC;
01609                         break;
01610                 }
01611             }
01612         }
01613     }
01614 }

```

```

01609     }
01610
01611     if (block != GEM_STATE_REMOTE) {
01612         res_block.respond = true;
01613         res_block.reply = (uint8_t*) & H153[0]; // S10F3 send Terminal Display, Single, queue
01614         res_block.reply_length = sizeof(header153);
01615         H153[0].data[S10F3_TID_POS] = V.response.TID;
01616         terminal_format(display_remote);
01617         format_display_text(V.terminal); // move into H153 GEM message buffer
01618         V.response.mesgid = 1;
01619         V.sequences++;
01620         V.sid = 10;
01621         sequence_messages(V.sid); // send text message
01622         set_display_info(DIS_SEQUENCE_M);
01623         block = GEM_STATE_REMOTE;
01624     }
01625     V.ticker = TICKER_ZERO;
01626     break;
01627 case 14:
01628     if (block != GEM_STATE_REMOTE) {
01629         terminal_format(display_remote);
01630         format_display_text(V.terminal); // move into H153 GEM message buffer
01631         V.response.mesgid = 1;
01632         V.sequences++;
01633         V.sid = 10;
01634         sequence_messages(V.sid); // send text message
01635         set_display_info(DIS_SEQUENCE_M);
01636         block = GEM_STATE_REMOTE;
01637     }
01638     V.ticker = TICKER_ZERO;
01639     break;
01640 #ifdef DB2
01641 case 15:
01642 #endif
01643 case 16:
01644     block = GEM_STATE_OFFLINE;
01645     V.ticker = TICKER_HIGH;
01646     break;
01647 #ifdef DB2
01648 case 17:
01649 #endif
01650 case 18:
01651     block = GEM_STATE_ONLINE;
01652     V.ticker = TICKER_LOW;
01653     break;
01654 default:
01655     if (block == GEM_STATE_DISABLE) {
01656         block = GEM_STATE_COMM;
01657         V.ticker = TICKER_HIGH;
01658     }
01659     break;
01660 }
01661 break;
01662 case 5: // alarms
01663     switch (function) {
01664     default:
01665         V.alarm = function;
01666         break;
01667     }
01668     break;
01669 case 9:
01670     switch (function) {
01671     default:
01672         V.alarm = function;
01673         if (V.ticker != TICKER_HIGH) {
01674             V.ticker = TICKER_LOW;
01675         }
01676         break;
01677     }
01678     break;
01679 default:
01680     if (block == GEM_STATE_DISABLE) {
01681         block = GEM_STATE_COMM;
01682         V.ticker = TICKER_HIGH;
01683     }
01684     break;
01685 }
01686
01687 V.e_types = equipment;
01688 return(block);
01689 }
01690
01691 /*
01692 * send equipment emulation data in some state and timing conditions
01693 */
01694 void equip_tx(const uint8_t data)
01695 {

```

```

01696     static uint8_t pinger = 0;
01697
01698     if (++pinger > PINGER || V.g_state != GEM_STATE_ONLINE || V.t_l_state == LINK_STATE_NAK) {
01699         switch (V.euart) {
01700             case HOST_UART:
01701                 UART1_Write(data);
01702                 V.tx_total++;
01703                 break;
01704             default:
01705                 UART2_Write(data);
01706                 V.tx_total++;
01707                 break;
01708         }
01709         pinger = 0;
01710         DLED_Toggle();
01711     }
01712 }

```

6.29 gemsecs.h

```

00001 /*
00002  * File:    gemsecs.h
00003  * Author:  root
00004  *
00005  * Created on February 26, 2019, 2:58 PM
00006  */
00007
00008 #ifndef GEMSECS_H
00009 #define GEMSECS_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 #define ENQ 0x05
00016 #define EOT 0x04
00017 #define ACK 0x06
00018 #define NAK 0x15
00019
00020 #include <stdio.h>
00021 #include <string.h>
00022 #include "vconfig.h"
00023 #include "mcc_generated_files/mcc.h"
00024 #include "mcc_generated_files/uart1.h"
00025 #include "timers.h"
00026 #include "mydisplay.h"
00027 #include "msg_text.h"
00028
00029 #define HOST_UART 1
00030 #define EQUIP_UART 2
00031
00032 const char msg_gemcmds[] = "Host CMDs: M C R P O L S D E H F";
00033 const char msg_freecmds[] = "Port baud rate unlocked ";
00034 const char msg_gemremote[] = "Host CMDs: ENABLED REMOTE";
00035
00036
00037 typedef struct block10_type {
00038     uint32_t systemb;
00039     uint8_t bidl;
00040     uint8_t bidh : 7;
00041     uint8_t ebit : 1;
00042     uint8_t function;
00043     uint8_t stream : 7;
00044     uint8_t wbit : 1;
00045     uint8_t didl;
00046     uint8_t didh : 7;
00047     uint8_t rbit : 1;
00048 } block10_type;
00049
00050 typedef union block10 {
00051     uint8_t b[sizeof(block10_type)];
00052     block10_type block;
00053 } block10;
00054
00055 typedef struct header10 {
00056     uint16_t checksum;
00057     block10 block;
00058     uint8_t length;
00059 } header10;
00060
00061 typedef struct header12 {
00062     uint16_t checksum;
00063     uint8_t data[2];

```



```
00064         block10 block;
00065         uint8_t length;
00066     } header12;
00067
00068     typedef struct header13 {
00069         uint16_t checksum;
00070         uint8_t data[3];
00071         block10 block;
00072         uint8_t length;
00073     } header13;
00074
00075     typedef struct header14 {
00076         uint16_t checksum;
00077         uint8_t data[4];
00078         block10 block;
00079         uint8_t length;
00080     } header14;
00081
00082     typedef struct header17 {
00083         uint16_t checksum;
00084         uint8_t data[7];
00085         block10 block;
00086         uint8_t length;
00087     } header17;
00088
00089     typedef struct header18 {
00090         uint16_t checksum;
00091         uint8_t data[8];
00092         block10 block;
00093         uint8_t length;
00094     } header18;
00095
00096     typedef struct header24 {
00097         uint16_t checksum;
00098         uint8_t data[12];
00099         uint8_t datam[2];
00100         block10 block;
00101         uint8_t length;
00102     } header24;
00103
00104     typedef struct header26 {
00105         uint16_t checksum;
00106         uint8_t data[14];
00107         uint8_t datam[2];
00108         block10 block;
00109         uint8_t length;
00110     } header26;
00111
00112     typedef struct header27 {
00113         uint16_t checksum;
00114         uint8_t data[17];
00115         block10 block;
00116         uint8_t length;
00117     } header27;
00118
00119     typedef struct header33 {
00120         uint16_t checksum;
00121         uint8_t data[23];
00122         block10 block;
00123         uint8_t length;
00124     } header33;
00125
00126     typedef struct header153 {
00127         uint16_t checksum;
00128         uint8_t data[143];
00129         block10 block;
00130         uint8_t length;
00131     } header153;
00132
00133     typedef struct header254 {
00134         uint16_t checksum;
00135         uint8_t data[244];
00136         block10 block;
00137         uint8_t length;
00138     } header254;
00139
00140     typedef struct response_type {
00141         uint8_t *header;
00142         uint8_t length;
00143         uint8_t *reply;
00144         uint8_t reply_length;
00145         uint8_t respond : 1;
00146     } response_type;
00147
00148     typedef struct gem_message_type {
00149         header33 message;
00150         response_type block;
```

```

00151         uint16_t delay;
00152         uint8_t stack;
00153     } gem_message_type;
00154
00155     typedef struct gem_display_type {
00156         header153 message;
00157         response_type block;
00158         uint16_t delay;
00159         uint8_t stack;
00160     } gem_display_type;
00161
00162     uint16_t block_checksum(uint8_t *, const uint16_t);
00163     uint16_t run_checksum(const uint8_t, const bool);
00164     LINK_STATES m_protocol(LINK_STATES *);
00165     LINK_STATES r_protocol(LINK_STATES *);
00166     LINK_STATES t_protocol(LINK_STATES *);
00167     void hb_message(void);
00168     void terminal_format(DISPLAY_TYPES);
00169     uint16_t format_display_text(const char *);
00170     P_CODES s10f1_opcmd(void);
00171     uint16_t s6f11_opcmd(void);
00172     response_type secs_II_message(const uint8_t, const uint8_t);
00173     bool sequence_messages(const uint8_t);
00174     bool gem_messages(response_type *, const uint8_t);
00175     void secs_II_monitor_message(const uint8_t, const uint8_t, const uint16_t);
00176     GEM_STATES secs_gem_state(const uint8_t, const uint8_t);
00177     void equip_tx(const uint8_t);
00178
00179 #ifdef __cplusplus
00180 }
00181 #endif
00182
00183 #endif /* GEMSECS_H */
00184

```

6.30 secs_q84.X/main.c File Reference

Lets get going with the code.

```

#include <stdio.h>
#include <string.h>
#include "mcc_generated_files/mcc.h"
#include "mcc_generated_files/uart1.h"
#include "eadog.h"
#include "gemsecs.h"
#include "timers.h"
#include "mconfig.h"
#include "mydisplay.h"
#include "rs232.h"

```

Macros

- #define `__DEFINED_int24_t`
- #define `M_TRACE` do { LATDbits.LATD5 = ~LATDbits.LATD5; } while(0)
- #define `I_TRACE` RB7_Toggle()

Typedefs

- typedef signed long long `int24_t`

Functions

- void `oneseq_io` (void)
- void `main` (void)
- char `spinners` (uint8_t shape, const uint8_t reset)

Variables

- struct [spi_link_type](#) [spi_link](#)
 - const char * [build_date](#) = `__DATE__`
 - const char * [build_time](#) = `__TIME__`
 - const char * [GEM_TEXT](#) []
 - [V_data](#) [V](#)
 - [B_type](#) [B](#)
 - [header10](#) [H10](#) []
 - [header12](#) [H12](#) []
 - [header13](#) [H13](#) []
 - [header14](#) [H14](#) []
 - [header17](#) [H17](#) []
 - [header26](#) [H26](#) []
 - [header33](#) [H33](#) []
 - const [header33](#) [HC33](#) []
 - [header153](#) [H153](#) []
 - [header254](#) [H254](#) []
 - [gem_message_type](#) [S](#) [10]
 - [gem_display_type](#) [D](#) [2]
 - [header10](#) [r_block](#)
 - volatile uint16_t [tickCount](#) [TMR_COUNT] = {0}
- TMR6 Generated Driver File.*
- volatile uint8_t [mode_sw](#) = false
 - volatile uint8_t [faker](#)

6.30.1 Detailed Description

Lets get going with the code.

Main application

Definition in file [main.c](#).

6.30.2 Macro Definition Documentation

6.30.2.1 `__DEFINED_int24_t`

```
#define __DEFINED_int24_t
```

Definition at line [162](#) of file [main.c](#).

6.30.2.2 `I_TRACE`

```
#define I_TRACE RB7_Toggle()
```

Definition at line [178](#) of file [main.c](#).

6.30.2.3 M_TRACE

```
#define M_TRACE do { LATDbits.LATD5 = ~LATDbits.LATD5; } while(0)
```

Definition at line 177 of file [main.c](#).

6.30.3 Typedef Documentation

6.30.3.1 int24_t

```
typedef signed long long int24_t
```

Definition at line 161 of file [main.c](#).

6.30.4 Function Documentation

6.30.4.1 main()

```
void main (  
    void )
```

Speed locking setup code. Use a few EEPROM bytes to cycle or lock the serial port baud rate during a power-up. 9600 and 19200 are the normal speeds for SECS-I serial communications

Definition at line 783 of file [main.c](#).

6.30.4.2 onesec_io()

```
void onesec_io (  
    void )
```

Definition at line 1235 of file [main.c](#).

6.30.4.3 spinners()

```
char spinners (  
    uint8_t shape,  
    const uint8_t reset)
```

Definition at line 1244 of file [main.c](#).

6.30.5 Variable Documentation

6.30.5.1 B

[B_type](#) B

Initial value:

```
= {  
    .one_sec_flag = false,  
    .display_update = false,  
    .dim_delay = DIM_DELAY,  
}
```

Definition at line 231 of file [main.c](#).

6.30.5.2 build_date

```
const char* build_date = __DATE__
```

Definition at line 185 of file [main.c](#).

6.30.5.3 build_time

```
const char * build_time = __TIME__
```

Definition at line 185 of file [main.c](#).

6.30.5.4 D

```
gem_display_type D[2]
```

Definition at line 771 of file [main.c](#).

6.30.5.5 faker

```
volatile uint8_t faker
```

Definition at line 776 of file [main.c](#).

6.30.5.6 GEM_TEXT

```
const char* GEM_TEXT[ ]
```

Initial value:

```
= {  
    "DISABLE",  
    "COMM  ",  
    "OFFLINE",  
    "ONLINE  ",  
    "REMOTE ",  
    "ERROR  "  
}
```

Definition at line 187 of file [main.c](#).

6.30.5.7 H10

```
header10 H10[ ]
```

Definition at line 237 of file [main.c](#).

6.30.5.8 H12

```
header12 H12[ ]
```

Definition at line 321 of file [main.c](#).

6.30.5.9 H13

`header13` H13[]

Definition at line 354 of file [main.c](#).

6.30.5.10 H14

`header14` H14[]

Initial value:

```
= {  
    {  
        .length = 14,  
        .block.block.rbit = 0,  
        .block.block.didh = 0,  
        .block.block.didl = 0,  
        .block.block.wbit = 0,  
        .block.block.stream = 1,  
        .block.block.function = 4,  
        .block.block.ebit = 1,  
        .block.block.bidh = 0,  
        .block.block.bidl = 1,  
        .block.block.systemb = 1,  
        .data[3] = 0x01,  
        .data[2] = 0x11,  
        .data[1] = 0x41,  
        .data[0] = 0x00,  
    },  
}
```

Definition at line 421 of file [main.c](#).

6.30.5.11 H153

`header153` H153[]

Definition at line 636 of file [main.c](#).

6.30.5.12 H17

`header17` H17[]

Initial value:

```
= {  
    {  
        .length = 17,  
        .block.block.rbit = 0,  
        .block.block.didh = 0,  
        .block.block.didl = 0,  
        .block.block.wbit = 0,  
        .block.block.stream = 1,  
        .block.block.function = 14,  
        .block.block.ebit = 1,  
        .block.block.bidh = 0,  
        .block.block.bidl = 1,  
        .block.block.systemb = 1,  
        .data[6] = 0x01,  
        .data[5] = 0x02,  
        .data[4] = 0x21,  
        .data[3] = 0x01,  
        .data[2] = 0x00,  
        .data[1] = 0x01,  
        .data[0] = 0x00,  
    },  
}
```

Definition at line 441 of file [main.c](#).

6.30.5.13 H254

`header254` H254[]

Initial value:

```
= {  
    {  
        .length = 254,  
        .block.block.rbit = 1,  
        .block.block.didh = 0,  
        .block.block.didl = 0xEF,  
        .block.block.wbit = 1,  
        .block.block.stream = 10,  
        .block.block.function = 3,  
        .block.block.ebit = 1,  
        .block.block.bidh = 0,  
        .block.block.bidl = 1,  
        .block.block.systemb = 1,  
        .checksum = 0x2019,  
        .data[1] = 0x19,  
        .data[0] = 0x57,  
    },  
}
```

Definition at line 751 of file [main.c](#).

6.30.5.14 H26

`header26` H26[]

Initial value:

```
= {  
    {  
        .length = 26,  
        .block.block.rbit = 0,  
        .block.block.didh = 0,  
        .block.block.didl = 0,  
        .block.block.wbit = 0,  
        .block.block.stream = 2,  
        .block.block.function = 18,  
        .block.block.ebit = 1,  
        .block.block.bidh = 0,  
        .block.block.bidl = 1,  
        .block.block.systemb = 1,  
        .data = "01091108460000",  
        .datam[1] = 0x41,  
        .datam[0] = 14,  
    },  
}
```

Definition at line 484 of file [main.c](#).

6.30.5.15 H33

`header33` H33[]

Definition at line 522 of file [main.c](#).

6.30.5.16 HC33

const `header33` HC33[]

Definition at line 561 of file [main.c](#).

6.30.5.17 mode_sw

```
volatile uint8_t mode_sw = false
```

Definition at line 776 of file [main.c](#).

6.30.5.18 r_block

```
header10 r_block
```

Definition at line 773 of file [main.c](#).

6.30.5.19 S

```
gem_message_type S[10]
```

Definition at line 770 of file [main.c](#).

6.30.5.20 spi_link

```
struct spi_link_type spi_link [extern]
```

Definition at line 19 of file [eadog.c](#).

6.30.5.21 tickCount

```
volatile uint16_t tickCount[TMR_COUNT] = {0}
```

TMR6 Generated Driver File.

@Company Microchip Technology Inc.

@File Name [tmr6.c](#)

@Summary This is the generated driver implementation file for the TMR6 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for TMR6. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F47Q84 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files

Definition at line 775 of file [main.c](#).

6.30.5.22 V

```
V_data V
```

Definition at line 196 of file [main.c](#).

6.31 main.c

[Go to the documentation of this file.](#)

```

00001 /*
00002  * SECS-II RS232 link tester
00003  */
00004
00005 /*
00006  * UART1 HOST RS-232 comms, ANALOG header, TX-PIN 7 MB_TX, RX-PIN 6 MB_RX
00007  * UART2 Equipment Testing RS-232 comms, TTL serial HEADER, TX-PIN 2 PC_TX, RX-PIN 3 PC_RX
00008  */
00009
00010 // PIC18F47Q84 Configuration Bit Settings
00011 // 'C' source line config statements
00012 // CONFIG1
00013 #pragma config FEXTOSC = ECH      // External Oscillator Selection (EC (external clock) above 8 MHz)
00014 #pragma config RSTOSC = EXTOSC_4PLL// Reset Oscillator Selection (EXTOSC with 4x PLL, with EXTOSC
operating per FEXTOSC bits)
00015
00016 // CONFIG2
00017 #pragma config CLKOUTEN = ON      // Clock out Enable bit (CLKOUT function is enabled)
00018 #pragma config PR1WAY = OFF      // PRLOCKED One-Way Set Enable bit (PRLOCKED bit can be set and
cleared repeatedly)
00019 #pragma config CSWEN = ON        // Clock Switch Enable bit (Writing to NOSC and NDIV is allowed)
00020 #pragma config JTAGEN = OFF      // JTAG Enable bit (Disable JTAG Boundary Scan mode, JTAG pins revert
to user functions)
00021 #pragma config FCMEN = ON        // Fail-Safe Clock Monitor Enable bit (Fail-Safe Clock Monitor
enabled)
00022 #pragma config FCMENP = ON       // Fail-Safe Clock Monitor -Primary XTAL Enable bit (FSCM timer will
set FSCMP bit and OSFIF interrupt on Primary XTAL failure)
00023 #pragma config FCMENS = ON       // Fail-Safe Clock Monitor -Secondary XTAL Enable bit (FSCM timer will
set FSCMS bit and OSFIF interrupt on Secondary XTAL failure)
00024
00025 // CONFIG3
00026 #pragma config MCLRE = EXTMCLR   // MCLR Enable bit (If LVP = 0, MCLR pin is MCLR; If LVP = 1, RE3 pin
function is MCLR )
00027 #pragma config PWRTS = PWRT_OFF  // Power-up timer selection bits (PWRT is disabled)
00028 #pragma config MVECEN = ON       // Multi-vector enable bit (Multi-vector enabled, Vector table used
for interrupts)
00029 #pragma config IVT1WAY = ON      // IVTLOCK bit One-way set enable bit (IVTLOCKED bit can be cleared
and set only once)
00030 #pragma config LPBORN = OFF      // Low Power BOR Enable bit (Low-Power BOR disabled)
00031 #pragma config BOREN = SBORDIS   // Brown-out Reset Enable bits (Brown-out Reset enabled , SBORN bit
is ignored)
00032
00033 // CONFIG4
00034 #pragma config BORV = VBOR_1P9  // Brown-out Reset Voltage Selection bits (Brown-out Reset Voltage
(VBOR) set to 1.9V)
00035 #pragma config ZCD = OFF         // ZCD Disable bit (ZCD module is disabled. ZCD can be enabled by
setting the ZCDSEN bit of ZCDCON)
00036 #pragma config PPS1WAY = OFF     // PPSLOCK bit One-Way Set Enable bit (PPSLOCKED bit can be set and
cleared repeatedly (subject to the unlock sequence))
00037 #pragma config STVREN = ON       // Stack Full/Underflow Reset Enable bit (Stack full/underflow will
cause Reset)
00038 #pragma config LVP = OFF         // Low Voltage Programming Enable bit (HV on MCLR/VPP must be used for
programming)
00039 #pragma config XINST = OFF       // Extended Instruction Set Enable bit (Extended Instruction Set and
Indexed Addressing Mode disabled)
00040
00041 // CONFIG5
00042 #pragma config WDTCP5 = WDTCP5_31// WDT Period selection bits (Divider ratio 1:65536; software control
of WDTPS)
00043 #pragma config WDTE = OFF        // WDT operating mode (WDT Disabled; SWDTEN is ignored)
00044
00045 // CONFIG6
00046 #pragma config WDTW5 = WDTW5_7// WDT Window Select bits (window always open (100%); software
control; keyed access not required)
00047 #pragma config WDTCC5 = SC       // WDT input clock selector (Software Control)
00048
00049 // CONFIG7
00050 #pragma config BBSIZE = BBSIZE_512// Boot Block Size selection bits (Boot Block size is 512 words)
00051 #pragma config BBEN = OFF        // Boot Block enable bit (Boot block disabled)
00052 #pragma config SAFEN = OFF       // Storage Area Flash enable bit (SAF disabled)
00053 #pragma config DEBUG = OFF       // Background Debugger (Background Debugger disabled)
00054
00055 // CONFIG8
00056 #pragma config WRTB = OFF        // Boot Block Write Protection bit (Boot Block not Write protected)
00057 #pragma config WRTC = OFF        // Configuration Register Write Protection bit (Configuration
registers not Write protected)
00058 #pragma config WRTE = OFF        // Data EEPROM Write Protection bit (Data EEPROM not Write protected)
00059 #pragma config WRTSAF = OFF      // SAF Write protection bit (SAF not Write Protected)
00060 #pragma config WRTAPP = OFF      // Application Block write protection bit (Application Block not write
protected)
00061
00062 // CONFIG9

```

```

00063 #pragma config BOOTPINSEL = RC5 // CRC on boot output pin selection (CRC on boot output pin is RC5)
00064 #pragma config BPEN = OFF // CRC on boot output pin enable bit (CRC on boot output pin disabled)
00065 #pragma config ODCON = OFF // CRC on boot output pin open drain bit (Pin drives both high-going
and low-going signals)
00066
00067 // CONFIG10
00068 #pragma config CP = ON // PFM and Data EEPROM Code Protection bit (PFM and Data EEPROM code
protection enabled)
00069
00070 // CONFIG11
00071 #pragma config BOOTSCEN = OFF // CRC on boot scan enable for boot area (CRC on boot will not include
the boot area of program memory in its calculation)
00072 #pragma config BOOTCOE = HALT // CRC on boot Continue on Error for boot areas bit (CRC on boot will
stop device if error is detected in boot areas)
00073 #pragma config APPSCEN = OFF // CRC on boot application code scan enable (CRC on boot will not
include the application area of program memory in its calculation)
00074 #pragma config SAFSCEN = OFF // CRC on boot SAF area scan enable (CRC on boot will not include the
SAF area of program memory in its calculation)
00075 #pragma config DATASCEN = OFF // CRC on boot Data EEPROM scan enable (CRC on boot will not include
data EEPROM in its calculation)
00076 #pragma config CFGSCEN = OFF // CRC on boot Config fuses scan enable (CRC on boot will not include
the configuration fuses in its calculation)
00077 #pragma config COE = HALT // CRC on boot Continue on Error for non-boot areas bit (CRC on boot
will stop device if error is detected in non-boot areas)
00078 #pragma config BOOTPOR = OFF // Boot on CRC Enable bit (CRC on boot will not run)
00079
00080 // CONFIG12
00081 #pragma config BCRCPOLT = hFF // Boot Sector Polynomial for CRC on boot bits 31-24 (Bits 31:24 of
BCRCPOL are 0xFF)
00082
00083 // CONFIG13
00084 #pragma config BCRCPOLU = hFF // Boot Sector Polynomial for CRC on boot bits 23-16 (Bits 23:16 of
BCRCPOL are 0xFF)
00085
00086 // CONFIG14
00087 #pragma config BCRCPOLH = hFF // Boot Sector Polynomial for CRC on boot bits 15-8 (Bits 15:8 of
BCRCPOL are 0xFF)
00088
00089 // CONFIG15
00090 #pragma config BCRCPOLL = hFF // Boot Sector Polynomial for CRC on boot bits 7-0 (Bits 7:0 of
BCRCPOL are 0xFF)
00091
00092 // CONFIG16
00093 #pragma config BCRSEEDT = hFF // Boot Sector Seed for CRC on boot bits 31-24 (Bits 31:24 of BCRSEED
are 0xFF)
00094
00095 // CONFIG17
00096 #pragma config BCRSEEDU = hFF // Boot Sector Seed for CRC on boot bits 23-16 (Bits 23:16 of BCRSEED
are 0xFF)
00097
00098 // CONFIG18
00099 #pragma config BCRSEEDH = hFF // Boot Sector Seed for CRC on boot bits 15-8 (Bits 15:8 of BCRSEED
are 0xFF)
00100
00101 // CONFIG19
00102 #pragma config BCRSEEDL = hFF // Boot Sector Seed for CRC on boot bits 7-0 (Bits 7:0 of BCRSEED are
0xFF)
00103
00104 // CONFIG20
00105 #pragma config BRCEREST = hFF // Boot Sector Expected Result for CRC on boot bits 31-24 (Bits 31:24
of BRCERES are 0xFF)
00106
00107 // CONFIG21
00108 #pragma config BRCERESU = hFF // Boot Sector Expected Result for CRC on boot bits 23-16 (Bits 23:16
of BRCERES are 0xFF)
00109
00110 // CONFIG22
00111 #pragma config BRCERESH = hFF // Boot Sector Expected Result for CRC on boot bits 15-8 (Bits 15:8 of
BRCERES are 0xFF)
00112
00113 // CONFIG23
00114 #pragma config BRCERESL = hFF // Boot Sector Expected Result for CRC on boot bits 7-0 (Bits 7:0 of
BRCERES are 0xFF)
00115
00116 // CONFIG24
00117 #pragma config CRCPOLT = hFF // Non-Boot Sector Polynomial for CRC on boot bits 31-24 (Bits 31:24
of CRCPOL are 0xFF)
00118
00119 // CONFIG25
00120 #pragma config CRCPOLU = hFF // Non-Boot Sector Polynomial for CRC on boot bits 23-16 (Bits 23:16
of CRCPOL are 0xFF)
00121
00122 // CONFIG26
00123 #pragma config CRCPOLH = hFF // Non-Boot Sector Polynomial for CRC on boot bits 15-8 (Bits 15:8 of
CRCPOL are 0xFF)
00124
00125 // CONFIG27

```

```

00126 #pragma config CRCPOLL = hFF    // Non-Boot Sector Polynomial for CRC on boot bits 7-0 (Bits 7:0 of
    CRCPOL are 0xFF)
00127
00128 // CONFIG28
00129 #pragma config CRCSEEDT = hFF    // Non-Boot Sector Seed for CRC on boot bits 31-24 (Bits 31:24 of
    CRCSEED are 0xFF)
00130
00131 // CONFIG29
00132 #pragma config CRCSEEDU = hFF    // Non-Boot Sector Seed for CRC on boot bits 23-16 (Bits 23:16 of
    CRCSEED are 0xFF)
00133
00134 // CONFIG30
00135 #pragma config CRCSEEDH = hFF    // Non-Boot Sector Seed for CRC on boot bits 15-8 (Bits 15:8 of
    CRCSEED are 0xFF)
00136
00137 // CONFIG31
00138 #pragma config CRCSEEDL = hFF    // Non-Boot Sector Seed for CRC on boot bits 7-0 (Bits 7:0 of CRCSEED
    are 0xFF)
00139
00140 // CONFIG32
00141 #pragma config CRCEREST = hFF    // Non-Boot Sector Expected Result for CRC on boot bits 31-24 (Bits
    31:24 of CRCERES are 0xFF)
00142
00143 // CONFIG33
00144 #pragma config CRCERESU = hFF    // Non-Boot Sector Expected Result for CRC on boot bits 23-16 (Bits
    23:16 of CRCERES are 0xFF)
00145
00146 // CONFIG34
00147 #pragma config CRCERESH = hFF    // Non-Boot Sector Expected Result for CRC on boot bits 15-8 (Bits
    15:8 of CRCERES are 0xFF)
00148
00149 // CONFIG35
00150 #pragma config CRCERESL = hFF    // Non-Boot Sector Expected Result for CRC on boot bits 7-0 (Bits 7:0
    of CRCERES are 0xFF)
00151
00152 // #pragma config statements should precede project file includes.
00153 // Use project enums instead of #define for ON and OFF.
00154
00155 #pragma warning disable 520
00156 #pragma warning disable 1090
00157 #pragma warning disable 1498
00158 #pragma warning disable 2053
00159
00160 #ifndef __DEFINED_int24_t
00161 typedef signed long long int24_t;
00162 #define __DEFINED_int24_t
00163 #endif
00164
00165 #include <stdio.h>
00166 #include <string.h>
00167 #include "mcc_generated_files/mcc.h"
00168 #include "mcc_generated_files/uart1.h"
00169 #include "eadog.h"
00170 #include "gemsecs.h"
00171 #include "timers.h"
00172 #include "mconfig.h"
00173 #include "mydisplay.h"
00174 #include "rs232.h"
00175
00176 #ifdef TRACE
00177 #define M_TRACE do { LATDbits.LATD5 = ~LATDbits.LATD5; } while(0)
00178 #define I_TRACE RB7_Toggle()
00179 #else
00180 #define M_TRACE ""
00181 #define I_TRACE ""
00182 #endif
00183
00184 extern struct spi_link_type spi_link;
00185 const char *build_date = __DATE__, *build_time = __TIME__;
00186
00187 const char * GEM_TEXT [] = {
00188     "DISABLE",
00189     "COMM   ",
00190     "OFFLINE",
00191     "ONLINE  ",
00192     "REMOTE  ",
00193     "ERROR   "
00194 };
00195
00196 V_data V = {
00197     .error = LINK_ERROR_NONE,
00198     .abort = LINK_ERROR_NONE,
00199     .msg_error = MSG_ERROR_RESET,
00200     .uart = HOST_UART,
00201     .g_state = GEM_STATE_DISABLE,
00202     .e_types = GEM_GENERIC,
00203     .ticker = TICKER_ZERO,

```

```

00204     .checksum_error = 0,
00205     .all_errors = 0,
00206     .timer_error = 0,
00207     .response.info = DIS_STR,
00208     .response.log_num = 0,
00209     .response.log_seq = 0,
00210     .response.host_display_ack = false,
00211     .queue = false,
00212     .stack = 0, // 0 no messages, 1-10 messages in queue
00213     .sid = 1,
00214     .help_id = 0,
00215     .ping_count = 0,
00216     .sequences = 0,
00217     .set_sequ = false,
00218     .euart = EQUIP_UART,
00219     .tx_total = 0,
00220     .rx_total = 0,
00221     .failed_receive = RECV_ERROR_NONE,
00222     .failed_send = SEND_ERROR_NONE,
00223     .vterm = MAIN_VTERM,
00224     .tx_rs232 = 'O',
00225     .rx_rs232 = 'O',
00226     .debug = false,
00227     .rerror = false,
00228     .help = false,
00229 };
00230
00231 B_type B = {
00232     .one_sec_flag = false,
00233     .display_update = false,
00234     .dim_delay = DIM_DELAY,
00235 };
00236
00237 header10 H10[] = {
00238     { // S1F1 send 'are you there?' from host to equipment
00239         .length = 10,
00240         .block.block.rbit = 0,
00241         .block.block.didh = 0,
00242         .block.block.didl = 0,
00243         .block.block.wbit = 1,
00244         .block.block.stream = 1,
00245         .block.block.function = 1,
00246         .block.block.ebit = 1,
00247         .block.block.bidh = 0,
00248         .block.block.bidl = 1,
00249         .block.block.systemb = 1,
00250     },
00251     { // all stream and function header receive buffer from equipment
00252         .length = 10,
00253     },
00254     { // S1F0 send 'ABORT' from host
00255         .length = 10,
00256         .block.block.rbit = 0,
00257         .block.block.didh = 0,
00258         .block.block.didl = 0,
00259         .block.block.wbit = 0,
00260         .block.block.stream = 1,
00261         .block.block.function = 0,
00262         .block.block.ebit = 1,
00263         .block.block.bidh = 0,
00264         .block.block.bidl = 1,
00265         .block.block.systemb = 1,
00266     },
00267     { // S1F1 send 'are you there?' from equipment to host
00268         .length = 10,
00269         .block.block.rbit = 1,
00270         .block.block.didh = 0,
00271         .block.block.didl = 0,
00272         .block.block.wbit = 1,
00273         .block.block.stream = 1,
00274         .block.block.function = 1,
00275         .block.block.ebit = 1,
00276         .block.block.bidh = 0,
00277         .block.block.bidl = 1,
00278         .block.block.systemb = 1,
00279     },
00280     { // S2F17 send 'date and time request?' from host
00281         .length = 10,
00282         .block.block.rbit = 0,
00283         .block.block.didh = 0,
00284         .block.block.didl = 0,
00285         .block.block.wbit = 1,
00286         .block.block.stream = 2,
00287         .block.block.function = 17,
00288         .block.block.ebit = 1,
00289         .block.block.bidh = 0,
00290         .block.block.bidl = 1,

```

```

00291         .block.block.systemb = 1,
00292     },
00293     { // S1F15 send 'request off-line ' from host
00294         .length = 10,
00295         .block.block.rbit = 0,
00296         .block.block.didh = 0,
00297         .block.block.didl = 0,
00298         .block.block.wbit = 1,
00299         .block.block.stream = 1,
00300         .block.block.function = 15,
00301         .block.block.ebit = 1,
00302         .block.block.bidh = 0,
00303         .block.block.bidl = 1,
00304         .block.block.systemb = 1,
00305     },
00306     { // S1F17 send 'request on-line ' from host
00307         .length = 10,
00308         .block.block.rbit = 0,
00309         .block.block.didh = 0,
00310         .block.block.didl = 0,
00311         .block.block.wbit = 1,
00312         .block.block.stream = 1,
00313         .block.block.function = 17,
00314         .block.block.ebit = 1,
00315         .block.block.bidh = 0,
00316         .block.block.bidl = 1,
00317         .block.block.systemb = 1,
00318     },
00319 };
00320
00321 header12 H12[] = {
00322     { // S1F2 send 'yes, we are here ' from host
00323         .length = 12,
00324         .block.block.rbit = 0,
00325         .block.block.didh = 0,
00326         .block.block.didl = 0,
00327         .block.block.wbit = 0,
00328         .block.block.stream = 1,
00329         .block.block.function = 2,
00330         .block.block.ebit = 1,
00331         .block.block.bidh = 0,
00332         .block.block.bidl = 1,
00333         .block.block.systemb = 1,
00334         .data[1] = 1,
00335         .data[0] = 0,
00336     },
00337     { // S1F13 send 'online request ' from host to equipment
00338         .length = 12,
00339         .block.block.rbit = 0,
00340         .block.block.didh = 0,
00341         .block.block.didl = 0,
00342         .block.block.wbit = 1,
00343         .block.block.stream = 1,
00344         .block.block.function = 13,
00345         .block.block.ebit = 1,
00346         .block.block.bidh = 0,
00347         .block.block.bidl = 1,
00348         .block.block.systemb = 1,
00349         .data[1] = 1,
00350         .data[0] = 0,
00351     },
00352 };
00353
00354 header13 H13[] = {
00355     { // S6F12 send 'online' reply from host
00356         .length = 13,
00357         .block.block.rbit = 0,
00358         .block.block.didh = 0,
00359         .block.block.didl = 0,
00360         .block.block.wbit = 0,
00361         .block.block.stream = 6,
00362         .block.block.function = 12,
00363         .block.block.ebit = 1,
00364         .block.block.bidh = 0,
00365         .block.block.bidl = 1,
00366         .block.block.systemb = 1,
00367         .data[2] = 0x21,
00368         .data[1] = 0x01,
00369         .data[0] = 0x00,
00370     },
00371     { // S10F2 send 'terminal' reply from host
00372         .length = 13,
00373         .block.block.rbit = 0,
00374         .block.block.didh = 0,
00375         .block.block.didl = 0,
00376         .block.block.wbit = 0,
00377         .block.block.stream = 10,

```

```

00378         .block.block.function = 2,
00379         .block.block.ebit = 1,
00380         .block.block.bidh = 0,
00381         .block.block.bidl = 1,
00382         .block.block.systemb = 1,
00383         .data[2] = 0x21,
00384         .data[1] = 0x01,
00385         .data[0] = 0x00,
00386     },
00387     { // S5F2 send 'Alarm Report Ack' reply from host
00388         .length = 13,
00389         .block.block.rbit = 0,
00390         .block.block.didh = 0,
00391         .block.block.didl = 0,
00392         .block.block.wbit = 0,
00393         .block.block.stream = 5,
00394         .block.block.function = 2,
00395         .block.block.ebit = 1,
00396         .block.block.bidh = 0,
00397         .block.block.bidl = 1,
00398         .block.block.systemb = 1,
00399         .data[2] = 0x21,
00400         .data[1] = 0x01,
00401         .data[0] = 0x00,
00402     },
00403     { // S2F26 send 'loopback' reply from host
00404         .length = 13,
00405         .block.block.rbit = 0,
00406         .block.block.didh = 0,
00407         .block.block.didl = 0,
00408         .block.block.wbit = 0,
00409         .block.block.stream = 2,
00410         .block.block.function = 26,
00411         .block.block.ebit = 1,
00412         .block.block.bidh = 0,
00413         .block.block.bidl = 1,
00414         .block.block.systemb = 1,
00415         .data[2] = 0x21,
00416         .data[1] = 0x01,
00417         .data[0] = 0xff, // binary data
00418     },
00419 };
00420
00421 header14 H14[] = {
00422     { // S1F4 send 'status response ' from host
00423         .length = 14,
00424         .block.block.rbit = 0,
00425         .block.block.didh = 0,
00426         .block.block.didl = 0,
00427         .block.block.wbit = 0,
00428         .block.block.stream = 1,
00429         .block.block.function = 4,
00430         .block.block.ebit = 1,
00431         .block.block.bidh = 0,
00432         .block.block.bidl = 1,
00433         .block.block.systemb = 1,
00434         .data[3] = 0x01,
00435         .data[2] = 0x11,
00436         .data[1] = 0x41,
00437         .data[0] = 0x00,
00438     },
00439 };
00440
00441 header17 H17[] = {
00442     { // S1F14 send 'online response ' from host to equipment
00443         .length = 17,
00444         .block.block.rbit = 0,
00445         .block.block.didh = 0,
00446         .block.block.didl = 0,
00447         .block.block.wbit = 0,
00448         .block.block.stream = 1,
00449         .block.block.function = 14,
00450         .block.block.ebit = 1,
00451         .block.block.bidh = 0,
00452         .block.block.bidl = 1,
00453         .block.block.systemb = 1,
00454         .data[6] = 0x01,
00455         .data[5] = 0x02,
00456         .data[4] = 0x21,
00457         .data[3] = 0x01,
00458         .data[2] = 0x00,
00459         .data[1] = 0x01,
00460         .data[0] = 0x00,
00461     },
00462 };
00463
00464 #ifndef Y2KTIME

```

```

00465 header24 H24[] = {
00466     { // S2F18 send 'host time ' from host to equipment non-y2k
00467         .length = 24,
00468         .block.block.rbit = 0,
00469         .block.block.didh = 0,
00470         .block.block.didl = 0,
00471         .block.block.wbit = 0,
00472         .block.block.stream = 2,
00473         .block.block.function = 18,
00474         .block.block.ebit = 1,
00475         .block.block.bidh = 0,
00476         .block.block.bidl = 1,
00477         .block.block.systemb = 1,
00478         .data = "010911084600",
00479         .datam[1] = 0x41,
00480         .datam[0] = 12,
00481     },
00482 };
00483 #else
00484 header26 H26[] = {
00485     { // S2F18 send 'host time ' from host to equipment y2k
00486         .length = 26,
00487         .block.block.rbit = 0,
00488         .block.block.didh = 0,
00489         .block.block.didl = 0,
00490         .block.block.wbit = 0,
00491         .block.block.stream = 2,
00492         .block.block.function = 18,
00493         .block.block.ebit = 1,
00494         .block.block.bidh = 0,
00495         .block.block.bidl = 1,
00496         .block.block.systemb = 1,
00497         .data = "01091108460000",
00498         .datam[1] = 0x41,
00499         .datam[0] = 14,
00500     },
00501 };
00502 #endif
00503
00504 #if defined(DB2) || defined(FAKER)
00505 header27 H27[] = {
00506     { // S1F13 send 'online request ' from equipment to host for TESTING
00507         .length = 27,
00508         .block.block.rbit = 1,
00509         .block.block.didh = 0,
00510         .block.block.didl = 0,
00511         .block.block.wbit = 1,
00512         .block.block.stream = 1,
00513         .block.block.function = 13,
00514         .block.block.ebit = 1,
00515         .block.block.bidh = 0,
00516         .block.block.bidl = 1,
00517         .block.block.systemb = 1,
00518     },
00519 };
00520 #endif
00521
00522 header33 H33[] = {
00523     { // S2F41 'ready load-lock ' command from host to equipment, read/write
00524         .length = 33,
00525         .block.block.rbit = 0,
00526         .block.block.didh = 0,
00527         .block.block.didl = 0,
00528         .block.block.wbit = 1,
00529         .block.block.stream = 2,
00530         .block.block.function = 41,
00531         .block.block.ebit = 1,
00532         .block.block.bidh = 0,
00533         .block.block.bidl = 1,
00534         .block.block.systemb = 1,
00535         .data[22] = 0x01, // list 2 items
00536         .data[21] = 0x02,
00537         .data[20] = 0x41, // 2 ascii data
00538         .data[19] = 0x02,
00539         .data[18] = 0x31, // 1
00540         .data[17] = 0x37, // 7
00541         .data[16] = 0x01, // list 1 item
00542         .data[15] = 0x01,
00543         .data[14] = 0x01, // list 2 items
00544         .data[13] = 0x02,
00545         .data[12] = 0x41, // 8 ascii data
00546         .data[11] = 0x08,
00547         .data[10] = 0x4c, // L
00548         .data[9] = 0x4f, // O
00549         .data[8] = 0x41, // A
00550         .data[7] = 0x44, // D
00551         .data[6] = 0x4c, // L

```

```

00552         .data[5] = 0x4f, // O
00553         .data[4] = 0x43, // C
00554         .data[3] = 0x4b, // K
00555         .data[2] = 0xa5, // 1 byte integer unsigned
00556         .data[1] = 0x01, // length 1
00557         .data[0] = 0x01, // value
00558     },
00559 };
00560
00561 const header33 HC33[] = { //template for vii80 stack commands, read-only
00562     { // S2F41 'ready load-lock ' command from host to equipment
00563         .length = 33,
00564         .block.block.rbit = 0,
00565         .block.block.didh = 0,
00566         .block.block.didl = 0,
00567         .block.block.wbit = 1,
00568         .block.block.stream = 2,
00569         .block.block.function = 41,
00570         .block.block.ebit = 1,
00571         .block.block.bidh = 0,
00572         .block.block.bidl = 1,
00573         .block.block.systemb = 1,
00574         .data[22] = 0x01, // list 2 items
00575         .data[21] = 0x02,
00576         .data[20] = 0x41, // 2 ascii data
00577         .data[19] = 0x02,
00578         .data[18] = 0x31, // 1
00579         .data[17] = 0x37, // 7
00580         .data[16] = 0x01, // list 1 item
00581         .data[15] = 0x01,
00582         .data[14] = 0x01, // list 2 items
00583         .data[13] = 0x02,
00584         .data[12] = 0x41, // 8 ascii data
00585         .data[11] = 0x08,
00586         .data[10] = 0x4c, // L
00587         .data[9] = 0x4f, // O
00588         .data[8] = 0x41, // A
00589         .data[7] = 0x44, // D
00590         .data[6] = 0x4c, // L
00591         .data[5] = 0x4f, // O
00592         .data[4] = 0x43, // C
00593         .data[3] = 0x4b, // K
00594         .data[2] = 0xa5, // 1 byte integer unsigned
00595         .data[1] = 0x01, // length 1
00596         .data[0] = 0x01, // value
00597     },
00598     { // S2F41 'vent/open load-lock ' command from host to equipment
00599         .length = 33,
00600         .block.block.rbit = 0,
00601         .block.block.didh = 0,
00602         .block.block.didl = 0,
00603         .block.block.wbit = 1,
00604         .block.block.stream = 2,
00605         .block.block.function = 41,
00606         .block.block.ebit = 1,
00607         .block.block.bidh = 0,
00608         .block.block.bidl = 1,
00609         .block.block.systemb = 1,
00610         .data[22] = 0x01, // list 2 items
00611         .data[21] = 0x02,
00612         .data[20] = 0x41, // 2 ascii data
00613         .data[19] = 0x02,
00614         .data[18] = 0x31, // 1
00615         .data[17] = 0x32, // 2
00616         .data[16] = 0x01, // list 1 item
00617         .data[15] = 0x01,
00618         .data[14] = 0x01, // list 2 items
00619         .data[13] = 0x02,
00620         .data[12] = 0x41, // 8 ascii data
00621         .data[11] = 0x08,
00622         .data[10] = 0x4c, // L
00623         .data[9] = 0x4f, // O
00624         .data[8] = 0x41, // A
00625         .data[7] = 0x44, // D
00626         .data[6] = 0x4c, // L
00627         .data[5] = 0x4f, // O
00628         .data[4] = 0x43, // C
00629         .data[3] = 0x4b, // K
00630         .data[2] = 0xa5, // 1 byte integer unsigned
00631         .data[1] = 0x01, // length 1
00632         .data[0] = 0x01, // value
00633     },
00634 };
00635
00636 header153 H153[] = {
00637     { // S10F3 send 'terminal text display ' command from host to equipment
00638         .length = 153,

```



```

00639         .block.block.rbit = 0,
00640         .block.block.didh = 0,
00641         .block.block.didl = 0,
00642         .block.block.wbit = 1,
00643         .block.block.stream = 10,
00644         .block.block.function = 3,
00645         .block.block.ebit = 1,
00646         .block.block.bidh = 0,
00647         .block.block.bidl = 1,
00648         .block.block.systemb = 1,
00649         .data[142] = 0x01,
00650         .data[141] = 0x02,
00651         .data[140] = 0x21,
00652         .data[139] = 0x01,
00653         .data[138] = DEFAULT_TID,
00654         .data[137] = 0x41,
00655         .data[136] = 44,
00656         .data[35] = ' ',
00657         .data[34] = 'F',
00658         .data[33] = 'R',
00659         .data[32] = 'E',
00660         .data[31] = 'D',
00661         .data[30] = '1',
00662         .data[29] = '2',
00663         .data[28] = '3',
00664         .data[27] = '0',
00665         .data[26] = '0',
00666         .data[25] = '4',
00667         .data[24] = 'B',
00668         .data[23] = 'B',
00669         .data[22] = 'B',
00670         .data[21] = 'B',
00671         .data[20] = 'R',
00672         .data[19] = 'O',
00673         .data[18] = 'O',
00674         .data[17] = 'K',
00675         .data[16] = 'S',
00676         .data[15] = '1',
00677         .data[14] = '2',
00678         .data[13] = 'E',
00679         .data[12] = 'D',
00680         .data[11] = '1',
00681         .data[10] = '2',
00682         .data[9] = '3',
00683         .data[8] = '4',
00684         .data[7] = 'B',
00685         .data[6] = 'B',
00686         .data[5] = 'B',
00687         .data[4] = 'B',
00688         .data[3] = 'R',
00689         .data[2] = 'O',
00690         .data[1] = 'O',
00691         .data[0] = 'K',
00692     },
00693     { // S10F5 send 'multi-line text display ' command from host to equipment
00694         .length = 153,
00695         .block.block.rbit = 0,
00696         .block.block.didh = 0,
00697         .block.block.didl = 0,
00698         .block.block.wbit = 1,
00699         .block.block.stream = 10,
00700         .block.block.function = 5,
00701         .block.block.ebit = 1,
00702         .block.block.bidh = 0,
00703         .block.block.bidl = 1,
00704         .block.block.systemb = 1,
00705         .data[142] = 0x01,
00706         .data[141] = 0x02,
00707         .data[140] = 0x21,
00708         .data[139] = 0x01,
00709         .data[138] = DEFAULT_TID,
00710         .data[137] = 0x01,
00711         .data[136] = 0x02,
00712         .data[135] = 0x41,
00713         .data[134] = 9,
00714         .data[33] = '*',
00715         .data[32] = 'F',
00716         .data[31] = 'R',
00717         .data[30] = 'E',
00718         .data[29] = 'D',
00719         .data[28] = '1',
00720         .data[27] = '2',
00721         .data[26] = '3',
00722         .data[25] = '4',
00723         .data[24] = 0x41,
00724         .data[23] = 23,
00725         .data[22] = '*',

```

```

00726     .data[21] = 'B',
00727     .data[20] = 'R',
00728     .data[19] = 'K',
00729     .data[18] = 'S',
00730     .data[17] = '1',
00731     .data[16] = '2',
00732     .data[15] = 'E',
00733     .data[14] = 'D',
00734     .data[13] = '1',
00735     .data[12] = '2',
00736     .data[11] = '3',
00737     .data[10] = '4',
00738     .data[9] = 'B',
00739     .data[8] = 'B',
00740     .data[7] = 'B',
00741     .data[6] = 'B',
00742     .data[5] = 'R',
00743     .data[4] = 'O',
00744     .data[3] = 'O',
00745     .data[2] = 'K',
00746     .data[1] = 'S',
00747     .data[0] = 'o',
00748 },
00749 };
00750
00751 header254 H254[] = {
00752     { // general message buffer for parsing
00753         .length = 254,
00754         .block.block.rbit = 1,
00755         .block.block.didh = 0,
00756         .block.block.didl = 0xEF,
00757         .block.block.wbit = 1,
00758         .block.block.stream = 10,
00759         .block.block.function = 3,
00760         .block.block.ebit = 1,
00761         .block.block.bidh = 0,
00762         .block.block.bidl = 1,
00763         .block.block.systemb = 1,
00764         .checksum = 0x2019,
00765         .data[1] = 0x19,
00766         .data[0] = 0x57,
00767     },
00768 };
00769
00770 gem_message_type S[10]; // a queue message stack
00771 gem_display_type D[2]; // equipment display message stack
00772
00773 header10 r_block;
00774
00775 volatile uint16_t tickCount[TMR_COUNT] = {0};
00776 volatile uint8_t mode_sw = false, faker;
00777 void onesec_io(void);
00778
00783 void main(void)
00784 {
00785     UI_STATES mode; /* link configuration host/equipment/etc ... */
00786     char * s, * speed_text;
00787     uint8_t temp_lock = false;
00788
00789     // Initialize the device
00790     SYSTEM_Initialize();
00791
00792     // Enable high priority global interrupts
00793     INTERRUPT_GlobalInterruptHighEnable();
00794
00795     // Enable low priority global interrupts.
00796     INTERRUPT_GlobalInterruptLowEnable();
00797
00798     mconfig_init(); // zero the entire text buffer
00799
00800
00801
00802     V.ui_state = UI_STATE_INIT;
00803     mode = UI_STATE_HOST;
00804
00805     TMR2_StartTimer();
00806     TMR5_SetInterruptHandler(onesec_io);
00807     TMR5_StartTimer();
00808     TMR6_StartTimer();
00809     ADC_SelectContext(CONTEXT_1);
00810
00816     V.speed_spin = DATAEE_ReadByte(UART_SPEED_LOCK_EADR);
00817     V.uart_speed_fast = DATAEE_ReadByte(UART_SPEED_EADR);
00818     DATAEE_WriteByte(UART_SPEED_LOCK_EADR, temp_lock);
00819     DATAEE_WriteByte(UART_SPEED_EADR, V.uart_speed_fast + 1);
00820
00821     if (V.uart_speed_fast % 2 == 0) { // Even/Odd selection for just two speeds

```

```

00822     speed_text = "Locked 9600bps";
00823 } else {
00824     speed_text = "Locked 19200bps";
00825 }
00826 // as soon as you see all LEDS ON, power down, quickly POWER CYCLE to LOCK baud rate
00827 MLED_SetHigh();
00828 RLED_SetHigh();
00829 DLED_SetHigh();
00830 WaitMs(TDELAY);
00831 RLED_SetLow(); // start complete power-up serial speed setups, LEDS OFF
00832 MLED_SetLow();
00833 DLED_SetLow();
00834 temp_lock = true;
00835 if (V.speed_spin) { // update the speed lock status byte
00836     DATAEE_WriteByte(UART_SPEED_LOCK_EADR, temp_lock);
00837 }
00838 DATAEE_WriteByte(UART_SPEED_EADR, V.uart_speed_fast); // update the speed setting byte
00839
00840 if (V.speed_spin) { // serial speed with alternate with every power cycle
00841     /*
00842      * get saved state of serial speed flag
00843      */
00844     V.uart_speed_fast = DATAEE_ReadByte(UART_SPEED_EADR);
00845     if (V.uart_speed_fast == 0xFF) { // programmer fill number
00846         V.uart_speed_fast=0;
00847         DATAEE_WriteByte(UART_SPEED_EADR, V.uart_speed_fast); // start at zero
00848     }
00849     if (V.uart_speed_fast % 2 == 0) {
00850         UART2_Initialize19200();
00851         UART1_Initialize19200();
00852         speed_text = "19200bps";
00853     } else {
00854         UART2_Initialize();
00855         UART1_Initialize();
00856         speed_text = "9600bps";
00857     }
00858     /*
00859      * Alternate the speed setting with each restart
00860      */
00861     DATAEE_WriteByte(UART_SPEED_EADR, ++V.uart_speed_fast);
00862     DATAEE_WriteByte(UART_SPEED_LOCK_EADR, V.speed_spin);
00863 } else { // serial port speed is locked
00864     V.uart_speed_fast = DATAEE_ReadByte(UART_SPEED_EADR); // just read and set the speed setting
00865     if (V.uart_speed_fast % 2 == 0) {
00866         UART2_Initialize();
00867         UART1_Initialize();
00868     } else {
00869         UART2_Initialize19200();
00870         UART1_Initialize19200();
00871     }
00872 }
00873
00874 /*
00875  * master processing I/O loop
00876  */
00877 while (true) {
00878     M_TRACE;
00879     if (!faker++) {
00880 #ifndef FAKER
00881         V.euart = EQUIP_UART;
00882         equip_tx(ENQ); // simulate equipment comm data
00883 #endif
00884     }
00885     /*
00886      * protocol state machine for HOST emulation
00887      */
00888     switch (V.ui_state) {
00889     case UI_STATE_INIT:
00890         init_display();
00891         eaDogM_CursorOff();
00892
00893         set_vterm(V.vterm); // set to buffer 0
00894         snprintf(get_vterm_ptr(0, MAIN_VTERM), MAX_TEXT, "Port %s", speed_text);
00895         snprintf(get_vterm_ptr(1, MAIN_VTERM), MAX_TEXT, "Port %s", speed_text);
00896         snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "Port %s", speed_text);
00897         snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "Port %s", speed_text);
00898         refresh_lcd();
00899         WaitMs(LDELAY);
00900
00901         V.ui_state = mode;
00902         V.s_state = SEQ_STATE_INIT;
00903         srand(1957);
00904         set_vterm(V.vterm); // set to buffer 0
00905         snprintf(V.info, MAX_INFO, " Terminal Info");
00906         snprintf(get_vterm_ptr(0, MAIN_VTERM), MAX_TEXT, " RVI HOST TESTER %u ",
00907             V.uart_speed_fast & 0x01);
00907         snprintf(get_vterm_ptr(1, MAIN_VTERM), MAX_TEXT, " Version %s", VER);

```

```

00908         snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, " NSASPOOK          ");
00909         snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, " %s          ", (char *)
build_date);
00910         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " INFO          ");
00911         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " Version %s          ", VER);
00912         snprintf(get_vterm_ptr(2, INFO_VTERM), MAX_TEXT, " VTERM INFO          ");
00913         snprintf(get_vterm_ptr(3, INFO_VTERM), MAX_TEXT, " %s          ", (char *)
build_date);
00914         snprintf(get_vterm_ptr(0, HELP_VTERM), MAX_TEXT, " HELP Build %s          ", VER);
00915         snprintf(get_vterm_ptr(1, HELP_VTERM), MAX_TEXT, " Version %s          ", VER);
00916         snprintf(get_vterm_ptr(2, HELP_VTERM), MAX_TEXT, " VTERM HELP          ");
00917         snprintf(get_vterm_ptr(3, HELP_VTERM), MAX_TEXT, " %s          ", (char *)
build_date);
00918         snprintf(get_vterm_ptr(0, DEBUG_VTERM), MAX_TEXT, " DEBUG          ");
00919         snprintf(get_vterm_ptr(1, DEBUG_VTERM), MAX_TEXT, " Version %s          ", VER);
00920         snprintf(get_vterm_ptr(2, DEBUG_VTERM), MAX_TEXT, " VTERM DEBUG          ");
00921         snprintf(get_vterm_ptr(3, DEBUG_VTERM), MAX_TEXT, " %s          ", (char *)
build_date);
00922         refresh_lcd();
00923         WaitMs(TDELAY);
00924         StartTimer(TMR_DISPLAY, DDELAY);
00925         StartTimer(TMR_SEQ, 10000);
00926         StartTimer(TMR_INFO, TDELAY);
00927         StartTimer(TMR_FLIPPER, DFLIP);
00928         StartTimer(TMR_HELPDIS, TDELAY);
00929         StartTimer(TMR_SEQ, SEQDELAY);
00930         StartTimer(TMR_HELP, TDELAY);
00931         snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, " UI_STATE_INIT          ");
00932         break;
00933         case UI_STATE_HOST: // equipment starts communications to host
00934 #ifdef FAKER
00935         snprintf(get_vterm_ptr(0, MAIN_VTERM), MAX_TEXT, "FAKER T%lu R%lu          ", V.tx_total,
V.rx_total);
00936 #else
00937 #if defined(DB1) && defined(DB2) && defined(DB3) && defined(DB3)
00938         snprintf(get_vterm_ptr(0, MAIN_VTERM), MAX_TEXT, "1UI_STATE_HOST 2EQIP ");
00939 #else
00940         snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "Equip %u SID %u %c:%c          ",
V.e_types, V.sid, V.rx_rs232, V.tx_rs232);
00941 #endif
00942 #endif
00943         switch (V.s_state) {
00944         case SEQ_STATE_INIT:
00945             V.r_l_state = LINK_STATE_IDLE;
00946             V.t_l_state = LINK_STATE_IDLE;
00947 #ifdef FAKER
00948             V.s_state = SEQ_STATE_TX;
00949 #else
00950             V.s_state = SEQ_STATE_RX;
00951 #endif
00952             if ((V.error == LINK_ERROR_NONE) && (V.abort == LINK_ERROR_NONE)) {
00953                 if (V.debug) {
00954                     snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "H254 %d, T%d          ",
sizeof(header254), V.testing);
00955                     } else {
00956 #ifdef FAKER
00957                     snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "EQUI: %ld G:%s          #",
V.ticks, GEM_TEXT[V.g_state]);
00958 #else
00959                     snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "HOST: %ld G:%s          #",
V.ticks, GEM_TEXT[V.g_state]);
00960 #endif
00961                     }
00962             }
00963 #ifdef DB1
00964             WaitMs(50);
00965             UART1_put_buffer(ENQ);
00966 #endif
00967             break;
00968         case SEQ_STATE_RX:
00969             /*
00970             * receive message from equipment
00971             */
00972             if (r_protocol(&V.r_l_state) == LINK_STATE_DONE) {
00973                 set_display_info(DIS_STR);
00974                 s = get_vterm_ptr(0, MAIN_VTERM);
00975                 if (V.stream == 9) { // error message from equipment
00976                     V.msg_error = V.function;
00977                     snprintf(s, MAX_TEXT, " S%dF%d Err %d          ", V.stream, V.function,
V.all_errors);
00978                 } else {
00979                     V.msg_error = MSG_ERROR_NONE;
00980                     snprintf(s, MAX_TEXT, " S%dF%d # Rx %d          ", V.stream, V.function,
V.all_errors);
00981                 }
00982                 s[MAX_LINE] = 0;
00983                 s[SPIN_CHAR] = spinners(3, false);

```

```

00984 #ifdef DB1
00985         WaitMs(5);
00986 #endif
00987         if (V.wbit) { // check for receive only messages
00988             V.s_state = SEQ_STATE_TX;
00989             V.failed_send = SEND_ERROR_NONE;
00990             V.t_l_state = LINK_STATE_IDLE;
00991         } else { // don't send a reply
00992             V.s_state = SEQ_STATE_TRIGGER;
00993         }
00994     }
00995     if (V.r_l_state == LINK_STATE_ERROR)
00996         V.s_state = SEQ_STATE_ERROR;
00997     break;
00998 case SEQ_STATE_TX:
00999     snprintf(get_vterm_ptr(3, MAIN_VTERM), MAX_TEXT, "SEQ_STATE_TX");
01000     // eaDogM_WriteStringAtPos(3, 0, "SEQ_STATE_TX");
01001     /*
01002     * send response message to equipment
01003     */
01004     if (t_protocol(&V.t_l_state) == LINK_STATE_DONE) {
01005 #ifdef FAKER
01006         V.s_state = SEQ_STATE_RX;
01007 #else
01008         V.s_state = SEQ_STATE_TRIGGER;
01009 #endif
01010     }
01011     if (V.t_l_state == LINK_STATE_ERROR)
01012         V.s_state = SEQ_STATE_ERROR;
01013     break;
01014 case SEQ_STATE_TRIGGER:
01015     set_display_info(DIS_STR);
01016     s = get_vterm_ptr(0, MAIN_VTERM);
01017     if (V.queue) {
01018         V.r_l_state = LINK_STATE_IDLE;
01019         V.t_l_state = LINK_STATE_IDLE;
01020         V.s_state = SEQ_STATE_TX;
01021         snprintf(s, MAX_TEXT, "S%dF%d # OK %d Q Tx %lu", V.stream, V.function,
01022             V.e_types, V.tx_total);
01023     } else {
01024         V.s_state = SEQ_STATE_DONE;
01025         snprintf(s, MAX_TEXT, "S%dF%d # OK %d Tx %lu", V.stream, V.function,
01026             V.e_types, V.tx_total);
01027     }
01028     s[MAX_LINE] = 0;
01029     s[SPIN_CHAR] = spinners(3, false);
01030     break;
01031 case SEQ_STATE_DONE:
01032     V.s_state = SEQ_STATE_INIT;
01033     break;
01034 case SEQ_STATE_ERROR:
01035     default:
01036         snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "SEQ_STATE_ERROR");
01037         V.s_state = SEQ_STATE_INIT;
01038         snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "E%d A%d T%d G:%s #", V.error,
01039             V.abort, V.timer_error, GEM_TEXT[V.g_state]);
01040         refresh_lcd();
01041         WaitMs(TDELAY);
01042         break;
01043     }
01044     if ((V.error == LINK_ERROR_NONE) && (V.abort == LINK_ERROR_NONE)) {
01045         if (TimerDone(TMR_DISPLAY)) { // limit update rate
01046             if (V.debug) {
01047                 snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "CEID %d, Mesg %c%c %d",
01048                     V.response.ceid, V.response.ack[7], V.response.ack[8], (uint8_t) V.response.ack[6]);
01049             } else {
01050 #ifdef FAKER
01051                 snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "EQUI: %ld G:%s", V.ticks, GEM_TEXT[V.g_state]);
01052             #else
01053                 snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "HOST: %ld G:%s", V.ticks, GEM_TEXT[V.g_state]);
01054             #endif
01055         }
01056     }
01057     /*
01058     * HeartBeat ping or sequence during idle times
01059     */
01060     if ((V.g_state == GEM_STATE_REMOTE) && (V.s_state == SEQ_STATE_RX) && !V.queue) {
01061         if ((V.r_l_state == LINK_STATE_IDLE) && (V.t_l_state == LINK_STATE_IDLE)) {
01062             if (TimerDone(TMR_HBIO)) {
01063                 set_display_info(DIS_STR); // reset display configuration
01064                 // send ping or sequence message
01065                 if (V.stack > 0) {
01066                     hb_message(); // prime the TX state machine
01067                     V.msg_error = MSG_ERROR_NONE;
01068                 }
01069             }
01070         }
01071     }

```

```

01065             V.ping_count = 0;
01066             V.error = LINK_ERROR_NONE;
01067             V.abort = LINK_ERROR_NONE;
01068         } else {
01069             StartTimer(TMR_HBIO, HBTI);
01070             if (V.ping_count++ > 4) {
01071                 set_display_info(DIS_STR);
01072                 hb_message();
01073                 snprintf(get_vterm_ptr(0, MAIN_VTERM), MAX_TEXT, "Ping P%d RTO %d
TX %lu      ", V.g_state, V.equip_timeout, V.tx_total);
01074                 refresh_lcd();
01075                 WaitMs(BDELAY);
01076                 V.ping_count = 0;
01077             }
01078             set_display_info(DIS_STR);
01079         }
01080     }
01081 }
01082 }
01083 }
01084 break;
01085 case UI_STATE_LOG: // monitor
01086     switch (V.s_state) {
01087     case SEQ_STATE_INIT:
01088         V.m_l_state = LINK_STATE_IDLE;
01089         V.s_state = SEQ_STATE_RX;
01090         if (V.debug)
01091             snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "CEID %d, Mesg %c%c %d      ",
V.response.ceid, V.response.ack[7], V.response.ack[8], (uint8_t) V.response.ack[6]);
01092     else
01093         snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "LOG: U%d G%d %d %d      #",
V.uart, V.g_state, V.timer_error, V.checksum_error);
01094
01095 #ifdef DB1
01096         if (SLED) {
01097             UART2_put_buffer(ENQ);
01098         } else {
01099             UART1_put_buffer(ENQ);
01100         }
01101 #endif
01102         break;
01103     case SEQ_STATE_RX:
01104         /*
01105          * receive rx and tx messages from comm link
01106          */
01107         if (m_protocol(&V.m_l_state) == LINK_STATE_DONE) {
01108             snprintf(get_vterm_ptr(V.uart - 1, 0), MAX_TEXT, "S%dF%d #%ld OK      ",
V.stream, V.function, V.ticks);
01109             V.s_state = SEQ_STATE_TRIGGER;
01110         }
01111         if (V.m_l_state == LINK_STATE_ERROR)
01112             V.s_state = SEQ_STATE_ERROR;
01113         break;
01114     case SEQ_STATE_TRIGGER:
01115         V.s_state = SEQ_STATE_DONE;
01116         break;
01117     case SEQ_STATE_DONE:
01118     case SEQ_STATE_ERROR:
01119     default:
01120         V.s_state = SEQ_STATE_INIT;
01121         break;
01122     }
01123     if (V.debug)
01124         snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "Equip type %d      ",
V.e_types);
01125     else
01126         snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "LOG: U%d G%d %d %d      #", V.uart,
V.g_state, V.timer_error, V.checksum_error);
01127     break;
01128     case UI_STATE_ERROR:
01129     default:
01130         V.ui_state = UI_STATE_INIT;
01131         break;
01132     }
01133     if (V.ticks) {
01134         if (V.failed_receive != RECV_ERROR_NONE) {
01135             if (V.error == LINK_ERROR_CHECKSUM) {
01136             }
01137         } else {
01138         }
01139         if (V.failed_send != SEND_ERROR_NONE) {
01140             if (V.error == LINK_ERROR_CHECKSUM) {
01141             }
01142         } else {
01143         }
01144     }
01145 }

```

```

01146         if (mode != UI_STATE_LOG) {
01147             if (TimerDone(TMR_DISPLAY)) { // limit update rate
01148                 static uint8_t switcher = INFO_VTERM;
01149                 if (TimerDone(TMR_HELPDIS)) {
01150                     set_display_info(DIS_STR);
01151                 }
01152                 snprintf(get_vterm_ptr(1, MAIN_VTERM), MAX_TEXT, "%d %d T%d %d C%d S%d      #",
01153                     V.r_l_state, V.failed_receive, V.t_l_state, V.failed_send, V.checksum_error, V.stack);
01154                 ADC_DischargeSampleCapacitor();
01155                 ADC_StartConversion(channel_ANA1);
01156                 while (!ADC_IsConversionDone()) {
01157                     };
01158                 if (ADC_IsConversionDone()) {
01159                     V.v_tx_line = ADC_GetConversionResult();
01160                 };
01161                 ADC_DischargeSampleCapacitor();
01162                 ADC_StartConversion(channel_ANA2);
01163                 while (!ADC_IsConversionDone()) {
01164                     };
01165                 if (ADC_IsConversionDone()) {
01166                     V.v_rx_line = ADC_GetConversionResult();
01167                 };
01168                 // convert ADC values to char for display
01169                 update_rs232_line_status();
01170                 StartTimer(TMR_DISPLAY, DDELAY);
01171                 if (V.vterm_switch++ > (SWITCH_VTERM)) {
01172                     set_vterm(switcher);
01173                     if (V.vterm_switch > (SWITCH_VTERM + V.ticker + SWITCH_DURATION)) {
01174                         switcher++;
01175                         if ((switcher & 0x03) == HELP_VTERM) { // mask [0..3]
01176                             switcher = INFO_VTERM; // skip short display of the main vterm
01177                         }
01178                         V.vterm_switch = 0;
01179                     }
01180                 } else {
01181                     set_vterm(V.vterm);
01182                 }
01183                 /*
01184                 * update info screen data points
01185                 */
01186                 snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, "I RS %3dV:%c %3dV:%c
01187 ", V.rx_volts, V.rx_rs232, V.tx_volts, V.tx_rs232);
01188                 snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, "RXB %lu NAK %lu
01189 ", V.rx_total, V.brn_total);
01190                 snprintf(get_vterm_ptr(2, INFO_VTERM), MAX_TEXT, "TXB %lu NAK %lu
01191 ", V.tx_total, V.btn_total);
01192                 snprintf(get_vterm_ptr(3, INFO_VTERM), MAX_TEXT, "Seq %lu Blks R%lu T%lu
01193 ", V.ticks, V.bt_total, V.br_total);
01194                 snprintf(get_vterm_ptr(0, DEBUG_VTERM), MAX_TEXT, "D S%uF%u SB%lu %d%d%d
01195 ", V.stream, V.function, V.systemb, V.rbit, V.wbit, V.ebit);
01196                 snprintf(get_vterm_ptr(1, DEBUG_VTERM), MAX_TEXT, "RX CKSUM 0x%04X
01197 ", V.r_checksum);
01198                 snprintf(get_vterm_ptr(2, DEBUG_VTERM), MAX_TEXT, "%s
01199 ", V.info);
01200                 snprintf(get_vterm_ptr(3, DEBUG_VTERM), MAX_TEXT, "Hlp %u Dbg %u Rer %u
01201 ", V.help, V.debug, V.error);
01202
01203                 /*
01204                 * don't default update the LCD when displaying HELP text
01205                 */
01206                 if (!V.set_sequ) {
01207                     refresh_lcd();
01208 #ifdef FRAME_OVERRUN
01209                     UART1_Write(0x19);
01210                     UART2_Write(0x57);
01211 #endif
01212                 }
01213             }
01214         }
01215     }
01216
01217     /*
01218     * show help messages if flag is set for timer duration
01219     */
01220     if (V.set_sequ) {
01221         if (TimerDone(TMR_HELP)) {
01222             V.set_sequ = false;
01223             set_vterm(V.vterm);
01224             refresh_lcd();
01225         } else {
01226             set_vterm(HELP_VTERM);
01227             refresh_lcd();
01228         }
01229     }
01230
01231     if (V.help && TimerDone(TMR_SEQ)) {
01232         StartTimer(TMR_SEQ, SEQDELAY);
01233     }

```

```

01224         StartTimer(TMR_HELP, TDELAY);
01225         V.set_sequ = true;
01226         check_help(false);
01227     }
01228     M_TRACE;
01229 }
01230 }
01231
01232 /*
01233  * run LED and status indicators
01234  */
01235 void onesec_io(void)
01236 {
01237     RLED_Toggle();
01238     MLED_SetLow();
01239     DLED_SetLow();
01240     B.one_sec_flag = true;
01241 }
01242
01243 /* Misc ACSII spinner character generator, stores position for each shape */
01244 char spinners(uint8_t shape, const uint8_t reset)
01245 {
01246     static uint8_t s[MAX_SHAPES];
01247     char c;
01248
01249     if (shape > (MAX_SHAPES - 1))
01250         shape = 0;
01251     if (reset)
01252         s[shape] = 0;
01253     c = spin[shape][s[shape]];
01254     if (++s[shape] >= strlen(spin[shape]))
01255         s[shape] = 0;
01256
01257     return c;
01258 }
01259

```

6.32 adc.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "adc.h"
00053
00054 //Pointers to ADC interrupt handlers
00055 //User can use them in application code to initialize with custom ISRs
00056 static void (*ADC_ConversionComplete_ISR)(void);
00057 static void (*ADC_Context1Threshld_ISR)(void);
00058 static void (*ADC_ActiveClockTuning_ISR)(void);
00059
00060 static void ADC_DefaultADI_ISR(void);
00061 static void ADC_DefaultContext1Threshold_ISR(void);
00062 static void ADC_DefaultActiveClockTuning_ISR(void);
00063
00064
00065 void ADC_Initialize(void)
00066 {

```



```

00067 //ADACT disabled;
00068 ADACT = 0x00;
00069
00070 //ADCCS FOSC/2;
00071 ADCLK = 0x00;
00072
00073 //ADC charge pump control
00074 ADCP = 0x00;
00075
00076
00077 /*****
00078 *          Configure Context-1          *
00079 *****/
00080 ADCTX = 0x0;
00081
00082 //ADLTHL 0;
00083 ADLTHL = 0x00;
00084
00085 //ADLTHH 0;
00086 ADLTHH = 0x00;
00087
00088 //ADUTHL 0;
00089 ADUTHL = 0x00;
00090
00091 //ADUTHH 0;
00092 ADUTHH = 0x00;
00093
00094 //ADSTPTL 0;
00095 ADSTPTL = 0x00;
00096
00097 //ADSTPTH 0;
00098 ADSTPTH = 0x00;
00099
00100 //ADACCL 0;
00101 ADACCL = 0x00;
00102
00103 //ADACCH 0;
00104 ADACCH = 0x00;
00105
00106 //ADACCU 0;
00107 ADACCU = 0x00;
00108
00109 //ADCNT 0;
00110 ADCNT = 0x00;
00111
00112 //ADRPT 0;
00113 ADRPT = 0x00;
00114
00115 //ADRESL 0;
00116 ADRESL = 0x00;
00117
00118 //ADRESH 0;
00119 ADRESH = 0x00;
00120
00121 //ADCHS ANA1;
00122 ADPCH = 0x01;
00123
00124 //ADACQL 1;
00125 ADACQL = 0x01;
00126
00127 //ADACQH 0;
00128 ADACQH = 0x00;
00129
00130 //ADCAP Additional uC disabled;
00131 ADCAP = 0x00;
00132
00133 //ADPREL 0;
00134 ADPREL = 0x00;
00135
00136 //ADPREH 0;
00137 ADPREH = 0x00;
00138
00139 //ADCONT disabled;
00140 ADCON0 = 0x00;
00141
00142 //ADPPOL Vss; ADIPEN disabled; ADGPOL digital_low; ADDSEN disabled;
00143 ADCON1 = 0x00;
00144
00145 //ADPSIS RES; ADCRS 1; ADACLR disabled; ADMD Basic_mode;
00146 ADCON2 = 0x10;
00147
00148 //ADCALC First derivative of Single measurement; ADSOI ADGO not cleared; ADTMD disabled;
00149 ADCON3 = 0x00;
00150
00151 //ADMATH registers not updated;
00152 ADSTAT = 0x00;
00153

```

```

00154     //ADNREF VSS; ADPREF VDD;
00155     ADREF = 0x00;
00156
00157     //CHEN channel content is not included; SSI scan sequence continues;
00158     ADCSEL1 = 0x00;
00159
00160     //Clear ADC Interrupt Flag
00161     PIR1bits.ADIF = 0;
00162
00163     //Clear ADC active clock tuning interrupt flag
00164     PIR1bits.ACTIF = 0;
00165
00166     //Clear ADC Context Threshold Interrupt Flag
00167     PIR2bits.ADCH1IF = 0;
00168
00169
00170     //Configure interrupt handlers
00171     ADC_SetADIIInterruptHandler(ADC_DefaultADI_ISR);
00172     ADC_SetContext1ThresholdInterruptHandler(ADC_DefaultContext1Threshold_ISR);
00173     ADC_SetActiveClockTuningInterruptHandler(ADC_DefaultActiveClockTuning_ISR);
00174
00175     //ADON enabled; CSEN disabled; ADCS Frc; ADFM right; GO_nDONE stop;
00176     ADCON0 = 0x94;
00177 }
00178
00179 inline void ADC_EnableChannelSequencer(void)
00180 {
00181     ADCON0bits.CSEN = 1;
00182 }
00183
00184 inline void ADC_DisableChannelSequencer(void)
00185 {
00186     ADCON0bits.CSEN = 0;
00187 }
00188
00189 inline void ADC_StartChannelSequencer(void)
00190 {
00191     ADCON0bits.GO = 1;
00192 }
00193
00194 inline void ADC_SelectContext(ADC_context_t context)
00195 {
00196     ADCTX = context;
00197 }
00198
00199 void ADC_EnableChannelScan(ADC_context_t context)
00200 {
00201     switch (context)
00202     {
00203     case CONTEXT_1:
00204         ADCSEL1bits.CHEN = 1;
00205         break;
00206     default:
00207         break;
00208     }
00209 }
00210
00211 void ADC_DisableChannelScan(ADC_context_t context)
00212 {
00213     switch (context)
00214     {
00215     case CONTEXT_1:
00216         ADCSEL1bits.CHEN = 0;
00217         break;
00218     default:
00219         break;
00220     }
00221 }
00222
00223 void ADC_StartConversion(ADC_channel_t channel)
00224 {
00225     //Select the A/D channel
00226     ADPCH = channel;
00227
00228     //Turn ON the ADC module
00229     ADCON0bits.ON = 1;
00230
00231     //Start the conversion
00232     ADCON0bits.GO = 1;
00233 }
00234
00235 bool ADC_IsConversionDone(void)
00236 {
00237     return (bool) (!ADCON0bits.GO);
00238 }
00239
00240 adc_result_t ADC_GetConversionResult(void)

```

```

00241 {
00242     //Return result of A/D conversion
00243     return ((adc_result_t) ((ADRESH << 8) + ADRESL));
00244 }
00245
00246 adc_result_t ADC_GetSingleConversion(ADC_channel_t channel)
00247 {
00248     //Select the A/D channel
00249     ADPCH = channel;
00250
00251     //Turn ON the ADC module
00252     ADCON0bits.ON = 1;
00253
00254     //Disable the continuous mode
00255     ADCON0bits.CONT = 0;
00256
00257     //Start the conversion
00258     ADCON0bits.GO = 1;
00259
00260     //Wait for the conversion to finish
00261     while(ADCON0bits.GO)
00262     {
00263     }
00264 }
00265
00266     return ((adc_result_t) ((ADRESH << 8) + ADRESL));
00267 }
00268
00269 inline void ADC_StopConversion(void)
00270 {
00271     //Reset the ADGO bit to manually stop conversion
00272     ADCON0bits.GO = 0;
00273 }
00274
00275 inline void ADC_SetStopOnInterrupt(void)
00276 {
00277     ADCON3bits.SOI = 1;
00278 }
00279
00280 inline void ADC_DischargeSampleCapacitor(void)
00281 {
00282     //Set ADC channel to AVss
00283     ADPCH = 0x3b;
00284 }
00285
00286 void ADC_LoadAcquisitionRegister(uint16_t acquisitionValue)
00287 {
00288     ADACQH = acquisitionValue >> 8;
00289     ADACQL = acquisitionValue;
00290 }
00291
00292 void ADC_SetPrechargeTime(uint16_t prechargeTime)
00293 {
00294     ADPREH = prechargeTime >> 8;
00295     ADPREL = prechargeTime;
00296 }
00297
00298 inline void ADC_SetRepeatCount(uint8_t repeatCount)
00299 {
00300     ADRPT = repeatCount;
00301 }
00302
00303 uint8_t ADC_GetCurrentCountofConversions(void)
00304 {
00305     return ADCNT;
00306 }
00307
00308 inline void ADC_ClearAccumulator(void)
00309 {
00310     ADCON2bits.ACLR = 1;
00311 }
00312
00313 uint24_t ADC_GetAccumulatorValue(void)
00314 {
00315     return ((uint24_t) ((ADACCH << 8) + ADACCL));
00316 }
00317
00318 bool ADC_HasAccumulatorOverflowed(void)
00319 {
00320     return ADSTATbits.ADAOV;
00321 }
00322
00323 uint16_t ADC_GetFilterValue(void)
00324 {
00325     return ((uint16_t) ((ADFLTRH << 8) + ADFLTRL));
00326 }
00327

```

```

00328 uint16_t ADC_GetPreviousResult(void)
00329 {
00330     return ((uint16_t) ((ADPREVH « 8) + ADPREVL));
00331 }
00332
00333 void ADC_DefineSetPoint(uint16_t setPoint)
00334 {
00335     ADSTPTH = setPoint « 8;
00336     ADSTPTL = setPoint;
00337 }
00338
00339 void ADC_SetUpperThreshold(uint16_t upperThreshold)
00340 {
00341     ADUTHH = upperThreshold « 8;
00342     ADUTHL = upperThreshold;
00343 }
00344
00345 void ADC_SetLowerThreshold(uint16_t lowerThreshold)
00346 {
00347     ADLTHH = lowerThreshold « 8;
00348     ADLTHL = lowerThreshold;
00349 }
00350
00351 uint16_t ADC_GetErrorCalculation(void)
00352 {
00353     return ((uint16_t) ((ADERRH « 8) + ADERRL));
00354 }
00355
00356 inline void ADC_EnableDoubleSampling(void)
00357 {
00358     ADCON1bits.DSEN = 1;
00359 }
00360
00361 inline void ADC_EnableContinuousConversion(void)
00362 {
00363     ADCON0bits.CONT = 1;
00364 }
00365
00366 inline void ADC_DisableContinuousConversion(void)
00367 {
00368     ADCON0bits.CONT = 0;
00369 }
00370
00371 bool ADC_HasErrorCrossedUpperThreshold(void)
00372 {
00373     return ADSTATbits.ADUTHR;
00374 }
00375
00376 bool ADC_HasErrorCrossedLowerThreshold(void)
00377 {
00378     return ADSTATbits.ADLTHR;
00379 }
00380
00381 uint8_t ADC_GetConversionStageStatus(void)
00382 {
00383     return ADSTATbits.ADSTAT;
00384 }
00385
00386 inline void ADC_EnableChargePump(void)
00387 {
00388     ADCPbits.CPON = 1;
00389 }
00390
00391 inline void ADC_DisableChargePump(void)
00392 {
00393     ADCPbits.CPON = 0;
00394 }
00395
00396 void __interrupt(irq(AD),base(8)) ADC_ADI_ISR(void)
00397 {
00398     PIR1bits.ADIF = 0;
00399     if (ADC_ConversionComplete_ISR != NULL)
00400         ADC_ConversionComplete_ISR();
00401 }
00402
00403 void __interrupt(irq(ACT),base(8)) ADC_ACTI_ISR(void)
00404 {
00405     PIR1bits.ACTIF = 0;
00406     if (ADC_ActiveClockTuning_ISR != NULL)
00407         ADC_ActiveClockTuning_ISR();
00408 }
00409
00410
00411 void __interrupt(irq(ADCH1),base(8)) ADC_ADCH1_ISR(void)
00412 {
00413     PIR2bits.ADCH1IF = 0;
00414     if (ADC_Context1Threshld_ISR != NULL)

```

```

00415         ADC_Context1Thereshld_ISR();
00416     }
00417
00418 void ADC_SetADIIInterruptHandler(void (* InterruptHandler)(void))
00419 {
00420     ADC_ConversionComplete_ISR = InterruptHandler;
00421 }
00422
00423 void ADC_SetContext1ThresholdInterruptHandler(void (* InterruptHandler)(void))
00424 {
00425     ADC_Context1Thereshld_ISR = InterruptHandler;
00426 }
00427
00428 void ADC_SetActiveClockTuningInterruptHandler(void (* InterruptHandler)(void))
00429 {
00430     ADC_ActiveClockTuning_ISR = InterruptHandler;
00431 }
00432
00433 static void ADC_DefaultADI_ISR(void)
00434 {
00435     //Add your interrupt code here or
00436     //Use ADC_SetADIIInterruptHandler() function to use Custom ISR
00437 }
00438
00439
00440 static void ADC_DefaultContext1Threshold_ISR(void)
00441 {
00442     //Add your interrupt code here or
00443     //Use ADC_SetContext1ThresholdInterruptHandler() function to use Custom ISR
00444 }
00445
00446 static void ADC_DefaultActiveClockTuning_ISR(void)
00447 {
00448     //Add your interrupt code here or
00449     //Use ADC_SetActiveClockTuningInterruptHandler() function to use Custom ISR
00450 }
00451

```

6.33 adc.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef ADC_H
00048 #define ADC_H
00049
00055
00056 #include <xc.h>
00057 #include <stdint.h>
00058 #include <stdbool.h>
00059
00065 typedef uint16_t adc_result_t;
00066
00072 typedef enum
00073 {
00074     channel_ANA1 = 0x1,
00075     channel_ANA2 = 0x2,
00076     channel_VSS = 0x3B,
00077     channel_Temp = 0x3C,

```

```
00078     channel_DAC1 = 0x3D,
00079     channel_FVR_Buffer1 = 0x3E,
00080     channel_FVR_Buffer2 = 0x3F
00081 } ADC_channel_t;
00082
00083 typedef enum
00084 {
00085     CONTEXT_1,
00086 } ADC_context_t;
00087
00088 void ADC_Initialize(void);
00089
00090 inline void ADC_EnableChannelSequencer(void);
00091
00092 inline void ADC_DisableChannelSequencer(void);
00093
00094 inline void ADC_StartChannelSequencer(void);
00095
00096 inline void ADC_SelectContext(ADC_context_t context);
00097
00098 void ADC_EnableChannelScan(ADC_context_t context);
00099
00100 void ADC_DisableChannelScan(ADC_context_t context);
00101
00102 void ADC_StartConversion(ADC_channel_t channel);
00103
00104 bool ADC_IsConversionDone(void);
00105
00106 adc_result_t ADC_GetConversionResult(void);
00107
00108 adc_result_t ADC_GetSingleConversion(ADC_channel_t channel);
00109
00110 inline void ADC_StopConversion(void);
00111
00112 inline void ADC_SetStopOnInterrupt(void);
00113
00114 inline void ADC_DischargeSampleCapacitor(void);
00115
00116 void ADC_LoadAcquisitionRegister(uint16_t acquisitionValue);
00117
00118 void ADC_SetPrechargeTime(uint16_t prechargeTime);
00119
00120 inline void ADC_SetRepeatCount(uint8_t repeatCount);
00121
00122 uint8_t ADC_GetCurrentCountofConversions(void);
00123
00124 inline void ADC_ClearAccumulator(void);
00125
00126 uint24_t ADC_GetAccumulatorValue(void);
00127
00128 bool ADC_HasAccumulatorOverflowed(void);
00129
00130 uint16_t ADC_GetFilterValue(void);
00131
00132 uint16_t ADC_GetPreviousResult(void);
00133
00134 void ADC_DefineSetPoint(uint16_t setPoint);
00135
00136 void ADC_SetUpperThreshold(uint16_t upperThreshold);
00137
00138 void ADC_SetLowerThreshold(uint16_t lowerThreshold);
00139
00140 uint16_t ADC_GetErrorCalculation(void);
00141
00142 inline void ADC_EnableDoubleSampling(void);
00143
00144 inline void ADC_EnableContinuousConversion(void);
00145
00146 inline void ADC_DisableContinuousConversion(void);
00147
00148 bool ADC_HasErrorCrossedUpperThreshold(void);
00149
00150 bool ADC_HasErrorCrossedLowerThreshold(void);
00151
00152 uint8_t ADC_GetConversionStageStatus(void);
00153
00154 inline void ADC_EnableChargePump(void);
00155
00156 inline void ADC_DisableChargePump(void);
00157
00158 void ADC_SetADIIInterruptHandler(void (* InterruptHandler)(void));
00159
00160 void ADC_SetActiveClockTuningInterruptHandler(void (* InterruptHandler)(void));
00161
```

```

01045 void ADC_SetContext1ThresholdInterruptHandler(void (* InterruptHandler)(void));
01046
01047
01074 void ADC_SetContext2ThresholdInterruptHandler(void (* InterruptHandler)(void));
01075
01076
01103 void ADC_SetContext3ThresholdInterruptHandler(void (* InterruptHandler)(void));
01104
01105
01132 void ADC_SetContext4ThresholdInterruptHandler(void (* InterruptHandler)(void));
01133
01137 #endif //ADC_H

```

6.34 clc1.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "clc1.h"
00053
00057
00058 void CLC1_Initialize(void)
00059 {
00060     // Set the CLC1 to the options selected in the User Interface
00061
00062     // LC1G1POL not_inverted; LC1G2POL not_inverted; LC1G3POL not_inverted; LC1G4POL not_inverted;
00062     LC1POL not_inverted;
00063     CLC1POL = 0x00;
00064     // LC1D1S FOSC;
00065     CLC1SEL0 = 0x04;
00066     // LC1D2S FOSC;
00067     CLC1SEL1 = 0x04;
00068     // LC1D3S FOSC;
00069     CLC1SEL2 = 0x04;
00070     // LC1D4S FOSC;
00071     CLC1SEL3 = 0x04;
00072     // LC1G1D3N disabled; LC1G1D2N disabled; LC1G1D4N disabled; LC1G1D1T disabled; LC1G1D3T disabled;
00072     LC1G1D2T disabled; LC1G1D4T disabled; LC1G1D1N disabled;
00073     CLC1GLS0 = 0x00;
00074     // LC1G2D2N disabled; LC1G2D1N disabled; LC1G2D4N disabled; LC1G2D3N disabled; LC1G2D2T disabled;
00074     LC1G2D1T disabled; LC1G2D4T disabled; LC1G2D3T disabled;
00075     CLC1GLS1 = 0x00;
00076     // LC1G3D1N disabled; LC1G3D2N disabled; LC1G3D3N disabled; LC1G3D4N disabled; LC1G3D1T disabled;
00076     LC1G3D2T disabled; LC1G3D3T disabled; LC1G3D4T disabled;
00077     CLC1GLS2 = 0x00;
00078     // LC1G4D1N disabled; LC1G4D2N disabled; LC1G4D3N disabled; LC1G4D4N disabled; LC1G4D1T disabled;
00078     LC1G4D2T disabled; LC1G4D3T disabled; LC1G4D4T disabled;
00079     CLC1GLS3 = 0x00;
00080     // LC1EN enabled; INTN disabled; INTP disabled; MODE OR-XOR;
00081     CLC1CON = 0x81;
00082
00083 }
00084
00085
00086 bool CLC1_OutputStatusGet(void)
00087 {
00088     return (CLC1CONbits.LC1OUT);
00089 }

```

6.35 clc1.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029 party
00029 license terms applicable to your use of third party software (including open source software) that
00030
00030 may accompany Microchip software.
00031
00032 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035 FOR A PARTICULAR PURPOSE.
00036
00037 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042 CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043 OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044 SOFTWARE.
00045 */
00046
00047 #ifndef CLC1_H
00048 #define CLC1_H
00049
00053
00054 #include <xc.h>
00055 #include <stdint.h>
00056 #include <stdbool.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059     extern "C" {
00060
00061
00062 #endif
00063
00067
00091 void CLC1_Initialize(void);
00092
00112
00113 bool CLC1_OutputStatusGet(void);
00114
00115 #ifdef __cplusplus // Provide C++ Compatibility
00116     }
00117
00118 #endif
00119 #endif
00120
00121 #endif // CLC1_H
00125

```

6.36 clkref.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029 party
00029 license terms applicable to your use of third party software (including open source software) that
00030
00030 may accompany Microchip software.
00031
00032 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035 FOR A PARTICULAR PURPOSE.
00036
00037 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL

```



```

00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "clkref.h"
00053
00057
00058 void CLKREF_Initialize(void)
00059 {
00060     // CLK FOSC;
00061     CLKRCLK = 0x00;
00062     // DC 50% Duty Cycle; DIV BaseClock/16; CLKREN enabled;
00063     CLKRCON = 0x94;
00064 }

```

6.37 clkref.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef CLKREF_H
00048 #define CLKREF_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00066
00092 void CLKREF_Initialize(void);
00093
00094 #ifdef __cplusplus // Provide C++ Compatibility
00095     }
00096
00097 #endif
00098 #endif
00099
00100 #endif // CLKREF_H
00104
00105

```

6.38 device_config.c

```

00001
00023
00024 /*

```

```

00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
      license terms applicable to your use of third party software (including open source software) that
00030      may accompany Microchip software.
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035      FOR A PARTICULAR PURPOSE.
00036
00037      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044      SOFTWARE.
00045  */
00046
00047  // Configuration bits: selected in the GUI
00048
00049  // CONFIG1
00050  #pragma config FEXTOSC = ECH      // External Oscillator Selection->EC (external clock) above 8 MHz
00051  #pragma config RSTOSC = EXTOSC_4PLL // Reset Oscillator Selection->EXTOSC with 4x PLL, with EXTOSC
      operating per FEXTOSC bits
00052
00053  // CONFIG2
00054  #pragma config CLKOUTEN = ON      // Clock out Enable bit->CLKOUT function is enabled
00055  #pragma config PR1WAY = OFF      // PRLOCKED One-Way Set Enable bit->PRLOCKED bit can be set and cleared
      repeatedly
00056  #pragma config CSWEN = ON      // Clock Switch Enable bit->Writing to NOSC and NDIV is allowed
00057  #pragma config JTAGEN = OFF      // JTAG Enable bit->Disable JTAG Boundary Scan mode, JTAG pins revert
      to user functions
00058  #pragma config FCMEN = ON      // Fail-Safe Clock Monitor Enable bit->Fail-Safe Clock Monitor enabled
00059  #pragma config FCMENP = ON      // Fail-Safe Clock Monitor -Primary XTAL Enable bit->FSCM timer will set
      FSCMP bit and OSFIF interrupt on Primary XTAL failure
00060  #pragma config FCMENS = ON      // Fail-Safe Clock Monitor -Secondary XTAL Enable bit->FSCM timer will
      set FSCMS bit and OSFIF interrupt on Secondary XTAL failure
00061
00062  // CONFIG3
00063  #pragma config MCLRE = EXTMCLR    // MCLR Enable bit->If LVP = 0, MCLR pin is MCLR; If LVP = 1, RE3
      pin function is MCLR
00064  #pragma config PWRTS = PWRT_OFF    // Power-up timer selection bits->PWRT is disabled
00065  #pragma config MVEECN = ON      // Multi-vector enable bit->Multi-vector enabled, Vector table used for
      interrupts
00066  #pragma config IVT1WAY = ON      // IVTLOCK bit One-way set enable bit->IVTLOCKED bit can be cleared and
      set only once
00067  #pragma config LPBORN = OFF      // Low Power BOR Enable bit->Low-Power BOR disabled
00068  #pragma config BOREN = SBORDIS    // Brown-out Reset Enable bits->Brown-out Reset enabled , SBORN bit
      is ignored
00069
00070  // CONFIG4
00071  #pragma config BORV = VBOR_1P9    // Brown-out Reset Voltage Selection bits->Brown-out Reset Voltage
      (VBOR) set to 1.9V
00072  #pragma config ZCD = OFF      // ZCD Disable bit->ZCD module is disabled. ZCD can be enabled by setting
      the ZCDSEN bit of ZCDCON
00073  #pragma config PPS1WAY = OFF      // PPSLOCK bit One-Way Set Enable bit->PPSLOCKED bit can be set and
      cleared repeatedly (subject to the unlock sequence)
00074  #pragma config STVREN = ON      // Stack Full/Underflow Reset Enable bit->Stack full/underflow will
      cause Reset
00075  #pragma config LVP = OFF      // Low Voltage Programming Enable bit->HV on MCLR/VPP must be used for
      programming
00076  #pragma config XINST = OFF      // Extended Instruction Set Enable bit->Extended Instruction Set and
      Indexed Addressing Mode disabled
00077
00078  // CONFIG5
00079  #pragma config WDTCP5 = WDTCP5_31 // WDT Period selection bits->Divider ratio 1:65536; software
      control of WDTCP5
00080  #pragma config WDTE = OFF      // WDT operating mode->WDT Disabled; SWDTEN is ignored
00081
00082  // CONFIG6
00083  #pragma config WDTW5 = WDTW5_7    // WDT Window Select bits->window always open (100%); software
      control; keyed access not required
00084  #pragma config WDTCC5 = SC      // WDT input clock selector->Software Control
00085
00086  // CONFIG7
00087  #pragma config BBSIZE = BBSIZE_512 // Boot Block Size selection bits->Boot Block size is 512 words
00088  #pragma config BBEN = OFF      // Boot Block enable bit->Boot block disabled
00089  #pragma config SAFEN = OFF      // Storage Area Flash enable bit->SAF disabled
00090
00091  // CONFIG8
00092  #pragma config WRTB = OFF      // Boot Block Write Protection bit->Boot Block not Write protected

```

```
00093 #pragma config WRTC = OFF      // Configuration Register Write Protection bit->Configuration registers
    not Write protected
00094 #pragma config WRTD = OFF      // Data EEPROM Write Protection bit->Data EEPROM not Write protected
00095 #pragma config WRTSAF = OFF    // SAF Write protection bit->SAF not Write Protected
00096 #pragma config WRTAPP = OFF    // Application Block write protection bit->Application Block not write
    protected
00097
00098 // CONFIG9
00099 #pragma config BOOTPINSEL = RC5  // CRC on boot output pin selection->CRC on boot output pin is RC5
00100 #pragma config BPEN = OFF      // CRC on boot output pin enable bit->CRC on boot output pin disabled
00101 #pragma config ODCON = OFF     // CRC on boot output pin open drain bit->Pin drives both high-going and
    low-going signals
00102
00103 // CONFIG10
00104 #pragma config CP = ON        // PFM and Data EEPROM Code Protection bit->PFM and Data EEPROM code
    protection enabled
00105
00106 // CONFIG11
00107 #pragma config BOOTSCEN = OFF   // CRC on boot scan enable for boot area->CRC on boot will not
    include the boot area of program memory in its calculation
00108 #pragma config BOOTCOE = HALT   // CRC on boot Continue on Error for boot areas bit->CRC on boot will
    stop device if error is detected in boot areas
00109 #pragma config APPSCEN = OFF    // CRC on boot application code scan enable->CRC on boot will not
    include the application area of program memory in its calculation
00110 #pragma config SAFSCEN = OFF    // CRC on boot SAF area scan enable->CRC on boot will not include the
    SAF area of program memory in its calculation
00111 #pragma config DATASCEN = OFF   // CRC on boot Data EEPROM scan enable->CRC on boot will not include
    data EEPROM in its calculation
00112 #pragma config CFGSCEN = OFF    // CRC on boot Config fuses scan enable->CRC on boot will not include
    the configuration fuses in its calculation
00113 #pragma config COE = HALT      // CRC on boot Continue on Error for non-boot areas bit->CRC on boot will
    stop device if error is detected in non-boot areas
00114 #pragma config BOOTPOR = OFF    // Boot on CRC Enable bit->CRC on boot will not run
00115
00116 // CONFIG12
00117 #pragma config BCRCPOLT = hFF   // Boot Sector Polynomial for CRC on boot bits 31-24->Bits 31:24 of
    BCRCPOL are 0xFF
00118
00119 // CONFIG13
00120 #pragma config BCRCPOLU = hFF   // Boot Sector Polynomial for CRC on boot bits 23-16->Bits 23:16 of
    BCRCPOL are 0xFF
00121
00122 // CONFIG14
00123 #pragma config BCRCPOLH = hFF   // Boot Sector Polynomial for CRC on boot bits 15-8->Bits 15:8 of
    BCRCPOL are 0xFF
00124
00125 // CONFIG15
00126 #pragma config BCRCPOLL = hFF   // Boot Sector Polynomial for CRC on boot bits 7-0->Bits 7:0 of
    BCRCPOL are 0xFF
00127
00128 // CONFIG16
00129 #pragma config BCRCEEDT = hFF   // Boot Sector Seed for CRC on boot bits 31-24->Bits 31:24 of
    BCRCEED are 0xFF
00130
00131 // CONFIG17
00132 #pragma config BCRCEEDU = hFF   // Boot Sector Seed for CRC on boot bits 23-16->Bits 23:16 of
    BCRCEED are 0xFF
00133
00134 // CONFIG18
00135 #pragma config BCRCEEDH = hFF   // Boot Sector Seed for CRC on boot bits 15-8->Bits 15:8 of BCRCEED
    are 0xFF
00136
00137 // CONFIG19
00138 #pragma config BCRCEEDL = hFF   // Boot Sector Seed for CRC on boot bits 7-0->Bits 7:0 of BCRCEED
    are 0xFF
00139
00140 // CONFIG24
00141 #pragma config CRCPOLT = hFF    // Non-Boot Sector Polynomial for CRC on boot bits 31-24->Bits 31:24
    of CRCPOL are 0xFF
00142
00143 // CONFIG25
00144 #pragma config CRCPOLU = hFF    // Non-Boot Sector Polynomial for CRC on boot bits 23-16->Bits 23:16
    of CRCPOL are 0xFF
00145
00146 // CONFIG26
00147 #pragma config CRCPOLH = hFF    // Non-Boot Sector Polynomial for CRC on boot bits 15-8->Bits 15:8 of
    CRCPOL are 0xFF
00148
00149 // CONFIG27
00150 #pragma config CRCPOLL = hFF    // Non-Boot Sector Polynomial for CRC on boot bits 7-0->Bits 7:0 of
    CRCPOL are 0xFF
00151
00152 // CONFIG28
00153 #pragma config CRCSEEDT = hFF   // Non-Boot Sector Seed for CRC on boot bits 31-24->Bits 31:24 of
    CRCSEED are 0xFF
00154
00155 // CONFIG29
```

```

00156 #pragma config CRCSEEDU = hFF      // Non-Boot Sector Seed for CRC on boot bits 23-16->Bits 23:16 of
      CRCSEED are 0xFF
00157
00158 // CONFIG30
00159 #pragma config CRCSEEDH = hFF      // Non-Boot Sector Seed for CRC on boot bits 15-8->Bits 15:8 of
      CRCSEED are 0xFF
00160
00161 // CONFIG31
00162 #pragma config CRCSEEDL = hFF      // Non-Boot Sector Seed for CRC on boot bits 7-0->Bits 7:0 of CRCSEED
      are 0xFF

```

6.39 device_config.h

```

00001
00023
00024 /*
00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
      license terms applicable to your use of third party software (including open source software) that
00030      may accompany Microchip software.
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035      FOR A PARTICULAR PURPOSE.
00036
00037      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044      SOFTWARE.
00045 */
00046
00047 #ifndef DEVICE_CONFIG_H
00048 #define DEVICE_CONFIG_H
00049
00050 #define _XTAL_FREQ 64000000
00051
00052 #endif /* DEVICE_CONFIG_H */

```

6.40 dma1.c

```

00001
00023
00024 /*
00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
      license terms applicable to your use of third party software (including open source software) that
00030      may accompany Microchip software.
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035      FOR A PARTICULAR PURPOSE.
00036
00037      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044      SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>

```

```

00052 #include "dma1.h"
00053
00054 void (*DMA1_SCNTI_IRQHandler)(void);
00055 void (*DMA1_ORI_IRQHandler)(void);
00056
00060
00061 void DMA1_Initialize(void)
00062 {
00063     //DMA Instance Selection : 0x00
00064     DMASELECT = 0x00;
00065     //Source Address : lcd_dma_buf
00066     DMA1SSA = (volatile unsigned short) &lcd_dma_buf;
00067     //Destination Address : &SPI1TXB
00068     DMA1DSA = (volatile unsigned short) &SPI1TXB;
00069     //DMODE unchanged; DSTP not cleared; SMR GPR; SMOE incremented; SSTP not cleared;
00070     DMA1CON1 = 0x02;
00071     //Source Message Size : 1
00072     DMA1SSZ = 1;
00073     //Destination Message Size : 1
00074     DMA1DSZ = 1;
00075     //Start Trigger : SIRQ SPI1TX;
00076     DMA1SIRQ = 0x19;
00077     //Abort Trigger : AIRQ None;
00078     DMA1AIRQ = 0x00;
00079
00080     // Clear Destination Count Interrupt Flag bit
00081     PIR2bits.DMA1DCNTIF = 0;
00082     // Clear Source Count Interrupt Flag bit
00083     PIR2bits.DMA1SCNTIF = 0;
00084     // Clear Abort Interrupt Flag bit
00085     PIR2bits.DMA1AIF = 0;
00086     // Clear Overrun Interrupt Flag bit
00087     PIR2bits.DMA1ORIF = 0;
00088
00089     PIE2bits.DMA1DCNTIE = 0;
00090     PIE2bits.DMA1SCNTIE = 1;
00091     DMA1_SetSCNTI_IRQHandler(DMA1_Default_IRQHandler);
00092     PIE2bits.DMA1AIE = 0;
00093     PIE2bits.DMA1ORIE = 1;
00094     DMA1_SetORI_IRQHandler(DMA1_Default_IRQHandler);
00095
00096     //EN enabled; SIRQEN enabled; DGO not in progress; AIRQEN disabled;
00097     DMA1CON0 = 0xC0;
00098
00099 }
00100
00101 void DMA1_SelectSourceRegion(uint8_t region)
00102 {
00103     DMASELECT = 0x00;
00104     DMA1CON1bits.SMR = region;
00105 }
00106
00107 void DMA1_SetSourceAddress(uint24_t address)
00108 {
00109     DMASELECT = 0x00;
00110     DMA1SSA = address;
00111 }
00112
00113 void DMA1_SetDestinationAddress(uint16_t address)
00114 {
00115     DMASELECT = 0x00;
00116     DMA1DSA = address;
00117 }
00118
00119 void DMA1_SetSourceSize(uint16_t size)
00120 {
00121     DMASELECT = 0x00;
00122     DMA1SSZ = size;
00123 }
00124
00125 void DMA1_SetDestinationSize(uint16_t size)
00126 {
00127     DMASELECT = 0x00;
00128     DMA1DSZ = size;
00129 }
00130
00131 uint24_t DMA1_GetSourcePointer(void)
00132 {
00133     DMASELECT = 0x00;
00134     return DMA1SPTR;
00135 }
00136
00137 uint16_t DMA1_GetDestinationPointer(void)
00138 {
00139     DMASELECT = 0x00;
00140     return DMA1DPTR;
00141 }

```

```

00142
00143 void DMA1_SetStartTrigger(uint8_t sirq)
00144 {
00145     DMASELECT = 0x00;
00146     DManSIRQ = sirq;
00147 }
00148
00149 void DMA1_SetAbortTrigger(uint8_t airq)
00150 {
00151     DMASELECT = 0x00;
00152     DManAIRQ = airq;
00153 }
00154
00155 void DMA1_StartTransfer(void)
00156 {
00157     DMASELECT = 0x00;
00158     DManCON0bits.DGO = 1;
00159 }
00160
00161 void DMA1_StartTransferWithTrigger(void)
00162 {
00163     DMASELECT = 0x00;
00164     DManCON0bits.SIRQEN = 1;
00165 }
00166
00167 void DMA1_StopTransfer(void)
00168 {
00169     DMASELECT = 0x00;
00170     DManCON0bits.SIRQEN = 0;
00171     DManCON0bits.DGO = 0;
00172 }
00173
00174 void DMA1_SetDMAPriority(uint8_t priority)
00175 {
00176     // This function is dependant on the PRLWAY CONFIG bit
00177     PRLOCK = 0x55;
00178     PRLOCK = 0xAA;
00179     PRLOCKbits.PRLOCKED = 0;
00180     DMA1PR = priority;
00181     PRLOCK = 0x55;
00182     PRLOCK = 0xAA;
00183     PRLOCKbits.PRLOCKED = 1;
00184 }
00185
00186 void __interrupt (irq (IRQ_DMA1SCNT), base(8)) DMA1_DMASCNTI_ISR()
00187 {
00188     // Clear the source count interrupt flag
00189     PIR2bits.DMA1SCNTIF = 0;
00190
00191     if (DMA1_SCNTI_InterruptHandler)
00192         DMA1_SCNTI_InterruptHandler();
00193 }
00194
00195 void DMA1_SetSCNTIInterruptHandler(void (* InterruptHandler)(void))
00196 {
00197     DMA1_SCNTI_InterruptHandler = InterruptHandler;
00198 }
00199
00200 void __interrupt (irq (IRQ_DMA1OR), base(8)) DMA1_DMAORI_ISR()
00201 {
00202     // Clear the source count interrupt flag
00203     PIR2bits.DMA1ORIF = 0;
00204
00205     if (DMA1_ORI_InterruptHandler)
00206         DMA1_ORI_InterruptHandler();
00207 }
00208
00209 void DMA1_SetORIInterruptHandler(void (* InterruptHandler)(void))
00210 {
00211     DMA1_ORI_InterruptHandler = InterruptHandler;
00212 }
00213
00214 void DMA1_DefaultInterruptHandler(void) {
00215     // add your DMA1 interrupt custom code
00216     // or set custom function using DMA1_SetSCNTIInterruptHandler() /DMA1_SetDCNTIInterruptHandler()
    //DMA1_SetAIIInterruptHandler() /DMA1_SetORIInterruptHandler()
00217 }

```

6.41 dma1.h

```

00001
00023
00024 /*

```

```

00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
      license terms applicable to your use of third party software (including open source software) that
00030      may accompany Microchip software.
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035      FOR A PARTICULAR PURPOSE.
00036
00037      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044      SOFTWARE.
00045  */
00046
00047  #ifndef DMA1_H
00048  #define DMA1_H
00049
00053  #include <stdint.h>
00054
00055  uint8_t lcd_dma_buf[32];
00056
00063  void DMA1_Initialize(void);
00064
00070  void DMA1_SelectSourceRegion(uint8_t region);
00071
00077  void DMA1_SetSourceAddress(uint24_t address);
00078
00084  void DMA1_SetDestinationAddress(uint16_t address);
00085
00091  void DMA1_SetSourceSize(uint16_t size);
00092
00098  void DMA1_SetDestinationSize(uint16_t size);
00099
00105  uint24_t DMA1_GetSourcePointer(void);
00106
00112  uint16_t DMA1_GetDestinationPointer(void);
00113
00119  void DMA1_SetStartTrigger(uint8_t sirq);
00120
00126  void DMA1_SetAbortTrigger(uint8_t airq);
00127
00133  void DMA1_StartTransfer(void);
00134
00140  void DMA1_StartTransferWithTrigger(void);
00141
00147  void DMA1_StopTransfer(void);
00148
00154  void DMA1_SetDMAPriority(uint8_t priority);
00155
00161  void DMA1_SetSCNTIInterruptHandler(void (* InterruptHandler)(void));
00162
00163
00164
00170  void DMA1_SetORIInterruptHandler(void (* InterruptHandler)(void));
00176  void DMA1_DefaultInterruptHandler(void);
00177  #endif //DMA1_H

```

6.42 dma2.c

```

00001
00023
00024  /*
00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029      party
      license terms applicable to your use of third party software (including open source software) that
00030      may accompany Microchip software.
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY

```

```

00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "dma2.h"
00053
00057
00061 void DMA2_Initialize(void)
00062 {
00063     DMA2SSA = 0x001000; //set source start address
00064     DMA2DSA = 0x3FBB; //set destination start address
00065     DMA2CON1 = 0x03; //set control register1
00066     DMA2SSZ = 0x0001; //set source size
00067     DMA2DSZ = 0x0001; //set destination size
00068     DMA2SIRQ = 0x46; //set DMA Transfer Trigger Source
00069     DMA2AIRQ = 0x3D; //set DMA Transfer abort Source
00070
00071     PIR5bits.DMA2DCNTIF =0; // clear Destination Count Interrupt Flag bit
00072     PIR5bits.DMA2SCNTIF =0; // clear Source Count Interrupt Flag bit
00073     PIR5bits.DMA2AIF =0; // clear abort Interrupt Flag bit
00074     PIR5bits.DMA2ORIF =0; // clear overrun Interrupt Flag bit
00075
00076     PIE5bits.DMA2DCNTIE =0; // disable Destination Count 0 Interrupt
00077     PIE5bits.DMA2SCNTIE =0; // disable Source Count Interrupt
00078     PIE5bits.DMA2AIE =0; // disable abort Interrupt
00079     PIE5bits.DMA2ORIE =0; // disable overrun Interrupt
00080
00081     asm("BCF INTCON0,7");
00082
00083     asm ("BANKSEL PRLOCK");
00084     asm ("MOVLW 0x55");
00085     asm ("MOVWF PRLOCK");
00086     asm ("MOVLW 0xAA");
00087     asm ("MOVWF PRLOCK");
00088     asm ("BSF PRLOCK, 0");
00089
00090     asm("BSF INTCON0,7");
00091
00092     DMA2CON0 = 0x00; //set control register0
00093 }
00094
00095
00096
00097

```

6.43 dma2.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.

```



```

00045 */
00046
00050 #ifndef DMA2_H
00051 #define DMA2_H
00052
00053 #include <xc.h>
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056 #include <stdlib.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059
00060     extern "C" {
00061
00062 #endif
00063
00067 void DMA2_Initialize(void);
00115
00116
00117
00118
00119 #ifdef __cplusplus // Provide C++ Compatibility
00120
00121     }
00122 #endif
00123
00124 #endif // dma2.h

```

6.44 dsm.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "dsm.h"
00053
00057
00058 void DSM_Initialize(void)
00059 {
00060     // set the DSM to the options selected in the User Interface
00061
00062     // Modulator Output Pin Polarity is not inverted
00063     MD1CON0bits.MD1OPOL = 0;
00064
00065     // Modulation Low Carrier Synchronization is enabled
00066     MD1CON1bits.MD1CLSYNC = 1;
00067
00068     // Modulation Low Carrier Polarity is not inverted
00069     MD1CON1bits.MD1CLPOL = 0;
00070
00071     // Modulation High Carrier Synchronization is enabled
00072     MD1CON1bits.MD1CHSYNC = 1;
00073
00074     // Modulation High Carrier Polarity is not inverted
00075     MD1CON1bits.MD1CHPOL = 0;
00076

```

```

00077 // MD1MS UART1_TX;
00078 MD1SRC = 0x13;
00079
00080 // MD1CH NCO1OUT;
00081 MD1CARH = 0x0E;
00082
00083 // MD1CL CCP1;
00084 MD1CARL = 0x04;
00085
00086 // Enable the Modulator
00087 MD1CON0bits.MD1EN = 1;
00088 }
00089
00090 void DSM_ManualModulationSet (void)
00091 {
00092     // Set MDBIT to High, Modulator uses high carrier source
00093     MD1CON0bits.MD1BIT = 1;
00094 }
00095
00096 void DSM_ManualModulationClear (void)
00097 {
00098     // Set MDBIT to Low, Modulator uses low carrier source
00099     MD1CON0bits.MD1BIT = 0;
00100 }
00101
00102 void DSM_ManualModulationToggle (void)
00103 {
00104     // Toggle the MDBIT
00105     MD1CON0bits.MD1BIT ^= 1;
00106 }
00107
00108 void DSM_ModulationStart (void)
00109 {
00110     // Enables module and begins modulation
00111     MD1CON0bits.MD1EN = 1;
00112 }
00113
00114 void DSM_ModulationStop (void)
00115 {
00116     // Disables module and stops modulation
00117     MD1CON0bits.MD1EN = 0;
00118 }

```

6.45 dsm.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef DSM_H
00048 #define DSM_H
00049
00053
00054 #include <stdint.h>
00055
00056 #ifdef __cplusplus // Provide C++ Compatibility
00057
00058     extern "C" {
00059
00060 #endif

```

```

00061
00065
00090 void DSM_Initialize(void);
00091
00115 void DSM_ManualModulationSet (void);
00116
00140 void DSM_ManualModulationClear (void);
00164 void DSM_ManualModulationToggle (void);
00165
00189 void DSM_ModulationStart (void);
00190
00216 void DSM_ModulationStop (void);
00217
00218 #ifdef __cplusplus // Provide C++ Compatibility
00219
00220     }
00221
00222 #endif
00223
00224 #endif // DSM_H

```

6.46 ext_int.c

```

00001
00023
00027 #include <xc.h>
00028 #include "ext_int.h"
00029 #include "interrupt_manager.h"
00030 #include "pin_manager.h"
00031 #include "../vconfig.h"
00032
00033
00034 void (*INT0_InterruptHandler)(void);
00035 void (*INT1_InterruptHandler)(void);
00036 void (*INT2_InterruptHandler)(void);
00037
00038 void __interrupt(irq(INT0), base(8)) INT0_ISR()
00039 {
00040     EXT_INT0_InterruptFlagClear();
00041
00042     // Callback function gets called everytime this ISR executes
00043     INT0_CallBack();
00044 }
00045
00046 void INT0_CallBack(void)
00047 {
00048     // Add your custom callback code here
00049     if (INT0_InterruptHandler) {
00050         INT0_InterruptHandler();
00051     }
00052 }
00053
00054 void INT0_SetInterruptHandler(void (* InterruptHandler)(void))
00055 {
00056     INT0_InterruptHandler = InterruptHandler;
00057 }
00058
00059 void INT0_DefaultInterruptHandler(void)
00060 {
00061     // add your INT0 interrupt custom code
00062     // or set custom function using INT0_SetInterruptHandler()
00063 }
00064
00065 void __interrupt(irq(INT1), base(8)) INT1_ISR()
00066 {
00067     EXT_INT1_InterruptFlagClear();
00068
00069     // Callback function gets called everytime this ISR executes
00070     INT1_CallBack();
00071 }
00072
00073 void INT1_CallBack(void)
00074 {
00075     // Add your custom callback code here
00076     if (INT1_InterruptHandler) {
00077         INT1_InterruptHandler();
00078     }
00079 }
00080
00081 void INT1_SetInterruptHandler(void (* InterruptHandler)(void))
00082 {
00083     INT1_InterruptHandler = InterruptHandler;
00084 }

```

```

00085
00086 void INT1_DefaultInterruptHandler(void)
00087 {
00088     // add your INT1 interrupt custom code
00089     // or set custom function using INT1_SetInterruptHandler()
00090 }
00091
00092 void __interrupt(irq(INT2), base(8)) INT2_ISR()
00093 {
00094     EXT_INT2_InterruptFlagClear();
00095
00096     // Callback function gets called everytime this ISR executes
00097     INT2_CallBack();
00098 }
00099
00100 void INT2_CallBack(void)
00101 {
00102     // Add your custom callback code here
00103     if (INT2_InterruptHandler) {
00104         INT2_InterruptHandler();
00105     }
00106 }
00107
00108 void INT2_SetInterruptHandler(void (* InterruptHandler)(void))
00109 {
00110     INT2_InterruptHandler = InterruptHandler;
00111 }
00112
00113 void INT2_DefaultInterruptHandler(void)
00114 {
00115     // add your INT2 interrupt custom code
00116     // or set custom function using INT2_SetInterruptHandler()
00117 }
00118
00119 void EXT_INT_Initialize(void)
00120 {
00121
00122     // Clear the interrupt flag
00123     // Set the external interrupt edge detect
00124     EXT_INT0_InterruptFlagClear();
00125     EXT_INT0_fallingEdgeSet();
00126     // Set Default Interrupt Handler
00127     INT0_SetInterruptHandler(INT0_DefaultInterruptHandler);
00128     EXT_INT0_InterruptEnable();
00129
00130
00131     // Clear the interrupt flag
00132     // Set the external interrupt edge detect
00133     EXT_INT1_InterruptFlagClear();
00134     EXT_INT1_fallingEdgeSet();
00135     // Set Default Interrupt Handler
00136     INT1_SetInterruptHandler(INT1_DefaultInterruptHandler);
00137     EXT_INT1_InterruptEnable();
00138
00139
00140     // Clear the interrupt flag
00141     // Set the external interrupt edge detect
00142     EXT_INT2_InterruptFlagClear();
00143     EXT_INT2_fallingEdgeSet();
00144     // Set Default Interrupt Handler
00145     INT2_SetInterruptHandler(INT2_DefaultInterruptHandler);
00146     EXT_INT2_InterruptEnable();
00147
00148 }
00149

```

6.47 ext_int.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
    party
00029     license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS

```

```

00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef _EXT_INT_H
00048 #define _EXT_INT_H
00049
00053 #include <xc.h>
00054
00055 // Provide C++ Compatibility
00056 #ifdef __cplusplus
00057 extern "C" {
00058
00060 #endif
00061
00093 #define EXT_INT0_InterruptFlagClear()      (PIR1bits.INT0IF = 0)
00094
00126 #define EXT_INT0_InterruptDisable()      (PIE1bits.INT0IE = 0)
00127
00157 #define EXT_INT0_InterruptEnable()      (PIE1bits.INT0IE = 1)
00158
00188 #define EXT_INT0_risingEdgeSet()      (INTCON0bits.INT0EDG = 1)
00189
00219 #define EXT_INT0_fallingEdgeSet()      (INTCON0bits.INT0EDG = 0)
00248 #define EXT_INT1_InterruptFlagClear()      (PIR5bits.INT1IF = 0)
00249
00281 #define EXT_INT1_InterruptDisable()      (PIE5bits.INT1IE = 0)
00282
00312 #define EXT_INT1_InterruptEnable()      (PIE5bits.INT1IE = 1)
00313
00343 #define EXT_INT1_risingEdgeSet()      (INTCON0bits.INT1EDG = 1)
00344
00374 #define EXT_INT1_fallingEdgeSet()      (INTCON0bits.INT1EDG = 0)
00403 #define EXT_INT2_InterruptFlagClear()      (PIR7bits.INT2IF = 0)
00404
00436 #define EXT_INT2_InterruptDisable()      (PIE7bits.INT2IE = 0)
00437
00467 #define EXT_INT2_InterruptEnable()      (PIE7bits.INT2IE = 1)
00468
00498 #define EXT_INT2_risingEdgeSet()      (INTCON0bits.INT2EDG = 1)
00499
00529 #define EXT_INT2_fallingEdgeSet()      (INTCON0bits.INT2EDG = 0)
00562 void EXT_INT_Initialize(void);
00563
00567
00590 void INT0_CallBack(void);
00591
00613 void INT0_SetInterruptHandler(void (* InterruptHandler)(void));
00614
00637 extern void (*INT0_InterruptHandler)(void);
00638
00661 void INT0_DefaultInterruptHandler(void);
00662
00685 void INT1_CallBack(void);
00686
00708 void INT1_SetInterruptHandler(void (* InterruptHandler)(void));
00709
00732 extern void (*INT1_InterruptHandler)(void);
00733
00756 void INT1_DefaultInterruptHandler(void);
00757
00780 void INT2_CallBack(void);
00781
00803 void INT2_SetInterruptHandler(void (* InterruptHandler)(void));
00804
00827 extern void (*INT2_InterruptHandler)(void);
00828
00851 void INT2_DefaultInterruptHandler(void);
00852
00853 // Provide C++ Compatibility
00854 #ifdef __cplusplus
00855
00856 }
00857
00858 #endif
00859 #endif

```

6.48 interrupt_manager.c

```

00001
00025
00026 /*
00027      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00028
00029      Subject to your compliance with these terms, you may use Microchip software and any
00030      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00031      party
      license terms applicable to your use of third party software (including open source software) that
00032      may accompany Microchip software.
00033
00034      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00035      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00036      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00037      FOR A PARTICULAR PURPOSE.
00038
00039      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00040      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00041      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00042      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00043      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00044      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00045      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00046      SOFTWARE.
00047 */
00048
00049 #include "interrupt_manager.h"
00050 #include "mcc.h"
00051
00052 void INTERRUPT_Initialize (void)
00053 {
00054     INTCON0bits.IPEN = 1;
00055
00056     bool state = (unsigned char)GIE;
00057     GIE = 0;
00058     IVTLOCK = 0x55;
00059     IVTLOCK = 0xAA;
00060     IVTLOCKbits.IVTLOCKED = 0x00; // unlock IVT
00061
00062     IVTBASEU = 0;
00063     IVTBASEH = 0;
00064     IVTBASEL = 8;
00065
00066     IVTLOCK = 0x55;
00067     IVTLOCK = 0xAA;
00068     IVTLOCKbits.IVTLOCKED = 0x01; // lock IVT
00069
00070     GIE = state;
00071
00072     // Assign peripheral interrupt priority vectors
00073     IPR2bits.DMA1SCNTIP = 1;
00074     IPR2bits.DMA1ORIP = 1;
00075     IPR8bits.U2TXIP = 1;
00076     IPR8bits.U2RXIP = 1;
00077     IPR4bits.U1TXIP = 1;
00078     IPR4bits.U1RXIP = 1;
00079     IPR15bits.TMR6IP = 0;
00080     IPR8bits.TMR5IP = 0;
00081 }
00082
00083 void __interrupt(irq(default),base(8)) Default_ISR()
00084 {
00085 }
00086

```

6.49 interrupt_manager.h

```

00001
00025
00026 /*
00027      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00028
00029      Subject to your compliance with these terms, you may use Microchip software and any
00030      derivatives exclusively with Microchip products. It is your responsibility to comply with third
00031      party
      license terms applicable to your use of third party software (including open source software) that
00032      may accompany Microchip software.
00033

```

```

00034     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00035     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00036     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00037     FOR A PARTICULAR PURPOSE.
00038
00039     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00040     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00041     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00042     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00043     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00044     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00045     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00046     SOFTWARE.
00047  */
00048
00049 #ifndef INTERRUPT_MANAGER_H
00050 #define INTERRUPT_MANAGER_H
00051
00062 #define INTERRUPT_GlobalInterruptHighEnable() (INTCON0bits.GIEH = 1)
00063
00074 #define INTERRUPT_GlobalInterruptHighDisable() (INTCON0bits.GIEH = 0)
00075
00086 #define INTERRUPT_GlobalInterruptLowEnable() (INTCON0bits.GIEL = 1)
00087
00098 #define INTERRUPT_GlobalInterruptLowDisable() (INTCON0bits.GIEL = 0)
00109 void INTERRUPT_Initialize(void);
00110
00111 #endif // INTERRUPT_MANAGER_H

```

6.50 mcc.c

```

00001
00023
00024  /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045  */
00046
00047 #include "mcc.h"
00048
00049
00050 void SYSTEM_Initialize(void)
00051 {
00052     INTERRUPT_Initialize();
00053     PMD_Initialize();
00054     ADC_Initialize();
00055     PIN_MANAGER_Initialize();
00056     OSCILLATOR_Initialize();
00057     TMR6_Initialize();
00058     DMA1_Initialize();
00059     TMR5_Initialize();
00060     TMR2_Initialize();
00061     UART2_Initialize();
00062     UART1_Initialize();
00063     SPI1_Initialize();
00064     PWM2_16BIT_Initialize();
00065     SystemArbiter_Initialize();
00066 }
00067
00068 void OSCILLATOR_Initialize(void)
00069 {
00070     // NOSC EXTOSC    with 4x PLL; NDIV 1;
00071     OSCCON1 = 0x20;

```

```

00072 // CSW HOLD may proceed; SOSCPWR Low power;
00073 OSCCON3 = 0x00;
00074 // MFOEN disabled; LFOEN disabled; ADOEN disabled; PLLEN enabled; SOSSEN disabled; EXTTOEN
disabled; HFOEN disabled;
00075 OSCEN = 0x01;
00076 // HFFRQ 64_MHz;
00077 OSCFRQ = 0x08;
00078 // TUN 0;
00079 OSCTUNE = 0x00;
00080 // Wait for PLL to stabilize
00081 while(PLLR == 0)
00082 {
00083 }
00084 }
00085
00086 void PMD_Initialize(void)
00087 {
00088 // CLKRMD CLKR enabled; SYSCMD SYSCLK enabled; SCANMD SCANNER enabled; FVRMD FVR enabled; IOCMD
IOC enabled; CRCMD CRC enabled; HLVDMD HLVD enabled;
00089 PMD0 = 0x00;
00090 // TMR0MD TMR0 enabled; TMR1MD TMR1 enabled; TMR4MD TMR4 enabled; SMT1MD SMT1 enabled; TMR5MD TMR5
enabled; TMR2MD TMR2 enabled; TMR3MD TMR3 enabled; TMR6MD TMR6 enabled;
00091 PMD1 = 0x00;
00092 // TU1MD UT16A enabled; CANMD CAN enabled; TU2MD UT16B enabled;
00093 PMD2 = 0x00;
00094 // ZCDMD ZCD enabled; DACMD DAC enabled; ADCMD ADC enabled; ACTMD ACT enabled; CM2MD CM2 enabled;
CM1MD CM1 enabled;
00095 PMD3 = 0x00;
00096 // NCO1MD NCO1 enabled; NCO2MD NCO2 enabled; DSM1MD DSM1 enabled; CWG3MD CWG3 enabled; CWG2MD CWG2
enabled; CWG1MD CWG1 enabled; NCO3MD NCO3 enabled;
00097 PMD4 = 0x00;
00098 // CCP2MD CCP2 enabled; CCP1MD CCP1 enabled; PWM2MD PWM2 enabled; CCP3MD CCP3 enabled; PWM1MD PWM1
enabled; PWM4MD PWM4 enabled; PWM3MD PWM3 enabled;
00099 PMD5 = 0x00;
00100 // U5MD UART5 enabled; U4MD UART4 enabled; U3MD UART3 enabled; U2MD UART2 enabled; U1MD UART1
enabled; SPI2MD SPI2 enabled; SPI1MD SPI1 enabled; I2C1MD I2C1 enabled;
00101 PMD6 = 0x00;
00102 // CLC5MD CLC5 enabled; CLC6MD CLC6 enabled; CLC3MD CLC3 enabled; CLC4MD CLC4 enabled; CLC7MD CLC7
enabled; CLC8MD CLC8 enabled; CLC1MD CLC1 enabled; CLC2MD CLC2 enabled;
00103 PMD7 = 0x00;
00104 // DMA5MD DMA5 enabled; DMA6MD DMA6 enabled; DMA8MD DMA8 enabled; DMA7MD DMA7 enabled; DMA1MD DMA1
enabled; DMA2MD DMA2 enabled; DMA3MD DMA3 enabled; DMA4MD DMA4 enabled;
00105 PMD8 = 0x00;
00106 }
00107
00108
00109 void SystemArbiter_Initialize(void)
00110 {
00111 // This function is dependant on the PR1WAY CONFIG bit
00112 PRLOCK = 0x55;
00113 PRLOCK = 0xAA;
00114 PRLOCKbits.PRLOCKED = 1;
00115 }

```

6.51 mcc.h

```

00001
00023
00024 /*
00025 (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027 Subject to your compliance with these terms, you may use Microchip software and any
00028 derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029 license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031
00032 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035 FOR A PARTICULAR PURPOSE.
00036
00037 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042 CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043 OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044 SOFTWARE.
00045 */
00046

```



```

00047 #ifndef MCC_H
00048 #define MCC_H
00049 #include <xc.h>
00050 #include "device_config.h"
00051 #include "pin_manager.h"
00052 #include <stdint.h>
00053 #include <stdbool.h>
00054 #include <conio.h>
00055 #include "interrupt_manager.h"
00056 #include "adc.h"
00057 #include "tmr5.h"
00058 #include "tmr6.h"
00059 #include "dmal.h"
00060 #include "tmr2.h"
00061 #include "memory.h"
00062 #include "uart2.h"
00063 #include "uart1.h"
00064 #include "spi1.h"
00065 #include "pwm2_16bit.h"
00066
00067
00068
00080 void SYSTEM_Initialize(void);
00081
00093 void OSCILLATOR_Initialize(void);
00094
00106 void PMD_Initialize(void);
00107
00118 void SystemArbiter_Initialize(void);
00119
00120 #endif /* MCC_H */

```

6.52 memory.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050 #include <xc.h>
00051 #include "memory.h"
00052
00056
00057 //128-words of Buffer RAM for PIC18F47Q84 is available at 0x3700
00058 uint16_t bufferRAM __at(0x3700);
00059
00060 uint8_t FLASH_ReadByte(uint32_t flashAddr)
00061 {
00062     //Set TBLPTR with the target byte address
00063     TBLPTRU = (uint8_t) ((flashAddr & 0x00FF0000) >> 16);
00064     TBLPTRH = (uint8_t) ((flashAddr & 0x0000FF00) >> 8);
00065     TBLPTRL = (uint8_t) (flashAddr & 0x000000FF);
00066
00067     //Perform table read to move one byte from NVM to TABLAT
00068     asm("TBLRD");
00069
00070     return (TABLAT);
00071 }
00072
00073 uint16_t FLASH_ReadWord(uint32_t flashAddr)
00074 {

```

```

00075     uint8_t readWordL, readWordH;
00076
00077     //Set TBLPTR with the target byte address
00078     TBLPTRU = (uint8_t) ((flashAddr & 0x00FF0000) » 16);
00079     TBLPTRH = (uint8_t) ((flashAddr & 0x0000FF00) » 8);
00080     TBLPTRL = (uint8_t) (flashAddr & 0x000000FF);
00081
00082     //Perform table read to move low byte from NVM to TABLAT
00083     asm("TBLRD*+");
00084     readWordL = TABLAT;
00085
00086     //Perform table read to move high byte from NVM to TABLAT
00087     asm("TBLRD");
00088     readWordH = TABLAT;
00089
00090     return (((uint16_t) readWordH « 8) | (readWordL));
00091 }
00092
00093 void FLASH_ReadPage(uint32_t flashAddr)
00094 {
00095     uint8_t GIEBitValue = INTCON0bits.GIE; // Save interrupt enable
00096
00097     //Set NVMADR with the target word address
00098     NVMADRU = (uint8_t) ((flashAddr & 0x00FF0000) » 16);
00099     NVMADRH = (uint8_t) ((flashAddr & 0x0000FF00) » 8);
00100     NVMADRL = (uint8_t) (flashAddr & 0x000000FF);
00101
00102     //Set the NVMCMD control bits for Page Read operation
00103     NVMCON1bits.NVMCMD = 0b010;
00104
00105     //Disable all interrupt
00106     INTCON0bits.GIE = 0;
00107
00108     //Perform the unlock sequence
00109     NVMLOCK = 0x55;
00110     NVMLOCK = 0xAA;
00111
00112     //Start page read and wait for the operation to complete
00113     NVMCON0bits.GO = 1;
00114     while (NVMCON0bits.GO);
00115
00116     //Restore the interrupts
00117     INTCON0bits.GIE = GIEBitValue;
00118
00119     //Set the NVMCMD control bits for Word Read operation to avoid accidental writes
00120     NVMCON1bits.NVMCMD = 0b000;
00121 }
00122
00123 void FLASH_WritePage(uint32_t flashAddr)
00124 {
00125     uint8_t GIEBitValue = INTCON0bits.GIE; // Save interrupt enable
00126
00127     //Set NVMADR with the target word address
00128     NVMADRU = (uint8_t) ((flashAddr & 0x00FF0000) » 16);
00129     NVMADRH = (uint8_t) ((flashAddr & 0x0000FF00) » 8);
00130     NVMADRL = (uint8_t) (flashAddr & 0x000000FF);
00131
00132     //Set the NVMCMD control bits for Write Page operation
00133     NVMCON1bits.NVMCMD = 0b101;
00134
00135     //Disable all interrupt
00136     INTCON0bits.GIE = 0;
00137
00138     //Perform the unlock sequence
00139     NVMLOCK = 0x55;
00140     NVMLOCK = 0xAA;
00141
00142     //Start page programming and wait for the operation to complete
00143     NVMCON0bits.GO = 1;
00144     while (NVMCON0bits.GO);
00145
00146     //Restore the interrupts
00147     INTCON0bits.GIE = GIEBitValue;
00148
00149     //Set the NVMCMD control bits for Word Read operation to avoid accidental writes
00150     NVMCON1bits.NVMCMD = 0b000;
00151 }
00152
00153 void FLASH_WriteWord(uint32_t flashAddr, uint16_t word)
00154 {
00155     uint16_t *bufferRamPtr = (uint16_t*) & bufferRAM;
00156     uint32_t blockStartAddr = (uint32_t) (flashAddr & ((END_FLASH - 1) ^ ((ERASE_FLASH_BLOCKSIZE * 2)
- 1)));
00157     uint8_t offset = (uint8_t) ((flashAddr & ((ERASE_FLASH_BLOCKSIZE * 2) - 1)) / 2);
00158
00159     //Read existing block into Buffer RAM
00160     FLASH_ReadPage(blockStartAddr);

```

```

00161
00162 //Erase the given block
00163 FLASH_EraseBlock(blockStartAddr);
00164
00165 //Modify Buffer RAM for the given word to be written to Program Flash Memory
00166 bufferRamPtr += offset;
00167 *bufferRamPtr = word;
00168
00169 //Write Buffer RAM contents to given Program Flash Memory block
00170 FLASH_WritePage(blockStartAddr);
00171 }
00172
00173 int8_t FLASH_WriteBlock(uint32_t flashAddr, uint16_t *flashWrBufPtr)
00174 {
00175     uint16_t *bufferRamPtr = (uint16_t*) & bufferRAM;
00176     uint32_t blockStartAddr = (uint32_t) (flashAddr & ((END_FLASH - 1) ^ ((ERASE_FLASH_BLOCKSIZE * 2)
00177 - 1)));
00178     uint16_t i; // change to uint16_t so i in for loop will not always be true
00179     //Block write must start at the beginning of a row
00180     if (flashAddr != blockStartAddr)
00181     {
00182         return -1;
00183     }
00184     //Copy application buffer contents to Buffer RAM
00185     for (i = 0; i < ERASE_FLASH_BLOCKSIZE; i++)
00186     {
00187         *bufferRamPtr++ = flashWrBufPtr[i];
00188     }
00189     //Erase the given block
00190     FLASH_EraseBlock(flashAddr);
00191     //Write Buffer RAM contents to given Program Flash Memory block
00192     FLASH_WritePage(flashAddr);
00193     return 0;
00194 }
00195
00196 void FLASH_EraseBlock(uint32_t flashAddr)
00197 {
00198     uint32_t blockStartAddr = (uint32_t) (flashAddr & ((END_FLASH - 1) ^ ((ERASE_FLASH_BLOCKSIZE * 2)
00199 - 1)));
00200     uint8_t GIEBitValue = INTCON0bits.GIE;
00201     //The NVMADR[21:8] bits point to the page being erased.
00202     //The NVMADR[7:0] bits are ignored
00203     NVMADRU = (uint8_t) ((blockStartAddr & 0x00FF0000) >> 16);
00204     NVMADRH = (uint8_t) ((blockStartAddr & 0x0000FF00) >> 8);
00205     //Set the NVMCMD control bits for Erase Page operation
00206     NVMCON1bits.NVMCMD = 0b110;
00207     //Disable all interrupts
00208     INTCON0bits.GIE = 0;
00209     //Perform the unlock sequence
00210     NVMLOCK = 0x55;
00211     NVMLOCK = 0xAA;
00212     //Start page erase and wait for the operation to complete
00213     NVMCON0bits.GO = 1;
00214     while (NVMCON0bits.GO);
00215     //Restore the interrupts
00216     INTCON0bits.GIE = GIEBitValue;
00217     //Set the NVMCMD control bits for Word Read operation to avoid accidental writes
00218     NVMCON1bits.NVMCMD = 0b000;
00219 }
00220
00221 void DATAEE_WriteByte(uint16_t bAddr, uint8_t bData)
00222 {
00223     uint8_t GIEBitValue = INTCON0bits.GIE;
00224     //Set NVMADR with the target word address (0x380000 - 0x3803FF)
00225     NVMADRU = 0x38;
00226     NVMADRH = (uint8_t) ((bAddr & 0xFF00) >> 8);
00227     NVMADRL = (uint8_t) (bAddr & 0x00FF);
00228     //Load NVMDATL with desired byte
00229     NVMDATL = bData;
00230     //Set the NVMCMD control bits for DFM Byte Write operation
00231     NVMCON1bits.NVMCMD = 0b011;
00232 }

```

```

00246 //Disable all interrupts
00247 INTCON0bits.GIE = 0;
00248
00249 //Perform the unlock sequence and start Page Erase
00250 NVMLLOCK = 0x55;
00251 NVMLLOCK = 0xAA;
00252
00253 //Start DFM write and wait for the operation to complete
00254 NVMCON0bits.GO = 1;
00255 while (NVMCON0bits.GO);
00256
00257 //Restore all the interrupts
00258 INTCON0bits.GIE = GIEBitValue;
00259
00260 //Set the NVMCMD control bits for Word Read operation to avoid accidental writes
00261 NVMCON1bits.NVMCMD = 0b000;
00262 }
00263
00264 uint8_t DATAEE_ReadByte(uint16_t bAdd)
00265 {
00266     //Set NVMDR with the target word address (0x380000 - 0x3803FF)
00267     NVMDR = 0x38;
00268     NVMDRH = (uint8_t) ((bAdd & 0xFF00) >> 8);
00269     NVMDRL = (uint8_t) (bAdd & 0x00FF);
00270
00271     //Set the NVMCMD control bits for DFM Byte Read operation
00272     NVMCON1bits.NVMCMD = 0b000;
00273     NVMCON0bits.GO = 1;
00274
00275     return NVMDATL;
00276 }
00277
00278 void __interrupt (irq(NVM),base(8)) MEMORY_ISR()
00279 {
00280     /* TODO : Add interrupt handling code */
00281     PIR1bits.NVMIF = 0;
00282 }

```

6.53 memory.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef MEMORY_H
00048 #define MEMORY_H
00049
00053
00054 #include <xc.h>
00055 #include <stdint.h>
00056
00060
00061 #define WRITE_FLASH_BLOCKSIZE    256
00062 #define ERASE_FLASH_BLOCKSIZE    256
00063 #define END_FLASH                0x020000
00064
00065
00069
00081 uint8_t FLASH_ReadByte(uint32_t flashAddr);
00082

```

```

00094 uint16_t FLASH_ReadWord(uint32_t flashAddr);
00095
00116 void FLASH_ReadPage(uint32_t flashAddr);
00117
00138 void FLASH_WritePage(uint32_t flashAddr);
00139
00151 void FLASH_WriteWord(uint32_t flashAddr, uint16_t word);
00152
00183 int8_t FLASH_WriteBlock(uint32_t flashAddr, uint16_t *flashWrBufPtr);
00184
00195 void FLASH_EraseBlock(uint32_t flashAddr);
00196
00200
00212 void DATAEE_WriteByte(uint16_t bAdd, uint8_t bData);
00213
00225 uint8_t DATAEE_ReadByte(uint16_t bAdd);
00226 #endif // MEMORY_H

```

6.54 nco1.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "nco1.h"
00053
00057
00058 void NCO1_Initialize (void)
00059 {
00060     // Set the NCO to the options selected in the GUI
00061     // EN disabled; POL active_hi; PFM FDC_mode;
00062     NCO1CON = 0x00;
00063     // CKS FOSC; PWS 1_clk;
00064     NCO1CLK = 0x00;
00065     //
00066     NCO1ACCU = 0x00;
00067     //
00068     NCO1ACCH = 0x00;
00069     //
00070     NCO1ACCL = 0x00;
00071     //
00072     NCO1INC0 = 0x06;
00073     //
00074     NCO1INCH = 0xC7;
00075     //
00076     NCO1INCL = 0xAE;
00077
00078     // Enable the NCO module
00079     NCO1CONbits.EN = 1;
00080
00081 }
00082
00083 bool NCO1_GetOutputStatus(void)
00084 {
00085     // Return output status on accumulator over flow
00086     return (NCO1CONbits.OUT);
00087 }
00091

```

6.55 nco1.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029 party
00029 license terms applicable to your use of third party software (including open source software) that
00030
00030 may accompany Microchip software.
00031
00032 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035 FOR A PARTICULAR PURPOSE.
00036
00037 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042 CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043 OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044 SOFTWARE.
00045 */
00046
00047 #ifndef NCO1_H
00048 #define NCO1_H
00049
00053
00054 #include <xc.h>
00055 #include <stdbool.h>
00056 #include <stdint.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059     extern "C" {
00060
00061
00062 #endif
00063
00067
00093 void NCO1_Initialize(void);
00094
00128 bool NCO1_GetOutputStatus(void);
00129
00130 #ifdef __cplusplus // Provide C++ Compatibility
00131     }
00132
00133
00134 #endif
00135
00136 #endif //NCO1_H
00140
00141

```

6.56 pin_manager.c

```

00001
00025
00026 /*
00027     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00028
00029     Subject to your compliance with these terms, you may use Microchip software and any
00030     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00031 party
00031 license terms applicable to your use of third party software (including open source software) that
00032
00032 may accompany Microchip software.
00033
00034 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00035 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00036 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00037 FOR A PARTICULAR PURPOSE.
00038
00039 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00040 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00041 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00042 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00043 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL

```

```

00044     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00045     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00046     SOFTWARE.
00047 */
00048
00049 #include "pin_manager.h"
00050
00051
00052
00053
00054
00055 void PIN_MANAGER_Initialize(void)
00056 {
00060     LATE = 0x01;
00061     LATD = 0xC8;
00062     LATA = 0x00;
00063     LATB = 0x0E;
00064     LATC = 0x02;
00065
00069     TRISE = 0x06;
00070     TRISA = 0xFF;
00071     TRISB = 0xF1;
00072     TRISC = 0xD5;
00073     TRISD = 0x31;
00074
00078     ANSELD = 0x30;
00079     ANSELC = 0xC4;
00080     ANSELB = 0xD1;
00081     ANSELE = 0x06;
00082     ANSELA = 0xFF;
00083
00087     WPUD = 0x09;
00088     WPUE = 0x00;
00089     WPUB = 0x20;
00090     WPUA = 0x00;
00091     WPUC = 0x01;
00092
00096     ODCONE = 0x00;
00097     ODCONA = 0x00;
00098     ODCONB = 0x00;
00099     ODCONC = 0x00;
00100     ODCOND = 0x00;
00101
00105     SLRCONA = 0xFF;
00106     SLRCONB = 0xFF;
00107     SLRCONC = 0xFF;
00108     SLRCOND = 0xFF;
00109     SLRCONC = 0x07;
00110
00114     INLVLA = 0xFF;
00115     INLVLB = 0xFF;
00116     INLVLC = 0xFF;
00117     INLVLD = 0xFF;
00118     INLVLE = 0x0F;
00119
00120
00121
00122
00123
00124
00125
00126
00127     U2RXPPS = 0x18;    //RD0->UART2:RX2;
00128     RD2PPS = 0x1B;    //RD2->PWM2_16BIT:PWM22;
00129     SPI1SCKPPS = 0x13; //RC3->SPI1:SCK1;
00130     RC3PPS = 0x31;    //RC3->SPI1:SCK1;
00131     RC1PPS = 0x20;    //RC1->UART1:TX1;
00132     RD1PPS = 0x1A;    //RD1->PWM2_16BIT:PWM21;
00133     RC5PPS = 0x32;    //RC5->SPI1:SD01;
00134     U1RXPPS = 0x10;    //RC0->UART1:RX1;
00135     RD6PPS = 0x23;    //RD6->UART2:TX2;
00136     SPI1SDIPPS = 0x14; //RC4->SPI1:SDI1;
00137 }
00138
00139 void PIN_MANAGER_IOC(void)
00140 {
00141 }
00142

```

6.57 pin_manager.h

```

00001
00023

```

```

00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef PIN_MANAGER_H
00048 #define PIN_MANAGER_H
00049
00050 #include <xc.h>
00051
00052 #define INPUT      1
00053 #define OUTPUT     0
00054
00055 #define HIGH       1
00056 #define LOW        0
00057
00058 #define ANALOG     1
00059 #define DIGITAL    0
00060
00061 #define PULL_UP_ENABLED      1
00062 #define PULL_UP_DISABLED    0
00063
00064 // get/set channel_ANA1 aliases
00065 #define channel_ANA1_TRIS      TRISAbits.TRISA1
00066 #define channel_ANA1_LAT      LATAbits.LATA1
00067 #define channel_ANA1_PORT      PORTAbits.RA1
00068 #define channel_ANA1_WPU      WPUAbits.WPUA1
00069 #define channel_ANA1_OD       ODCONAbits.ODCA1
00070 #define channel_ANA1_ANS      ANSELAbits.ANSELA1
00071 #define channel_ANA1_SetHigh() do { LATAbits.LATA1 = 1; } while(0)
00072 #define channel_ANA1_SetLow()  do { LATAbits.LATA1 = 0; } while(0)
00073 #define channel_ANA1_Toggle()  do { LATAbits.LATA1 = ~LATAbits.LATA1; } while(0)
00074 #define channel_ANA1_GetValue() PORTAbits.RA1
00075 #define channel_ANA1_SetDigitalInput() do { TRISAbits.TRISA1 = 1; } while(0)
00076 #define channel_ANA1_SetDigitalOutput() do { TRISAbits.TRISA1 = 0; } while(0)
00077 #define channel_ANA1_SetPullup() do { WPUAbits.WPUA1 = 1; } while(0)
00078 #define channel_ANA1_ResetPullup() do { WPUAbits.WPUA1 = 0; } while(0)
00079 #define channel_ANA1_SetPushPull() do { ODCONAbits.ODCA1 = 0; } while(0)
00080 #define channel_ANA1_SetOpenDrain() do { ODCONAbits.ODCA1 = 1; } while(0)
00081 #define channel_ANA1_SetAnalogMode() do { ANSELAbits.ANSELA1 = 1; } while(0)
00082 #define channel_ANA1_SetDigitalMode() do { ANSELAbits.ANSELA1 = 0; } while(0)
00083
00084 // get/set channel_ANA2 aliases
00085 #define channel_ANA2_TRIS      TRISAbits.TRISA2
00086 #define channel_ANA2_LAT      LATAbits.LATA2
00087 #define channel_ANA2_PORT      PORTAbits.RA2
00088 #define channel_ANA2_WPU      WPUAbits.WPUA2
00089 #define channel_ANA2_OD       ODCONAbits.ODCA2
00090 #define channel_ANA2_ANS      ANSELAbits.ANSELA2
00091 #define channel_ANA2_SetHigh() do { LATAbits.LATA2 = 1; } while(0)
00092 #define channel_ANA2_SetLow()  do { LATAbits.LATA2 = 0; } while(0)
00093 #define channel_ANA2_Toggle()  do { LATAbits.LATA2 = ~LATAbits.LATA2; } while(0)
00094 #define channel_ANA2_GetValue() PORTAbits.RA2
00095 #define channel_ANA2_SetDigitalInput() do { TRISAbits.TRISA2 = 1; } while(0)
00096 #define channel_ANA2_SetDigitalOutput() do { TRISAbits.TRISA2 = 0; } while(0)
00097 #define channel_ANA2_SetPullup() do { WPUAbits.WPUA2 = 1; } while(0)
00098 #define channel_ANA2_ResetPullup() do { WPUAbits.WPUA2 = 0; } while(0)
00099 #define channel_ANA2_SetPushPull() do { ODCONAbits.ODCA2 = 0; } while(0)
00100 #define channel_ANA2_SetOpenDrain() do { ODCONAbits.ODCA2 = 1; } while(0)
00101 #define channel_ANA2_SetAnalogMode() do { ANSELAbits.ANSELA2 = 1; } while(0)
00102 #define channel_ANA2_SetDigitalMode() do { ANSELAbits.ANSELA2 = 0; } while(0)
00103
00104 // get/set MLED aliases
00105 #define MLED_TRIS      TRISBbits.TRISB1
00106 #define MLED_LAT      LATBbits.LATB1
00107 #define MLED_PORT      PORTBbits.RB1

```



```

00112 #define MLED_WPU                WPUBbits.WPUB1
00113 #define MLED_OD                  ODCONBbits.ODCB1
00114 #define MLED_ANS                 ANSELBbits.ANSELB1
00115 #define MLED_SetHigh()           do { LATBbits.LATB1 = 1; } while(0)
00116 #define MLED_SetLow()            do { LATBbits.LATB1 = 0; } while(0)
00117 #define MLED_Toggle()            do { LATBbits.LATB1 = ~LATBbits.LATB1; } while(0)
00118 #define MLED_GetValue()          PORTBbits.RB1
00119 #define MLED_SetDigitalInput()    do { TRISBbits.TRISB1 = 1; } while(0)
00120 #define MLED_SetDigitalOutput()   do { TRISBbits.TRISB1 = 0; } while(0)
00121 #define MLED_SetPullup()          do { WPUBbits.WPUB1 = 1; } while(0)
00122 #define MLED_ResetPullup()        do { WPUBbits.WPUB1 = 0; } while(0)
00123 #define MLED_SetPushPull()        do { ODCONBbits.ODCB1 = 0; } while(0)
00124 #define MLED_SetOpenDrain()        do { ODCONBbits.ODCB1 = 1; } while(0)
00125 #define MLED_SetAnalogMode()      do { ANSELBbits.ANSELB1 = 1; } while(0)
00126 #define MLED_SetDigitalMode()     do { ANSELBbits.ANSELB1 = 0; } while(0)
00127
00128 // get/set RLED aliases
00129 #define RLED_TRIS                TRISBbits.TRISB2
00130 #define RLED_LAT                 LATBbits.LATB2
00131 #define RLED_PORT                PORTBbits.RB2
00132 #define RLED_WPU                WPUBbits.WPUB2
00133 #define RLED_OD                  ODCONBbits.ODCB2
00134 #define RLED_ANS                 ANSELBbits.ANSELB2
00135 #define RLED_SetHigh()           do { LATBbits.LATB2 = 1; } while(0)
00136 #define RLED_SetLow()            do { LATBbits.LATB2 = 0; } while(0)
00137 #define RLED_Toggle()            do { LATBbits.LATB2 = ~LATBbits.LATB2; } while(0)
00138 #define RLED_GetValue()          PORTBbits.RB2
00139 #define RLED_SetDigitalInput()    do { TRISBbits.TRISB2 = 1; } while(0)
00140 #define RLED_SetDigitalOutput()   do { TRISBbits.TRISB2 = 0; } while(0)
00141 #define RLED_SetPullup()          do { WPUBbits.WPUB2 = 1; } while(0)
00142 #define RLED_ResetPullup()        do { WPUBbits.WPUB2 = 0; } while(0)
00143 #define RLED_SetPushPull()        do { ODCONBbits.ODCB2 = 0; } while(0)
00144 #define RLED_SetOpenDrain()        do { ODCONBbits.ODCB2 = 1; } while(0)
00145 #define RLED_SetAnalogMode()      do { ANSELBbits.ANSELB2 = 1; } while(0)
00146 #define RLED_SetDigitalMode()     do { ANSELBbits.ANSELB2 = 0; } while(0)
00147
00148 // get/set DLED aliases
00149 #define DLED_TRIS                TRISBbits.TRISB3
00150 #define DLED_LAT                 LATBbits.LATB3
00151 #define DLED_PORT                PORTBbits.RB3
00152 #define DLED_WPU                WPUBbits.WPUB3
00153 #define DLED_OD                  ODCONBbits.ODCB3
00154 #define DLED_ANS                 ANSELBbits.ANSELB3
00155 #define DLED_SetHigh()           do { LATBbits.LATB3 = 1; } while(0)
00156 #define DLED_SetLow()            do { LATBbits.LATB3 = 0; } while(0)
00157 #define DLED_Toggle()            do { LATBbits.LATB3 = ~LATBbits.LATB3; } while(0)
00158 #define DLED_GetValue()          PORTBbits.RB3
00159 #define DLED_SetDigitalInput()    do { TRISBbits.TRISB3 = 1; } while(0)
00160 #define DLED_SetDigitalOutput()   do { TRISBbits.TRISB3 = 0; } while(0)
00161 #define DLED_SetPullup()          do { WPUBbits.WPUB3 = 1; } while(0)
00162 #define DLED_ResetPullup()        do { WPUBbits.WPUB3 = 0; } while(0)
00163 #define DLED_SetPushPull()        do { ODCONBbits.ODCB3 = 0; } while(0)
00164 #define DLED_SetOpenDrain()        do { ODCONBbits.ODCB3 = 1; } while(0)
00165 #define DLED_SetAnalogMode()      do { ANSELBbits.ANSELB3 = 1; } while(0)
00166 #define DLED_SetDigitalMode()     do { ANSELBbits.ANSELB3 = 0; } while(0)
00167
00168 // get/set RB0 aliases
00169 #define RB0_TRIS                 TRISBbits.TRISB5
00170 #define RB0_LAT                  LATBbits.LATB5
00171 #define RB0_PORT                 PORTBbits.RB5
00172 #define RB0_WPU                  WPUBbits.WPUB5
00173 #define RB0_OD                   ODCONBbits.ODCB5
00174 #define RB0_ANS                  ANSELBbits.ANSELB5
00175 #define RB0_SetHigh()            do { LATBbits.LATB5 = 1; } while(0)
00176 #define RB0_SetLow()             do { LATBbits.LATB5 = 0; } while(0)
00177 #define RB0_Toggle()             do { LATBbits.LATB5 = ~LATBbits.LATB5; } while(0)
00178 #define RB0_GetValue()           PORTBbits.RB5
00179 #define RB0_SetDigitalInput()     do { TRISBbits.TRISB5 = 1; } while(0)
00180 #define RB0_SetDigitalOutput()    do { TRISBbits.TRISB5 = 0; } while(0)
00181 #define RB0_SetPullup()           do { WPUBbits.WPUB5 = 1; } while(0)
00182 #define RB0_ResetPullup()         do { WPUBbits.WPUB5 = 0; } while(0)
00183 #define RB0_SetPushPull()         do { ODCONBbits.ODCB5 = 0; } while(0)
00184 #define RB0_SetOpenDrain()        do { ODCONBbits.ODCB5 = 1; } while(0)
00185 #define RB0_SetAnalogMode()       do { ANSELBbits.ANSELB5 = 1; } while(0)
00186 #define RB0_SetDigitalMode()      do { ANSELBbits.ANSELB5 = 0; } while(0)
00187
00188 // get/set RC0 procedures
00189 #define RC0_SetHigh()            do { LATCbits.LATC0 = 1; } while(0)
00190 #define RC0_SetLow()             do { LATCbits.LATC0 = 0; } while(0)
00191 #define RC0_Toggle()             do { LATCbits.LATC0 = ~LATCbits.LATC0; } while(0)
00192 #define RC0_GetValue()           PORTCbits.RC0
00193 #define RC0_SetDigitalInput()     do { TRISCbits.TRISC0 = 1; } while(0)
00194 #define RC0_SetDigitalOutput()    do { TRISCbits.TRISC0 = 0; } while(0)
00195 #define RC0_SetPullup()           do { WPUCbits.WPUC0 = 1; } while(0)
00196 #define RC0_ResetPullup()         do { WPUCbits.WPUC0 = 0; } while(0)
00197 #define RC0_SetAnalogMode()       do { ANSELcbits.ANSELC0 = 1; } while(0)
00198 #define RC0_SetDigitalMode()      do { ANSELcbits.ANSELC0 = 0; } while(0)

```

```

00199
00200 // get/set RC1 procedures
00201 #define RC1_SetHigh() do { LATCbits.LATC1 = 1; } while(0)
00202 #define RC1_SetLow() do { LATCbits.LATC1 = 0; } while(0)
00203 #define RC1_Toggle() do { LATCbits.LATC1 = ~LATCbits.LATC1; } while(0)
00204 #define RC1_GetValue() PORTCbits.RC1
00205 #define RC1_SetDigitalInput() do { TRISCbits.TRISC1 = 1; } while(0)
00206 #define RC1_SetDigitalOutput() do { TRISCbits.TRISC1 = 0; } while(0)
00207 #define RC1_SetPullup() do { WPUCbits.WPUC1 = 1; } while(0)
00208 #define RC1_ResetPullup() do { WPUCbits.WPUC1 = 0; } while(0)
00209 #define RC1_SetAnalogMode() do { ANSELcbits.ANSELc1 = 1; } while(0)
00210 #define RC1_SetDigitalMode() do { ANSELcbits.ANSELc1 = 0; } while(0)
00211
00212 // get/set RC3 procedures
00213 #define RC3_SetHigh() do { LATCbits.LATC3 = 1; } while(0)
00214 #define RC3_SetLow() do { LATCbits.LATC3 = 0; } while(0)
00215 #define RC3_Toggle() do { LATCbits.LATC3 = ~LATCbits.LATC3; } while(0)
00216 #define RC3_GetValue() PORTCbits.RC3
00217 #define RC3_SetDigitalInput() do { TRISCbits.TRISC3 = 1; } while(0)
00218 #define RC3_SetDigitalOutput() do { TRISCbits.TRISC3 = 0; } while(0)
00219 #define RC3_SetPullup() do { WPUCbits.WPUC3 = 1; } while(0)
00220 #define RC3_ResetPullup() do { WPUCbits.WPUC3 = 0; } while(0)
00221 #define RC3_SetAnalogMode() do { ANSELcbits.ANSELc3 = 1; } while(0)
00222 #define RC3_SetDigitalMode() do { ANSELcbits.ANSELc3 = 0; } while(0)
00223
00224 // get/set RC4 procedures
00225 #define RC4_SetHigh() do { LATCbits.LATC4 = 1; } while(0)
00226 #define RC4_SetLow() do { LATCbits.LATC4 = 0; } while(0)
00227 #define RC4_Toggle() do { LATCbits.LATC4 = ~LATCbits.LATC4; } while(0)
00228 #define RC4_GetValue() PORTCbits.RC4
00229 #define RC4_SetDigitalInput() do { TRISCbits.TRISC4 = 1; } while(0)
00230 #define RC4_SetDigitalOutput() do { TRISCbits.TRISC4 = 0; } while(0)
00231 #define RC4_SetPullup() do { WPUCbits.WPUC4 = 1; } while(0)
00232 #define RC4_ResetPullup() do { WPUCbits.WPUC4 = 0; } while(0)
00233 #define RC4_SetAnalogMode() do { ANSELcbits.ANSELc4 = 1; } while(0)
00234 #define RC4_SetDigitalMode() do { ANSELcbits.ANSELc4 = 0; } while(0)
00235
00236 // get/set RC5 procedures
00237 #define RC5_SetHigh() do { LATCbits.LATC5 = 1; } while(0)
00238 #define RC5_SetLow() do { LATCbits.LATC5 = 0; } while(0)
00239 #define RC5_Toggle() do { LATCbits.LATC5 = ~LATCbits.LATC5; } while(0)
00240 #define RC5_GetValue() PORTCbits.RC5
00241 #define RC5_SetDigitalInput() do { TRISCbits.TRISC5 = 1; } while(0)
00242 #define RC5_SetDigitalOutput() do { TRISCbits.TRISC5 = 0; } while(0)
00243 #define RC5_SetPullup() do { WPUCbits.WPUC5 = 1; } while(0)
00244 #define RC5_ResetPullup() do { WPUCbits.WPUC5 = 0; } while(0)
00245 #define RC5_SetAnalogMode() do { ANSELcbits.ANSELc5 = 1; } while(0)
00246 #define RC5_SetDigitalMode() do { ANSELcbits.ANSELc5 = 0; } while(0)
00247
00248 // get/set RD0 procedures
00249 #define RD0_SetHigh() do { LATDbits.LATD0 = 1; } while(0)
00250 #define RD0_SetLow() do { LATDbits.LATD0 = 0; } while(0)
00251 #define RD0_Toggle() do { LATDbits.LATD0 = ~LATDbits.LATD0; } while(0)
00252 #define RD0_GetValue() PORTDbits.RD0
00253 #define RD0_SetDigitalInput() do { TRISDbits.TRISD0 = 1; } while(0)
00254 #define RD0_SetDigitalOutput() do { TRISDbits.TRISD0 = 0; } while(0)
00255 #define RD0_SetPullup() do { WPUDbits.WPUD0 = 1; } while(0)
00256 #define RD0_ResetPullup() do { WPUDbits.WPUD0 = 0; } while(0)
00257 #define RD0_SetAnalogMode() do { ANSELDbits.ANSELd0 = 1; } while(0)
00258 #define RD0_SetDigitalMode() do { ANSELDbits.ANSELd0 = 0; } while(0)
00259
00260 // get/set RD1 procedures
00261 #define RD1_SetHigh() do { LATDbits.LATD1 = 1; } while(0)
00262 #define RD1_SetLow() do { LATDbits.LATD1 = 0; } while(0)
00263 #define RD1_Toggle() do { LATDbits.LATD1 = ~LATDbits.LATD1; } while(0)
00264 #define RD1_GetValue() PORTDbits.RD1
00265 #define RD1_SetDigitalInput() do { TRISDbits.TRISD1 = 1; } while(0)
00266 #define RD1_SetDigitalOutput() do { TRISDbits.TRISD1 = 0; } while(0)
00267 #define RD1_SetPullup() do { WPUDbits.WPUD1 = 1; } while(0)
00268 #define RD1_ResetPullup() do { WPUDbits.WPUD1 = 0; } while(0)
00269 #define RD1_SetAnalogMode() do { ANSELDbits.ANSELd1 = 1; } while(0)
00270 #define RD1_SetDigitalMode() do { ANSELDbits.ANSELd1 = 0; } while(0)
00271
00272 // get/set RD2 procedures
00273 #define RD2_SetHigh() do { LATDbits.LATD2 = 1; } while(0)
00274 #define RD2_SetLow() do { LATDbits.LATD2 = 0; } while(0)
00275 #define RD2_Toggle() do { LATDbits.LATD2 = ~LATDbits.LATD2; } while(0)
00276 #define RD2_GetValue() PORTDbits.RD2
00277 #define RD2_SetDigitalInput() do { TRISDbits.TRISD2 = 1; } while(0)
00278 #define RD2_SetDigitalOutput() do { TRISDbits.TRISD2 = 0; } while(0)
00279 #define RD2_SetPullup() do { WPUDbits.WPUD2 = 1; } while(0)
00280 #define RD2_ResetPullup() do { WPUDbits.WPUD2 = 0; } while(0)
00281 #define RD2_SetAnalogMode() do { ANSELDbits.ANSELd2 = 1; } while(0)
00282 #define RD2_SetDigitalMode() do { ANSELDbits.ANSELd2 = 0; } while(0)
00283
00284 // get/set CS aliases
00285 #define CS_TRIS TRISDbits.TRISD3

```

```

00286 #define CS_LAT          LATDbits.LATD3
00287 #define CS_PORT          PORTDbits.RD3
00288 #define CS_WPU           WPUDbits.WPUD3
00289 #define CS_OD            ODCONDbits.ODCD3
00290 #define CS_ANS           ANSELDbits.ANSELD3
00291 #define CS_SetHigh()     do { LATDbits.LATD3 = 1; } while(0)
00292 #define CS_SetLow()      do { LATDbits.LATD3 = 0; } while(0)
00293 #define CS_Toggle()      do { LATDbits.LATD3 = ~LATDbits.LATD3; } while(0)
00294 #define CS_GetValue()    PORTDbits.RD3
00295 #define CS_SetDigitalInput() do { TRISDbits.TRISD3 = 1; } while(0)
00296 #define CS_SetDigitalOutput() do { TRISDbits.TRISD3 = 0; } while(0)
00297 #define CS_SetPullup()   do { WPUDbits.WPUD3 = 1; } while(0)
00298 #define CS_ResetPullup() do { WPUDbits.WPUD3 = 0; } while(0)
00299 #define CS_SetPushPull() do { ODCONDbits.ODCD3 = 0; } while(0)
00300 #define CS_SetOpenDrain() do { ODCONDbits.ODCD3 = 1; } while(0)
00301 #define CS_SetAnalogMode() do { ANSELDbits.ANSELD3 = 1; } while(0)
00302 #define CS_SetDigitalMode() do { ANSELDbits.ANSELD3 = 0; } while(0)
00303
00304 // get/set RD6 procedures
00305 #define RD6_SetHigh()     do { LATDbits.LATD6 = 1; } while(0)
00306 #define RD6_SetLow()      do { LATDbits.LATD6 = 0; } while(0)
00307 #define RD6_Toggle()      do { LATDbits.LATD6 = ~LATDbits.LATD6; } while(0)
00308 #define RD6_GetValue()    PORTDbits.RD6
00309 #define RD6_SetDigitalInput() do { TRISDbits.TRISD6 = 1; } while(0)
00310 #define RD6_SetDigitalOutput() do { TRISDbits.TRISD6 = 0; } while(0)
00311 #define RD6_SetPullup()   do { WPUDbits.WPUD6 = 1; } while(0)
00312 #define RD6_ResetPullup() do { WPUDbits.WPUD6 = 0; } while(0)
00313 #define RD6_SetAnalogMode() do { ANSELDbits.ANSELD6 = 1; } while(0)
00314 #define RD6_SetDigitalMode() do { ANSELDbits.ANSELD6 = 0; } while(0)
00315
00316 // get/set DEBUG1 aliases
00317 #define DEBUG1_TRIS      TRISDbits.TRISD7
00318 #define DEBUG1_LAT       LATDbits.LATD7
00319 #define DEBUG1_PORT      PORTDbits.RD7
00320 #define DEBUG1_WPU       WPUDbits.WPUD7
00321 #define DEBUG1_OD        ODCONDbits.ODCD7
00322 #define DEBUG1_ANS       ANSELDbits.ANSELD7
00323 #define DEBUG1_SetHigh() do { LATDbits.LATD7 = 1; } while(0)
00324 #define DEBUG1_SetLow()  do { LATDbits.LATD7 = 0; } while(0)
00325 #define DEBUG1_Toggle()  do { LATDbits.LATD7 = ~LATDbits.LATD7; } while(0)
00326 #define DEBUG1_GetValue() PORTDbits.RD7
00327 #define DEBUG1_SetDigitalInput() do { TRISDbits.TRISD7 = 1; } while(0)
00328 #define DEBUG1_SetDigitalOutput() do { TRISDbits.TRISD7 = 0; } while(0)
00329 #define DEBUG1_SetPullup() do { WPUDbits.WPUD7 = 1; } while(0)
00330 #define DEBUG1_ResetPullup() do { WPUDbits.WPUD7 = 0; } while(0)
00331 #define DEBUG1_SetPushPull() do { ODCONDbits.ODCD7 = 0; } while(0)
00332 #define DEBUG1_SetOpenDrain() do { ODCONDbits.ODCD7 = 1; } while(0)
00333 #define DEBUG1_SetAnalogMode() do { ANSELDbits.ANSELD7 = 1; } while(0)
00334 #define DEBUG1_SetDigitalMode() do { ANSELDbits.ANSELD7 = 0; } while(0)
00335
00336 // get/set RELAY0 aliases
00337 #define RELAY0_TRIS      TRISEbits.TRISE0
00338 #define RELAY0_LAT       LATEbits.LATE0
00339 #define RELAY0_PORT      PORTEbits.RE0
00340 #define RELAY0_WPU       WPUEbits.WPUE0
00341 #define RELAY0_OD        ODCONEbits.ODCE0
00342 #define RELAY0_ANS       ANSELEbits.ANSELE0
00343 #define RELAY0_SetHigh() do { LATEbits.LATE0 = 1; } while(0)
00344 #define RELAY0_SetLow()  do { LATEbits.LATE0 = 0; } while(0)
00345 #define RELAY0_Toggle()  do { LATEbits.LATE0 = ~LATEbits.LATE0; } while(0)
00346 #define RELAY0_GetValue() PORTEbits.RE0
00347 #define RELAY0_SetDigitalInput() do { TRISEbits.TRISE0 = 1; } while(0)
00348 #define RELAY0_SetDigitalOutput() do { TRISEbits.TRISE0 = 0; } while(0)
00349 #define RELAY0_SetPullup() do { WPUEbits.WPUE0 = 1; } while(0)
00350 #define RELAY0_ResetPullup() do { WPUEbits.WPUE0 = 0; } while(0)
00351 #define RELAY0_SetPushPull() do { ODCONEbits.ODCE0 = 0; } while(0)
00352 #define RELAY0_SetOpenDrain() do { ODCONEbits.ODCE0 = 1; } while(0)
00353 #define RELAY0_SetAnalogMode() do { ANSELEbits.ANSELE0 = 1; } while(0)
00354 #define RELAY0_SetDigitalMode() do { ANSELEbits.ANSELE0 = 0; } while(0)
00355
00366 void PIN_MANAGER_Initialize (void);
00367
00368
00369
00370
00371 #endif // PIN_MANAGER_H

```

6.58 pwm2_16bit.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.

```

```

00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "pwm2_16bit.h"
00053
00054 //Pointers to PWM2_16BIT interrupt handlers
00055 //User can use them in application code to initialize with custom ISRs
00056 static void (*PWM2_16BIT_Slice1Output1_InterruptHandler)(void); //SaP1IF and hence PWMxIF is set
00057 static void (*PWM2_16BIT_Slice1Output2_InterruptHandler)(void); //SaP2IF and hence PWMxIF is set
00058 static void (*PWM2_16BIT_Period_InterruptHandler)(void); //PWMxPIF is set
00059 static void PWM2_16BIT_Slice1Output1_DefaultInterruptHandler(void);
00060 static void PWM2_16BIT_Slice1Output2_DefaultInterruptHandler(void);
00061 static void PWM2_16BIT_Period_DefaultInterruptHandler(void);
00062
00063 void PWM2_16BIT_Initialize(void)
00064 {
00065     //PWMMERS External Reset Disabled;
00066     PWM2ERS = 0x00;
00067
00068     //PWMCLK FOSC;
00069     PWM2CLK = 0x02;
00070
00071     //PWMLDS Autoload disabled;
00072     PWM2LDS = 0x00;
00073
00074     //PWMPRL 63;
00075     PWM2PRL = 0x3F;
00076
00077     //PWMPRH 6;
00078     PWM2PRH = 0x06;
00079
00080     //PWMCPRE Prescale by 4;
00081     PWM2CPRE = 0x03;
00082
00083     //PWMPIPOS No postscale;
00084     PWM2PIPOS = 0x00;
00085
00086     //PWMS1P2IF PWM2 output match did not occur; PWMS1P1IF PWM1 output match did not occur;
00087     PWM2GIR = 0x00;
00088
00089     //PWMS1P2IE disabled; PWMS1P1IE disabled;
00090     PWM2GIE = 0x00;
00091
00092     //PWMPOL2 disabled; PWMPOL1 disabled; PWMPDEN disabled; PWMMODE Left aligned mode;
00093     PWM2S1CFG = 0x00;
00094
00095     //PWMS1P1L 32;
00096     PWM2S1P1L = 0x20;
00097
00098     //PWMS1P1H 3;
00099     PWM2S1P1H = 0x03;
00100
00101     //PWMS1P2L 32;
00102     PWM2S1P2L = 0x20;
00103
00104     //PWMS1P2H 3;
00105     PWM2S1P2H = 0x03;
00106
00107     //Clear PWM2_16BIT period interrupt flag
00108     PIR5bits.PWM2PIF = 0;
00109
00110     //Clear PWM2_16BIT interrupt flag
00111     PIR5bits.PWM2IF = 0;
00112
00113     //Clear PWM2_16BIT slice 1, output 1 interrupt flag

```

```

00114     PWM2GIRbits.S1P1IF = 0;
00115
00116     //Clear PWM2_16BIT slice 1, output 2 interrupt flag
00117     PWM2GIRbits.S1P2IF = 0;
00118
00119     //PWM2_16BIT interrupt enable bit
00120     PIE5bits.PWM2IE = 0;
00121
00122     //PWM2_16BIT period interrupt enable bit
00123     PIE5bits.PWM2PIE = 0;
00124
00125     //Set default interrupt handlers
00126     PWM2_16BIT_Slice1Output1_SetInterruptHandler(PWM2_16BIT_Slice1Output1_DefaultInterruptHandler);
00127     PWM2_16BIT_Slice1Output2_SetInterruptHandler(PWM2_16BIT_Slice1Output2_DefaultInterruptHandler);
00128     PWM2_16BIT_Period_SetInterruptHandler(PWM2_16BIT_Period_DefaultInterruptHandler);
00129
00130     //PWMEN enabled; PWMLD disabled; PWMERSPOL disabled; PWMERSNOW disabled;
00131     PWM2CON = 0x80;
00132 }
00133
00134 void PWM2_16BIT_Enable()
00135 {
00136     PWM2CON |= _PWM2CON_EN_MASK;
00137 }
00138
00139 void PWM2_16BIT_Disable()
00140 {
00141     PWM2CON &= (~_PWM2CON_EN_MASK);
00142 }
00143
00144 void PWM2_16BIT_WritePeriodRegister(uint16_t periodCount)
00145 {
00146     PWM2PRL = (uint8_t)periodCount;
00147     PWM2PRH = (uint8_t)(periodCount >> 8);
00148 }
00149
00150 void PWM2_16BIT_SetSlice1Output1DutyCycleRegister(uint16_t registerValue)
00151 {
00152     PWM2S1P1L = (uint8_t)(registerValue);
00153     PWM2S1P1H = (uint8_t)(registerValue >> 8);
00154 }
00155
00156 void PWM2_16BIT_SetSlice1Output2DutyCycleRegister(uint16_t registerValue)
00157 {
00158     PWM2S1P2L = (uint8_t)(registerValue);
00159     PWM2S1P2H = (uint8_t)(registerValue >> 8);
00160 }
00161
00162 void PWM2_16BIT_LoadBufferRegisters(void)
00163 {
00164     //Load the period and duty cycle registers on the next period event
00165     PWM2CONbits.LD = 1;
00166 }
00167
00168 void __interrupt (irq(PWM2),base(8)) PWM2_16BIT_PWM1_ISR()
00169 {
00170     PIR5bits.PWM2IF = 0;
00171     if((PWM2GIEbits.S1P1IE == 1) && (PWM2GIRbits.S1P1IF == 1))
00172     {
00173         PWM2GIRbits.S1P1IF = 0;
00174         if(PWM2_16BIT_Slice1Output1_InterruptionHandler != NULL)
00175             PWM2_16BIT_Slice1Output1_InterruptionHandler();
00176     }
00177     else if((PWM2GIEbits.S1P2IE == 1) && (PWM2GIRbits.S1P2IF == 1))
00178     {
00179         PWM2GIRbits.S1P2IF = 0;
00180         if(PWM2_16BIT_Slice1Output2_InterruptionHandler != NULL)
00181             PWM2_16BIT_Slice1Output2_InterruptionHandler();
00182     }
00183 }
00184
00185 void __interrupt (irq(PWM2PR),base(8)) PWM2_16BIT_PWMPI_ISR()
00186 {
00187     PIR5bits.PWM2PIF = 0;
00188     if(PWM2_16BIT_Period_InterruptionHandler != NULL)
00189         PWM2_16BIT_Period_InterruptionHandler();
00190 }
00191
00192 void PWM2_16BIT_Slice1Output1_SetInterruptHandler(void (* InterruptHandler)(void))
00193 {
00194     PWM2_16BIT_Slice1Output1_InterruptionHandler = InterruptHandler;
00195 }
00196
00197 void PWM2_16BIT_Slice1Output2_SetInterruptHandler(void (* InterruptHandler)(void))
00198 {
00199     PWM2_16BIT_Slice1Output2_InterruptionHandler = InterruptHandler;
00200 }

```

```

00201
00202 void PWM2_16BIT_Period_SetInterruptHandler(void (* InterruptHandler)(void))
00203 {
00204     PWM2_16BIT_Period_InterruptHandler = InterruptHandler;
00205 }
00206
00207 static void PWM2_16BIT_Slice1Output1_DefaultInterruptHandler(void)
00208 {
00209     //Add your interrupt code here or
00210     //Use PWM2_16BIT_Slice1Output1_SetInterruptHandler() function to use Custom ISR
00211 }
00212
00213 static void PWM2_16BIT_Slice1Output2_DefaultInterruptHandler(void)
00214 {
00215     //Add your interrupt code here or
00216     //Use PWM2_16BIT_Slice1Output2_SetInterruptHandler() function to use Custom ISR
00217 }
00218
00219 static void PWM2_16BIT_Period_DefaultInterruptHandler(void)
00220 {
00221     //Add your interrupt code here or
00222     //Use PWM2_16BIT_Period_SetInterruptHandler() function to use Custom ISR
00223 }

```

6.59 pwm2_16bit.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef PWM2_16BIT_H
00048 #define PWM2_16BIT_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>
00056
00063 void PWM2_16BIT_Initialize(void);
00064
00070 void PWM2_16BIT_Enable();
00071
00079 void PWM2_16BIT_Disable();
00080
00096 void PWM2_16BIT_WritePeriodRegister(uint16_t periodCount);
00097
00114 void PWM2_16BIT_SetSlice1Output1DutyCycleRegister(uint16_t value);
00115
00131 void PWM2_16BIT_SetSlice1Output2DutyCycleRegister(uint16_t value);
00132
00148 void PWM2_16BIT_LoadBufferRegisters(void);
00149
00150
00169 void PWM2_16BIT_Slice1Output1_SetInterruptHandler(void (* InterruptHandler)(void));
00170
00189 void PWM2_16BIT_Slice1Output2_SetInterruptHandler(void (* InterruptHandler)(void));
00190
00209 void PWM2_16BIT_Period_SetInterruptHandler(void (* InterruptHandler)(void));
00210
00211 #endif //PWM2_16BIT_H

```

6.60 pwm8.c

```

00001
00023
00024  /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00032     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00033     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00034     FOR A PARTICULAR PURPOSE.
00035
00036     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00037     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00038     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00039     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00040     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00041     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00042     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00043     SOFTWARE.
00044 */
00045
00046
00050
00051 #include <xc.h>
00052 #include "pwm8.h"
00053
00057
00058 void PWM8_Initialize(void)
00059 {
00060     // Set the PWM to the options selected in the PIC10 / PIC12 / PIC16 / PIC18 MCUs.
00061     // PWM8POL active_hi; PWM8EN enabled;
00062     PWM8CON = 0x80;
00063
00064     // DC 0;
00065     PWM8DCH = 0x00;
00066
00067     // DC 1;
00068     PWM8DCL = 0x40;
00069
00070     // Select timer
00071     CCPTMRS1bits.P8TSEL = 1;
00072 }
00073
00074 void PWM8_LoadDutyValue(uint16_t dutyValue)
00075 {
00076     // Writing to 8 MSBs of PWM duty cycle in PWMDCH register
00077     PWM8DCH = (dutyValue & 0x03FC)»2;
00078
00079     // Writing to 2 LSBs of PWM duty cycle in PWMDCL register
00080     PWM8DCL = (dutyValue & 0x0003)«6;
00081 }

```

6.61 pwm8.h

```

00001
00023
00024  /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00032     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00033     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00034     FOR A PARTICULAR PURPOSE.
00035
00036     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00037     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00038     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00039     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO

```

```

00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef PWM8_H
00048 #define PWM8_H
00049
00053
00054 #include <xc.h>
00055 #include <stdint.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058
00059     extern "C" {
00060
00061 #endif
00062
00066
00067 #define PWM8_INITIALIZE_DUTY_VALUE    1
00068
00072
00102 void PWM8_Initialize(void);
00103
00129 void PWM8_LoadDutyValue(uint16_t dutyValue);
00130
00131
00132 #ifdef __cplusplus // Provide C++ Compatibility
00133
00134     }
00135
00136 #endif
00137
00138 #endif //PWM8_H

```

6.62 spi1.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
    party
00029     license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #include "spi1.h"
00048 #include <xc.h>
00049
00050 typedef struct {
00051     uint8_t con0;
00052     uint8_t con1;
00053     uint8_t con2;
00054     uint8_t baud;
00055     uint8_t operation;
00056 } spi1_configuration_t;
00057
00058
00059 //con0 == SPIxCON0, con1 == SPIxCON1, con2 == SPIxCON2, baud == SPIxBAUD, operation == Master/Slave
00060 static const spi1_configuration_t spi1_configuration[] = {
00061     { 0x83, 0x20, 0x3, 0x4, 0 }
00062 };
00063
00064 void SPI1_Initialize(void)

```



```

00065 {
00066     //EN enabled; LSBF MSb first; MST bus master; BMODE every byte;
00067     SPI1CON0 = 0x83;
00068     //SMP Middle; CKE Idle to active; CKP Idle:High, Active:Low; FST disabled; SSP active high; SDIP
    active high; SDOP active high;
00069     SPI1CON1 = 0x20;
00070     //SSET disabled; TXR required for a transfer; RXR suspended if the RxFIFO is full;
00071     SPI1CON2 = 0x03;
00072     //CLKSEL MFINTOSC;
00073     SPI1CLK = 0x02;
00074     //BAUD 4;
00075     SPI1BAUD = 0x04;
00076     TRISCbits.TRISC3 = 0;
00077 }
00078
00079 bool SPI1_Open(spi1_modes_t spi1UniqueConfiguration)
00080 {
00081     if(!SPI1CON0bits.EN)
00082     {
00083         SPI1CON0 = spi1_configuration[spi1UniqueConfiguration].con0;
00084         SPI1CON1 = spi1_configuration[spi1UniqueConfiguration].con1;
00085         SPI1CON2 = spi1_configuration[spi1UniqueConfiguration].con2 | (_SPI1CON2_SPI1RXR_MASK |
    _SPI1CON2_SPI1TXR_MASK);
00086         SPI1BAUD = spi1_configuration[spi1UniqueConfiguration].baud;
00087         TRISCbits.TRISC3 = spi1_configuration[spi1UniqueConfiguration].operation;
00088         SPI1CON0bits.EN = 1;
00089         return true;
00090     }
00091     return false;
00092 }
00093
00094 void SPI1_Close(void)
00095 {
00096     SPI1CON0bits.EN = 0;
00097 }
00098
00099 uint8_t SPI1_ExchangeByte(uint8_t data)
00100 {
00101     SPI1TCNTL = 1;
00102     SPI1TXB = data;
00103     while(!PIR3bits.SPI1RXIF);
00104     return SPI1RXB;
00105 }
00106
00107 void SPI1_ExchangeBlock(void *block, size_t blockSize)
00108 {
00109     uint8_t *data = block;
00110     while(blockSize--)
00111     {
00112         SPI1TCNTL = 1;
00113         SPI1TXB = *data;
00114         while(!PIR3bits.SPI1RXIF);
00115         *data++ = SPI1RXB;
00116     }
00117 }
00118
00119 // Half Duplex SPI Functions
00120 void SPI1_WriteBlock(void *block, size_t blockSize)
00121 {
00122     uint8_t *data = block;
00123     while(blockSize--)
00124     {
00125         SPI1_ExchangeByte(*data++);
00126     }
00127 }
00128
00129 void SPI1_ReadBlock(void *block, size_t blockSize)
00130 {
00131     uint8_t *data = block;
00132     while(blockSize--)
00133     {
00134         *data++ = SPI1_ExchangeByte(0);
00135     }
00136 }
00137
00138 void SPI1_WriteByte(uint8_t byte)
00139 {
00140     SPI1TXB = byte;
00141 }
00142
00143 uint8_t SPI1_ReadByte(void)
00144 {
00145     return SPI1RXB;
00146 }

```

6.63 spi1.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029 party
00029 license terms applicable to your use of third party software (including open source software) that
00030
00030 may accompany Microchip software.
00031
00032 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035 FOR A PARTICULAR PURPOSE.
00036
00037 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042 CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043 OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044 SOFTWARE.
00045 */
00046
00047 #ifndef SPI1_MASTER_H
00048 #define SPI1_MASTER_H
00049
00053
00054 #include <stdio.h>
00055 #include <stdint.h>
00056 #include <stdbool.h>
00057
00058 /* SPI interfaces */
00059 typedef enum {
00060     SPI1_DEFAULT
00061 } spil_modes_t;
00062
00063 void SPI1_Initialize(void);
00064 bool SPI1_Open(spil_modes_t spilUniqueConfiguration);
00065 void SPI1_Close(void);
00066 uint8_t SPI1_ExchangeByte(uint8_t data);
00067 void SPI1_ExchangeBlock(void *block, size_t blockSize);
00068 void SPI1_WriteBlock(void *block, size_t blockSize);
00069 void SPI1_ReadBlock(void *block, size_t blockSize);
00070 void SPI1_WriteByte(uint8_t byte);
00071 uint8_t SPI1_ReadByte(void);
00072
00073 #endif //SPI1_H

```

6.64 tmr2.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029 party
00029 license terms applicable to your use of third party software (including open source software) that
00030
00030 may accompany Microchip software.
00031
00032 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035 FOR A PARTICULAR PURPOSE.
00036
00037 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042 CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043 OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044 SOFTWARE.
00045 */

```

```

00046
00050
00051 #include <xc.h>
00052 #include "tmr2.h"
00053
00057
00061
00062 void TMR2_Initialize(void)
00063 {
00064     // Set TMR2 to the options selected in the User Interface
00065
00066     // T2CS FOSC/4;
00067     T2CLKCON = 0x01;
00068
00069     // T2PSYNC Not Synchronized; T2MODE Software control; T2CKPOL Rising Edge; T2CKSYNC Not
    Synchronized;
00070     T2HLT = 0x00;
00071
00072     // T2RSEL T2CKIPPS pin;
00073     T2RST = 0x00;
00074
00075     // PR2 99;
00076     T2PR = 0x63;
00077
00078     // TMR2 0;
00079     T2TMR = 0x00;
00080
00081     // Clearing IF flag.
00082     PIR3bits.TMR2IF = 0;
00083
00084     // T2CKPS 1:1; T2OUTPS 1:16; TMR2ON on;
00085     T2CON = 0x8F;
00086 }
00087
00088 void TMR2_ModeSet(TMR2_HLT_MODE mode)
00089 {
00090     // Configure different types HLT mode
00091     T2HLTbits.MODE = mode;
00092 }
00093
00094 void TMR2_ExtResetSourceSet(TMR2_HLT_EXT_RESET_SOURCE reset)
00095 {
00096     //Configure different types of HLT external reset source
00097     T2RSTbits.RSEL = reset;
00098 }
00099
00100 void TMR2_Start(void)
00101 {
00102     // Start the Timer by writing to TMRxON bit
00103     T2CONbits.TMR2ON = 1;
00104 }
00105
00106 void TMR2_StartTimer(void)
00107 {
00108     TMR2_Start();
00109 }
00110
00111 void TMR2_Stop(void)
00112 {
00113     // Stop the Timer by writing to TMRxON bit
00114     T2CONbits.TMR2ON = 0;
00115 }
00116
00117 void TMR2_StopTimer(void)
00118 {
00119     TMR2_Stop();
00120 }
00121
00122 uint8_t TMR2_Counter8BitGet(void)
00123 {
00124     uint8_t readVal;
00125
00126     readVal = TMR2;
00127
00128     return readVal;
00129 }
00130
00131 uint8_t TMR2_ReadTimer(void)
00132 {
00133     return TMR2_Counter8BitGet();
00134 }
00135
00136 void TMR2_Counter8BitSet(uint8_t timerVal)
00137 {
00138     // Write to the Timer2 register
00139     TMR2 = timerVal;
00140 }

```

```

00141
00142 void TMR2_WriteTimer(uint8_t timerVal)
00143 {
00144     TMR2_Counter8BitSet(timerVal);
00145 }
00146
00147 void TMR2_Period8BitSet(uint8_t periodVal)
00148 {
00149     PR2 = periodVal;
00150 }
00151
00152 void TMR2_LoadPeriodRegister(uint8_t periodVal)
00153 {
00154     TMR2_Period8BitSet(periodVal);
00155 }
00156
00157 bool TMR2_HasOverflowOccured(void)
00158 {
00159     // check if overflow has occurred by checking the TMRIF bit
00160     bool status = PIR3bits.TMR2IF;
00161     if(status)
00162     {
00163         // Clearing IF flag.
00164         PIR3bits.TMR2IF = 0;
00165     }
00166     return status;
00167 }

```

6.65 tmr2.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef TMR2_H
00048 #define TMR2_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00061 #endif
00062
00066
00078
00079 typedef enum
00080 {
00081
00082     /* Roll-over Pulse mode clears the TMRx upon TMRx = PRx, then continue running.
00083     ON bit must be set and is not affected by Resets
00084     */
00085
00086     /* Roll-over Pulse mode indicates that Timer starts
00087     immediately upon ON = 1 (Software Control)
00088     */
00089     TMR2_ROP_STARTS_TMROn,

```

```
00090
00091 /* Roll-over Pulse mode indicates that the Timer starts
00092    when ON = 1 and TMRx_ers = 1. Stops when TMRx_ers = 0
00093    */
00094 TMR2_ROP_STARTS_TMRON_ERSHIGH,
00095
00096 /* Roll-over Pulse mode indicates that the Timer starts
00097    when ON = 1 and TMRx_ers = 0. Stops when TMRx_ers = 1
00098    */
00099 TMR2_ROP_STARTS_TMRON_ERSLOW,
00100
00101 /* Roll-over Pulse mode indicates that the Timer resets
00102    upon rising or falling edge of TMRx_ers
00103    */
00104 TMR2_ROP_RESETS_ERSBOTHEDGE,
00105
00106 /* Roll-over Pulse mode indicates that the Timer resets
00107    upon rising edge of TMRx_ers
00108    */
00109 TMR2_ROP_RESETS_ERSRISINGEDGE,
00110
00111 /* Roll-over Pulse mode indicates that the Timer resets
00112    upon falling edge of TMRx_ers
00113    */
00114 TMR2_ROP_RESETS_ERSFALLINGEDGE,
00115
00116 /* Roll-over Pulse mode indicates that the Timer resets
00117    upon TMRx_ers = 0
00118    */
00119 TMR2_ROP_RESETS_ERSLOW,
00120
00121 /* Roll-over Pulse mode indicates that the Timer resets
00122    upon TMRx_ers = 1
00123    */
00124 TMR2_ROP_RESETS_ERSHIGH,
00125
00126 /*In all One-Shot mode the timer resets and the ON bit is
00127    cleared when the timer value matches the PRx period
00128    value. The ON bit must be set by software to start
00129    another timer cycle.
00130    */
00131
00132 /* One shot mode indicates that the Timer starts
00133    immediately upon ON = 1 (Software Control)
00134    */
00135 TMR2_OS_STARTS_TMRON,
00136
00137 /* One shot mode indicates that the Timer starts
00138    when a rising edge is detected on the TMRx_ers
00139    */
00140 TMR2_OS_STARTS_ERSRISINGEDGE ,
00141
00142 /* One shot mode indicates that the Timer starts
00143    when a falling edge is detected on the TMRx_ers
00144    */
00145 TMR2_OS_STARTS_ERSFALLINGEDGE ,
00146
00147 /* One shot mode indicates that the Timer starts
00148    when either a rising or falling edge is detected on TMRx_ers
00149    */
00150 TMR2_OS_STARTS_ERSBOTHEDGE,
00151
00152 /* One shot mode indicates that the Timer starts
00153    upon first TMRx_ers rising edge and resets on all
00154    subsequent TMRx_ers rising edges
00155    */
00156 TMR2_OS_STARTS_ERSFIRSTRISINGEDGE,
00157
00158 /* One shot mode indicates that the Timer starts
00159    upon first TMRx_ers falling edge and restarts on all
00160    subsequent TMRx_ers falling edges
00161    */
00162 TMR2_OS_STARTS_ERSFIRSTFALLINGEDGE,
00163
00164 /* One shot mode indicates that the Timer starts
00165    when a rising edge is detected on the TMRx_ers,
00166    resets upon TMRx_ers = 0
00167    */
00168 TMR2_OS_STARTS_ERSRISINGEDGEDETECT,
00169 /* One shot mode indicates that the Timer starts
00170    when a falling edge is detected on the TMRx_ers,
00171    resets upon TMRx_ers = 1
00172    */
00173 TMR2_OS_STARTS_ERSFALLINGEDGEDETECT,
00174
00175 /* One shot mode indicates that the Timer starts
00176    when a TMRx_ers = 1, ON =1 and resets upon TMRx_ers =0
```

```

00177     */
00178     TMR2_OS_STARTS_TMRON_ERSHIGH = 0x16,
00179
00180     /* One shot mode indicates that the Timer starts
00181        when a TMRx_ers = 0, ON = 1 and resets upon TMRx_ers =1
00182     */
00183     TMR2_OS_STARTS_TMRON_ERSLOW = 0x17,
00184
00185     /*In all Mono-Stable mode the ON bit must be initially set, but
00186        not cleared upon the TMRx = PRx, and the timer will start upon
00187        an TMRx_ers start event following TMRx = PRx.*/
00188
00189     /* Mono Stable mode indicates that the Timer starts
00190        when a rising edge is detected on the TMRx_ers and ON = 1
00191     */
00192     TMR2_MS_STARTS_TMRON_ERSRISINGEDGEDETECT = 0x11,
00193
00194     /* Mono Stable mode indicates that the Timer starts
00195        when a falling edge is detected on the TMRx_ers and ON = 1
00196     */
00197     TMR2_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT = 0x12,
00198
00199     /* Mono Stable mode indicates that the Timer starts
00200        when either a rising or falling edge is detected on TMRx_ers
00201        and ON = 1
00202     */
00203     TMR2_MS_STARTS_TMRON_ERSBOTHEDGE = 0x13
00204
00205 } TMR2_HLT_MODE;
00206
00219
00220 typedef enum
00221 {
00222     /* T2INPPS is the Timer external reset source
00223     */
00224     TMR2_T2INPPS,
00225
00226     /* Reserved enum cannot be used
00227     */
00228     TMR2_RESERVED,
00229
00230     /* Timer4 Postscale is the Timer external reset source
00231     */
00232     TMR2_T4POSTSCALED,
00233
00234     /* Timer6 Postscale is the Timer external reset source
00235     */
00236     TMR2_T6POSTSCALED,
00237
00238     /* CCP1_OUT is the Timer external reset source
00239     */
00240     TMR2_CCP1_OUT,
00241
00242     /* CCP2_OUT is the Timer external reset source
00243     */
00244     TMR2_CCP2_OUT,
00245
00246     /* CCP3_OUT is the Timer external reset source
00247     */
00248     TMR2_CCP3_OUT,
00249
00250     /* PWM1S1P1_out is the Timer external reset source
00251     */
00252     TMR2_PWM1S1P1_OUT,
00253
00254     /* PWM1S1P2_out is the Timer external reset source
00255     */
00256     TMR2_PWM1S1P2_OUT,
00257
00258     /* PWM2S1P1_out is the Timer external reset source
00259     */
00260     TMR2_PWM2S1P1_OUT,
00261
00262     /* PWM2S1P2_out is the Timer external reset source
00263     */
00264     TMR2_PWM2S1P2_OUT,
00265
00266     /* PWM3S1P1_out is the Timer external reset source
00267     */
00268     TMR2_PWM3S1P1_OUT,
00269
00270     /* PWM3S1P2_out is the Timer external reset source
00271     */
00272     TMR2_PWM3S1P2_OUT,
00273
00274     /* Reserved enum cannot be used
00275     */

```

```
00276     TMR2_RESERVED_2,
00277
00278     /* Reserved enum cannot be used
00279     */
00280     TMR2_RESERVED_3,
00281
00282     /* CMP1_OUT is the Timer external reset source
00283     */
00284     TMR2_CMP1_OUT,
00285
00286     /* CMP2_OUT is the Timer external reset source
00287     */
00288     TMR2_CMP2_OUT,
00289
00290     /* ZCD_Output is the Timer external reset source
00291     */
00292     TMR2_ZCD_OUTPUT,
00293
00294     /* CLC1_out is the Timer external reset source
00295     */
00296     TMR2_CLC1_OUT,
00297
00298     /* CLC2_out is the Timer external reset source
00299     */
00300     TMR2_CLC2_OUT,
00301
00302     /* CLC3_out is the Timer external reset source
00303     */
00304     TMR2_CLC3_OUT,
00305
00306     /* CLC4_out is the Timer external reset source
00307     */
00308     TMR2_CLC4_OUT,
00309
00310     /* CLC5_out is the Timer external reset source
00311     */
00312     TMR2_CLC5_OUT,
00313
00314     /* CLC6_out is the Timer external reset source
00315     */
00316     TMR2_CLC6_OUT,
00317
00318     /* CLC7_out is the Timer external reset source
00319     */
00320     TMR2_CLC7_OUT,
00321
00322     /* CLC8_out is the Timer external reset source
00323     */
00324     TMR2_CLC8_OUT,
00325
00326     /* UART1_rx_edge is the Timer external reset source
00327     */
00328     TMR2_UART1_RX_EDGE,
00329
00330     /* UART1_tx_edge is the Timer external reset source
00331     */
00332     TMR2_UART1_TX_EDGE,
00333
00334     /* UART2_rx_edge is the Timer external reset source
00335     */
00336     TMR2_UART2_RX_EDGE,
00337
00338     /* UART2_tx_edge is the Timer external reset source
00339     */
00340     TMR2_UART2_TX_EDGE,
00341
00342     /* UART3_rx_edge is the Timer external reset source
00343     */
00344     TMR2_UART3_RX_EDGE,
00345
00346     /* UART3_tx_edge is the Timer external reset source
00347     */
00348     TMR2_UART3_TX_EDGE,
00349
00350     /* UART4_rx_edge is the Timer external reset source
00351     */
00352     TMR2_UART4_RX_EDGE,
00353
00354     /* UART4_tx_edge is the Timer external reset source
00355     */
00356     TMR2_UART4_TX_EDGE,
00357
00358     /* UART5_rx_edge is the Timer external reset source
00359     */
00360     TMR2_UART5_RX_EDGE,
00361
00362     /* UART5_tx_edge is the Timer external reset source
```

```

00363      */
00364      TMR2_UART5_TX_EDGE,
00365
00366      /* Reserved enum cannot be used
00367      */
00368      TMR2_RESERVED_4
00369
00370
00371 } TMR2_HLT_EXT_RESET_SOURCE;
00372
00373
00377
00381
00413 void TMR2_Initialize(void);
00414
00449 void TMR2_ModeSet (TMR2_HLT_MODE mode);
00450
00484 void TMR2_ExtResetSourceSet (TMR2_HLT_EXT_RESET_SOURCE reset);
00485
00513 void TMR2_Start(void);
00514
00542 void TMR2_StartTimer(void);
00543
00574 void TMR2_Stop(void);
00575
00606 void TMR2_StopTimer(void);
00607
00641 uint8_t TMR2_Counter8BitGet(void);
00642
00676 uint8_t TMR2_ReadTimer(void);
00677
00715 void TMR2_Counter8BitSet(uint8_t timerVal);
00716
00754 void TMR2_WriteTimer(uint8_t timerVal);
00755
00806 void TMR2_Period8BitSet(uint8_t periodVal);
00807
00858 void TMR2_LoadPeriodRegister(uint8_t periodVal);
00859
00896 bool TMR2_HasOverflowOccured(void);
00897
00898 #ifdef __cplusplus // Provide C++ Compatibility
00899
00900     }
00901
00902 #endif
00903
00904 #endif // TMR2_H

```

6.66 tmr5.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "tmr5.h"
00053 #include "interrupt_manager.h"
00054

```



```
00058 volatile uint16_t timer5ReloadVal;
00059 void (*TMR5_InterruptHandler)(void);
00060
00064
00065 void TMR5_Initialize(void)
00066 {
00067     //Set the Timer to the options selected in the GUI
00068
00069     //T5GE disabled; T5GTM disabled; T5GPOL low; T5GGO done; T5GSPM disabled;
00070     T5GCON = 0x00;
00071
00072     //GSS T5G_pin;
00073     T5GATE = 0x00;
00074
00075     //CS FOSC/4;
00076     T5CLK = 0x01;
00077
00078     //TMR5H 99;
00079     TMR5H = 0x63;
00080
00081     //TMR5L 192;
00082     TMR5L = 0xC0;
00083
00084     // Clearing IF flag before enabling the interrupt.
00085     PIR8bits.TMR5IF = 0;
00086
00087     // Load the TMR value to reload variable
00088     timer5ReloadVal=(uint16_t)((TMR5H « 8) | TMR5L);
00089
00090     // Enabling TMR5 interrupt.
00091     PIE8bits.TMR5IE = 1;
00092
00093     // Set Default Interrupt Handler
00094     TMR5_SetInterruptHandler(TMR5_DefaultInterruptHandler);
00095
00096     // CKPS 1:8; NOT_SYNC synchronize; TMR5ON enabled; T5RD16 disabled;
00097     T5CON = 0x31;
00098 }
00099
00100 void TMR5_StartTimer(void)
00101 {
00102     // Start the Timer by writing to TMRxON bit
00103     T5CONbits.TMR5ON = 1;
00104 }
00105
00106 void TMR5_StopTimer(void)
00107 {
00108     // Stop the Timer by writing to TMRxON bit
00109     T5CONbits.TMR5ON = 0;
00110 }
00111
00112 uint16_t TMR5_ReadTimer(void)
00113 {
00114     uint16_t readVal;
00115     uint8_t readValHigh;
00116     uint8_t readValLow;
00117
00118     T5CONbits.T5RD16 = 1;
00119
00120     readValLow = TMR5L;
00121     readValHigh = TMR5H;
00122
00123     readVal = ((uint16_t)readValHigh « 8) | readValLow;
00124
00125     return readVal;
00126 }
00127
00128 void TMR5_WriteTimer(uint16_t timerVal)
00129 {
00130     if (T5CONbits.NOT_SYNC == 1)
00131     {
00132         // Stop the Timer by writing to TMRxON bit
00133         T5CONbits.TMR5ON = 0;
00134
00135         // Write to the Timer5 register
00136         TMR5H = (uint8_t)(timerVal » 8);
00137         TMR5L = (uint8_t)timerVal;
00138
00139         // Start the Timer after writing to the register
00140         T5CONbits.TMR5ON =1;
00141     }
00142     else
00143     {
00144         // Write to the Timer5 register
00145         TMR5H = (uint8_t)(timerVal » 8);
00146         TMR5L = (uint8_t)timerVal;
00147     }
}
```

```

00148 }
00149
00150 void TMR5_Reload(void)
00151 {
00152     TMR5_WriteTimer(timer5ReloadVal);
00153 }
00154
00155 void TMR5_StartSinglePulseAcquisition(void)
00156 {
00157     T5GCONbits.T5GGO = 1;
00158 }
00159
00160 uint8_t TMR5_CheckGateValueStatus(void)
00161 {
00162     return (T5GCONbits.T5GVAL);
00163 }
00164
00165 void __interrupt(irq(TMR5),base(8),low_priority) TMR5_ISR()
00166 {
00167     static volatile unsigned int CountCallBack = 0;
00168
00169     // Clear the TMR5 interrupt flag
00170     PIR8bits.TMR5IF = 0;
00171     TMR5_WriteTimer(timer5ReloadVal);
00172
00173     // callback function - called every 50th pass
00174     if (++CountCallBack >= TMR5_INTERRUPT_TICKER_FACTOR)
00175     {
00176         // ticker function call
00177         TMR5_CallBack();
00178
00179         // reset ticker counter
00180         CountCallBack = 0;
00181     }
00182 }
00183
00184 void TMR5_CallBack(void)
00185 {
00186     // Add your custom callback code here
00187     if(TMR5_InterruptHandler)
00188     {
00189         TMR5_InterruptHandler();
00190     }
00191 }
00192
00193 void TMR5_SetInterruptHandler(void (* InterruptHandler)(void)){
00194     TMR5_InterruptHandler = InterruptHandler;
00195 }
00196
00197 void TMR5_DefaultInterruptHandler(void){
00198     // add your TMR5 interrupt custom code
00199     // or set custom function using TMR5_SetInterruptHandler()
00200 }
00201

```

6.67 tmr5.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */

```

```

00046
00047 #ifndef TMR5_H
00048 #define TMR5_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00063 #define TMR5_INTERRUPT_TICKER_FACTOR    50
00064
00068
00101 void TMR5_Initialize(void);
00102
00130 void TMR5_StartTimer(void);
00131
00162 void TMR5_StopTimer(void);
00163
00197 uint16_t TMR5_ReadTimer(void);
00198
00236 void TMR5_WriteTimer(uint16_t timerVal);
00237
00272 void TMR5_Reload(void);
00273
00311 void TMR5_StartSinglePulseAcquisition(void);
00312
00350 uint8_t TMR5_CheckGateValueStatus(void);
00351
00368 void TMR5_CallBack(void);
00369
00386 void TMR5_SetInterruptHandler(void (* InterruptHandler)(void));
00387
00404 extern void (*TMR5_InterruptHandler)(void);
00405
00422 void TMR5_DefaultInterruptHandler(void);
00423
00424 #ifdef __cplusplus // Provide C++ Compatibility
00425     }
00426 }
00427
00428 #endif
00429
00430 #endif // TMR5_H

```

6.68 tmr6.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "tmr6.h"
00053 #include "interrupt_manager.h"
00054 #include "../timers.h"

```

```

00055
00056 extern volatile uint16_t tickCount[TMR_COUNT];
00060
00061 void (*TMR6_InterruptHandler)(void);
00062
00066
00067 void TMR6_Initialize(void)
00068 {
00069     // Set TMR6 to the options selected in the User Interface
00070
00071     // T6CS FOSC/4;
00072     T6CLKCON = 0x01;
00073
00074     // T6PSYNC Not Synchronized; T6MODE Software control; T6CKPOL Rising Edge; T6CKSYNC Not
    Synchronized;
00075     T6HLT = 0x00;
00076
00077     // T6RSEL T6CKIPPS pin;
00078     T6RST = 0x00;
00079
00080     // PR6 127;
00081     T6PR = 0x7F;
00082
00083     // TMR6 0;
00084     T6TMR = 0x00;
00085
00086     // Clearing IF flag before enabling the interrupt.
00087     PIR15bits.TMR6IF = 0;
00088
00089     // Enabling TMR6 interrupt.
00090     PIE15bits.TMR6IE = 1;
00091
00092     // Set Default Interrupt Handler
00093     TMR6_SetInterruptHandler(TMR6_DefaultInterruptHandler);
00094
00095     // T6CKPS 1:16; T6OUTPS 1:4; TMR6ON on;
00096     T6CON = 0xC3;
00097 }
00098
00099 void TMR6_ModeSet(TMR6_HLT_MODE mode)
00100 {
00101     // Configure different types HLT mode
00102     T6HLTbits.MODE = mode;
00103 }
00104
00105 void TMR6_ExtResetSourceSet(TMR6_HLT_EXT_RESET_SOURCE reset)
00106 {
00107     //Configure different types of HLT external reset source
00108     T6RSTbits.RSEL = reset;
00109 }
00110
00111 void TMR6_Start(void)
00112 {
00113     // Start the Timer by writing to TMRxON bit
00114     T6CONbits.TMR6ON = 1;
00115 }
00116
00117 void TMR6_StartTimer(void)
00118 {
00119     TMR6_Start();
00120 }
00121
00122 void TMR6_Stop(void)
00123 {
00124     // Stop the Timer by writing to TMRxON bit
00125     T6CONbits.TMR6ON = 0;
00126 }
00127
00128 void TMR6_StopTimer(void)
00129 {
00130     TMR6_Stop();
00131 }
00132
00133 uint8_t TMR6_Counter8BitGet(void)
00134 {
00135     uint8_t readVal;
00136
00137     readVal = TMR6;
00138
00139     return readVal;
00140 }
00141
00142 uint8_t TMR6_ReadTimer(void)
00143 {
00144     return TMR6_Counter8BitGet();
00145 }
00146

```

```

00147 void TMR6_Counter8BitSet(uint8_t timerVal)
00148 {
00149     // Write to the Timer6 register
00150     TMR6 = timerVal;
00151 }
00152
00153 void TMR6_WriteTimer(uint8_t timerVal)
00154 {
00155     TMR6_Counter8BitSet(timerVal);
00156 }
00157
00158 void TMR6_Period8BitSet(uint8_t periodVal)
00159 {
00160     PR6 = periodVal;
00161 }
00162
00163 void TMR6_LoadPeriodRegister(uint8_t periodVal)
00164 {
00165     TMR6_Period8BitSet(periodVal);
00166 }
00167
00168 void __interrupt(irq(TMR6),base(8),low_priority) TMR6_ISR()
00169 {
00170
00171     // clear the TMR6 interrupt flag
00172     PIR15bits.TMR6IF = 0;
00173
00174     // ticker function call;
00175     // ticker is 1 -> Callback function gets called everytime this ISR executes
00176     TMR6_CallBack();
00177 }
00178
00179 void TMR6_CallBack(void)
00180 {
00181     // Add your custom callback code here
00182     // this code executes every TMR6_INTERRUPT_TICKER_FACTOR periods of TMR6
00183     if(TMR6_InterruptHandler)
00184     {
00185         TMR6_InterruptHandler();
00186     }
00187 }
00188
00189 void TMR6_SetInterruptHandler(void (* InterruptHandler)(void)){
00190     TMR6_InterruptHandler = InterruptHandler;
00191 }
00192
00193 void TMR6_DefaultInterruptHandler(void)
00194 {
00195     uint8_t i;
00196     // add your TMR6 interrupt custom code
00197     // or set custom function using TMR6_SetInterruptHandler()
00198
00199     //Decrement each software timer
00200     for (i = 0; i < TMR_COUNT; i++) {
00201         if (tickCount[i] != 0) {
00202             tickCount[i]--;
00203         }
00204     }
00205 }
00206

```

6.69 tmr6.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP

```

```

00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef TMR6_H
00048 #define TMR6_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00066
00078
00079 typedef enum
00080 {
00081
00082     /* Roll-over Pulse mode clears the TMRx upon TMRx = PRx, then continue running.
00083     ON bit must be set and is not affected by Resets
00084     */
00085
00086     /* Roll-over Pulse mode indicates that Timer starts
00087     immediately upon ON = 1 (Software Control)
00088     */
00089     TMR6_ROP_STARTS_TMRON,
00090
00091     /* Roll-over Pulse mode indicates that the Timer starts
00092     when ON = 1 and TMRx_ers = 1. Stops when TMRx_ers = 0
00093     */
00094     TMR6_ROP_STARTS_TMRON_ERSHIGH,
00095
00096     /* Roll-over Pulse mode indicates that the Timer starts
00097     when ON = 1 and TMRx_ers = 0. Stops when TMRx_ers = 1
00098     */
00099     TMR6_ROP_STARTS_TMRON_ERSLOW,
00100
00101     /* Roll-over Pulse mode indicates that the Timer resets
00102     upon rising or falling edge of TMRx_ers
00103     */
00104     TMR6_ROP_RESETS_ERSEBOTHEDGE,
00105
00106     /* Roll-over Pulse mode indicates that the Timer resets
00107     upon rising edge of TMRx_ers
00108     */
00109     TMR6_ROP_RESETS_ERSRISINGEDGE,
00110
00111     /* Roll-over Pulse mode indicates that the Timer resets
00112     upon falling edge of TMRx_ers
00113     */
00114     TMR6_ROP_RESETS_ERSEFALLINGEDGE,
00115
00116     /* Roll-over Pulse mode indicates that the Timer resets
00117     upon TMRx_ers = 0
00118     */
00119     TMR6_ROP_RESETS_ERSELOW,
00120
00121     /* Roll-over Pulse mode indicates that the Timer resets
00122     upon TMRx_ers = 1
00123     */
00124     TMR6_ROP_RESETS_ERSEHIGH,
00125
00126     /*In all One-Shot mode the timer resets and the ON bit is
00127     cleared when the timer value matches the PRx period
00128     value. The ON bit must be set by software to start
00129     another timer cycle.
00130     */
00131
00132     /* One shot mode indicates that the Timer starts
00133     immediately upon ON = 1 (Software Control)
00134     */
00135     TMR6_OS_STARTS_TMRON,
00136
00137     /* One shot mode indicates that the Timer starts
00138     when a rising edge is detected on the TMRx_ers
00139     */
00140     TMR6_OS_STARTS_ERSESRISINGEDGE ,
00141
00142     /* One shot mode indicates that the Timer starts
00143     when a falling edge is detected on the TMRx_ers

```

```

00144     */
00145     TMR6_OS_STARTS_ERSFALLINGEDGE ,
00146
00147     /* One shot mode indicates that the Timer starts
00148     when either a rising or falling edge is detected on TMRx_ers
00149     */
00150     TMR6_OS_STARTS_ERSBOTHEDGE,
00151
00152     /* One shot mode indicates that the Timer starts
00153     upon first TMRx_ers rising edge and resets on all
00154     subsequent TMRx_ers rising edges
00155     */
00156     TMR6_OS_STARTS_ERSFIRSTRISINGEDGE,
00157
00158     /* One shot mode indicates that the Timer starts
00159     upon first TMRx_ers falling edge and restarts on all
00160     subsequent TMRx_ers falling edges
00161     */
00162     TMR6_OS_STARTS_ERSFIRSTFALLINGEDGE,
00163
00164     /* One shot mode indicates that the Timer starts
00165     when a rising edge is detected on the TMRx_ers,
00166     resets upon TMRx_ers = 0
00167     */
00168     TMR6_OS_STARTS_ERSRISINGEDGEDETECT,
00169     /* One shot mode indicates that the Timer starts
00170     when a falling edge is detected on the TMRx_ers,
00171     resets upon TMRx_ers = 1
00172     */
00173     TMR6_OS_STARTS_ERSFALLINGEDGEDETECT,
00174
00175     /* One shot mode indicates that the Timer starts
00176     when a TMRx_ers = 1, ON = 1 and resets upon TMRx_ers = 0
00177     */
00178     TMR6_OS_STARTS_TMRON_ERSHIGH = 0x16,
00179
00180     /* One shot mode indicates that the Timer starts
00181     when a TMRx_ers = 0, ON = 1 and resets upon TMRx_ers = 1
00182     */
00183     TMR6_OS_STARTS_TMRON_ERSLOW = 0x17,
00184
00185     /*In all Mono-Stable mode the ON bit must be initially set, but
00186     not cleared upon the TMRx = PRx, and the timer will start upon
00187     an TMRx_ers start event following TMRx = PRx.*/
00188
00189     /* Mono Stable mode indicates that the Timer starts
00190     when a rising edge is detected on the TMRx_ers and ON = 1
00191     */
00192     TMR6_MS_STARTS_TMRON_ERSRISINGEDGEDETECT = 0x11,
00193
00194     /* Mono Stable mode indicates that the Timer starts
00195     when a falling edge is detected on the TMRx_ers and ON = 1
00196     */
00197     TMR6_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT = 0x12,
00198
00199     /* Mono Stable mode indicates that the Timer starts
00200     when either a rising or falling edge is detected on TMRx_ers
00201     and ON = 1
00202     */
00203     TMR6_MS_STARTS_TMRON_ERSBOTHEDGE = 0x13
00204
00205 } TMR6_HLT_MODE;
00206
00219
00220 typedef enum
00221 {
00222     /* T6INPPS is the Timer external reset source
00223     */
00224     TMR6_T6INPPS,
00225
00226     /* Timer2 Postscale is the Timer external reset source
00227     */
00228     TMR6_T2POSTSCALED,
00229
00230     /* Timer4 Postscale is the Timer external reset source
00231     */
00232     TMR6_T4POSTSCALED,
00233
00234     /* Reserved enum cannot be used
00235     */
00236     TMR6_RESERVED,
00237
00238     /* CCP1_OUT is the Timer external reset source
00239     */
00240     TMR6_CCP1_OUT,
00241
00242     /* CCP2_OUT is the Timer external reset source

```

```
00243     */
00244     TMR6_CCP2_OUT,
00245
00246     /* CCP3_OUT is the Timer external reset source
00247     */
00248     TMR6_CCP3_OUT,
00249
00250     /* PWM1S1P1_out is the Timer external reset source
00251     */
00252     TMR6_PWM1S1P1_OUT,
00253
00254     /* PWM1S1P2_out is the Timer external reset source
00255     */
00256     TMR6_PWM1S1P2_OUT,
00257
00258     /* PWM2S1P1_out is the Timer external reset source
00259     */
00260     TMR6_PWM2S1P1_OUT,
00261
00262     /* PWM2S1P2_out is the Timer external reset source
00263     */
00264     TMR6_PWM2S1P2_OUT,
00265
00266     /* PWM3S1P1_out is the Timer external reset source
00267     */
00268     TMR6_PWM3S1P1_OUT,
00269
00270     /* PWM3S1P2_out is the Timer external reset source
00271     */
00272     TMR6_PWM3S1P2_OUT,
00273
00274     /* Reserved enum cannot be used
00275     */
00276     TMR6_RESERVED_2,
00277
00278     /* Reserved enum cannot be used
00279     */
00280     TMR6_RESERVED_3,
00281
00282     /* CMP1_OUT is the Timer external reset source
00283     */
00284     TMR6_CMP1_OUT,
00285
00286     /* CMP2_OUT is the Timer external reset source
00287     */
00288     TMR6_CMP2_OUT,
00289
00290     /* ZCD_Output is the Timer external reset source
00291     */
00292     TMR6_ZCD_OUTPUT,
00293
00294     /* CLC1_out is the Timer external reset source
00295     */
00296     TMR6_CLC1_OUT,
00297
00298     /* CLC2_out is the Timer external reset source
00299     */
00300     TMR6_CLC2_OUT,
00301
00302     /* CLC3_out is the Timer external reset source
00303     */
00304     TMR6_CLC3_OUT,
00305
00306     /* CLC4_out is the Timer external reset source
00307     */
00308     TMR6_CLC4_OUT,
00309
00310     /* CLC5_out is the Timer external reset source
00311     */
00312     TMR6_CLC5_OUT,
00313
00314     /* CLC6_out is the Timer external reset source
00315     */
00316     TMR6_CLC6_OUT,
00317
00318     /* CLC7_out is the Timer external reset source
00319     */
00320     TMR6_CLC7_OUT,
00321
00322     /* CLC8_out is the Timer external reset source
00323     */
00324     TMR6_CLC8_OUT,
00325
00326     /* UART1_rx_edge is the Timer external reset source
00327     */
00328     TMR6_UART1_RX_EDGE,
00329
```



```

00330     /* UART1_tx_edge is the Timer external reset source
00331     */
00332     TMR6_UART1_TX_EDGE,
00333
00334     /* UART2_rx_edge is the Timer external reset source
00335     */
00336     TMR6_UART2_RX_EDGE,
00337
00338     /* UART2_tx_edge is the Timer external reset source
00339     */
00340     TMR6_UART2_TX_EDGE,
00341
00342     /* UART3_rx_edge is the Timer external reset source
00343     */
00344     TMR6_UART3_RX_EDGE,
00345
00346     /* UART3_tx_edge is the Timer external reset source
00347     */
00348     TMR6_UART3_TX_EDGE,
00349
00350     /* UART4_rx_edge is the Timer external reset source
00351     */
00352     TMR6_UART4_RX_EDGE,
00353
00354     /* UART4_tx_edge is the Timer external reset source
00355     */
00356     TMR6_UART4_TX_EDGE,
00357
00358     /* UART5_rx_edge is the Timer external reset source
00359     */
00360     TMR6_UART5_RX_EDGE,
00361
00362     /* UART5_tx_edge is the Timer external reset source
00363     */
00364     TMR6_UART5_TX_EDGE,
00365
00366     /* Reserved enum cannot be used
00367     */
00368     TMR6_RESERVED_4
00369
00370 } TMR6_HLT_EXT_RESET_SOURCE;
00371
00372
00373
00377
00381
00413 void TMR6_Initialize(void);
00414
00449 void TMR6_ModeSet(TMR6_HLT_MODE mode);
00450
00484 void TMR6_ExtResetSourceSet(TMR6_HLT_EXT_RESET_SOURCE reset);
00485
00513 void TMR6_Start(void);
00514
00542 void TMR6_StartTimer(void);
00543
00574 void TMR6_Stop(void);
00575
00606 void TMR6_StopTimer(void);
00607
00641 uint8_t TMR6_Counter8BitGet(void);
00642
00676 uint8_t TMR6_ReadTimer(void);
00677
00715 void TMR6_Counter8BitSet(uint8_t timerVal);
00716
00754 void TMR6_WriteTimer(uint8_t timerVal);
00755
00806 void TMR6_Period8BitSet(uint8_t periodVal);
00807
00858 void TMR6_LoadPeriodRegister(uint8_t periodVal);
00859
00860
00877 void TMR6_CallBack(void);
00894 void TMR6_SetInterruptHandler(void (* InterruptHandler)(void));
00895
00912 extern void (*TMR6_InterruptHandler)(void);
00913
00930 void TMR6_DefaultInterruptHandler(void);
00931
00932
00933 #ifdef __cplusplus // Provide C++ Compatibility
00934
00935     }
00936
00937 #endif
00938

```

```
00939 #endif // TMR6_H
```

6.70 uart1.c

```
00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050 #include <xc.h>
00051 #include "uart1.h"
00052 #include "interrupt_manager.h"
00053
00057 #define UART1_TX_BUFFER_SIZE 64
00058 #define UART1_RX_BUFFER_SIZE 255
00059
00063
00064 static volatile uint8_t uart1TxHead = 0;
00065 static volatile uint8_t uart1TxTail = 0;
00066 static volatile uint8_t uart1TxBuffer[UART1_TX_BUFFER_SIZE];
00067 volatile uint8_t uart1TxBufferRemaining;
00068
00069 static volatile uint8_t uart1RxHead = 0;
00070 static volatile uint8_t uart1RxTail = 0;
00071 static volatile uint8_t uart1RxBuffer[UART1_RX_BUFFER_SIZE];
00072 static volatile uart1_status_t uart1RxStatusBuffer[UART1_RX_BUFFER_SIZE];
00073 volatile uint8_t uart1RxCount;
00074 static volatile uart1_status_t uart1RxLastError;
00075
00079 void (*UART1_FramingErrorHandler)(void);
00080 void (*UART1_OverrunErrorHandler)(void);
00081 void (*UART1_ErrorHandler)(void);
00082
00083 void UART1_DefaultFramingErrorHandler(void);
00084 void UART1_DefaultOverrunErrorHandler(void);
00085 void UART1_DefaultErrorHandler(void);
00086
00087 void UART1_Initialize(void)
00088 {
00089     // Disable interrupts before changing states
00090     PIE4bits.U1RXIE = 0;
00091     UART1_SetRxInterruptHandler(UART1_Receive_ISR);
00092     PIE4bits.U1TXIE = 0;
00093     UART1_SetTxInterruptHandler(UART1_Transmit_ISR);
00094
00095     // Set the UART1 module to the options selected in the user interface.
00096
00097     // P1L 0;
00098     U1P1L = 0x00;
00099
00100     // P1H 0;
00101     U1P1H = 0x00;
00102
00103     // P2L 0;
00104     U1P2L = 0x00;
00105
00106     // P2H 0;
00107     U1P2H = 0x00;
00108
00109     // P3L 0;
00110     U1P3L = 0x00;
```

```

00111
00112     // P3H 0;
00113     U1P3H = 0x00;
00114
00115     // BRGS high speed; MODE Asynchronous 8-bit mode; RXEN enabled; TXEN enabled; ABDEN disabled;
00116     U1CON0 = 0xB0;
00117
00118     // RXBIMD Set RXBKIF on rising RX input; BRKOVr disabled; WUE disabled; SENDB disabled; ON
00119     enabled;
00120     U1CON1 = 0x80;
00121
00122     // TXPOL not inverted; FLO off; COEN Checksum Mode 0; RXPOL not inverted; RUNOVF RX input shifter
00123     stops all activity; STP Transmit 1Stop bit, receiver verifies first Stop bit;
00124     U1CON2 = 0x00;
00125
00126     // BRGL 130;
00127     U1BRGL = 0x82; // 82
00128
00129     // BRGH 6;
00130     U1BRGH = 0x06; // 06
00131
00132     // STPMD in middle of first Stop bit; TXWRE No error;
00133     U1FIFO = 0x00;
00134
00135     // ABDIF Auto-baud not enabled or not complete; WUIF WUE not enabled by software; ABDIE disabled;
00136     U1UIR = 0x00;
00137
00138     // ABDOVF Not overflowed; TXCIF 0; RXBKIF No Break detected; RXFOIF not overflowed; CERIF No
00139     Checksum error;
00140     U1ERRIR = 0x00;
00141
00142     // TXCIE disabled; FERRIE disabled; TXMTIE disabled; ABDOVE disabled; CERIE disabled; RXFOIE
00143     disabled; PERIE disabled; RXBKIE disabled;
00144     U1ERRIE = 0x00;
00145
00146     UART1_SetFramingErrorHandler(UART1_DefaultFramingErrorHandler);
00147     UART1_SetOverrunErrorHandler(UART1_DefaultOverrunErrorHandler);
00148     UART1_SetErrorHandler(UART1_DefaultErrorHandler);
00149
00150     uart1RxLastError.status = 0;
00151
00152     // initializing the driver state
00153     uart1TxHead = 0;
00154     uart1TxTail = 0;
00155     uart1TxBufferRemaining = sizeof(uart1TxBuffer);
00156     uart1RxHead = 0;
00157     uart1RxTail = 0;
00158     uart1RxCount = 0;
00159
00160     // enable receive interrupt
00161     PIE4bits.U1RXIE = 1;
00162 }
00163
00164 bool UART1_is_rx_ready(void)
00165 {
00166     return(uart1RxCount ? true : false);
00167 }
00168
00169 bool UART1_is_tx_ready(void)
00170 {
00171     return(uart1TxBufferRemaining ? true : false);
00172 }
00173
00174 bool UART1_is_tx_done(void)
00175 {
00176     return U1ERRIRbits.TXMTIF;
00177 }
00178
00179 uart1_status_t UART1_get_last_status(void)
00180 {
00181     return uart1RxLastError;
00182 }
00183
00184 uint8_t UART1_Read(void)
00185 {
00186     uint8_t readValue = 0;
00187
00188     while (0 == uart1RxCount) {
00189     }
00190
00191     uart1RxLastError = uart1RxStatusBuffer[uart1RxTail];
00192
00193     readValue = uart1RxBuffer[uart1RxTail++];
00194     if (sizeof(uart1RxBuffer) <= uart1RxTail) {
00195         uart1RxTail = 0;
00196     }
00197 }

```

```

00194     PIE4bits.U1RXIE = 0;
00195     uart1RxCount--;
00196     PIE4bits.U1RXIE = 1;
00197
00198     return readValue;
00199 }
00200
00201 void UART1_Write(uint8_t txData)
00202 {
00203     while (0 == uart1TxBufferRemaining) {
00204     }
00205
00206     if (0 == PIE4bits.U1TXIE) {
00207         U1TXB = txData;
00208     } else {
00209         PIE4bits.U1TXIE = 0;
00210         uart1TxBuffer[uart1TxHead++] = txData;
00211         if (sizeof(uart1TxBuffer) <= uart1TxHead) {
00212             uart1TxHead = 0;
00213         }
00214         uart1TxBufferRemaining--;
00215     }
00216     PIE4bits.U1TXIE = 1;
00217 }
00218
00219 void __interrupt(irq(U1TX), base(8)) UART1_tx_vect_isr()
00220 {
00221     if (UART1_TxInterruptHandler) {
00222         UART1_TxInterruptHandler();
00223     }
00224 }
00225
00226 void __interrupt(irq(U1RX), base(8)) UART1_rx_vect_isr()
00227 {
00228     if (UART1_RxInterruptHandler) {
00229         UART1_RxInterruptHandler();
00230     }
00231 }
00232
00233 void UART1_Transmit_ISR(void)
00234 {
00235     // use this default transmit interrupt handler code
00236     if (sizeof(uart1TxBuffer) > uart1TxBufferRemaining) {
00237         U1TXB = uart1TxBuffer[uart1TxTail++];
00238         if (sizeof(uart1TxBuffer) <= uart1TxTail) {
00239             uart1TxTail = 0;
00240         }
00241         uart1TxBufferRemaining++;
00242     } else {
00243         PIE4bits.U1TXIE = 0;
00244     }
00245
00246     // or set custom function using UART1_SetTxInterruptHandler()
00247 }
00248
00249 void UART1_Receive_ISR(void)
00250 {
00251     // use this default receive interrupt handler code
00252     uart1RxStatusBuffer[uart1RxHead].status = 0;
00253
00254     if (U1ERRIRbits.FERIF) {
00255         uart1RxStatusBuffer[uart1RxHead].ferr = 1;
00256         UART1_FramingErrorHandler();
00257     }
00258
00259     if (U1ERRIRbits.RXFOIF) {
00260         uart1RxStatusBuffer[uart1RxHead].oerr = 1;
00261         UART1_OverrunErrorHandler();
00262     }
00263
00264     if (uart1RxStatusBuffer[uart1RxHead].status) {
00265         UART1_ErrorHandler();
00266     } else {
00267         UART1_RxDataHandler();
00268     }
00269
00270     // or set custom function using UART1_SetRxInterruptHandler()
00271 }
00272
00273 void UART1_RxDataHandler(void)
00274 {
00275     // use this default receive interrupt handler code
00276     uart1RxBuffer[uart1RxHead++] = U1RXB;
00277     if (sizeof(uart1RxBuffer) <= uart1RxHead) {
00278         uart1RxHead = 0;
00279     }
00280     uart1RxCount++;

```

```

00281 }
00282
00283 void UART1_DefaultFramingErrorHandler(void)
00284 {
00285 }
00286
00287 void UART1_DefaultOverrunErrorHandler(void)
00288 {
00289 }
00290
00291 void UART1_DefaultErrorHandler(void)
00292 {
00293     UART1_RxDataHandler();
00294 }
00295
00296 void UART1_SetFramingErrorHandler(void (* interruptHandler)(void))
00297 {
00298     UART1_FramingErrorHandler = interruptHandler;
00299 }
00300
00301 void UART1_SetOverrunErrorHandler(void (* interruptHandler)(void))
00302 {
00303     UART1_OverrunErrorHandler = interruptHandler;
00304 }
00305
00306 void UART1_SetErrorHandler(void (* interruptHandler)(void))
00307 {
00308     UART1_ErrorHandler = interruptHandler;
00309 }
00310
00311 void UART1_SetRxInterruptHandler(void (* InterruptHandler)(void))
00312 {
00313     UART1_RxInterruptHandler = InterruptHandler;
00314 }
00315
00316 void UART1_SetTxInterruptHandler(void (* InterruptHandler)(void))
00317 {
00318     UART1_TxInterruptHandler = InterruptHandler;
00319 }
00320
00321 /* stuff the uart1 receive buffer with testing data */
00322 void UART1_put_buffer(uint8_t bufData)
00323 {
00324     INTERRUPT_GlobalInterruptHighDisable();
00325     INTERRUPT_GlobalInterruptLowDisable();
00326     PIE4bits.U1RXIE = 0;
00327     uart1RxBuffer[uart1RxHead++] = bufData;
00328     if (sizeof(uart1RxBuffer) <= uart1RxHead) {
00329         uart1RxHead = 0;
00330     }
00331
00332     uart1RxCount++;
00333     PIE4bits.U1RXIE = 1;
00334     INTERRUPT_GlobalInterruptLowEnable();
00335     INTERRUPT_GlobalInterruptHighEnable();
00336 }
00337
00338 /*
00339  * on the fly bps selection
00340  */
00341 void UART1_Initialize19200(void)
00342 {
00343     // Disable interrupts before changing states
00344     PIE4bits.U1RXIE = 0;
00345     UART1_SetRxInterruptHandler(UART1_Receive_ISR);
00346     PIE4bits.U1TXIE = 0;
00347     UART1_SetTxInterruptHandler(UART1_Transmit_ISR);
00348     PIE4bits.U1EIE = 0;
00349
00350
00351     // Set the UART1 module to the options selected in the user interface.
00352
00353     // P1L 0;
00354     U1P1L = 0x00;
00355
00356     // P1H 0;
00357     U1P1H = 0x00;
00358
00359     // P2L 0;
00360     U1P2L = 0x00;
00361
00362     // P2H 0;
00363     U1P2H = 0x00;
00364
00365     // P3L 0;
00366     U1P3L = 0x00;
00367

```

```

00368     // P3H 0;
00369     U1P3H = 0x00;
00370
00371     // BRGS high speed; MODE Asynchronous 8-bit mode; RXEN enabled; TXEN enabled; ABDEN disabled;
00372     U1CON0 = 0xB0;
00373
00374     // RXBIMD Set RXBKIF on rising RX input; BRKOVr disabled; WUE disabled; SENDB disabled; ON
    enabled;
00375     U1CON1 = 0x80;
00376
00377     // TXPOL not inverted; FLO off; COEN Checksum Mode 0; RXPOL not inverted; RUNOVF RX input shifter
    stops all activity; STP Transmit 1Stop bit, receiver verifies first Stop bit;
00378     U1CON2 = 0x00;
00379
00380     // BRGL 64;
00381     U1BRGL = 0x40;
00382
00383     // BRGH 3;
00384     U1BRGH = 0x03;
00385
00386     // STPMD in middle of first Stop bit; TXWRE No error;
00387     U1FIFO = 0x00;
00388
00389     // ABDIF Auto-baud not enabled or not complete; WUIF WUE not enabled by software; ABDIE disabled;
00390     U1UIR = 0x00;
00391
00392     // ABDOVF Not overflowed; TXCIF 0; RXBKIF No Break detected; RXFOIF not overflowed; CERIF No
    Checksum error;
00393     U1ERRIR = 0x00;
00394
00395     // TXCIE disabled; FERIC disabled; TXMTIE disabled; ABDOVF disabled; CERIE disabled; RXFOIE
    disabled; PERIE disabled; RXBKIE disabled;
00396     U1ERRIE = 0x00;
00397
00398
00399     UART1_SetOverrunErrorHandler(UART1_DefaultOverrunErrorHandler);
00400     UART1_SetErrorHandler(UART1_DefaultErrorHandler);
00401
00402     uart1RxLastError.status = 0;
00403
00404     // initializing the driver state
00405     uart1TxHead = 0;
00406     uart1TxTail = 0;
00407     uart1TxBufferRemaining = sizeof(uart1TxBuffer);
00408     uart1RxHead = 0;
00409     uart1RxTail = 0;
00410     uart1RxCount = 0;
00411
00412     // enable receive interrupt
00413     PIE4bits.U1RXIE = 1;
00414     // enable error interrupt
00415     PIE4bits.U1EIE = 1;
00416 }

```

6.71 uart1.h

```

00001
00023
00024  /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
    party
00029     license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045  */
00046

```

```

00047 #ifndef UART1_H
00048 #define UART1_H
00049
00053
00054 #include <xc.h>
00055 #include <stdbool.h>
00056 #include <stdint.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059     extern "C" {
00060
00061
00062 #endif
00063
00067
00068 #define UART1_DataReady (UART1_is_rx_ready())
00069
00073
00074 typedef union {
00075     struct {
00076         unsigned perr : 1;
00077         unsigned ferr : 1;
00078         unsigned oerr : 1;
00079         unsigned reserved : 5;
00080     };
00081     uint8_t status;
00082 }uart1_status_t;
00083
00087 extern volatile uint8_t uart1TxBufferRemaining;
00088 extern volatile uint8_t uart1RxCount;
00089
00093
00115 void UART1_Initialize(void);
00116
00163 bool UART1_is_rx_ready(void);
00164
00211 bool UART1_is_tx_ready(void);
00212
00258 bool UART1_is_tx_done(void);
00259
00306 uart1_status_t UART1_get_last_status(void);
00307
00355 uint8_t UART1_Read(void);
00356
00380 void UART1_Write(uint8_t txData);
00381
00401 void UART1_Transmit_ISR(void);
00402
00422 void UART1_Receive_ISR(void);
00423
00443 void UART1_RxDataHandler(void);
00444
00461 void UART1_SetFramingErrorHandler(void (* interruptHandler)(void));
00462
00479 void UART1_SetOverrunErrorHandler(void (* interruptHandler)(void));
00480
00497 void UART1_SetErrorHandler(void (* interruptHandler)(void));
00498
00499
00500
00517 void (*UART1_RxInterruptHandler)(void);
00518
00535 void (*UART1_TxInterruptHandler)(void);
00536
00537
00538
00555 void UART1_SetRxInterruptHandler(void (* InterruptHandler)(void));
00556
00573 void UART1_SetTxInterruptHandler(void (* InterruptHandler)(void));
00574
00575 void UART1_put_buffer(uint8_t);
00576
00577 #ifdef __cplusplus // Provide C++ Compatibility
00578     }
00579
00580
00581 #endif
00582
00583 #endif // UART1_H

```

6.72 uart2.c

```
00001
```

```

00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050 #include <xc.h>
00051 #include "uart2.h"
00052 #include "interrupt_manager.h"
00053
00057 #define UART2_TX_BUFFER_SIZE 64
00058 #define UART2_RX_BUFFER_SIZE 255
00059
00063
00064 static volatile uint8_t uart2TxHead = 0;
00065 static volatile uint8_t uart2TxTail = 0;
00066 static volatile uint8_t uart2TxBuffer[UART2_TX_BUFFER_SIZE];
00067 volatile uint8_t uart2TxBufferRemaining;
00068
00069 static volatile uint8_t uart2RxHead = 0;
00070 static volatile uint8_t uart2RxTail = 0;
00071 static volatile uint8_t uart2RxBuffer[UART2_RX_BUFFER_SIZE];
00072 static volatile uart2_status_t uart2RxStatusBuffer[UART2_RX_BUFFER_SIZE];
00073 volatile uint8_t uart2RxCount;
00074 static volatile uart2_status_t uart2RxLastError;
00075
00079 void (*UART2_FramingErrorHandler)(void);
00080 void (*UART2_OverrunErrorHandler)(void);
00081 void (*UART2_ErrorHandler)(void);
00082
00083 void UART2_DefaultFramingErrorHandler(void);
00084 void UART2_DefaultOverrunErrorHandler(void);
00085 void UART2_DefaultErrorHandler(void);
00086
00087 void UART2_Initialize(void)
00088 {
00089     // Disable interrupts before changing states
00090     PIE8bits.U2RXIE = 0;
00091     UART2_SetRxInterruptHandler(UART2_Receive_ISR);
00092     PIE8bits.U2TXIE = 0;
00093     UART2_SetTxInterruptHandler(UART2_Transmit_ISR);
00094
00095     // Set the UART2 module to the options selected in the user interface.
00096
00097     // P1L 0;
00098     U2P1L = 0x00;
00099
00100     // P1H 0;
00101     U2P1H = 0x00;
00102
00103     // P2L 0;
00104     U2P2L = 0x00;
00105
00106     // P2H 0;
00107     U2P2H = 0x00;
00108
00109     // P3L 0;
00110     U2P3L = 0x00;
00111
00112     // P3H 0;
00113     U2P3H = 0x00;
00114
00115     // BRGS high speed; MODE Asynchronous 8-bit mode; RXEN enabled; TXEN enabled; ABDEN disabled;
00116     U2CON0 = 0xB0;
00117
00118     // RXBIMD Set RXBKIF on rising RX input; BRKOVr disabled; WUE disabled; SENDB disabled; ON
    enabled;

```



```

00119     U2CON1 = 0x80;
00120
00121     // TXPOL not inverted; FLO off; C0EN Checksum Mode 0; RXPOL not inverted; RUNOVF RX input shifter
stops all activity; STP Transmit 1Stop bit, receiver verifies first Stop bit;
00122     U2CON2 = 0x00;
00123
00124     // BRGL 130;
00125     U2BRGL = 0x82;
00126
00127     // BRGH 6;
00128     U2BRGH = 0x06;
00129
00130     // STPMD in middle of first Stop bit; TXWRE No error;
00131     U2FIFO = 0x00;
00132
00133     // ABDIF Auto-baud not enabled or not complete; WUIF WUE not enabled by software; ABDIE disabled;
00134     U2UIR = 0x00;
00135
00136     // ABDOVF Not overflowed; TXCIF 0; RXBKIF No Break detected; RXFOIF not overflowed; CERIF No
Checksum error;
00137     U2ERRIR = 0x00;
00138
00139     // TXCIE disabled; FERRIE disabled; TXMTIE disabled; ABDOVE disabled; CERIE disabled; RXFOIE
disabled; PERIE disabled; RXBKIE disabled;
00140     U2ERRIE = 0x00;
00141
00142
00143     UART2_SetFramingErrorHandler(UART2_DefaultFramingErrorHandler);
00144     UART2_SetOverrunErrorHandler(UART2_DefaultOverrunErrorHandler);
00145     UART2_SetErrorHandler(UART2_DefaultErrorHandler);
00146
00147     uart2RxLastError.status = 0;
00148
00149     // initializing the driver state
00150     uart2TxHead = 0;
00151     uart2TxTail = 0;
00152     uart2TxBufferRemaining = sizeof(uart2TxBuffer);
00153     uart2RxHead = 0;
00154     uart2RxTail = 0;
00155     uart2RxCount = 0;
00156
00157     // enable receive interrupt
00158     PIE8bits.U2RXIE = 1;
00159 }
00160
00161 bool UART2_is_rx_ready(void)
00162 {
00163     return(uart2RxCount ? true : false);
00164 }
00165
00166 bool UART2_is_tx_ready(void)
00167 {
00168     return(uart2TxBufferRemaining ? true : false);
00169 }
00170
00171 bool UART2_is_tx_done(void)
00172 {
00173     return U2ERRIRbits.TXMTIF;
00174 }
00175
00176 uart2_status_t UART2_get_last_status(void)
00177 {
00178     return uart2RxLastError;
00179 }
00180
00181 uint8_t UART2_Read(void)
00182 {
00183     uint8_t readValue = 0;
00184
00185     while (0 == uart2RxCount) {
00186     }
00187
00188     uart2RxLastError = uart2RxStatusBuffer[uart2RxTail];
00189
00190     readValue = uart2RxBuffer[uart2RxTail++];
00191     if (sizeof(uart2RxBuffer) <= uart2RxTail) {
00192         uart2RxTail = 0;
00193     }
00194     PIE8bits.U2RXIE = 0;
00195     uart2RxCount--;
00196     PIE8bits.U2RXIE = 1;
00197
00198     return readValue;
00199 }
00200
00201 void UART2_Write(uint8_t txData)
00202 {

```

```

00203     while (0 == uart2TxBufferRemaining) {
00204     }
00205
00206     if (0 == PIE8bits.U2TXIE) {
00207         U2TXB = txData;
00208     } else {
00209         PIE8bits.U2TXIE = 0;
00210         uart2TxBuffer[uart2TxHead++] = txData;
00211         if (sizeof(uart2TxBuffer) <= uart2TxHead) {
00212             uart2TxHead = 0;
00213         }
00214         uart2TxBufferRemaining--;
00215     }
00216     PIE8bits.U2TXIE = 1;
00217 }
00218
00219 void __interrupt (irq(U2TX), base(8)) UART2_tx_vect_isr()
00220 {
00221     if (UART2_TxInterruptHandler) {
00222         UART2_TxInterruptHandler();
00223     }
00224 }
00225
00226 void __interrupt (irq(U2RX), base(8)) UART2_rx_vect_isr()
00227 {
00228     if (UART2_RxInterruptHandler) {
00229         UART2_RxInterruptHandler();
00230     }
00231 }
00232
00233 void UART2_Transmit_ISR(void)
00234 {
00235     // use this default transmit interrupt handler code
00236     if (sizeof(uart2TxBuffer) > uart2TxBufferRemaining) {
00237         U2TXB = uart2TxBuffer[uart2TxTail++];
00238         if (sizeof(uart2TxBuffer) <= uart2TxTail) {
00239             uart2TxTail = 0;
00240         }
00241         uart2TxBufferRemaining++;
00242     } else {
00243         PIE8bits.U2TXIE = 0;
00244     }
00245
00246     // or set custom function using UART2_SetTxInterruptHandler()
00247 }
00248
00249 void UART2_Receive_ISR(void)
00250 {
00251     // use this default receive interrupt handler code
00252     uart2RxStatusBuffer[uart2RxHead].status = 0;
00253
00254     if (U2ERRIRbits.FERIF) {
00255         uart2RxStatusBuffer[uart2RxHead].ferr = 1;
00256         UART2_FramingErrorHandler();
00257     }
00258
00259     if (U2ERRIRbits.RXFOIF) {
00260         uart2RxStatusBuffer[uart2RxHead].oerr = 1;
00261         UART2_OverrunErrorHandler();
00262     }
00263
00264     if (uart2RxStatusBuffer[uart2RxHead].status) {
00265         UART2_ErrorHandler();
00266     } else {
00267         UART2_RxDataHandler();
00268     }
00269
00270     // or set custom function using UART2_SetRxInterruptHandler()
00271 }
00272
00273 void UART2_RxDataHandler(void)
00274 {
00275     // use this default receive interrupt handler code
00276     uart2RxBuffer[uart2RxHead++] = U2RXB;
00277     if (sizeof(uart2RxBuffer) <= uart2RxHead) {
00278         uart2RxHead = 0;
00279     }
00280     uart2RxCount++;
00281 }
00282
00283 void UART2_DefaultFramingErrorHandler(void)
00284 {
00285 }
00286
00287 void UART2_DefaultOverrunErrorHandler(void)
00288 {
00289     U2ERRIRbits.RXFOIF = 0;

```

```

00290 }
00291
00292 void UART2_DefaultErrorHandler(void)
00293 {
00294     UART2_RxDataHandler();
00295 }
00296
00297 void UART2_SetFramingErrorHandler(void (* interruptHandler)(void))
00298 {
00299     UART2_FramingErrorHandler = interruptHandler;
00300 }
00301
00302 void UART2_SetOverrunErrorHandler(void (* interruptHandler)(void))
00303 {
00304     UART2_OverrunErrorHandler = interruptHandler;
00305 }
00306
00307 void UART2_SetErrorHandler(void (* interruptHandler)(void))
00308 {
00309     UART2_ErrorHandler = interruptHandler;
00310 }
00311
00312 void UART2_SetRxInterruptHandler(void (* InterruptHandler)(void))
00313 {
00314     UART2_RxInterruptHandler = InterruptHandler;
00315 }
00316
00317 void UART2_SetTxInterruptHandler(void (* InterruptHandler)(void))
00318 {
00319     UART2_TxInterruptHandler = InterruptHandler;
00320 }
00321
00322 /* stuff the uart2 receive buffer with testing data */
00323 void UART2_put_buffer(uint8_t bufData)
00324 {
00325     INTERRUPT_GlobalInterruptHighDisable();
00326     INTERRUPT_GlobalInterruptLowDisable();
00327     PIE8bits.U2RXIE = 0;
00328     uart2RxBuffer[uart2RxHead++] = bufData;
00329     if (sizeof(uart2RxBuffer) <= uart2RxHead) {
00330         uart2RxHead = 0;
00331     }
00332
00333     uart2RxCount++;
00334     PIE8bits.U2RXIE = 1;
00335     INTERRUPT_GlobalInterruptLowEnable();
00336     INTERRUPT_GlobalInterruptHighEnable();
00337 }
00338
00339 /*
00340  * on the fly bps selection
00341  */
00342 void UART2_Initialize19200(void)
00343 {
00344     // Disable interrupts before changing states
00345     PIE8bits.U2RXIE = 0;
00346     UART2_SetRxInterruptHandler(UART2_Receive_ISR);
00347     PIE8bits.U2TXIE = 0;
00348     UART2_SetTxInterruptHandler(UART2_Transmit_ISR);
00349
00350     // Set the UART2 module to the options selected in the user interface.
00351
00352     // P1L 0;
00353     U2P1L = 0x00;
00354
00355     // P1H 0;
00356     U2P1H = 0x00;
00357
00358     // P2L 0;
00359     U2P2L = 0x00;
00360
00361     // P2H 0;
00362     U2P2H = 0x00;
00363
00364     // P3L 0;
00365     U2P3L = 0x00;
00366
00367     // P3H 0;
00368     U2P3H = 0x00;
00369
00370     // BRGS high speed; MODE Asynchronous 8-bit mode; RXEN enabled; TXEN enabled; ABDEN disabled;
00371     U2CON0 = 0xB0;
00372
00373     // RXBIMD Set RXBKIF on rising RX input; BRKOVr disabled; WUE disabled; SENDB disabled; ON
    enabled;
00374     U2CON1 = 0x80;
00375

```

```

00376      // TXPOL not inverted; FLO off; C0EN Checksum Mode 0; RXPOL not inverted; RUNOVF RX input shifter
stops all activity; STP Transmit 1Stop bit, receiver verifies first Stop bit;
00377      U2CON2 = 0x00;
00378
00379      // BRGL 64;
00380      U2BRGL = 0x40;
00381
00382      // BRGH 3;
00383      U2BRGH = 0x03;
00384
00385      // STPMD in middle of first Stop bit; TXWRE No error;
00386      U2FIFO = 0x00;
00387
00388      // ABDIF Auto-baud not enabled or not complete; WUIF WUE not enabled by software; ABDIE disabled;
00389      U2UIR = 0x00;
00390
00391      // ABDOVF Not overflowed; TXCIF 0; RXBKIF No Break detected; RXFOIF not overflowed; CERIF No
Checksum error;
00392      U2ERRIR = 0x00;
00393
00394      // TXCIE disabled; FERIE disabled; TXMTIE disabled; ABDOVE disabled; CERIE disabled; RXFOIE
disabled; PERIE disabled; RXBKIE disabled;
00395      U2ERRIE = 0x00;
00396
00397
00398      UART2_SetFramingErrorHandler(UART2_DefaultFramingErrorHandler);
00399      UART2_SetOverrunErrorHandler(UART2_DefaultOverrunErrorHandler);
00400      UART2_SetErrorHandler(UART2_DefaultErrorHandler);
00401
00402      uart2RxLastError.status = 0;
00403
00404      // initializing the driver state
00405      uart2TxHead = 0;
00406      uart2TxTail = 0;
00407      uart2TxBufferRemaining = sizeof(uart2TxBuffer);
00408      uart2RxHead = 0;
00409      uart2RxTail = 0;
00410      uart2RxCount = 0;
00411
00412      // enable receive interrupt
00413      PIE8bits.U2RXIE = 1;
00414 }

```

6.73 uart2.h

```

00001
00023
00024 /*
00025      (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027      Subject to your compliance with these terms, you may use Microchip software and any
00028      derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029      license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031
00032      THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033      EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034      IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035      FOR A PARTICULAR PURPOSE.
00036
00037      IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038      INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039      WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040      HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041      THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042      CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043      OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044      SOFTWARE.
00045 */
00046
00047 #ifndef UART2_H
00048 #define UART2_H
00049
00053
00054 #include <xc.h>
00055 #include <stdbool.h>
00056 #include <stdint.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059
00060      extern "C" {

```

```

00061
00062 #endif
00063
00067
00068 #define UART2_DataReady (UART2_is_rx_ready())
00069
00073
00074 typedef union {
00075     struct {
00076         unsigned perr : 1;
00077         unsigned ferr : 1;
00078         unsigned oerr : 1;
00079         unsigned reserved : 5;
00080     };
00081     uint8_t status;
00082 }uart2_status_t;
00083
00087 extern volatile uint8_t uart2TxBufferRemaining;
00088 extern volatile uint8_t uart2RxCount;
00089
00093
00115 void UART2_Initialize(void);
00116
00163 bool UART2_is_rx_ready(void);
00164
00211 bool UART2_is_tx_ready(void);
00212
00258 bool UART2_is_tx_done(void);
00259
00306 uart2_status_t UART2_get_last_status(void);
00307
00355 uint8_t UART2_Read(void);
00356
00380 void UART2_Write(uint8_t txData);
00381
00382 void UART2_put_buffer(uint8_t);
00383
00403 void UART2_Transmit_ISR(void);
00404
00424 void UART2_Receive_ISR(void);
00425
00445 void UART2_RxDataHandler(void);
00446
00463 void UART2_SetFramingErrorHandler(void (* interruptHandler)(void));
00464
00481 void UART2_SetOverrunErrorHandler(void (* interruptHandler)(void));
00482
00499 void UART2_SetErrorHandler(void (* interruptHandler)(void));
00500
00501
00502
00519 void (*UART2_RxInterruptHandler)(void);
00520
00537 void (*UART2_TxInterruptHandler)(void);
00538
00539
00540
00557 void UART2_SetRxInterruptHandler(void (* InterruptHandler)(void));
00558
00575 void UART2_SetTxInterruptHandler(void (* InterruptHandler)(void));
00576
00577
00578
00579 #ifdef __cplusplus // Provide C++ Compatibility
00580
00581     }
00582
00583 #endif
00584
00585 #endif // UART2_H

```

6.74 mconfig.c

```

00001 #include "mconfig.h"
00002 #include "mydisplay.h"
00003 #include "msg_text.h"
00004 #include <string.h>
00005
00006 typedef struct D_data { // control data structure
00007     char lcd[MAX_VTERM][MAX_LCD_LINES][MAX_BUF + 1];
00008     uint8_t vterm : 2; // line to 4 possible lines
00009     D_CODES last_info;
00010 } D_data;

```

```

00011
00012 extern V_data V;
00013 static D_data D = {0};
00014
00015 /*
00016  * This MUST run first to clear the vterm buffers before using any other functions
00017  */
00018 void mconfig_init(void)
00019 {
00020     memset(&D, 0, sizeof(D));
00021 }
00022
00023 /*
00024  * hardware specific routines
00025  */
00026 void mode_lamp_dim(void)
00027 {
00028     set_lcd_dim(true);
00029 }
00030
00031 void mode_lamp_bright(void)
00032 {
00033     set_lcd_dim(false);
00034 }
00035
00036 /*
00037  * mode button help mode select
00038  * Not used
00039  */
00040 bool help_button(void)
00041 {
00042     if (!RB0_GetValue()) { // debounce and delay for button press
00043         V.help = true;
00044         if (TimerDone(TMR_HELP)) {
00045             return true;
00046         }
00047     }
00048
00049     if (!V.help) {
00050         StartTimer(TMR_HELP, BDELAY);
00051     }
00052
00053     return false;
00054 }
00055
00056 /*
00057  * Help Messages, show program compile date or extra message with flipper flag
00058  */
00059 void check_help(const bool flipper)
00060 {
00061     /*
00062      * show help display
00063      */
00064     if (V.help) {
00065         snprintf(get_vterm_ptr(0, HELP_VTERM), MAX_TEXT, " HELP Screen: %u          ", V.help_id +
00066 1);
00067         if (flipper) {
00068             snprintf(get_vterm_ptr(3, HELP_VTERM), MAX_TEXT, "HELP %s          ", build_date);
00069         } else {
00070             snprintf(get_vterm_ptr(3, HELP_VTERM), MAX_TEXT, "%s          ",
00071 T[V.help_id].extrams);
00072         }
00073         snprintf(get_vterm_ptr(2, HELP_VTERM), MAX_TEXT, "%s          ", T[V.help_id].display);
00074         snprintf(get_vterm_ptr(1, HELP_VTERM), MAX_TEXT, "%s          ", T[V.help_id].message);
00075         V.help_id++; // cycle help text messages to LCD
00076     }
00077 }
00078 /*
00079  * write character data to LCD from the selected vterm buffers
00080  * 20x4 lines, returns the current default terminal buffer number
00081  */
00082 uint8_t update_lcd(uint8_t vterm)
00083 {
00084     vterm = vterm & 0x03; // mask to 2 bits for buffer array
00085
00086     D.lcd[vterm][0][MAX_LINE] = 0;
00087     D.lcd[vterm][1][MAX_LINE] = 0;
00088     D.lcd[vterm][2][MAX_LINE] = 0;
00089     D.lcd[vterm][3][MAX_LINE] = 0;
00090     wait_lcd_done();
00091     eaDogM_WriteStringAtPos(0, 0, D.lcd[vterm][0]);
00092     wait_lcd_done();
00093     eaDogM_WriteStringAtPos(1, 0, D.lcd[vterm][1]);
00094     wait_lcd_done();
00095     eaDogM_WriteStringAtPos(2, 0, D.lcd[vterm][2]);
00096     wait_lcd_done();

```

```

00096     eaDogM_WriteStringAtPos(3, 0, D.lcd[vterm][3]);
00097     return D.vterm;
00098 }
00099
00100 /*
00101  * update the LCD screen with the current vterm buffers
00102  */
00103 uint8_t refresh_lcd(void)
00104 {
00105     return update_lcd(D.vterm);
00106 }
00107
00108 /*
00109  * set terminal window to 0..3 to internal vterm
00110  */
00111 uint8_t set_vterm(const uint8_t vterm)
00112 {
00113     D.vterm = vterm & 0x03;
00114     return D.vterm;
00115 }
00116
00117 /*
00118  * return pointer to vterm line buffer
00119  * Software diagnostic function, get address of pointers
00120  */
00121 char * get_vterm_ptr(uint8_t line, const uint8_t vterm)
00122 {
00123     return D.lcd[vterm & 0x03][line & 0x03];
00124 }
00125
00126 /*
00127  * 20x4 lines
00128  * Software diagnostic function, print address of pointers
00129  */
00130 void vterm_dump(void)
00131 {
00132     snprintf(V.buf, MAX_TEXT, "vterm %x:%x", (uint16_t) get_vterm_ptr(0, MAIN_VTERM),
00133             (uint16_t) get_vterm_ptr(0, INFO_VTERM));
00134     wait_lcd_done();
00135     eaDogM_WriteStringAtPos(0, MAIN_VTERM, V.buf);
00136     snprintf(V.buf, MAX_TEXT, "vterm %x:%x", (uint16_t) get_vterm_ptr(1, MAIN_VTERM),
00137             (uint16_t) get_vterm_ptr(1, INFO_VTERM));
00138     wait_lcd_done();
00139     eaDogM_WriteStringAtPos(1, MAIN_VTERM, V.buf);
00140     snprintf(V.buf, MAX_TEXT, "vterm %x:%x", (uint16_t) get_vterm_ptr(2, MAIN_VTERM),
00141             (uint16_t) get_vterm_ptr(2, INFO_VTERM));
00142     wait_lcd_done();
00143     eaDogM_WriteStringAtPos(2, MAIN_VTERM, V.buf);
00144     snprintf(V.buf, MAX_TEXT, "vterm %x:%x", (uint16_t) get_vterm_ptr(3, MAIN_VTERM),
00145             (uint16_t) get_vterm_ptr(3, INFO_VTERM));
00146     wait_lcd_done();
00147     eaDogM_WriteStringAtPos(3, MAIN_VTERM, V.buf);
00148     WaitMs(TDELAY);
00149 }
00150
00151 void vterm_sequence(void)
00152 {
00153     snprintf(get_vterm_ptr(2, INFO_VTERM), MAX_TEXT, " Mesg %d Stack %d", V.msg_error,
00154             V.stack);
00155     switch (V.response.info) {
00156     case DIS_LOG:
00157         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " S%dF%d log %d", V.function, V.response.log_seq & 0x03,
00158                 V.response.log_num);
00159         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " Stored #%d", V.response.log_num);
00160         break;
00161     case DIS_FREE:
00162         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " S%dF%d log %d", V.function, V.response.log_seq & 0x03,
00163                 V.response.log_num);
00164         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " Port Lock #%d", V.speed_spin);
00165         break;
00166     case DIS_LOAD:
00167         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " Ready LL", V.response.mcode);
00168         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " S2F41 #c", V.response.mcode);
00169         break;
00170     case DIS_PUMP:
00171         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " Pump LL", V.response.mcode);
00172         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " S2F41 #c", V.response.mcode);
00173         break;
00174     case DIS_UNLOAD:
00175         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " Open LL", V.response.mcode);
00176         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " S2F41 #c", V.response.mcode);
00177         break;
00178     case DIS_HELP:
00179         snprintf(get_vterm_ptr(0, HELP_VTERM), MAX_TEXT, "HELP %s", build_date);
00180         snprintf(get_vterm_ptr(1, HELP_VTERM), MAX_TEXT, "DISPLAY %s", build_time);
00181         break;
00182     }
00183 }

```

```

00175     case DIS_SEQUENCE:
00176         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " Load-lock num %d", V.lid);
00177         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " SEQUENCE %d", V.sequences);
00178         break;
00179     case DIS_SEQUENCE_M:
00180         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " Display num %d",
V.response.mesgid);
00181         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " SEQUENCE %d", V.sequences);
00182         break;
00183     case DIS_TERM:
00184         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, " Terminal %d", V.response.TID);
00185         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, " CMD %c %c Len %d",
V.response.mcode, V.response.mparm, V.response.cmdlen);
00186         break;
00187     case DIS_CLEAR:
00188     default:
00189         snprintf(get_vterm_ptr(0, MAIN_VTERM), MAX_TEXT, "
");
00190         snprintf(get_vterm_ptr(1, MAIN_VTERM), MAX_TEXT, "
");
00191         snprintf(get_vterm_ptr(0, INFO_VTERM), MAX_TEXT, "
");
00192         snprintf(get_vterm_ptr(1, INFO_VTERM), MAX_TEXT, "
");
00193         break;
00194     }
00195 }
00196
00197 /*
00198  * update possible command messages from equipment
00199  * not being used now
00200  */
00201 void MyeaDogM_WriteStringAtPos(const uint8_t r, const uint8_t c, char *strPtr)
00202 {
00203     wait_lcd_done();
00204     if (V.response.info == DIS_STR) {
00205         eaDogM_WriteStringAtPos(r, c, strPtr);
00206     } else {
00207         if (V.response.info == DIS_HELP) {
00208             snprintf(get_vterm_ptr(2, HELP_VTERM), MAX_TEXT, "%s", V.info);
00209             update_lcd(HELP_VTERM);
00210         } else {
00211             if ((V.response.info != DIS_STR)) {
00212                 update_lcd(INFO_VTERM);
00213             } else {
00214                 snprintf(get_vterm_ptr(2, MAIN_VTERM), MAX_TEXT, "%s", V.info);
00215                 update_lcd(MAIN_VTERM);
00216             }
00217         }
00218     }
00219     if ((V.response.info != DIS_STR) && TimerDone(TMR_INFO)) {
00220         V.response.info = DIS_STR;
00221     }
00222 }
00223
00224 /*
00225  * this is for possible message flipping with the HELP button
00226  */
00227 if (D.last_info == DIS_HELP && V.response.info != DIS_HELP) {
00228     // show some stuff, maybe
00229 }
00230
00231 D.last_info = V.response.info;
00232 }
00233
00234 inline D_CODES display_info(void)
00235 {
00236     return V.response.info;
00237 }
00238
00239 inline D_CODES display_help(void)
00240 {
00241     return V.response.help_temp;
00242 }
00243
00244 D_CODES set_display_info(const D_CODES new_response_info)
00245 {
00246     static D_CODES old_info = DIS_STR;
00247     if (TimerDone(TMR_INFO)) {
00248         old_info = V.response.info;
00249         V.response.info = new_response_info;
00250     }
00251     return old_info;
00252 }
00253
00254 D_CODES set_temp_display_help(const D_CODES new_response_info)
00255 {
00256     static D_CODES old_info;
00257     old_info = V.response.help_temp;

```



```

00260     V.response.help_temp = new_response_info;
00261     return old_info;
00262 }

```

6.75 mconfig.h

```

00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002 * and any derivatives exclusively with Microchip products.
00003 *
00004 * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005 * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006 * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007 * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008 * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009 *
00010 * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011 * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012 * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013 * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014 * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015 * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016 * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017 *
00018 * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019 * TERMS.
00020 */
00021
00022 /*
00023 * File:
00024 * Author:
00025 * Comments:
00026 * Revision history:
00027 */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef MCONFIG_H
00032 #define MCONFIG_H
00033
00034
00035 #include "mcc_generated_files/mcc.h"
00036 #include "vconfig.h"
00037
00038 void mconfig_init(void);
00039
00040 void mode_lamp_dim(void);
00041 void mode_lamp_bright(void);
00042
00043 #endif
00044

```

6.76 msg_text.h

```

00001 /*
00002 * File: msg_text.h
00003 * Author: root
00004 *
00005 * Created on July 15, 2019, 7:32 AM
00006 */
00007
00008 #ifndef MSG_TEXT_H
00009 #define MSG_TEXT_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014 #include "mconfig.h"
00015 #include "mydisplay.h"
00016
00017 typedef enum {
00018     display_message = 0,
00019     display_online,
00020     display_remote,
00021     display_gemhelp,
00022     display_free,
00023 } DISPLAY_TYPES;
00024
00025 /*

```

```

00026      * *****|*****|*****|*****|*****|***|
00027      */
00028      const char msg0[] = "Host A%d, R%d RF%d, T%d TF%d, C%d %d %s";
00029      const char msg1[] = "ONLINE A%d, R%d RF%d, T%d TF%d, C%d %d %s";
00030      const char msg2[] = "%s %d %s";
00031      const char msg_gemhelp[] = "%s %s";
00032      const char msg_free[] = "%s %s";
00033      const char msg99[] = "UNK FORMAT A%d, R%d RF%d, T%d TF%d, C%d %s";
00034
00035      V_help T[] = {
00036      {
00037          .message = "CMD: M, C",
00038          .display = "Message, Close",
00039          .extrams = "Text Message to Host",
00040      },
00041      {
00042          .message = "CMD: R, P, O",
00043          .display = "Ready, Pump, Open",
00044          .extrams = "Load Lock commands",
00045      },
00046      {
00047          .message = "CMD: L, S, D",
00048          .display = "Log, Sequence, Debug",
00049          .extrams = "",
00050      },
00051      {
00052          .message = "CMD: E, H, F",
00053          .display = "Errors, Help, Free",
00054          .extrams = "E: Test F: Speed",
00055      },
00056      };
00057
00058      #ifdef __cplusplus
00059      }
00060      #endif
00061
00062      #endif /* MSG_TEXT_H */
00063

```

6.77 mydisplay.h

```

00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002  * and any derivatives exclusively with Microchip products.
00003  *
00004  * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005  * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006  * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007  * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008  * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009  *
00010  * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011  * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012  * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013  * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014  * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015  * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016  * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017  *
00018  * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019  * TERMS.
00020  */
00021
00022 /*
00023  * File:
00024  * Author:
00025  * Comments:
00026  * Revision history:
00027  */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef XC_HEADER_TEMPLATE_H
00032 #define XC_HEADER_TEMPLATE_H
00033
00034 #include <xc.h> // include processor files - each processor file is guarded.
00035 #include <stdio.h>
00036 #include <string.h>
00037
00038 #include "vconfig.h"
00039 #include "eadog.h"
00040 #include "timers.h"
00041
00042 void MyeaDogM_WriteStringAtPos(const uint8_t, const uint8_t, char *);

```

```

00043 uint8_t update_lcd(uint8_t);
00044 uint8_t refresh_lcd(void);
00045 uint8_t set_vterm(const uint8_t);
00046 char * get_vterm_ptr(uint8_t, const uint8_t);
00047 void vterm_dump(void);
00048 void vterm_sequence(void);
00049 inline D_CODES display_info(void);
00050 inline D_CODES display_help(void);
00051 bool help_button(void);
00052 void check_help(const bool);
00053 D_CODES set_display_info(const D_CODES);
00054 D_CODES set_temp_display_help(const D_CODES);
00055
00056
00057 #ifdef __cplusplus
00058 extern "C" {
00059 #endif /* __cplusplus */
00060
00061     // TODO If C++ is being used, regular C code needs function names to have C
00062     // linkage so the functions can be used by the c code.
00063
00064 #ifdef __cplusplus
00065 }
00066 #endif /* __cplusplus */
00067
00068 #endif /* XC_HEADER_TEMPLATE_H */
00069

```

6.78 ringbufs.c

```

00001 #include <string.h>
00002 #include "ringbufs.h"
00003
00004 /*
00005  * general ring buffer fuctions from the internet
00006  */
00007 uint8_t modulo_inc(const uint8_t value, const uint8_t modulus)
00008 {
00009     uint8_t my_value = value + 1;
00010     if (my_value >= modulus) {
00011         my_value = 0;
00012     }
00013     return my_value;
00014 }
00015
00016 uint8_t modulo_dec(const uint8_t value, const uint8_t modulus)
00017 {
00018     uint8_t my_value = (0 == value) ? (modulus - 1) : (value - 1);
00019     return my_value;
00020 }
00021
00022 void ringBufS_init(volatile ringBufS_t *_this)
00023 {
00024     /*****
00025      The following clears:
00026      -> buf
00027      -> head
00028      -> tail
00029      -> count
00030      and sets head = tail
00031      ***/
00032     memset((void*)_this, 0, sizeof(*_this));
00033 }
00034
00035 int8_t ringBufS_empty(ringBufS_t *_this)
00036 {
00037     return (0 == _this->count);
00038 }
00039
00040 int8_t ringBufS_full(ringBufS_t *_this)
00041 {
00042     return (_this->count >= RBUF_SIZE);
00043 }
00044
00045 uint8_t ringBufS_get(ringBufS_t *_this)
00046 {
00047     uint8_t c;
00048     if (_this->count > 0) {
00049         c = _this->buf[_this->tail];
00050         _this->tail = modulo_inc(_this->tail, RBUF_SIZE);
00051         --_this->count;
00052     } else {
00053         c = 0; // return null with empty buffer

```

```

00054     }
00055     return(c);
00056 }
00057
00058 void ringBufS_put(ringBufS_t *_this, const uint8_t c)
00059 {
00060     if (_this->count < RBUF_SIZE) {
00061         _this->buf[_this->head] = c;
00062         _this->head = modulo_inc(_this->head, RBUF_SIZE);
00063         ++_this->count;
00064     }
00065 }
00066
00067 void ringBufS_put_dma(ringBufS_t *_this, const uint8_t c)
00068 {
00069     if (_this->count < RBUF_SIZE) {
00070         _this->buf[_this->head] = c;
00071         ++_this->head;
00072         ++_this->count;
00073     }
00074 }
00075
00076 void ringBufS_flush(ringBufS_t *_this, const int8_t clearBuffer)
00077 {
00078     _this->count = 0;
00079     _this->head = 0;
00080     _this->tail = 0;
00081     if (clearBuffer) {
00082         memset(_this->buf, 0, sizeof(_this->buf));
00083     }
00084 }
00085

```

6.79 ringbufs.h

```

00001 /*
00002  * File:   ringbufs.h
00003  * Author: rootswitcDC circuit breakerh debounce icdebounce icfuse type T
00004  *
00005  * Created on July 25, 2015, 2:03 PM
00006  */
00007
00008 #ifndef RINGBUFS_H
00009 #define RINGBUFS_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 #include "vconfig.h"
00016
00017 #define RBUF_SIZE 64
00018
00019 typedef struct ringBufS_t {
00020     uint8_t buf[RBUF_SIZE];
00021     uint8_t head;
00022     uint8_t tail;
00023     uint8_t count;
00024 } ringBufS_t;
00025
00026 void ringBufS_init(volatile ringBufS_t *_this);
00027 int8_t ringBufS_empty(ringBufS_t *_this);
00028 int8_t ringBufS_full(ringBufS_t *_this);
00029 uint8_t ringBufS_get(ringBufS_t *_this);
00030 void ringBufS_put(ringBufS_t *_this, const uint8_t c);
00031 void ringBufS_put_dma(ringBufS_t *_this, const uint8_t c);
00032 void ringBufS_flush(ringBufS_t *_this, const int8_t clearBuffer);
00033
00034
00035 #ifdef __cplusplus
00036 }
00037 #endif
00038
00039 #endif /* RINGBUFS_H */
00040

```

6.80 rs232.c

```

00001 #include "rs232.h"

```

```

00002
00003 static char line_status(const adc_result_t);
00004 extern struct V_data V;
00005
00006 /*
00007  * convert the raw ADC result into something close to the RS-232 wire voltages
00008  */
00009 void update_rs232_line_status(void)
00010 {
00011     V.tx_rs232 = line_status(V.v_tx_line);
00012     V.rx_rs232 = line_status(V.v_rx_line);
00013
00014     V.tx_volts = (adc_scale_zero + (int16_t) V.v_tx_line) / neg_scale; // negative scaling
00015     if (V.tx_volts > 0) {
00016         V.tx_volts = (adc_scale_zero + (int16_t) V.v_tx_line) / pos_scale; // positive scaling
00017     } else {
00018         if (V.tx_volts < line_zero_limit) { // check for ADC disconnect condition
00019             V.tx_volts = 0;
00020         }
00021     }
00022
00023     V.rx_volts = (adc_scale_zero + (int16_t) V.v_rx_line) / neg_scale;
00024     if (V.rx_volts > 0) {
00025         V.rx_volts = (adc_scale_zero + (int16_t) V.v_rx_line) / pos_scale;
00026     } else {
00027         if (V.rx_volts < line_zero_limit) {
00028             V.rx_volts = 0;
00029         }
00030     }
00031
00032 }
00033
00034 /*
00035  * convert raw ADC values tto RS232 line status indicators
00036  * D: Disconnected from anyload (device measurements error)
00037  * O: Open, Line receiver connected only voltage
00038  * M: Mark line condition
00039  * S: Space line condition
00040  */
00041
00042 static char line_status(const adc_result_t value)
00043 {
00044     char status_c = 'D';
00045
00046     if ((value < (LINE_OPEN_V - LINE_LIMIT_OPEN)) && (value > (LINE_RECV_V - LINE_LIMIT_OPEN))) {
00047         status_c = 'O';
00048     }
00049
00050     if ((value > (LINE_OPEN_V + LINE_LIMIT_LOW)) && (value < (LINE_SPACE_V + LINE_LIMIT_H))) {
00051         status_c = 'S';
00052     }
00053
00054     if ((value < (LINE_OPEN_V - LINE_LIMIT_MARK)) && (value > (LINE_MARK_V - LINE_LIMIT_H))) {
00055         status_c = 'M';
00056     }
00057
00058     return status_c;
00059 }

```

6.81 rs232.h

```

00001 /*
00002  * File:   rs232.h
00003  * Author: root
00004  *
00005  * Created on February 10, 2025, 3:52 PM
00006  */
00007
00008 #ifndef RS232_H
00009 #define RS232_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 #include "vconfig.h"
00016     const int16_t pos_scale = 40,
00017         neg_scale = 70,
00018         line_zero_limit = -24,
00019         adc_scale_zero = -2048;
00020     void update_rs232_line_status(void);
00021
00022 #ifdef __cplusplus

```

```

00023 }
00024 #endif
00025
00026 #endif /* RS232_H */
00027

```

6.82 timers.c

```

00001 #include <xc.h>
00002 #include <stdint.h>
00003 #include <stdbool.h>
00004 #include "vconfig.h"
00005 #include "timers.h"
00006
00007 extern volatile uint16_t tickCount[TMR_COUNT];
00008
00009 //*****
00010 // Start one of the software timers
00011
00012 inline void StartTimer(const uint8_t timer, const uint16_t count)
00013 {
00014     tickCount[timer] = count << 1; //Interrupt is every 500us but StartTimer() takes multiple of 1ms so
    multiply by 2
00015 }
00016
00017 //*****
00018 // Check if one of the software software timers has timed out
00019
00020 inline bool TimerDone(const uint8_t timer)
00021 {
00022     ClrWdt(); // reset the WDT timer
00023     if (tickCount[timer] == 0) { //Check if counted down to zero
00024         return true; //then return true
00025     }
00026     return false; //else return false
00027 }
00028
00029 //*****
00030 // Simple delay for n milliseconds (blocking)
00031
00032 void WaitMs(const uint16_t numMilliseconds)
00033 {
00034     StartTimer(TMR_INTERNAL, numMilliseconds); //Start software timer and wait for it to count down
00035     while (!TimerDone(TMR_INTERNAL)) {
00036         Nop();
00037         Nop();
00038         Nop();
00039         Nop();
00040         ClrWdt(); // reset the WDT timer
00041     }
00042 }
00043

```

6.83 timers.h

```

00001 #ifndef TIMERS_H
00002 #define TIMERS_H
00003
00004 #include <stdint.h>
00005 #include <stdbool.h>
00006
00007 //Software timers - use these to refer to timers rather than
00008 //integers. Add more as needed.
00009 //NOTE: 32767 ms max delay
00010
00011 enum APP_TIMERS {
00012     TMR_INTERNAL = 0, //Used in timers.c - do not remove or use elsewhere
00013     TMR_T1,
00014     TMR_T2,
00015     TMR_T3,
00016     TMR_T4,
00017     TMR_MC_TX,
00018     TMR_HBIO,
00019     TMR_INFO,
00020     TMR_HELP,

```

```

00021     TMR_HELPDIS,
00022     TMR_DISPLAY,
00023     TMR_SEQ,
00024     TMR_FLIPPER,
00025     //
00026     //(Add timers here as needed)
00027     //
00028     TMR_COUNT //number of timers - always keep at end of enum!
00029 };
00030
00031 inline void StartTimer(uint8_t timer, uint16_t count);
00032 inline bool TimerDone(uint8_t timer);
00033 void WaitMs(uint16_t numMilliseconds);
00034
00035 #endif //TIMERS_H

```

6.84 secs_q84.X/vconfig.h File Reference

Software version and a brief doc for each version changes.

```

#include <xc.h>
#include "mcc_generated_files/adc.h"
#include "mcc_generated_files/spi1.h"
#include "mcc_generated_files/pin_manager.h"

```

Data Structures

- struct [spi_link_type](#)
- struct [B_type](#)
- struct [terminal_type](#)
- struct [V_data](#)
- struct [V_help](#)

Macros

- #define [NHD](#)
- #define [VER](#) "2.16G"
- #define [SLED](#) MLED_LAT
enable two serial port internal Host Equipment RS-232 testing.
- #define [DLED](#) DLED_LAT
- #define [TRACE](#)
- #define [T1](#) 500
- #define [T2](#) 3000
- #define [T3](#) 5000
- #define [T4](#) 5000
- #define [HBTS](#) 5000
- #define [HBTL](#) 30000
- #define [RTY](#) 3
- #define [ERROR_CHECKSUM](#) 31500
- #define [ERROR_COMM](#) 31500
- #define [DEFAULT_TID](#) 1
- #define [TDELAY](#) 3000
- #define [SEQDELAY](#) 10000
- #define [LDELAY](#) 1000
- #define [SDELAY](#) 500

- #define `BDELAY` 300
- #define `DDELAY` 100
- #define `DFLIP` 1500
- #define `PINGER` 8
- #define `Y2KTIME`
- #define `TX_RESERVE` 59
- #define `V_OSCREEN` 93
- #define `V_SSCREEN` 94
- #define `E_OSCREEN` 81
- #define `E_SSCREEN` 81
- #define `S10F3_STR_POS` 135
- #define `S10F3_TID_POS` 138
- #define `MAX_STRLEN` 20
- #define `MAX_LINE` `MAX_STRLEN+1`
- #define `MAX_TEXT` `MAX_STRLEN+1`
- #define `MAX_HELP_STRLEN` 32
- #define `MAX_INFO` 63
- #define `MAX_BUF` 127
- #define `MAX_TERM` 159
- #define `MAX_VTERM` 4
- #define `MAX_LCD_LINES` 4
- #define `HELP_VTERM` 3
- #define `DEBUG_VTERM` 2
- #define `INFO_VTERM` 1
- #define `MAIN_VTERM` 0
- #define `SWITCH_VTERM` 70
- #define `SWITCH_DURATION` 32
- #define `LINE_OPEN_V` 2048
- #define `LINE_RECV_V` 1920
- #define `LINE_MARK_V` 1425
- #define `LINE_SPACE_V` 2390
- #define `LINE_LIMIT_H` 500
- #define `LINE_LIMIT_LOW` 100
- #define `LINE_LIMIT_MARK` 300
- #define `LINE_LIMIT_OPEN` 50
- #define `SPIN_CHAR` 19
- #define `UART_SPEED_EADR` 0x03F0
- #define `UART_SPEED_LOCK_EADR` 0x03F1
- #define `MAX_SHAPES` 6
- #define `SPIN_VAL_UPDATE` 5

Typedefs

- typedef struct B_type **B_type**
- typedef struct terminal_type **terminal_type**
- typedef struct V_data **V_data**
- typedef struct V_help **V_help**

Enumerations

- enum **P_CODES** {
CODE_TS = 0 , **CODE_TM** = 1 , **CODE_ONLOCAL** = 2 , **CODE_ONREMOTE** = 3 ,
CODE_OFFLINE = 4 , **CODE_DEBUG** , **CODE_LOG** , **CODE_LOAD** ,
CODE_UNLOAD , **CODE_PUMP** , **CODE_HELP** , **CODE_SEQUENCE** ,
CODE_ERROR , **CODE_FREE** , **CODE_ERR** }
- enum **D_CODES** {
DIS_STR = 0 , **DIS_TERM** , **DIS_LOG** , **DIS_LOAD** ,
DIS_UNLOAD , **DIS_PUMP** , **DIS_HELP** , **DIS_SEQUENCE** ,
DIS_SEQUENCE_M , **DIS_ERR** , **DIS_FREE** , **DIS_CLEAR** }
- enum **SEQ_STATES** {
SEQ_STATE_INIT = 0 , **SEQ_STATE_RX** , **SEQ_STATE_TX** , **SEQ_STATE_TRIGGER** ,
SEQ_STATE_QUEUE , **SEQ_STATE_DONE** , **SEQ_STATE_ERROR** }
- enum **UI_STATES** {
UI_STATE_INIT = 0 , **UI_STATE_HOST** , **UI_STATE_DEBUG** , **UI_STATE_LOG** ,
UI_STATE_ERROR }
- enum **GEM_STATES** {
GEM_STATE_DISABLE = 0 , **GEM_STATE_COMM** , **GEM_STATE_OFFLINE** , **GEM_STATE_ONLINE** ,
GEM_STATE_REMOTE , **GEM_STATE_ERROR** }
- enum **GEM_EQUIP** { **GEM_GENERIC** = 0 , **GEM_VII80** , **GEM_E220** , **GEM_ERROR** = 9 }
- enum **LINK_STATES** {
LINK_STATE_IDLE = 0 , **LINK_STATE_ENQ** , **LINK_STATE_EOT** , **LINK_STATE_ACK** ,
LINK_STATE_DONE , **LINK_STATE_NAK** , **LINK_STATE_ERROR** }
- enum **LINK_ERRORS** {
LINK_ERROR_NONE = 10 , **LINK_ERROR_T1** , **LINK_ERROR_T2** , **LINK_ERROR_T3** ,
LINK_ERROR_T4 , **LINK_ERROR_CHECKSUM** , **LINK_ERROR_NAK** , **LINK_ERROR_ABORT** ,
LINK_ERROR_SEND }
- enum **MSG_ERRORS** {
MSG_ERROR_NONE = 0 , **MSG_ERROR_ID** = 1 , **MSG_ERROR_STREAM** = 3 , **MSG_ERROR_↵**
FUNCTION = 5 ,
MSG_ERROR_DATA = 7 , **MSG_ERROR_TIMEOUT** = 9 , **MSG_ERROR_DATASIZE** = 11 , **MSG_ERROR_↵**
_RESET = 20 }
- enum **SEND_ERRORS** {
SEND_ERROR_NONE = 0 , **SEND_ERROR_ABORT** , **SEND_ERROR_EOT** , **SEND_ERROR_T2** ,
SEND_ERROR_T3 , **SEND_ERROR_DATA** }
- enum **RECV_ERRORS** {
RECV_ERROR_NONE = 0 , **RECV_ERROR_NAK** , **RECV_ERROR_EOT** , **RECV_ERROR_T2** ,
RECV_ERROR_T3 , **RECV_ERROR_CKSUM** , **RECV_ERROR_DATA** }
- enum **TICKER_VAL** { **TICKER_ZERO** = 0 , **TICKER_LOW** = 20 , **TICKER_HIGH** = 40 }

Functions

- char [spinners](#) (uint8_t, const uint8_t)
- void [UART1_Initialize19200](#) (void)
- void [UART2_Initialize19200](#) (void)

Variables

- const char * [GEM_TEXT](#) []
- const char [spin](#) [MAX_SHAPES][MAX_STRLEN]
- [B_type](#) B
- const char * [build_date](#)
- const char * [build_time](#)

6.84.1 Detailed Description

Software version and a brief doc for each version changes.

1.13G wafer load-lock control. 1.15 add specific equipment types V.e types. message sequencing in secs_II ↔ message. Version for 47Q84. uart to uart loopback testing with FAKER define. 2.02 more cleanups of code. 2.03 rework trace code. 2.04 program FAKER equipment side emulation. 2.05 Beta. 2.06 add TX bytes transmitted counter and display. 2.07 more info messages to equipment terminal display from host. 2.08 reformat LCD messages, add ADC 1 and 2 for RS232 line voltage checks. 2.09 add link stats for info screen. 2.10 data screens reformatting. 2.11 screen flipping timing adjustments. 2.12 remote set random error generation. 2.13 clear up display errors and make this a GAMMA software release version. 2.14 add 19200 or 9600 speed switching with reboot. 2.15 fix USART speed switching bugs. 2.16 switch to 16MHz external clock chip for 64MHz FOSC because CAN is not being used.

Definition in file [vconfig.h](#).

6.84.2 Macro Definition Documentation

6.84.2.1 BDELAY

```
#define BDELAY 300
```

Definition at line 86 of file [vconfig.h](#).

6.84.2.2 DBUG_VTERM

```
#define DBUG_VTERM 2
```

Definition at line 126 of file [vconfig.h](#).

6.84.2.3 DDELAY

```
#define DDELAY 100
```

Definition at line 87 of file [vconfig.h](#).

6.84.2.4 DEFAULT_TID

```
#define DEFAULT_TID 1
```

Definition at line 81 of file [vconfig.h](#).

6.84.2.5 DFLIP

```
#define DFLIP 1500
```

Definition at line 88 of file [vconfig.h](#).

6.84.2.6 DLED

```
#define DLED DLED_LAT
```

Definition at line 62 of file [vconfig.h](#).

6.84.2.7 E_OSCREEN

```
#define E_OSCREEN 81
```

Definition at line 105 of file [vconfig.h](#).

6.84.2.8 E_SSCREEN

```
#define E_SSCREEN 81
```

Definition at line 106 of file [vconfig.h](#).

6.84.2.9 ERROR_CHECKSUM

```
#define ERROR_CHECKSUM 31500
```

Definition at line 78 of file [vconfig.h](#).

6.84.2.10 ERROR_COMM

```
#define ERROR_COMM 31500
```

Definition at line 79 of file [vconfig.h](#).

6.84.2.11 HBTL

```
#define HBTL 30000
```

Definition at line 76 of file [vconfig.h](#).

6.84.2.12 HBTS

```
#define HBTS 5000
```

Definition at line 75 of file [vconfig.h](#).

6.84.2.13 HELP_VTERM

```
#define HELP_VTERM 3
```

Definition at line 125 of file [vconfig.h](#).

6.84.2.14 INFO_VTERM

```
#define INFO_VTERM 1
```

Definition at line 127 of file [vconfig.h](#).

6.84.2.15 LDELAY

```
#define LDELAY 1000
```

Definition at line 84 of file [vconfig.h](#).

6.84.2.16 LINE_LIMIT_H

```
#define LINE_LIMIT_H 500
```

Definition at line 137 of file [vconfig.h](#).

6.84.2.17 LINE_LIMIT_LOW

```
#define LINE_LIMIT_LOW 100
```

Definition at line 138 of file [vconfig.h](#).

6.84.2.18 LINE_LIMIT_MARK

```
#define LINE_LIMIT_MARK 300
```

Definition at line 139 of file [vconfig.h](#).

6.84.2.19 LINE_LIMIT_OPEN

```
#define LINE_LIMIT_OPEN 50
```

Definition at line 140 of file [vconfig.h](#).

6.84.2.20 LINE_MARK_V

```
#define LINE_MARK_V 1425
```

Definition at line 135 of file [vconfig.h](#).

6.84.2.21 LINE_OPEN_V

```
#define LINE_OPEN_V 2048
```

Definition at line 133 of file [vconfig.h](#).

6.84.2.22 LINE_RECV_V

```
#define LINE_RECV_V 1920
```

Definition at line 134 of file [vconfig.h](#).

6.84.2.23 LINE_SPACE_V

```
#define LINE_SPACE_V 2390
```

Definition at line 136 of file [vconfig.h](#).

6.84.2.24 MAIN_VTERM

```
#define MAIN_VTERM 0
```

Definition at line 128 of file [vconfig.h](#).

6.84.2.25 MAX_BUF

```
#define MAX_BUF 127
```

Definition at line 120 of file [vconfig.h](#).

6.84.2.26 MAX_HELP_STRLEN

```
#define MAX_HELP_STRLEN 32
```

Definition at line 117 of file [vconfig.h](#).

6.84.2.27 MAX_INFO

```
#define MAX_INFO 63
```

Definition at line 119 of file [vconfig.h](#).

6.84.2.28 MAX_LCD_LINES

```
#define MAX_LCD_LINES 4
```

Definition at line 123 of file [vconfig.h](#).

6.84.2.29 MAX_LINE

```
#define MAX_LINE MAX_STRLEN+1
```

Definition at line 115 of file [vconfig.h](#).

6.84.2.30 MAX_SHAPES

```
#define MAX_SHAPES 6
```

Definition at line 330 of file [vconfig.h](#).

6.84.2.31 MAX_STRLEN

```
#define MAX_STRLEN 20
```

Definition at line 114 of file [vconfig.h](#).

6.84.2.32 MAX_TERM

```
#define MAX_TERM 159
```

Definition at line 121 of file [vconfig.h](#).

6.84.2.33 MAX_TEXT

```
#define MAX_TEXT MAX_STRLEN+1
```

Definition at line 116 of file [vconfig.h](#).

6.84.2.34 MAX_VTERM

```
#define MAX_VTERM 4
```

Definition at line 122 of file [vconfig.h](#).

6.84.2.35 NHD

```
#define NHD
```

Definition at line 20 of file [vconfig.h](#).

6.84.2.36 PINGER

```
#define PINGER 8
```

Definition at line 90 of file [vconfig.h](#).

6.84.2.37 RTY

```
#define RTY 3
```

Definition at line 77 of file [vconfig.h](#).

6.84.2.38 S10F3_STR_POS

```
#define S10F3_STR_POS 135
```

Definition at line 108 of file [vconfig.h](#).

6.84.2.39 S10F3_TID_POS

```
#define S10F3_TID_POS 138
```

Definition at line 109 of file [vconfig.h](#).

6.84.2.40 SDELAY

```
#define SDELAY 500
```

Definition at line 85 of file [vconfig.h](#).

6.84.2.41 SEQDELAY

```
#define SEQDELAY 10000
```

Definition at line 83 of file [vconfig.h](#).

6.84.2.42 SLED

```
#define SLED MLED_LAT
```

enable two serial port internal Host Equipment RS-232 testing.

comment out FAKER for normal host mode

Definition at line 61 of file [vconfig.h](#).

6.84.2.43 SPIN_CHAR

```
#define SPIN_CHAR 19
```

Definition at line 142 of file [vconfig.h](#).

6.84.2.44 SPIN_VAL_UPDATE

```
#define SPIN_VAL_UPDATE 5
```

Definition at line 340 of file [vconfig.h](#).

6.84.2.45 SWITCH_DURATION

```
#define SWITCH_DURATION 32
```

Definition at line 131 of file [vconfig.h](#).

6.84.2.46 SWITCH_VTERM

```
#define SWITCH_VTERM 70
```

Definition at line 130 of file [vconfig.h](#).

6.84.2.47 T1

```
#define T1 500
```

Definition at line 71 of file [vconfig.h](#).

6.84.2.48 T2

```
#define T2 3000
```

Definition at line 72 of file [vconfig.h](#).

6.84.2.49 T3

```
#define T3 5000
```

Definition at line 73 of file [vconfig.h](#).

6.84.2.50 T4

```
#define T4 5000
```

Definition at line 74 of file [vconfig.h](#).

6.84.2.51 TDELAY

```
#define TDELAY 3000
```

Definition at line 82 of file [vconfig.h](#).

6.84.2.52 TRACE

```
#define TRACE
```

Definition at line 64 of file [vconfig.h](#).

6.84.2.53 TX_RESERVE

```
#define TX_RESERVE 59
```

Definition at line 98 of file [vconfig.h](#).

6.84.2.54 UART_SPEED_EADR

```
#define UART_SPEED_EADR 0x03F0
```

Definition at line 144 of file [vconfig.h](#).

6.84.2.55 UART_SPEED_LOCK_EADR

```
#define UART_SPEED_LOCK_EADR 0x03F1
```

Definition at line 145 of file [vconfig.h](#).

6.84.2.56 V_OSCREEN

```
#define V_OSCREEN 93
```

Definition at line 103 of file [vconfig.h](#).

6.84.2.57 V_SSCREEN

```
#define V_SSCREEN 94
```

Definition at line 104 of file [vconfig.h](#).

6.84.2.58 VER

```
#define VER "2.16G"
```

Definition at line 22 of file [vconfig.h](#).

6.84.2.59 Y2KTIME

```
#define Y2KTIME
```

Definition at line 92 of file [vconfig.h](#).

6.84.3 Enumeration Type Documentation

6.84.3.1 D_CODES

enum D_CODES

Definition at line 182 of file [vconfig.h](#).

6.84.3.2 GEM_EQUIP

enum GEM_EQUIP

Definition at line 235 of file [vconfig.h](#).

6.84.3.3 GEM_STATES

enum GEM_STATES

Definition at line 224 of file [vconfig.h](#).

6.84.3.4 LINK_ERRORS

enum LINK_ERRORS

Definition at line 252 of file [vconfig.h](#).

6.84.3.5 LINK_STATES

enum LINK_STATES

Definition at line 242 of file [vconfig.h](#).

6.84.3.6 MSG_ERRORS

enum MSG_ERRORS

Definition at line 264 of file [vconfig.h](#).

6.84.3.7 P_CODES

enum P_CODES

Definition at line 164 of file [vconfig.h](#).

6.84.3.8 RECV_ERRORS

enum RECV_ERRORS

Definition at line 284 of file [vconfig.h](#).

6.84.3.9 SEND_ERRORS

enum SEND_ERRORS

Definition at line 275 of file [vconfig.h](#).

6.84.3.10 SEQ_STATES

enum SEQ_STATES

Definition at line 206 of file [vconfig.h](#).

6.84.3.11 TICKER_VAL

enum TICKER_VAL

Definition at line 294 of file [vconfig.h](#).

6.84.3.12 UI_STATES

enum UI_STATES

Definition at line 216 of file [vconfig.h](#).

6.84.4 Function Documentation

6.84.4.1 spinners()

```
char spinners (
    uint8_t shape,
    const uint8_t reset) [extern]
```

Definition at line 1244 of file [main.c](#).

6.84.4.2 UART1_Initialize19200()

```
void UART1_Initialize19200 (
    void ) [extern]
```

Definition at line 341 of file [uart1.c](#).

6.84.4.3 UART2_Initialize19200()

```
void UART2_Initialize19200 (
    void ) [extern]
```

Definition at line 342 of file [uart2.c](#).

6.84.5 Variable Documentation

6.84.5.1 B

```
B_type B [extern]
```

Definition at line 231 of file [main.c](#).

6.84.5.2 build_date

```
const char* build_date
```

Definition at line 344 of file [vconfig.h](#).

6.84.5.3 build_time

```
const char * build_time
```

Definition at line 344 of file [vconfig.h](#).

6.84.5.4 GEM_TEXT

```
const char* GEM_TEXT[] [extern]
```

Definition at line 187 of file [main.c](#).

6.84.5.5 spin

```
const char spin[MAX_SHAPES][MAX_STRLEN]
```

Initial value:

```
= {
    "||//--",
    "||//--\\\\",
    "000000--",
    "vv<<^^>>",
    ". . * x # x # x X | | --",
    ". . o o o o o o"
}
```

Definition at line 331 of file [vconfig.h](#).

6.85 vconfig.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * File:    vconfig.h
00003  * Author:  root
00004  *
00005  * Created on February 19, 2019, 4:41 PM
00006  */
00007
00008 #ifndef VCONFIG_H
00009 #define VCONFIG_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00015 #include <xc.h>
00016 #include "mcc_generated_files/adc.h"
00017 #include "mcc_generated_files/spi1.h"
00018 #include "mcc_generated_files/pin_manager.h"
00019
00020 #define NHD          // SPI 20X4 display, nhd-0420d3z-nsw-bbw
00021
00022 #define VER "2.16G"
00023
00024 // #define FAKER          // emulate equipment protocol to host loopback
00025
00026 // #define FRAME_OVERRUN
00027
00028 // #define DB1
00029 // #define DB2
00030 // #define DB3
00031 // #define DB4
00032 // generate 'random' checksum/link errors to simulate rs-232 bit errors, set V.rerror to true in
the code init for V or the equipment can send a E terminal message to the host
00033
00034 #define SLED          MLED_LAT
00035 #define DLED          DLED_LAT
00036
00037 #define TRACE
00038 // EXT_IO PIN 5, RB6 PGC
00039 // EXT_IO pin 6, RB7 PGD, I_TRACE
00040 // EXT_IO pin 2, MISC_IO pin 5, RD5 M_TRACE
00041 // EXT_IO pin 3, MISC_IO pin 7, RD7 DEBUG1/E_TRACE
00042
00043 #define T1 500
00044 #define T2 3000
00045 #define T3 5000
00046 #define T4 5000
00047 #define HBTS 5000 // short ping delay
00048 #define HBTL 30000 // ping delay
00049 #define RTY 3
00050 #define ERROR_CHECKSUM 31500
00051 #define ERROR_COMM 31500
00052
00053 #define DEFAULT_TID 1
00054 #define TDELAY 3000
00055 #define SEQDELAY 10000
00056 #define LDELAY 1000
00057 #define SDELAY 500
00058 #define BDELAY 300
00059 #define DDELAY 100 // display update spacing
00060 #define DFLIP 1500 // display info flipping spacing
00061
00062 #define PINGER 8
00063
00064 #define Y2KTIME
00065
00066 /*
00067  * offsets in bytes
00068  */
00069
00070 #define TX_RESERVE 59
00071
00072 /*
00073  * Host message ACK CEIDs
00074  */
00075 #define V_OSCREEN 93
00076 #define V_SSCREEN 94
00077 #define E_OSCREEN 81
00078 #define E_SSCREEN 81
00079
00080 #define S10F3_STR_POS 135

```

```

00109 #define S10F3_TID_POS 138
00110
00111 /*
00112  * characters per line on the display
00113  */
00114 #define MAX_STRLEN 20
00115 #define MAX_LINE MAX_STRLEN+1
00116 #define MAX_TEXT MAX_STRLEN+1
00117 #define MAX_HELP_STRLEN 32
00118
00119 #define MAX_INFO 63
00120 #define MAX_BUF 127
00121 #define MAX_TERM 159
00122 #define MAX_VTERM 4
00123 #define MAX_LCD_LINES 4
00124
00125 #define HELP_VTERM 3
00126 #define DEBUG_VTERM 2
00127 #define INFO_VTERM 1
00128 #define MAIN_VTERM 0
00129
00130 #define SWITCH_VTERM 70 // time between main to info screen switches
00131 #define SWITCH_DURATION 32 // time in the info screen
00132
00133 #define LINE_OPEN_V 2048 // open wire
00134 #define LINE_RECV_V 1920 // connected to receiver input only
00135 #define LINE_MARK_V 1425 // xmit -8 volts
00136 #define LINE_SPACE_V 2390 // xmit +9 volts
00137 #define LINE_LIMIT_H 500
00138 #define LINE_LIMIT_LOW 100
00139 #define LINE_LIMIT_MARK 300
00140 #define LINE_LIMIT_OPEN 50
00141
00142 #define SPIN_CHAR 19
00143
00144 #define UART_SPEED_EADR 0x03F0 // offset from 0x380000
00145 #define UART_SPEED_LOCK_EADR 0x03F1 // offset from 0x380000
00146
00147 struct spi_link_type { // internal SPI state table
00148     uint8_t SPI_LCD : 1;
00149     uint8_t SPI_AUX : 1;
00150     uint8_t LCD_TIMER : 1;
00151     volatile uint8_t LCD_DATA : 1;
00152     uint16_t delay;
00153     uint8_t config;
00154     uint8_t * txbuf;
00155     volatile int32_t int_count;
00156 };
00157
00158 typedef struct B_type {
00159     volatile bool one_sec_flag;
00160     volatile uint16_t dim_delay;
00161     volatile bool display_update;
00162 } B_type;
00163
00164 typedef enum {
00165     CODE_TS = 0,
00166     CODE_TM = 1,
00167     CODE_ONLOCAL = 2, // CEID codes
00168     CODE_ONREMOTE = 3,
00169     CODE_OFFLINE = 4,
00170     CODE_DEBUG,
00171     CODE_LOG,
00172     CODE_LOAD,
00173     CODE_UNLOAD,
00174     CODE_PUMP,
00175     CODE_HELP,
00176     CODE_SEQUENCE,
00177     CODE_RERROR,
00178     CODE_FREE,
00179     CODE_ERR,
00180 } P_CODES;
00181
00182 typedef enum {
00183     DIS_STR = 0,
00184     DIS_TERM,
00185     DIS_LOG,
00186     DIS_LOAD,
00187     DIS_UNLOAD,
00188     DIS_PUMP,
00189     DIS_HELP,
00190     DIS_SEQUENCE,
00191     DIS_SEQUENCE_M,
00192     DIS_ERR,
00193     DIS_FREE,
00194     DIS_CLEAR,
00195 } D_CODES;

```

```
00196
00197     typedef struct terminal_type {
00198         uint8_t ack[32], msgid;
00199         uint8_t TID, mcode, mparm, cmdlen, log_seq;
00200         uint8_t host_display_ack : 1;
00201         D_CODES info, help_temp;
00202         uint16_t ceid;
00203         uint16_t log_num;
00204     } terminal_type;
00205
00206     typedef enum {
00207         SEQ_STATE_INIT = 0,
00208         SEQ_STATE_RX,
00209         SEQ_STATE_TX,
00210         SEQ_STATE_TRIGGER,
00211         SEQ_STATE_QUEUE,
00212         SEQ_STATE_DONE,
00213         SEQ_STATE_ERROR
00214     } SEQ_STATES;
00215
00216     typedef enum {
00217         UI_STATE_INIT = 0,
00218         UI_STATE_HOST,
00219         UI_STATE_DEBUG,
00220         UI_STATE_LOG,
00221         UI_STATE_ERROR
00222     } UI_STATES;
00223
00224     typedef enum {
00225         GEM_STATE_DISABLE = 0,
00226         GEM_STATE_COMM,
00227         GEM_STATE_OFFLINE,
00228         GEM_STATE_ONLINE,
00229         GEM_STATE_REMOTE,
00230         GEM_STATE_ERROR
00231     } GEM_STATES;
00232
00233     extern const char * GEM_TEXT [];
00234
00235     typedef enum {
00236         GEM_GENERIC = 0,
00237         GEM_VII80,
00238         GEM_F220,
00239         GEM_ERROR = 9
00240     } GEM_EQUIP;
00241
00242     typedef enum {
00243         LINK_STATE_IDLE = 0,
00244         LINK_STATE_ENQ,
00245         LINK_STATE_EOT,
00246         LINK_STATE_ACK,
00247         LINK_STATE_DONE,
00248         LINK_STATE_NAK,
00249         LINK_STATE_ERROR
00250     } LINK_STATES;
00251
00252     typedef enum {
00253         LINK_ERROR_NONE = 10,
00254         LINK_ERROR_T1,
00255         LINK_ERROR_T2,
00256         LINK_ERROR_T3,
00257         LINK_ERROR_T4,
00258         LINK_ERROR_CHECKSUM,
00259         LINK_ERROR_NAK,
00260         LINK_ERROR_ABORT,
00261         LINK_ERROR_SEND
00262     } LINK_ERRORS;
00263
00264     typedef enum {
00265         MSG_ERROR_NONE = 0,
00266         MSG_ERROR_ID = 1,
00267         MSG_ERROR_STREAM = 3,
00268         MSG_ERROR_FUNCTION = 5,
00269         MSG_ERROR_DATA = 7,
00270         MSG_ERROR_TIMEOUT = 9,
00271         MSG_ERROR_DATASIZE = 11,
00272         MSG_ERROR_RESET = 20
00273     } MSG_ERRORS;
00274
00275     typedef enum {
00276         SEND_ERROR_NONE = 0,
00277         SEND_ERROR_ABORT,
00278         SEND_ERROR_EOT,
00279         SEND_ERROR_T2,
00280         SEND_ERROR_T3,
00281         SEND_ERROR_DATA,
00282     } SEND_ERRORS;
```

```

00283
00284     typedef enum {
00285         RECV_ERROR_NONE = 0,
00286         RECV_ERROR_NAK,
00287         RECV_ERROR_EOT,
00288         RECV_ERROR_T2,
00289         RECV_ERROR_T3,
00290         RECV_ERROR_CKSUM,
00291         RECV_ERROR_DATA,
00292     } RECV_ERRORS;
00293
00294     typedef enum {
00295         TICKER_ZERO = 0,
00296         TICKER_LOW = 20,
00297         TICKER_HIGH = 40,
00298     } TICKER_VAL;
00299
00300     typedef struct V_data { // control data structure
00301         SEQ_STATES s_state;
00302         UI_STATES ui_state;
00303         GEM_STATES g_state;
00304         GEM_EQUIP e_types;
00305         LINK_STATES m_l_state;
00306         LINK_STATES r_l_state;
00307         LINK_STATES t_l_state;
00308         char buf[MAX_BUF + 1], terminal[MAX_TERM + 1], info[MAX_INFO + 1];
00309         uint32_t ticks, systemb, tx_total, rx_total, bt_total, brn_total, btn_total;
00310         int32_t testing;
00311         uint8_t stream, function, error, abort, msg_error, msg_ret, alarm;
00312         UI_STATES ui_sw;
00313         uint16_t r_checksum, t_checksum, checksum_error, timer_error, ping, mode_pwm, equip_timeout,
sequences, all_errors;
00314         uint8_t rbit : 1, wbit : 1, ebit : 1, failed_send : 4, failed_receive : 4;
00315         terminal_type response;
00316         uint8_t uart, llid, sid, ping_count, euart, vterm, vterm_switch, uart_speed_fast;
00317         volatile uint8_t ticker;
00318         bool flipper, queue, debug, help, stack, help_id, rerror, speed_spin, set_sequ;
00319         adc_result_t v_tx_line, v_rx_line;
00320         int16_t tx_volts, rx_volts;
00321         char tx_rs232, rx_rs232;
00322     } V_data;
00323
00324     typedef struct V_help {
00325         const char message[MAX_HELP_STRLEN], display[MAX_HELP_STRLEN], extrams[MAX_HELP_STRLEN];
00326     } V_help;
00327
00328     extern char spinners(uint8_t, const uint8_t);
00329     /* spinner defines */
00330     #define MAX_SHAPES 6
00331     const char spin[MAX_SHAPES][MAX_STRLEN] = {
00332         "||//--", // classic LCD version with no \ character
00333         "||//--\\", // classic
00334         "OOOOO--", // eye blink
00335         "vv<^>", // point spinner
00336         "..*x#x#XX||--", // warp portal
00337         "..ooOOoo" // ball bouncer
00338     };
00339
00340     #define SPIN_VAL_UPDATE 5
00341
00342     extern B_type B;
00343
00344     const char *build_date, *build_time;
00345     extern void UART1_Initialize19200(void);
00346     extern void UART2_Initialize19200(void);
00347
00348     #ifdef __cplusplus
00349     }
00350     #endif
00351
00352     #endif /* VCONFIG_H */
00353

```


Chapter 7

Examples

7.1 void

Section: MEMORY APIs.

Section: MEMORY APIs This routine reads one data byte from given address of Program Flash Memory

Returns

Data byte read from given Program Flash Memory address

Parameters

in	Address	of Program Flash Memory location to be read <code>main(void) { uint8_t readByte; SYSTEM_Initialize(); readByte = FLASH_ReadByte(0xFF00); }</code>
----	---------	---

7.2 Use

This routine reads one block from given address of Program Flash Memory into device Buffer RAM.

This routine reads one block from given address of Program Flash Memory into device Buffer RAM

Returns

None

Parameters

in	Starting	address of a Program Flash Memory block case: Read one page from given PFM location into device Buffer RAM using FLASH_ReadPage() Transfer the contents of device Buffer RAM to U1TXB using DMA2
----	----------	--

```
void main(void) {  
    SYSTEM_Initialize();
```

```
//Read one page from PFM into device Buffer RAM FLASH_ReadPage();
```

```
//Initialize DMA2 to copy data from device Buffer RAM to U1TXB register //Configure source size as 256 and destination size as 1 DMA2_Initialize(); }
```

7.3 uint16_t

This routine writes 128 words from given block start address.

This routine writes 128 words from given block start address

Returns

Status of operation

Parameters

in	<i>Starting</i>	address of a Program Flash Memory block
in	<i>Address</i>	of RAM buffer to be copied to Program Flash Memory block flashWrBufPtr[WRITE_FLASH_BLOCKSIZE] = { 0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x2, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x3, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x4, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x5, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x6, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x7, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x8, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0xa, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0xb, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0xc, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0xd, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0xe, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0xf, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0xf, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8 }; void main(void) { SYSTEM_Initialize(); FLASH_WriteBlock(0x1FF00,flashWrBufPtr); }

7.4 /root/vtouch_v2/secs_q84.X/mcc_generated_files/pwm2_16bit.h

Setter for slice 1, parameter 1 out interrupt handler.

Setter for slice 1, parameter 1 out interrupt handler

Returns

None

Parameters

<i>None</i>	
-------------	--

```
void Slice1Output1_CustomInterruptHandler(void) { //Custom interrupt handler code }
```

```
void main\(void\) { SYSTEM_Initialize();
```

```
//Use custom handler for Slice1Output1 interrupt PWM2_16BIT_Slice1Output1_SetInterruptHandler(Slice1↵  
Output1_CustomInterruptHandler); }
```

```
/*  
 (c) 2018 Microchip Technology Inc. and its subsidiaries.
```

```
Subject to your compliance with these terms, you may use Microchip software and any  
derivatives exclusively with Microchip products. It is your responsibility to comply with third party  
license terms applicable to your use of third party software (including open source software) that  
may accompany Microchip software.
```

THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.

*/

```
#ifndef PWM2_16BIT_H
#define PWM2_16BIT_H
```

```
#include <stdint.h>
#include <stdbool.h>
```

```
void PWM2_16BIT_Initialize(void);
```

```
void PWM2_16BIT_Enable();
```

```
void PWM2_16BIT_Disable();
```

```
void PWM2_16BIT_WritePeriodRegister(uint16_t periodCount);
```

```
void PWM2_16BIT_SetSlicelOutput1DutyCycleRegister(uint16_t value);
```

```
void PWM2_16BIT_SetSlicelOutput2DutyCycleRegister(uint16_t value);
```

```
void PWM2_16BIT_LoadBufferRegisters(void);
```

```
void PWM2_16BIT_SlicelOutput1_SetInterruptHandler(void (* InterruptHandler)(void));
```

```
void PWM2_16BIT_SlicelOutput2_SetInterruptHandler(void (* InterruptHandler)(void));
```

```
void PWM2_16BIT_Period_SetInterruptHandler(void (* InterruptHandler)(void));
```

```
#endif //PWM2_16BIT_H
```


Index

- [__DEFINED_int24_t](#)
 - [main.c, 107](#)
- [abort](#)
 - [V_data, 60](#)
- [ack](#)
 - [terminal_type, 54](#)
- [ADC3: Analog-to-Digital Converter with Context, 7](#)
 - [ADC_channel_t, 9](#)
 - [ADC_ClearAccumulator, 10](#)
 - [ADC_context_t, 9](#)
 - [ADC_DefineSetPoint, 10](#)
 - [ADC_DisableChannelScan, 10](#)
 - [ADC_DisableChannelSequencer, 11](#)
 - [ADC_DisableChargePump, 11](#)
 - [ADC_DisableContinuousConversion, 12](#)
 - [ADC_DischargeSampleCapacitor, 12](#)
 - [ADC_EnableChannelScan, 13](#)
 - [ADC_EnableChannelSequencer, 13](#)
 - [ADC_EnableChargePump, 14](#)
 - [ADC_EnableContinuousConversion, 14](#)
 - [ADC_EnableDoubleSampling, 15](#)
 - [ADC_GetAccumulatorValue, 15](#)
 - [ADC_GetConversionResult, 16](#)
 - [ADC_GetConversionStageStatus, 16](#)
 - [ADC_GetCurrentCountofConversions, 17](#)
 - [ADC_GetErrorCalculation, 17](#)
 - [ADC_GetFilterValue, 18](#)
 - [ADC_GetPreviousResult, 18](#)
 - [ADC_GetSingleConversion, 19](#)
 - [ADC_HasAccumulatorOverflowed, 19](#)
 - [ADC_HasErrorCrossedLowerThreshold, 20](#)
 - [ADC_HasErrorCrossedUpperThreshold, 20](#)
 - [ADC_Initialize, 21](#)
 - [ADC_IsConversionDone, 21](#)
 - [ADC_LoadAcquisitionRegister, 22](#)
 - [adc_result_t, 9](#)
 - [ADC_SelectContext, 22](#)
 - [ADC_SetActiveClockTuningInterruptHandler, 23](#)
 - [ADC_SetADInterruptHandler, 23](#)
 - [ADC_SetContext1ThresholdInterruptHandler, 24](#)
 - [ADC_SetContext2ThresholdInterruptHandler, 24](#)
 - [ADC_SetContext3ThresholdInterruptHandler, 25](#)
 - [ADC_SetContext4ThresholdInterruptHandler, 25](#)
 - [ADC_SetLowerThreshold, 26](#)
 - [ADC_SetPrechargeTime, 26](#)
 - [ADC_SetRepeatCount, 27](#)
 - [ADC_SetStopOnInterrupt, 27](#)
 - [ADC_SetUpperThreshold, 28](#)
 - [ADC_StartChannelSequencer, 28](#)
 - [ADC_StartConversion, 29](#)
 - [ADC_StopConversion, 29](#)
- [ADC_channel_t](#)
 - [ADC3: Analog-to-Digital Converter with Context, 9](#)
- [ADC_ClearAccumulator](#)
 - [ADC3: Analog-to-Digital Converter with Context, 10](#)
- [ADC_context_t](#)
 - [ADC3: Analog-to-Digital Converter with Context, 9](#)
- [ADC_DefineSetPoint](#)
 - [ADC3: Analog-to-Digital Converter with Context, 10](#)
- [ADC_DisableChannelScan](#)
 - [ADC3: Analog-to-Digital Converter with Context, 10](#)
- [ADC_DisableChannelSequencer](#)
 - [ADC3: Analog-to-Digital Converter with Context, 11](#)
- [ADC_DisableChargePump](#)
 - [ADC3: Analog-to-Digital Converter with Context, 11](#)
- [ADC_DisableContinuousConversion](#)
 - [ADC3: Analog-to-Digital Converter with Context, 12](#)
- [ADC_DischargeSampleCapacitor](#)
 - [ADC3: Analog-to-Digital Converter with Context, 12](#)
- [ADC_EnableChannelScan](#)
 - [ADC3: Analog-to-Digital Converter with Context, 13](#)
- [ADC_EnableChannelSequencer](#)
 - [ADC3: Analog-to-Digital Converter with Context, 13](#)
- [ADC_EnableChargePump](#)
 - [ADC3: Analog-to-Digital Converter with Context, 14](#)
- [ADC_EnableContinuousConversion](#)
 - [ADC3: Analog-to-Digital Converter with Context, 14](#)
- [ADC_EnableDoubleSampling](#)
 - [ADC3: Analog-to-Digital Converter with Context, 15](#)
- [ADC_GetAccumulatorValue](#)
 - [ADC3: Analog-to-Digital Converter with Context, 15](#)
- [ADC_GetConversionResult](#)
 - [ADC3: Analog-to-Digital Converter with Context, 16](#)
- [ADC_GetConversionStageStatus](#)
 - [ADC3: Analog-to-Digital Converter with Context, 16](#)
- [ADC_GetCurrentCountofConversions](#)
 - [ADC3: Analog-to-Digital Converter with Context, 17](#)
- [ADC_GetErrorCalculation](#)
 - [ADC3: Analog-to-Digital Converter with Context, 17](#)
- [ADC_GetFilterValue](#)
 - [ADC3: Analog-to-Digital Converter with Context, 18](#)
- [ADC_GetPreviousResult](#)
 - [ADC3: Analog-to-Digital Converter with Context, 18](#)
- [ADC_GetSingleConversion](#)
 - [ADC3: Analog-to-Digital Converter with Context, 19](#)
- [ADC_HasAccumulatorOverflowed](#)
 - [ADC3: Analog-to-Digital Converter with Context, 19](#)
- [ADC_HasErrorCrossedLowerThreshold](#)

- ADC3: Analog-to-Digital Converter with Context, [20](#)
- ADC_HasErrorCrossedUpperThreshold
 - ADC3: Analog-to-Digital Converter with Context, [20](#)
- ADC_Initialize
 - ADC3: Analog-to-Digital Converter with Context, [21](#)
- ADC_IsConversionDone
 - ADC3: Analog-to-Digital Converter with Context, [21](#)
- ADC_LoadAcquisitionRegister
 - ADC3: Analog-to-Digital Converter with Context, [22](#)
- adc_result_t
 - ADC3: Analog-to-Digital Converter with Context, [9](#)
- ADC_SelectContext
 - ADC3: Analog-to-Digital Converter with Context, [22](#)
- ADC_SetActiveClockTuningInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [23](#)
- ADC_SetADIIInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [23](#)
- ADC_SetContext1ThresholdInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [24](#)
- ADC_SetContext2ThresholdInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [24](#)
- ADC_SetContext3ThresholdInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [25](#)
- ADC_SetContext4ThresholdInterruptHandler
 - ADC3: Analog-to-Digital Converter with Context, [25](#)
- ADC_SetLowerThreshold
 - ADC3: Analog-to-Digital Converter with Context, [26](#)
- ADC_SetPrechargeTime
 - ADC3: Analog-to-Digital Converter with Context, [26](#)
- ADC_SetRepeatCount
 - ADC3: Analog-to-Digital Converter with Context, [27](#)
- ADC_SetStopOnInterrupt
 - ADC3: Analog-to-Digital Converter with Context, [27](#)
- ADC_SetUpperThreshold
 - ADC3: Analog-to-Digital Converter with Context, [28](#)
- ADC_StartChannelSequencer
 - ADC3: Analog-to-Digital Converter with Context, [28](#)
- ADC_StartConversion
 - ADC3: Analog-to-Digital Converter with Context, [29](#)
- ADC_StopConversion
 - ADC3: Analog-to-Digital Converter with Context, [29](#)
- alarm
 - V_data, [60](#)
- all_errors
 - V_data, [60](#)
- B
 - main.c, [108](#)
 - vconfig.h, [220](#)
- b
 - block10, [32](#)
- B_type, [31](#)
 - dim_delay, [31](#)
 - display_update, [31](#)
 - one_sec_flag, [31](#)
- baud
 - spi1_configuration_t, [52](#)
- BDELAY
 - vconfig.h, [210](#)
- bidh
 - block10_type, [33](#)
- bidl
 - block10_type, [33](#)
- block
 - block10, [32](#)
 - gem_display_type, [35](#)
 - gem_message_type, [36](#)
 - header10, [37](#)
 - header12, [38](#)
 - header13, [39](#)
 - header14, [40](#)
 - header153, [41](#)
 - header17, [42](#)
 - header18, [43](#)
 - header24, [44](#)
 - header254, [45](#)
 - header26, [46](#)
 - header27, [47](#)
 - header33, [48](#)
- block10, [32](#)
 - b, [32](#)
 - block, [32](#)
- block10_type, [32](#)
 - bidh, [33](#)
 - bidl, [33](#)
 - didh, [33](#)
 - didl, [33](#)
 - ebit, [33](#)
 - function, [33](#)
 - rbit, [33](#)
 - stream, [33](#)
 - systemb, [34](#)
 - wbit, [34](#)
- br_total
 - V_data, [61](#)
- brn_total
 - V_data, [61](#)
- bt_total
 - V_data, [61](#)
- btn_total
 - V_data, [61](#)
- buf
 - ringBufS_t, [50](#)
 - V_data, [61](#)
- build_date
 - main.c, [108](#)
 - vconfig.h, [220](#)
- build_time
 - main.c, [109](#)
 - vconfig.h, [220](#)
- ceid
 - terminal_type, [54](#)
- checksum
 - header10, [37](#)
 - header12, [38](#)
 - header13, [39](#)
 - header14, [40](#)

- header153, [41](#)
- header17, [42](#)
- header18, [43](#)
- header24, [44](#)
- header254, [45](#)
- header26, [46](#)
- header27, [47](#)
- header33, [48](#)
- checksum_error
 - V_data, [61](#)
- cmdlen
 - terminal_type, [55](#)
- con0
 - spi1_configuration_t, [52](#)
- con1
 - spi1_configuration_t, [52](#)
- con2
 - spi1_configuration_t, [52](#)
- config
 - spi_link_type, [53](#)
- count
 - ringBufS_t, [50](#)
- D
 - main.c, [109](#)
- D_CODES
 - vconfig.h, [218](#)
- D_data, [34](#)
 - last_info, [34](#)
 - lcd, [34](#)
 - vterm, [35](#)
- data
 - header12, [38](#)
 - header13, [39](#)
 - header14, [40](#)
 - header153, [41](#)
 - header17, [42](#)
 - header18, [43](#)
 - header24, [44](#)
 - header254, [45](#)
 - header26, [46](#)
 - header27, [48](#)
 - header33, [49](#)
- datam
 - header24, [44](#)
 - header26, [47](#)
- DEBUG_VTERM
 - vconfig.h, [210](#)
- DDELAY
 - vconfig.h, [210](#)
- debug
 - V_data, [61](#)
- DEFAULT_TID
 - vconfig.h, [210](#)
- delay
 - gem_display_type, [35](#)
 - gem_message_type, [36](#)
 - spi_link_type, [53](#)
- DFLIP
 - vconfig.h, [210](#)
- didh
 - block10_type, [33](#)
- didl
 - block10_type, [33](#)
- dim_delay
 - B_type, [31](#)
- display
 - V_help, [69](#)
- display_update
 - B_type, [31](#)
- DLED
 - vconfig.h, [210](#)
- E_OSCREEN
 - vconfig.h, [211](#)
- E_SSCREEN
 - vconfig.h, [211](#)
- e_types
 - V_data, [61](#)
- ebit
 - block10_type, [33](#)
 - V_data, [62](#)
- equip_timeout
 - V_data, [62](#)
- error
 - V_data, [62](#)
- ERROR_CHECKSUM
 - vconfig.h, [211](#)
- ERROR_COMM
 - vconfig.h, [211](#)
- euart
 - V_data, [62](#)
- extrams
 - V_help, [69](#)
- failed_receive
 - V_data, [62](#)
- failed_send
 - V_data, [62](#)
- faker
 - main.c, [109](#)
- ferr
 - uart1_status_t, [57](#)
 - uart2_status_t, [58](#)
- flipper
 - V_data, [62](#)
- function
 - block10_type, [33](#)
 - V_data, [62](#)
- g_state
 - V_data, [63](#)
- gem_display_type, [35](#)
 - block, [35](#)
 - delay, [35](#)
 - message, [35](#)
 - stack, [36](#)
- GEM_EQUIP

- vconfig.h, 218
- gem_message_type, 36
 - block, 36
 - delay, 36
 - message, 36
 - stack, 37
- GEM_STATES
 - vconfig.h, 218
- GEM_TEXT
 - main.c, 109
 - vconfig.h, 220
- H10
 - main.c, 109
- H12
 - main.c, 109
- H13
 - main.c, 109
- H14
 - main.c, 110
- H153
 - main.c, 110
- H17
 - main.c, 110
- H254
 - main.c, 110
- H26
 - main.c, 111
- H33
 - main.c, 111
- HBTL
 - vconfig.h, 211
- HBTS
 - vconfig.h, 211
- HC33
 - main.c, 111
- head
 - ringBufS_t, 51
- header
 - response_type, 49
- header10, 37
 - block, 37
 - checksum, 37
 - length, 37
- header12, 38
 - block, 38
 - checksum, 38
 - data, 38
 - length, 38
- header13, 39
 - block, 39
 - checksum, 39
 - data, 39
 - length, 39
- header14, 40
 - block, 40
 - checksum, 40
 - data, 40
 - length, 40
- header153, 41
 - block, 41
 - checksum, 41
 - data, 41
 - length, 41
- header17, 42
 - block, 42
 - checksum, 42
 - data, 42
 - length, 42
- header18, 43
 - block, 43
 - checksum, 43
 - data, 43
 - length, 43
- header24, 44
 - block, 44
 - checksum, 44
 - data, 44
 - datam, 44
 - length, 45
- header254, 45
 - block, 45
 - checksum, 45
 - data, 45
 - length, 46
- header26, 46
 - block, 46
 - checksum, 46
 - data, 46
 - datam, 47
 - length, 47
- header27, 47
 - block, 47
 - checksum, 47
 - data, 48
 - length, 48
- header33, 48
 - block, 48
 - checksum, 48
 - data, 49
 - length, 49
- help
 - V_data, 63
- help_id
 - V_data, 63
- help_temp
 - terminal_type, 55
- HELP_VTERM
 - vconfig.h, 211
- host_display_ack
 - terminal_type, 55
- I_TRACE
 - main.c, 107
- info
 - terminal_type, 55
 - V_data, 63
- INFO_VTERM

- vconfig.h, 211
- int24_t
 - main.c, 108
- int_count
 - spi_link_type, 53
- last_info
 - D_data, 34
- lcd
 - D_data, 34
- LCD_DATA
 - spi_link_type, 53
- LCD_TIMER
 - spi_link_type, 53
- LDELAY
 - vconfig.h, 212
- length
 - header10, 37
 - header12, 38
 - header13, 39
 - header14, 40
 - header153, 41
 - header17, 42
 - header18, 43
 - header24, 45
 - header254, 46
 - header26, 47
 - header27, 48
 - header33, 49
 - response_type, 49
- LINE_LIMIT_H
 - vconfig.h, 212
- LINE_LIMIT_LOW
 - vconfig.h, 212
- LINE_LIMIT_MARK
 - vconfig.h, 212
- LINE_LIMIT_OPEN
 - vconfig.h, 212
- LINE_MARK_V
 - vconfig.h, 212
- LINE_OPEN_V
 - vconfig.h, 212
- LINE_RECV_V
 - vconfig.h, 212
- LINE_SPACE_V
 - vconfig.h, 213
- LINK_ERRORS
 - vconfig.h, 218
- LINK_STATES
 - vconfig.h, 218
- llid
 - V_data, 63
- log_num
 - terminal_type, 55
- log_seq
 - terminal_type, 55
- m_l_state
 - V_data, 63
- M_TRACE
 - main.c, 107
- main
 - main.c, 108
- main.c
 - __DEFINED_int24_t, 107
 - B, 108
 - build_date, 108
 - build_time, 109
 - D, 109
 - faker, 109
 - GEM_TEXT, 109
 - H10, 109
 - H12, 109
 - H13, 109
 - H14, 110
 - H153, 110
 - H17, 110
 - H254, 110
 - H26, 111
 - H33, 111
 - HC33, 111
 - I_TRACE, 107
 - int24_t, 108
 - M_TRACE, 107
 - main, 108
 - mode_sw, 111
 - onesecond_io, 108
 - r_block, 112
 - S, 112
 - spi_link, 112
 - spinners, 108
 - tickCount, 112
 - V, 112
- MAIN_VTERM
 - vconfig.h, 213
- MAX_BUF
 - vconfig.h, 213
- MAX_HELP_STRLEN
 - vconfig.h, 213
- MAX_INFO
 - vconfig.h, 213
- MAX_LCD_LINES
 - vconfig.h, 213
- MAX_LINE
 - vconfig.h, 213
- MAX_SHAPES
 - vconfig.h, 213
- MAX_STRLEN
 - vconfig.h, 214
- MAX_TERM
 - vconfig.h, 214
- MAX_TEXT
 - vconfig.h, 214
- MAX_VTERM
 - vconfig.h, 214
- mcode
 - terminal_type, 55

msgid
 terminal_type, 55
 message
 gem_display_type, 35
 gem_message_type, 36
 V_help, 69
 mode_pwm
 V_data, 63
 mode_sw
 main.c, 111
 mparm
 terminal_type, 56
 msg_error
 V_data, 63
 MSG_ERRORS
 vconfig.h, 218
 msg_ret
 V_data, 64

 NHD
 vconfig.h, 214

 oerr
 uart1_status_t, 57
 uart2_status_t, 58
 one_sec_flag
 B_type, 31
 onesec_io
 main.c, 108
 operation
 spi1_configuration_t, 52

 P_CODES
 vconfig.h, 218
 perr
 uart1_status_t, 57
 uart2_status_t, 58
 ping
 V_data, 64
 ping_count
 V_data, 64
 PINGER
 vconfig.h, 214

 queue
 V_data, 64

 r_block
 main.c, 112
 r_checksum
 V_data, 64
 r_l_state
 V_data, 64
 rbit
 block10_type, 33
 V_data, 64
 RECV_ERRORS
 vconfig.h, 218
 reply

 response_type, 50
 reply_length
 response_type, 50
 rerror
 V_data, 64
 reserved
 uart1_status_t, 57
 uart2_status_t, 58
 respond
 response_type, 50
 response
 V_data, 65
 response_type, 49
 header, 49
 length, 49
 reply, 50
 reply_length, 50
 respond, 50
 ringBufS_t, 50
 buf, 50
 count, 50
 head, 51
 tail, 51
 RTY
 vconfig.h, 214
 rx_rs232
 V_data, 65
 rx_total
 V_data, 65
 rx_volts
 V_data, 65

S

 main.c, 112
 S10F3_STR_POS
 vconfig.h, 214
 S10F3_TID_POS
 vconfig.h, 215
 s_state
 V_data, 65
 SDELAY
 vconfig.h, 215
 secs_q84.X/0805L075WR/PADS/0805L075WR.c, 71
 secs_q84.X/0805L075WR/PADS/FUSC2114X125N.d,
 71
 secs_q84.X/build/default/production/eadog.p1.d, 72
 secs_q84.X/build/default/production/gemsecs.p1.d, 72
 secs_q84.X/build/default/production/main.p1.d, 72
 secs_q84.X/build/default/production/mcc_generated_files/adc.p1.d,
 73
 secs_q84.X/build/default/production/mcc_generated_files/device_config.p1.d,
 73
 secs_q84.X/build/default/production/mcc_generated_files/dma1.p1.d,
 73
 secs_q84.X/build/default/production/mcc_generated_files/interrupt_manage.p1.d,
 73
 secs_q84.X/build/default/production/mcc_generated_files/mcc.p1.d,
 73

- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/pin_manager.h, 159
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/pwm2_16bit.c, 163
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/pwm2_16bit.h, 166
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/pwm8.c, 167
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/pwm8.h, 167
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/spi1.c, 168
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/spi1.h, 170
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/tmr2.c, 170
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/tmr2.h, 172
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/tmr5.c, 176
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/tmr5.h, 178
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/tmr6.c, 179
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/tmr6.h, 181
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/uart1.c, 186
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/uart1.h, 190
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/uart2.c, 191
- secs_q84.X/build/default/production/mcc_generated_files/mcc_generated_files/uart2.h, 196
- secs_q84.X/build/default/production/mconfig.p1.d, 75
- secs_q84.X/build/default/production/rs232.p1.d, 75
- secs_q84.X/build/default/production/timers.p1.d, 75
- secs_q84.X/BZT52B5V6-TP/PADS/BZT52B5V6-TP.c, 75
- secs_q84.X/BZT52B5V6-TP/PADS/SOD3716X135N.d, 76
- secs_q84.X/dist/default/production/doprnt.d, 76
- secs_q84.X/eadog.c, 77
- secs_q84.X/eadog.h, 82
- secs_q84.X/gemsecs.c, 84
- secs_q84.X/gemsecs.h, 104
- secs_q84.X/main.c, 106, 113
- secs_q84.X/mcc_generated_files/adc.c, 128
- secs_q84.X/mcc_generated_files/adc.h, 133
- secs_q84.X/mcc_generated_files/clc1.c, 135
- secs_q84.X/mcc_generated_files/clc1.h, 136
- secs_q84.X/mcc_generated_files/clkref.c, 136
- secs_q84.X/mcc_generated_files/clkref.h, 137
- secs_q84.X/mcc_generated_files/device_config.c, 137
- secs_q84.X/mcc_generated_files/device_config.h, 140
- secs_q84.X/mcc_generated_files/dma1.c, 140
- secs_q84.X/mcc_generated_files/dma1.h, 142
- secs_q84.X/mcc_generated_files/dma2.c, 143
- secs_q84.X/mcc_generated_files/dma2.h, 144
- secs_q84.X/mcc_generated_files/dsm.c, 145
- secs_q84.X/mcc_generated_files/dsm.h, 146
- secs_q84.X/mcc_generated_files/ext_int.c, 147
- secs_q84.X/mcc_generated_files/ext_int.h, 148
- secs_q84.X/mcc_generated_files/interrupt_manager.c, 150
- secs_q84.X/mcc_generated_files/interrupt_manager.h, 150
- secs_q84.X/mcc_generated_files/mcc.c, 151
- secs_q84.X/mcc_generated_files/mcc.h, 152
- secs_q84.X/mcc_generated_files/memory.c, 153
- secs_q84.X/mcc_generated_files/memory.h, 156
- secs_q84.X/mcc_generated_files/nco1.c, 157
- secs_q84.X/mcc_generated_files/nco1.h, 158
- secs_q84.X/mcc_generated_files/pin_manager.c, 158
- secs_q84.X/mcc_generated_files/pin_manager.h, 159
- secs_q84.X/mcc_generated_files/pwm2_16bit.c, 163
- secs_q84.X/mcc_generated_files/pwm2_16bit.h, 166
- secs_q84.X/mcc_generated_files/pwm8.c, 167
- secs_q84.X/mcc_generated_files/pwm8.h, 167
- secs_q84.X/mcc_generated_files/spi1.c, 168
- secs_q84.X/mcc_generated_files/spi1.h, 170
- secs_q84.X/mcc_generated_files/tmr2.c, 170
- secs_q84.X/mcc_generated_files/tmr2.h, 172
- secs_q84.X/mcc_generated_files/tmr5.c, 176
- secs_q84.X/mcc_generated_files/tmr5.h, 178
- secs_q84.X/mcc_generated_files/tmr6.c, 179
- secs_q84.X/mcc_generated_files/tmr6.h, 181
- secs_q84.X/mcc_generated_files/uart1.c, 186
- secs_q84.X/mcc_generated_files/uart1.h, 190
- secs_q84.X/mcc_generated_files/uart2.c, 191
- secs_q84.X/mcc_generated_files/uart2.h, 196
- secs_q84.X/mconfig.c, 197
- secs_q84.X/mconfig.h, 201
- secs_q84.X/msg_text.h, 201
- secs_q84.X/mydisplay.h, 202
- secs_q84.X/ringbufs.c, 203
- secs_q84.X/ringbufs.h, 204
- secs_q84.X/rs232.c, 204
- secs_q84.X/rs232.h, 205
- secs_q84.X/timers.c, 206
- secs_q84.X/timers.h, 206
- secs_q84.X/vconfig.h, 207, 221
- SEND_ERRORS
 - vconfig.h, 219
- SEQ_STATES
 - vconfig.h, 219
- SEQDELAY
 - vconfig.h, 215
- sequences
 - V_data, 65
- set_sequ
 - V_data, 65
- sid
 - V_data, 65
- SLED
 - vconfig.h, 215
- speed_spin
 - V_data, 66
- spi1_configuration_t, 51
 - baud, 52
 - con0, 52
 - con1, 52
 - con2, 52
 - operation, 52
- SPI_AUX
 - spi_link_type, 53
- SPI_LCD
 - spi_link_type, 53
- spi_link
 - main.c, 112
- spi_link_type, 52
 - config, 53

- delay, [53](#)
 - int_count, [53](#)
 - LCD_DATA, [53](#)
 - LCD_TIMER, [53](#)
 - SPI_AUX, [53](#)
 - SPI_LCD, [53](#)
 - txbuf, [54](#)
- spin
 - vconfig.h, [220](#)
- SPIN_CHAR
 - vconfig.h, [215](#)
- SPIN_VAL_UPDATE
 - vconfig.h, [215](#)
- spinners
 - main.c, [108](#)
 - vconfig.h, [219](#)
- stack
 - gem_display_type, [36](#)
 - gem_message_type, [37](#)
 - V_data, [66](#)
- status
 - uart1_status_t, [57](#)
 - uart2_status_t, [58](#)
- stream
 - block10_type, [33](#)
 - V_data, [66](#)
- SWITCH_DURATION
 - vconfig.h, [215](#)
- SWITCH_VTERM
 - vconfig.h, [216](#)
- systemb
 - block10_type, [34](#)
 - V_data, [66](#)
- T1
 - vconfig.h, [216](#)
- T2
 - vconfig.h, [216](#)
- T3
 - vconfig.h, [216](#)
- T4
 - vconfig.h, [216](#)
- t_checksum
 - V_data, [66](#)
- t_l_state
 - V_data, [66](#)
- tail
 - ringBufS_t, [51](#)
- TDELAY
 - vconfig.h, [216](#)
- terminal
 - V_data, [66](#)
- terminal_type, [54](#)
 - ack, [54](#)
 - ceid, [54](#)
 - cmdlen, [55](#)
 - help_temp, [55](#)
 - host_display_ack, [55](#)
 - info, [55](#)
 - log_num, [55](#)
 - log_seq, [55](#)
 - mcode, [55](#)
 - mesgid, [55](#)
 - mparm, [56](#)
 - TID, [56](#)
- testing
 - V_data, [66](#)
- tickCount
 - main.c, [112](#)
- ticker
 - V_data, [67](#)
- TICKER_VAL
 - vconfig.h, [219](#)
- ticks
 - V_data, [67](#)
- TID
 - terminal_type, [56](#)
- timer_error
 - V_data, [67](#)
- TRACE
 - vconfig.h, [216](#)
- TX_RESERVE
 - vconfig.h, [216](#)
- tx_rs232
 - V_data, [67](#)
- tx_total
 - V_data, [67](#)
- tx_volts
 - V_data, [67](#)
- txbuf
 - spi_link_type, [54](#)
- uart
 - V_data, [67](#)
- UART1_Initialize19200
 - vconfig.h, [219](#)
- uart1_status_t, [56](#)
 - ferr, [57](#)
 - oerr, [57](#)
 - perr, [57](#)
 - reserved, [57](#)
 - status, [57](#)
- UART2_Initialize19200
 - vconfig.h, [219](#)
- uart2_status_t, [57](#)
 - ferr, [58](#)
 - oerr, [58](#)
 - perr, [58](#)
 - reserved, [58](#)
 - status, [58](#)
- UART_SPEED_EADR
 - vconfig.h, [217](#)
- uart_speed_fast
 - V_data, [67](#)
- UART_SPEED_LOCK_EADR
 - vconfig.h, [217](#)
- ui_state
 - V_data, [68](#)

- UI_STATES
 - vconfig.h, 219
- ui_sw
 - V_data, 68
- V
 - main.c, 112
- V_data, 59
 - abort, 60
 - alarm, 60
 - all_errors, 60
 - br_total, 61
 - brn_total, 61
 - bt_total, 61
 - btn_total, 61
 - buf, 61
 - checksum_error, 61
 - debug, 61
 - e_types, 61
 - ebit, 62
 - equip_timeout, 62
 - error, 62
 - euart, 62
 - failed_receive, 62
 - failed_send, 62
 - flipper, 62
 - function, 62
 - g_state, 63
 - help, 63
 - help_id, 63
 - info, 63
 - llid, 63
 - m_l_state, 63
 - mode_pwm, 63
 - msg_error, 63
 - msg_ret, 64
 - ping, 64
 - ping_count, 64
 - queue, 64
 - r_checksum, 64
 - r_l_state, 64
 - rbit, 64
 - rerror, 64
 - response, 65
 - rx_rs232, 65
 - rx_total, 65
 - rx_volts, 65
 - s_state, 65
 - sequences, 65
 - set_sequ, 65
 - sid, 65
 - speed_spin, 66
 - stack, 66
 - stream, 66
 - systemb, 66
 - t_checksum, 66
 - t_l_state, 66
 - terminal, 66
 - testing, 66
 - ticker, 67
 - ticks, 67
 - timer_error, 67
 - tx_rs232, 67
 - tx_total, 67
 - tx_volts, 67
 - uart, 67
 - uart_speed_fast, 67
 - ui_state, 68
 - ui_sw, 68
 - v_rx_line, 68
 - v_tx_line, 68
 - vterm, 68
 - vterm_switch, 68
 - wbit, 68
- V_help, 69
 - display, 69
 - extrams, 69
 - message, 69
- V_OSCREEN
 - vconfig.h, 217
- v_rx_line
 - V_data, 68
- V_SSCREEN
 - vconfig.h, 217
- v_tx_line
 - V_data, 68
- vconfig.h
 - B, 220
 - BDELAY, 210
 - build_date, 220
 - build_time, 220
 - D_CODES, 218
 - DEBUG_VTERM, 210
 - DDELAY, 210
 - DEFAULT_TID, 210
 - DFLIP, 210
 - DLED, 210
 - E_OSCREEN, 211
 - E_SSCREEN, 211
 - ERROR_CHECKSUM, 211
 - ERROR_COMM, 211
 - GEM_EQUIP, 218
 - GEM_STATES, 218
 - GEM_TEXT, 220
 - HBTL, 211
 - HBTS, 211
 - HELP_VTERM, 211
 - INFO_VTERM, 211
 - LDELAY, 212
 - LINE_LIMIT_H, 212
 - LINE_LIMIT_LOW, 212
 - LINE_LIMIT_MARK, 212
 - LINE_LIMIT_OPEN, 212
 - LINE_MARK_V, 212
 - LINE_OPEN_V, 212
 - LINE_RECV_V, 212
 - LINE_SPACE_V, 213

- LINK_ERRORS, [218](#)
- LINK_STATES, [218](#)
- MAIN_VTERM, [213](#)
- MAX_BUF, [213](#)
- MAX_HELP_STRLEN, [213](#)
- MAX_INFO, [213](#)
- MAX_LCD_LINES, [213](#)
- MAX_LINE, [213](#)
- MAX_SHAPES, [213](#)
- MAX_STRLEN, [214](#)
- MAX_TERM, [214](#)
- MAX_TEXT, [214](#)
- MAX_VTERM, [214](#)
- MSG_ERRORS, [218](#)
- NHD, [214](#)
- P_CODES, [218](#)
- PINGER, [214](#)
- RECV_ERRORS, [218](#)
- RTY, [214](#)
- S10F3_STR_POS, [214](#)
- S10F3_TID_POS, [215](#)
- SDELAY, [215](#)
- SEND_ERRORS, [219](#)
- SEQ_STATES, [219](#)
- SEQDELAY, [215](#)
- SLED, [215](#)
- spin, [220](#)
- SPIN_CHAR, [215](#)
- SPIN_VAL_UPDATE, [215](#)
- spinners, [219](#)
- SWITCH_DURATION, [215](#)
- SWITCH_VTERM, [216](#)
- T1, [216](#)
- T2, [216](#)
- T3, [216](#)
- T4, [216](#)
- TDELAY, [216](#)
- TICKER_VAL, [219](#)
- TRACE, [216](#)
- TX_RESERVE, [216](#)
- UART1_Initialize19200, [219](#)
- UART2_Initialize19200, [219](#)
- UART_SPEED_EADR, [217](#)
- UART_SPEED_LOCK_EADR, [217](#)
- UI_STATES, [219](#)
- V_OSCREEN, [217](#)
- V_SSCREEN, [217](#)
- VER, [217](#)
- Y2KTIME, [217](#)
- VER
 - vconfig.h, [217](#)
- vterm
 - D_data, [35](#)
 - V_data, [68](#)
- vterm_switch
 - V_data, [68](#)
- wbit
 - block10_type, [34](#)
- V_data, [68](#)
- Y2KTIME
 - vconfig.h, [217](#)